

IE437 Data-Driven Decision Making and Control
2023 Spring Semester

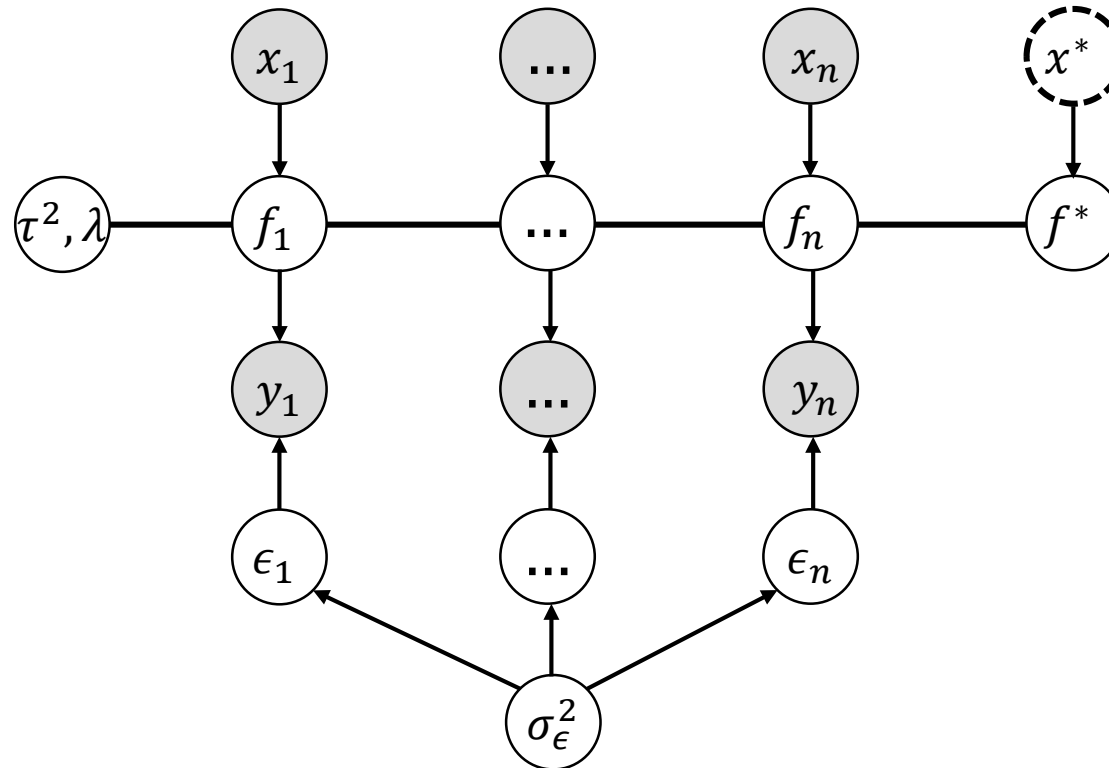
4. Bayesian Optimization

Jinkyoo Park
Associate Professor
Industrial and Systems Engineering Department

Gaussian Process Regression

Nonparametric Bayesian Regression

- Treat a target function $f(x)$ as a collection of random variables $\mathbf{f} = [f(x_1), \dots, f(x_n)]^T$



Input

$$\mathbf{X} = \{x_1, \dots, x_n\}$$

True function values

$$\mathbf{f} = \{f_1, \dots, f_n\}$$

Observation

$$\mathbf{y} = \{y_1, \dots, y_n\}$$

Noise

$$\boldsymbol{\epsilon} = \{\epsilon_1, \dots, \epsilon_n\}$$

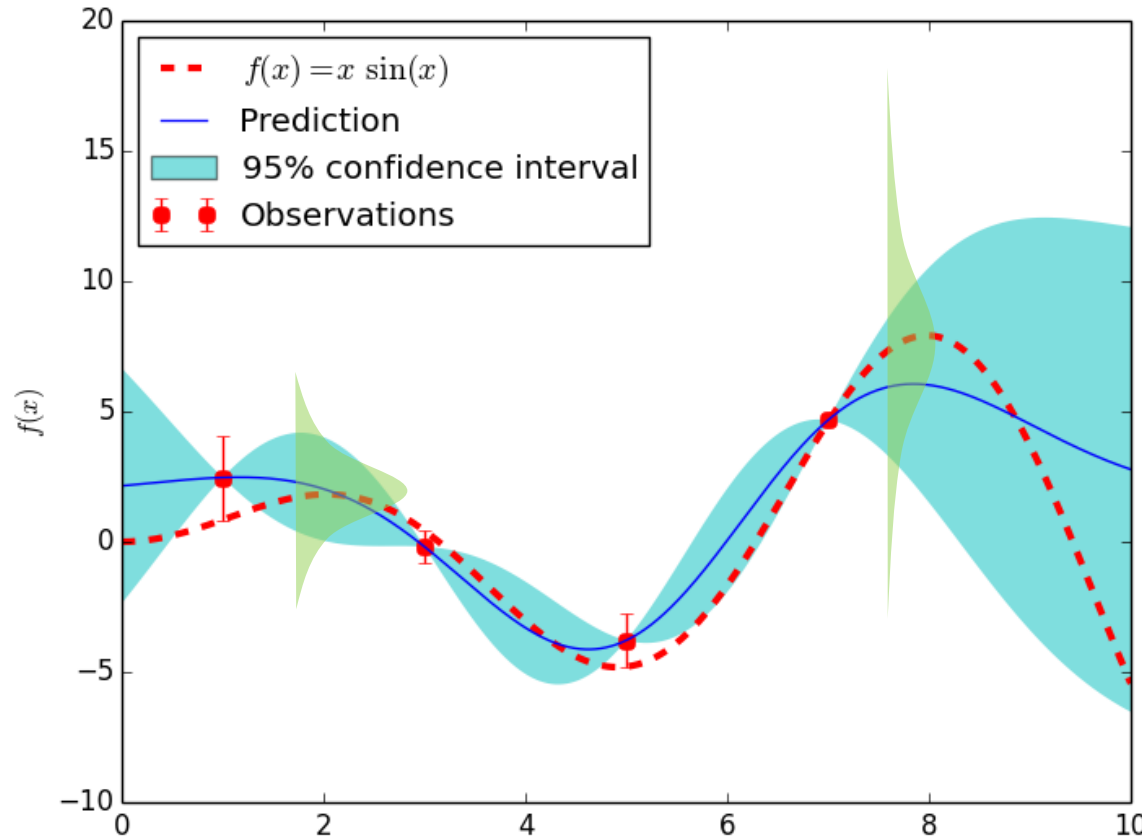
- Construct the distribution on unknown target value $f^* = f(x^*)$ corresponding x^* given

$$p(f^*|x^*, \mathbf{X}, \mathbf{y}) = \int_{\mathbf{f}} p(f^*|x^*, \mathbf{f}) \underline{p(\mathbf{f}|\mathbf{X}, \mathbf{y})} d\mathbf{f}$$

probability distribution over function $f(x)$, which
will be represented as Gaussian Process

Nonparametric Bayesian Regression

- Treat a target function $f(x)$ as a collection of random variables $\mathbf{f} = [f(x_1), \dots, f(x_n)]^T$



- Construct the distribution on unknown target value $f^* = f(x^*)$ corresponding x^* given

$$p(f^*|x^*, \mathbf{X}, \mathbf{y}) = \int_{\mathbf{f}} p(f^*|x^*, \mathbf{f}) \underline{p(\mathbf{f}|\mathbf{X}, \mathbf{y})} d\mathbf{f}$$

probability distribution over function $f(x)$, which will be represented as Gaussian Process

The Multivariate Normal Distribution (Lecture Note 2.1)

- **Notation**

$$Y = (Y_1, \dots, Y_k) \sim N(\mu, \Sigma)$$

$$p(y) = N(y|\mu, \Sigma) = \frac{1}{\sqrt{(2\pi)^k |\Sigma|}} e^{-\frac{1}{2}(y-\mu)^T \Sigma^{-1} (y-\mu)}, \quad \mathbf{y} \in \mathbb{R}^k$$

$$\mathbf{y} = (y_1, \dots, y_k): \text{vector}, \quad y_i \in [-\infty, \infty]$$

- **Parameters**

$$\mu = (\mu_1, \dots, \mu_k): \text{mean vector (location)}$$

$\Sigma \succcurlyeq 0$: positive semi – definite covariance matrix (scale) where $\Sigma_{i,j} = \text{cov}(y_i, y_j)$

- **Mean and variance**

$$\mathbb{E}[Y] = \mu$$

$$\text{Var}(Y) = \Sigma$$

The Multivariate Normal Distribution (Lecture Note 2.1)

- A random vector $Y = (Y_1, \dots, Y_k)$ is a Gaussian random vector (or $Y_1, \dots, Y_k$ are jointly Gaussian random variables) if the joint probability distribution function of the form

$$p(y) = N(y|\mu, \Sigma) = \frac{1}{\sqrt{(2\pi)^k |\Sigma|}} e^{-\frac{1}{2}(y-\mu)^T \Sigma^{-1} (y-\mu)}, \quad \mathbf{y} \in \mathbb{R}^k$$

Property 1.

For a Gaussian random variables, uncorrelation implies independence

- This can be verified by substituting $\sigma_{ij} = 0$ for all $i \neq j$ in the joint probability density function. Then Σ becomes diagonal and so does Σ^{-1} , and the joint probability density function reduces to the product of the marginal $Y_i \sim N(\mu_i, \sigma_{ii})$

The Multivariate Normal Distribution (Lecture Note 2.1)

Property 2.

Linear transformation of a Gaussian random variable yields a Gaussian random variable, that is, given any $m \times n$ matrix $\mathbf{A}$, where $m \leq n$ and $\mathbf{A}$ has full rank m , then

$$Z = \mathbf{A}Y \sim N(\mathbf{A}\mu, \mathbf{A}\Sigma\mathbf{A}^T)$$

Proof. Because

$$\mathbb{E}[Z] = \mathbb{E}[\mathbf{A}Y] = \mathbf{A}\mathbb{E}[Y] = \mathbf{A}\mu$$

$$\begin{aligned}\Sigma_Z &= \mathbb{E}[(Z - \mathbb{E}[Z])(Z - \mathbb{E}[Z])^T] \\ &= \mathbb{E}[(\mathbf{A}Y - \mathbf{A}\mu)(\mathbf{A}Y - \mathbf{A}\mu)^T] \\ &= \mathbf{A}\mathbb{E}[(Y - \mu)(Y - \mu)^T]\mathbf{A}^T = \mathbf{A}\Sigma\mathbf{A}^T\end{aligned}$$

This proof is not complete; we need to show the joint probability density function is in a right form.

The Multivariate Normal Distribution (Lecture Note 2.1)

Property 3.

Marginal of Gaussian random variables, that is, if Y is Gaussian random variables then for any subset $\{i_1, i_2, \dots, i_k\} \subset \{1, 2, \dots, n\}$ of indexes, the random variables

$$Z = \begin{bmatrix} Y_{i_1} \\ Y_{i_2} \\ \vdots \\ Y_{i_k} \end{bmatrix} \text{ is Gaussian random variable}$$

Converse is not generally true

- To show this, we use Property 2. For example, let $n = 3$ and $Z = \begin{bmatrix} Y_1 \\ Y_3 \end{bmatrix}$, we can express Z as a linear transformation of Y :

$$Z = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} Y_1 \\ Y_2 \\ Y_3 \end{bmatrix} = \begin{bmatrix} Y_1 \\ Y_3 \end{bmatrix}$$

- Therefore,

$$Z = N\left(\begin{bmatrix} \mu_1 \\ \mu_3 \end{bmatrix}, \begin{bmatrix} \sigma_{11} & \sigma_{13} \\ \sigma_{31} & \sigma_{33} \end{bmatrix}\right)$$

The Multivariate Normal Distribution (Lecture Note 2.1)

Property 4.

Conditional of a Gaussian random variables are Gaussians, more specifically, if

$$Z = \begin{bmatrix} Y_1 \\ Y_2 \end{bmatrix} \sim N \left(\begin{bmatrix} \mu_1 \\ \mu_2 \end{bmatrix}, \begin{bmatrix} \Sigma_{11} & \Sigma_{12} \\ \Sigma_{21} & \Sigma_{22} \end{bmatrix} \right)$$

where Y_1 is k -dim random variable and Y_2 is an $n - k$ dim random variable, then

$$\{Y_2 | Y_1 = y\} \sim N(\Sigma_{21}\Sigma_{11}^{-1}(y - \mu_1) + \mu_2, \Sigma_{22} - \Sigma_{21}\Sigma_{11}^{-1}\Sigma_{12})$$

- Example:

$$Z = \begin{bmatrix} Y_2 \\ Y_3 \end{bmatrix} \sim N \left(\begin{bmatrix} 1 \\ 2 \end{bmatrix}, \begin{bmatrix} 1 & 2 & 1 \\ 2 & 5 & 2 \\ 1 & 2 & 9 \end{bmatrix} \right)$$

$$\mathbb{E}[Z | Y_2 = y] = \begin{bmatrix} 2 \\ 1 \end{bmatrix} [1](y - 1) + \begin{bmatrix} 2 \\ 2 \end{bmatrix} = \begin{bmatrix} 2y \\ y + 1 \end{bmatrix}$$

$$\Sigma(Z | Y_2 = y) = \begin{bmatrix} 5 & 2 \\ 2 & 9 \end{bmatrix} - \begin{bmatrix} 2 \\ 1 \end{bmatrix} [1][2 \quad 1] = \begin{bmatrix} 1 & 0 \\ 0 & 8 \end{bmatrix}$$

Probability distributions over functions with finite domains

- Let $\mathcal{X} = \{x_1, x_2, \dots, x_n\}$ be any finite set of elements.
- Let consider the set $\mathcal{H}$ of all possible functions mapping from $\mathcal{X}$ to $\mathcal{R}$
- For example, one example of a function $f_0(\cdot) \in \mathcal{H}$ is given by

$$f_0(x_1) = 5, \quad f_0(x_2) = 2.3, \quad f_0(x_3) = -7, \quad \dots, \quad f_0(x_{n-1}) = -7, \quad f_0(x_n) = 8$$

- Since the domain of any $f(\cdot) \in \mathcal{H}$ has only m elements, we can always represent $f_0(\cdot)$ compactly as an n -dimensional vector, $\mathbf{f} = [f(x_1), f(x_2), \dots, f(x_n)]^T$.
- In order to specify a probability distribution over functions $f(\cdot) \in \mathcal{H}$, we must associate some “probability density” with each function in $\mathcal{H}$.
 - One natural way to do this is to express the probability distribution over $\mathcal{H}$ as the probability distribution over its vector representation $\mathbf{f}$.
 - For example, if $\mathbf{f} \sim \mathcal{N}(\boldsymbol{\mu}, \sigma^2 \mathbf{I})$, then this in turn implies a probability distribution over functions $f(\cdot)$, whose probability density function is given by

$$p(\mathbf{f}) = \prod_{i=1}^n \frac{1}{\sqrt{2\pi\sigma}} \exp\left(-\frac{1}{2\sigma^2} (f(x_i) - \mu_i)^2\right)$$

What is a Gaussian process?

- In the example above, we showed that probability distributions over functions with finite domains can be represented using a finite-dimensional multivariate Gaussian distribution over function outputs $f(x_1), f(x_2), \dots, f(x_n)$ at a finite number of input points $x_1, \dots, x_n$.
- How can we specify **probability distributions over functions** when the domain size may be infinite? For this, we turn to a fancier type of probability distribution known as a **Gaussian process**.
- Gaussian process (GP) is used as a prior distribution over infinite function.

Probability distributions over functions with infinite domains

- A stochastic process is a collection of random variables, $\{f(x): x \in \mathcal{X}\}$, indexed by elements from some set $\mathcal{X}$, known as the index set.
- A Gaussian process is a stochastic process such that any finite sub collection of random variables has a multivariate Gaussian distribution.
- In particular, a collection of random variables $\{f(x): x \in \mathcal{X}\}$ is said to be drawn from a Gaussian process with mean function $m(\cdot)$ and covariance function $k(\cdot, \cdot)$ if for any finite set of elements $x_1, \dots, x_n \in \mathcal{X}$, the associated finite set of random variables $f(x_1), f(x_2), \dots, f(x_n)$ have distribution

$$\begin{bmatrix} f(x_1) \\ \vdots \\ f(x_n) \end{bmatrix} \sim \text{N} \left(\begin{bmatrix} m(x_1) \\ \vdots \\ m(x_n) \end{bmatrix}, \begin{bmatrix} k(x_1, x_1) & \cdots & k(x_1, x_n) \\ \vdots & \ddots & \vdots \\ k(x_n, x_1) & \cdots & k(x_n, x_n) \end{bmatrix} \right)$$

Probability distributions over functions with infinite domains

- In particular, a collection of random variables $\{f(x): x \in \mathcal{X}\}$ is said to be drawn from a Gaussian process with mean function $m(\cdot)$ and covariance function $k(\cdot, \cdot)$ if for any finite set of elements $x_1, \dots, x_n \in \mathcal{X}$, the associated finite set of random variables $f(x_1), f(x_2), \dots, f(x_n)$ have distribution

$$\begin{bmatrix} f(x_1) \\ \vdots \\ f(x_n) \end{bmatrix} \sim \mathcal{N} \left(\begin{bmatrix} m(x_1) \\ \vdots \\ m(x_n) \end{bmatrix}, \begin{bmatrix} k(x_1, x_1) & \cdots & k(x_1, x_n) \\ \vdots & \ddots & \vdots \\ k(x_n, x_1) & \cdots & k(x_n, x_n) \end{bmatrix} \right)$$

- We denote using the notation

$$f(\cdot) \sim \mathcal{GP}(m(\cdot), k(\cdot, \cdot))$$

- mean function: $m(x) = \mathbb{E}[f(x)]$
- covariance function: $k(x, x') = \mathbb{E}[(f(x) - m(x))(f(x') - m(x'))]$
- Intuitively, one can think of a function $f(\cdot)$ drawn from a Gaussian process prior as an extremely high-dimensional vector drawn from an extremely high-dimensional multivariate Gaussian.

How to choose $m(x)$ and $k(x, x')$?

- $m(x)$ is planning as a prior for $f(x)$, describing the general trend in $f(x)$
 - Any real-valued function $m(\cdot)$ is acceptable.
 - In general, we set $m(\cdot) = 0$, since it does not hurt its expressibility but reduce the computation cost.
- For $k(x, x')$, it must be the case that for any set of elements $x_1, \dots, x_n \in \mathcal{X}$, the resulting covariance matrix

$$\mathbf{K} = \begin{bmatrix} k(x_1, x_1) & \cdots & k(x_1, x_n) \\ \vdots & \ddots & \vdots \\ k(x_n, x_1) & \cdots & k(x_n, x_n) \end{bmatrix}$$

is a valid covariance matrix corresponding to some multivariate Gaussian distribution.

- A standard result in probability theory states that this is true provided that $\mathbf{K}$ is **positive semidefinite**
- $k(x, x')$ describes the structure (smoothness) of a target function
- Various covariance function can be used depending on a target function

Gaussian process Regression

- Gaussian processes provide a method for modelling probability distributions over functions.
- From now on, we discuss how probability **distributions over functions (GP)** can be used in the framework of Bayesian regression.

- **Prior:**

$$p(\mathbf{f}) = \mathcal{GP}(m(\cdot), k(\cdot, \cdot))$$

- **Likelihood:**

$$y_i = f_i + \epsilon_i, \quad \epsilon_i \sim \mathcal{N}(0, \sigma_\epsilon^2) \rightarrow (\text{i.i.d.}) : p(\mathbf{y}|\mathbf{f}) = \mathcal{N}(\mathbf{f}_{1:n}, \sigma_\epsilon^2 \mathbf{I})$$

- **Posterior:**

$$p(\mathbf{f}|\mathbf{X}, \mathbf{y})$$

- **Posterior predictive distribution:**

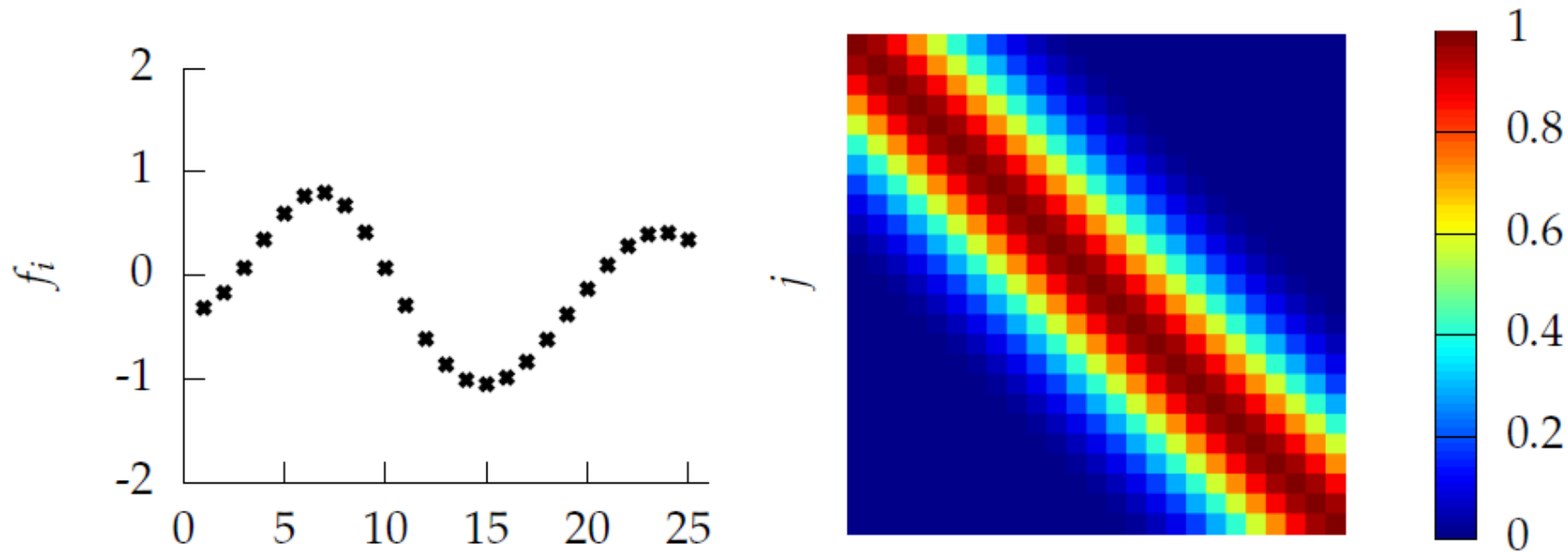
$$p(f^*|x^*, \mathbf{X}, \mathbf{y}) = \int_{\mathbf{f}} p(f^*|x^*, \mathbf{f}) p(\mathbf{f}|\mathbf{X}, \mathbf{y}) d\mathbf{f}$$

Intuition behind Gaussian process Regression

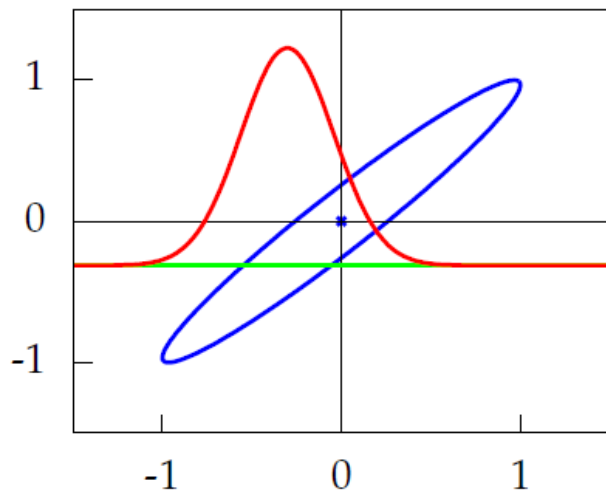
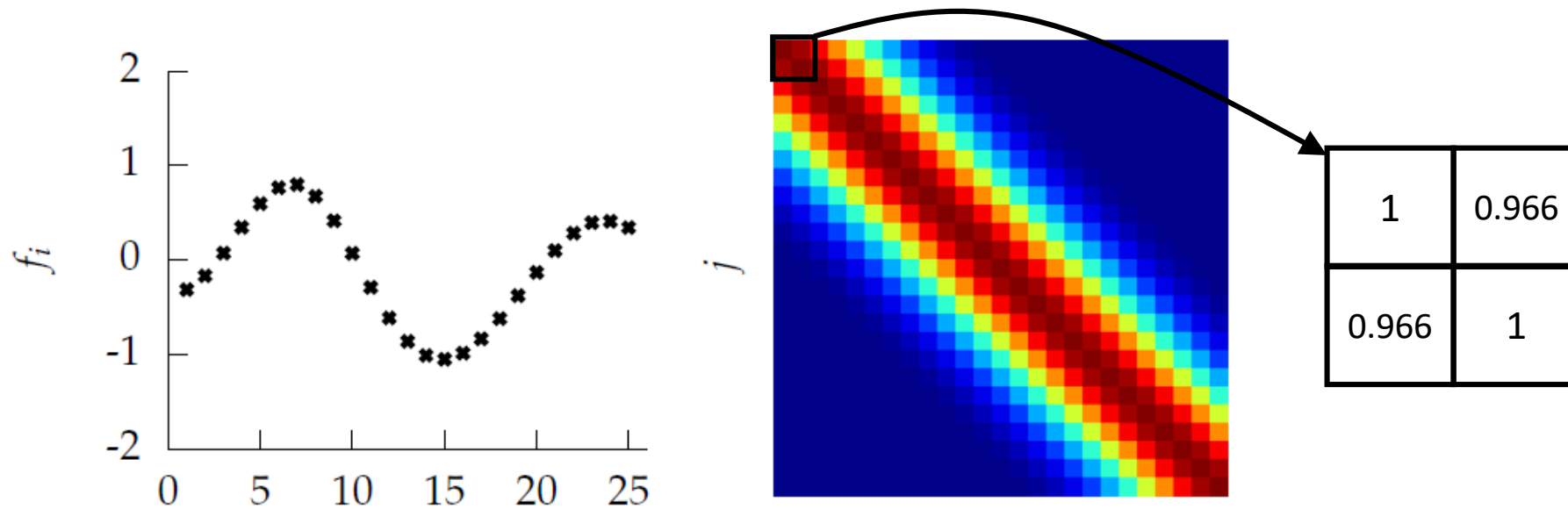
- Consider 25-dimensional random vector $\mathbf{f} = [f_1, f_2, \dots, f_{25}]$ that follow the GP prior

$$\begin{bmatrix} f_1 \\ \vdots \\ f_{25} \end{bmatrix} \sim \mathcal{N} \left(\mathbf{0}, \begin{bmatrix} k(x_1, x_1) & \cdots & k(x_1, x_{25}) \\ \vdots & \ddots & \vdots \\ k(x_{25}, x_1) & \cdots & k(x_{25}, x_{25}) \end{bmatrix} \right)$$

- One 25-dimensional samples are show bellow

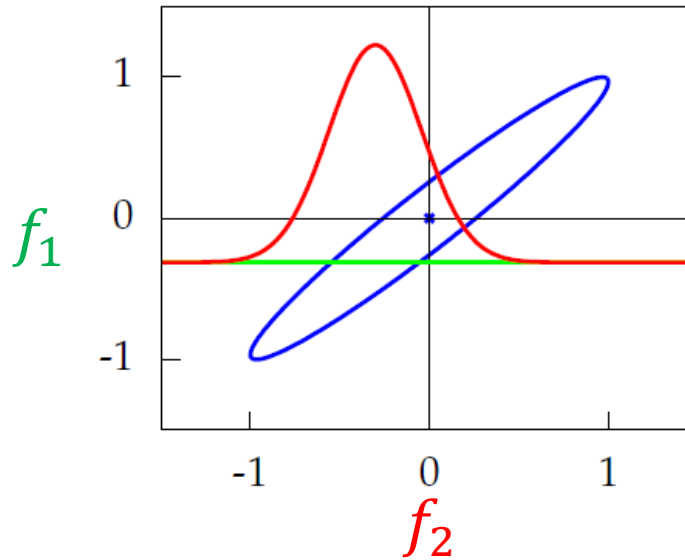


Intuition behind Gaussian process Regression



- The single contour of the Gaussian density represents the joint distribution $p(f_1, f_2)$.
- We observe that $f_1 = -0.313$
- Conditional density: $p(f_2 | f_1 = -0.313)$

Intuition behind Gaussian process Regression



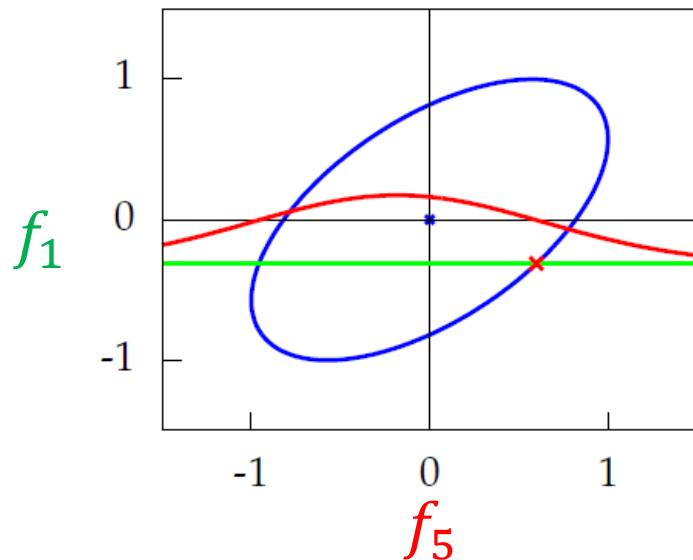
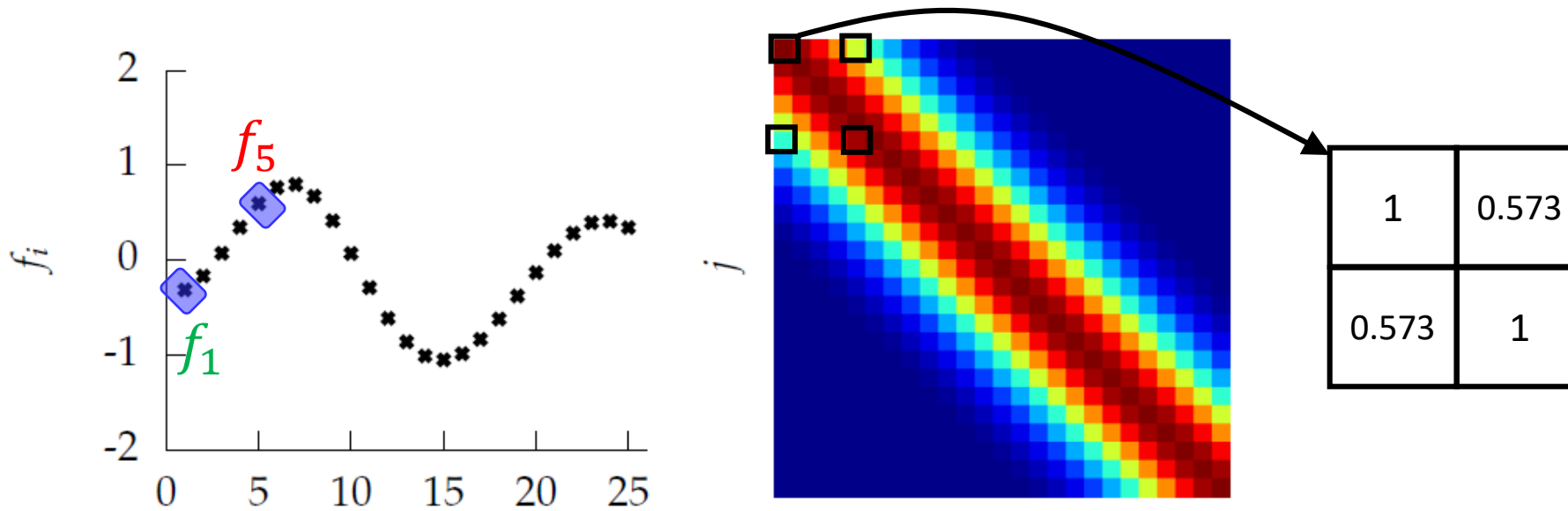
- Conditionals of a Gaussian random vector Z are Gaussians, more specifically, if

$$Z = \begin{bmatrix} f_1 \\ f_2 \end{bmatrix} \sim N \left(\begin{bmatrix} \mu_1 \\ \mu_2 \end{bmatrix}, \begin{bmatrix} \Sigma_{11} & \Sigma_{12} \\ \Sigma_{21} & \Sigma_{22} \end{bmatrix} \right)$$

then

$$\{f_2 | f_1 = y\} \sim N(\Sigma_{21}\Sigma_{11}^{-1}(y - \mu_1) + \mu_2, \Sigma_{22} - \Sigma_{21}\Sigma_{11}^{-1}\Sigma_{12})$$

Intuition behind Gaussian process Regression



- The single contour of the Gaussian density represents the joint distribution $p(f_1, f_5)$.
- We observe that $f_1 = -0.313$
- Conditional density: $p(f_5 | f_1 = -0.313)$

Gaussian process regression (Step by Step)

- Prior on the function values $\mathbf{f}_{1:n}$ is represented as Gaussian Process (GP)

$$p(\mathbf{f}_{1:n}) = \mathcal{GP}(m(\cdot), k(\cdot, \cdot)): \begin{bmatrix} f_1 \\ \vdots \\ f_n \end{bmatrix} \sim \mathcal{N} \left(\begin{bmatrix} m(x_1) \\ \vdots \\ m(x_n) \end{bmatrix}, \begin{bmatrix} k(x_1, x_1) & \cdots & k(x_1, x_n) \\ \vdots & \ddots & \vdots \\ k(x_n, x_1) & \cdots & k(x_n, x_n) \end{bmatrix} \right)$$

- $m(\cdot)$: mean function represents the overall trend of a target function
- $k(\cdot, \cdot)$: kernel function describes overall structure (smoothness) of a target function

$$\text{cov}(f_i, f_j) = \mathbb{E}[(f_i - m(x_i))(f_j - m(x_j))] \approx k(x_i, x_j)$$

- Likelihood is constructed base the assumption on the noise, i.e., i.i.d. Gaussian noise

$$y_i = f_i + \epsilon_i, \quad \epsilon_i \sim \mathcal{N}(0, \sigma_\epsilon^2) \rightarrow (\text{i.i.d.}) : p(\mathbf{y}_{1:n} | \mathbf{f}_{1:n}) = \mathcal{N}(\mathbf{f}_{1:n}, \sigma_\epsilon^2 \mathbf{I})$$

- Joint distribution based on Bayes' rule :

$$p(f, \mathbf{y}^{1:n}) = \int p(f, \mathbf{f}_{1:n}) p(\mathbf{y}^{1:n} | \mathbf{f}_{1:n}) d\mathbf{f}_{1:n} \rightarrow \begin{bmatrix} \mathbf{y}^{1:n} \\ f \end{bmatrix} \sim \mathcal{N} \left(\mathbf{0}, \begin{bmatrix} \mathbf{K} + \sigma_\epsilon^2 \mathbf{I} & \mathbf{k} \\ \mathbf{k}^T & k(x, x) \end{bmatrix} \right)$$

- Conditionalization $\rightarrow$ Posterior distribution on the function value $f = f(x)$ for x given $\mathcal{D} = \{\mathbf{x}_{1:n}, \mathbf{y}_{1:n}\}$

$$p(f | \mathcal{D}) = \mathcal{N}(\mu(x | \mathcal{D}), \sigma^2(x | \mathcal{D})) \rightarrow \begin{array}{l} \text{Mean : } \mu(x | \mathcal{D}) = \mathbf{k}^T (\mathbf{K} + \sigma_\epsilon^2 \mathbf{I})^{-1} \mathbf{y}_{1:n} \\ \text{Variance : } \sigma^2(x | \mathcal{D}) = k(x, x) - \mathbf{k}^T (\mathbf{K} + \sigma_\epsilon^2 \mathbf{I})^{-1} \mathbf{k} \end{array}$$

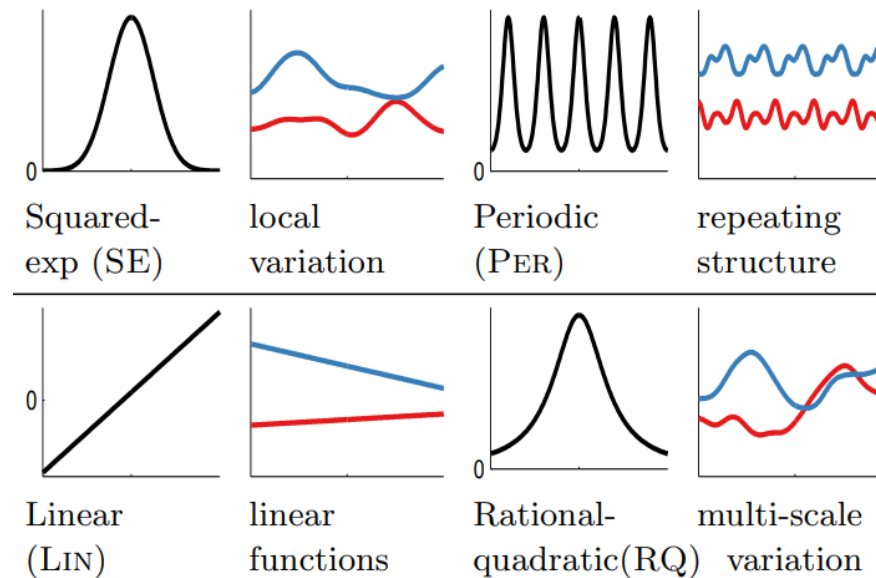
Gaussian process prior

- Prior on the function values $\mathbf{f}_{1:n}$ is represented as Gaussian Process (GP)

$$p(\mathbf{f}_{1:n}) = \mathcal{GP}(m(\cdot), k(\cdot, \cdot)): \begin{bmatrix} f_1 \\ \vdots \\ f_n \end{bmatrix} \sim \mathcal{N} \left(\begin{bmatrix} m(x_1) \\ \vdots \\ m(x_n) \end{bmatrix}, \begin{bmatrix} k(x_1, x_1) & \cdots & k(x_1, x_n) \\ \vdots & \ddots & \vdots \\ k(x_n, x_1) & \cdots & k(x_n, x_n) \end{bmatrix} \right)$$

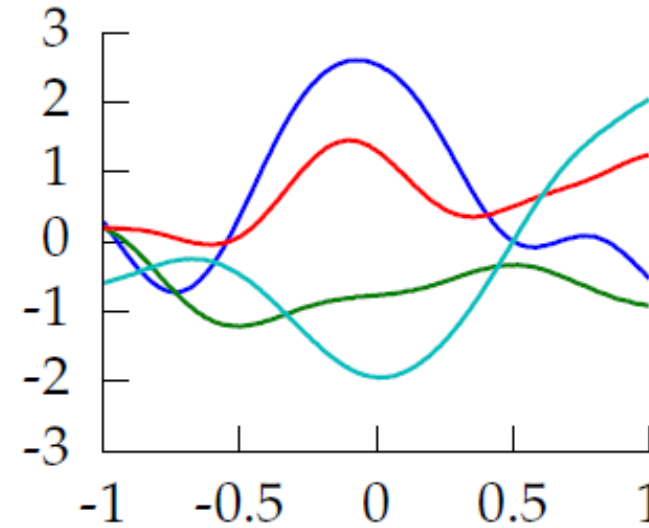
- $m(\cdot)$: mean function represents the overall trend of a target function
- $k(\cdot, \cdot)$: kernel function describes overall structure (smoothness) of a target function

$$\text{cov}(f_i, f_j) = \mathbb{E}[(f_i - m(x_i))(f_j - m(x_j))] \approx k(x_i, x_j)$$



Squared Exponential (SE) covariance

$$k(x, x') = \sigma_0^2 \exp \left[-\frac{1}{2} \left(\frac{x - x'}{\lambda} \right)^2 \right]$$



- Intuition: function variables close in input space are highly correlated, whilst those far away are uncorrelated
- Hyperparameters : λ and σ_0
 - λ : length scale
 - σ_0 : amplitude
- Stationary: $k(x, x') = k(|x - x'|)$: Invariant to translations
- Very smooth sample functions: infinitely differentiable

Automatic Relevance Determination

- The **ARD SE covariance function** for multi-dimensional inputs

$$k(x, x') = \sigma_0^2 \exp \left[-\frac{1}{2} \sum_{d=1}^D \left(\frac{x_d - x'_d}{\lambda_d} \right)^2 \right]$$

- Learn an individual length scale hyperparameter λ_d for each input dimension x_d
- λ_d determines the relevancy of input feature d to the regression
- If λ_d very large, then the feature is irrelevant

Matérn class of Covariance

- Matérn 3/2 covariance function : Ornstein-Uhlenbeck

$$k(x, x') = \alpha \exp\left(-\frac{|x - x'|}{2l^2}\right)$$

- Matérn 3/2 covariance function

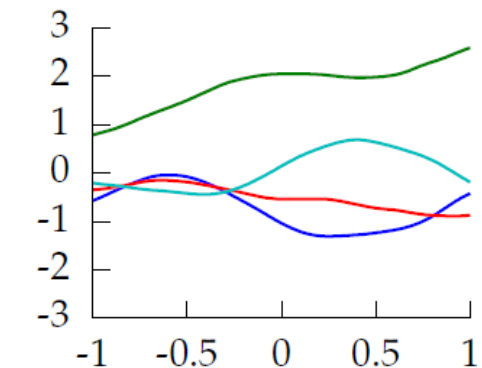
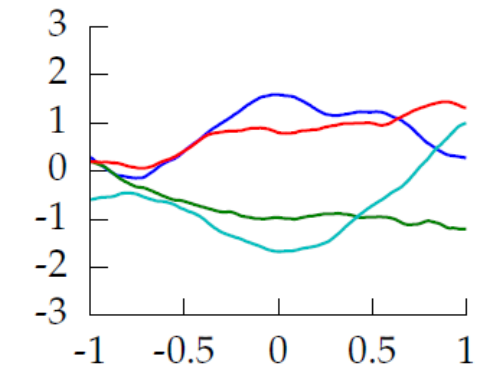
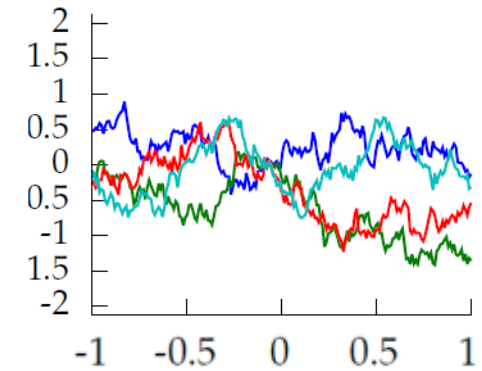
$$k(x, x') = \alpha (1 + \sqrt{3}r) \exp(-\sqrt{3}r)$$

where $r = \frac{\|x - x'\|_2}{l}$

- Matérn 3/2 covariance function

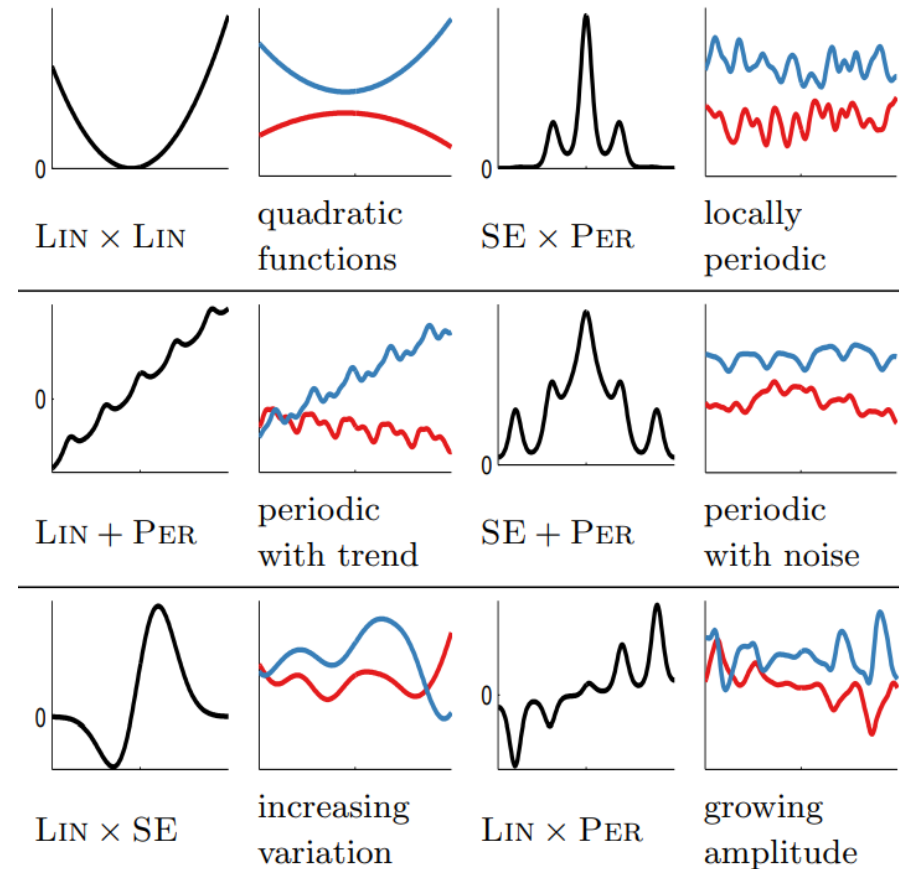
$$k(x, x') = \alpha \left(1 + \sqrt{5}r + \frac{5}{3}r^2\right) \exp(-\sqrt{5}r)$$

where $r = \frac{\|x - x'\|_2}{l}$



Constructing new covariance from old

- There are several ways to combine covariance:
- Sum: $k(x, x') = k_1(x, x') + k_2(x, x')$
 - Addition of independent processes
- Product: $k(x, x') = k_1(x, x')k_2(x, x')$
 - Product of independent processes



Gaussian process regression with Gaussian noise

- Prior on the function values $\mathbf{f}_{1:n}$ is represented as Gaussian Process (GP)

$$p(\mathbf{f}_{1:n}) = \mathcal{GP}(m(\cdot), k(\cdot, \cdot)): \begin{bmatrix} f_1 \\ \vdots \\ f_n \end{bmatrix} \sim \mathcal{N} \left(\begin{bmatrix} m(x_1) \\ \vdots \\ m(x_n) \end{bmatrix}, \begin{bmatrix} k(x_1, x_1) & \cdots & k(x_1, x_n) \\ \vdots & \ddots & \vdots \\ k(x_n, x_1) & \cdots & k(x_n, x_n) \end{bmatrix} \right)$$

- $m(\cdot)$: mean function represents the overall trend of a target function
- $k(\cdot, \cdot)$: kernel function describes overall structure (smoothness) of a target function

$$\text{cov}(f_i, f_j) = \mathbb{E}[(f_i - m(x_i))(f_j - m(x_j))] \approx k(x_i, x_j)$$

- Likelihood is constructed base the assumption on the noise, i.e., i.i.d. Gaussian noise

$$y_i = f_i + \epsilon_i, \quad \epsilon_i \sim \mathcal{N}(0, \sigma_\epsilon^2) \rightarrow (\text{i.i.d.}) : p(\mathbf{y}_{1:n} | \mathbf{f}_{1:n}) = \mathcal{N}(\mathbf{f}_{1:n}, \sigma_\epsilon^2 \mathbf{I})$$

Determination of Hyper-parameters (Training)

- Prior on the function values $\mathbf{f}_{1:n}$ is represented as Gaussian Process (GP)

$$p(\mathbf{f}_{1:n}) = \mathcal{GP}(m(\cdot), k(\cdot, \cdot)): \begin{bmatrix} f_1 \\ \vdots \\ f_n \end{bmatrix} \sim \mathcal{N} \left(\begin{bmatrix} m(x_1) \\ \vdots \\ m(x_n) \end{bmatrix}, \begin{bmatrix} k(x_1, x_1) & \cdots & k(x_1, x_n) \\ \vdots & \ddots & \vdots \\ k(x_n, x_1) & \cdots & k(x_n, x_n) \end{bmatrix} \right)$$

- $m(\cdot)$: mean function represents the overall trend of a target function
- $k(\cdot, \cdot)$: kernel function describes overall structure (smoothness) of a target function

$$\text{cov}(f_i, f_j) = \mathbb{E}[(f_i - m(x_i))(f_j - m(x_j))] \approx k(x_i, x_j)$$

- Likelihood is constructed base the assumption on the noise, i.e., i.i.d. Gaussian noise

$$y_i = f_i + \epsilon_i, \quad \epsilon_i \sim \mathcal{N}(0, \sigma_\epsilon^2) \rightarrow (\text{i.i.d.}) : p(\mathbf{y}_{1:n} | \mathbf{f}_{1:n}) = \mathcal{N}(\mathbf{f}_{1:n}, \sigma_\epsilon^2 \mathbf{I})$$

- The hyper-parameters that need to be optimized are

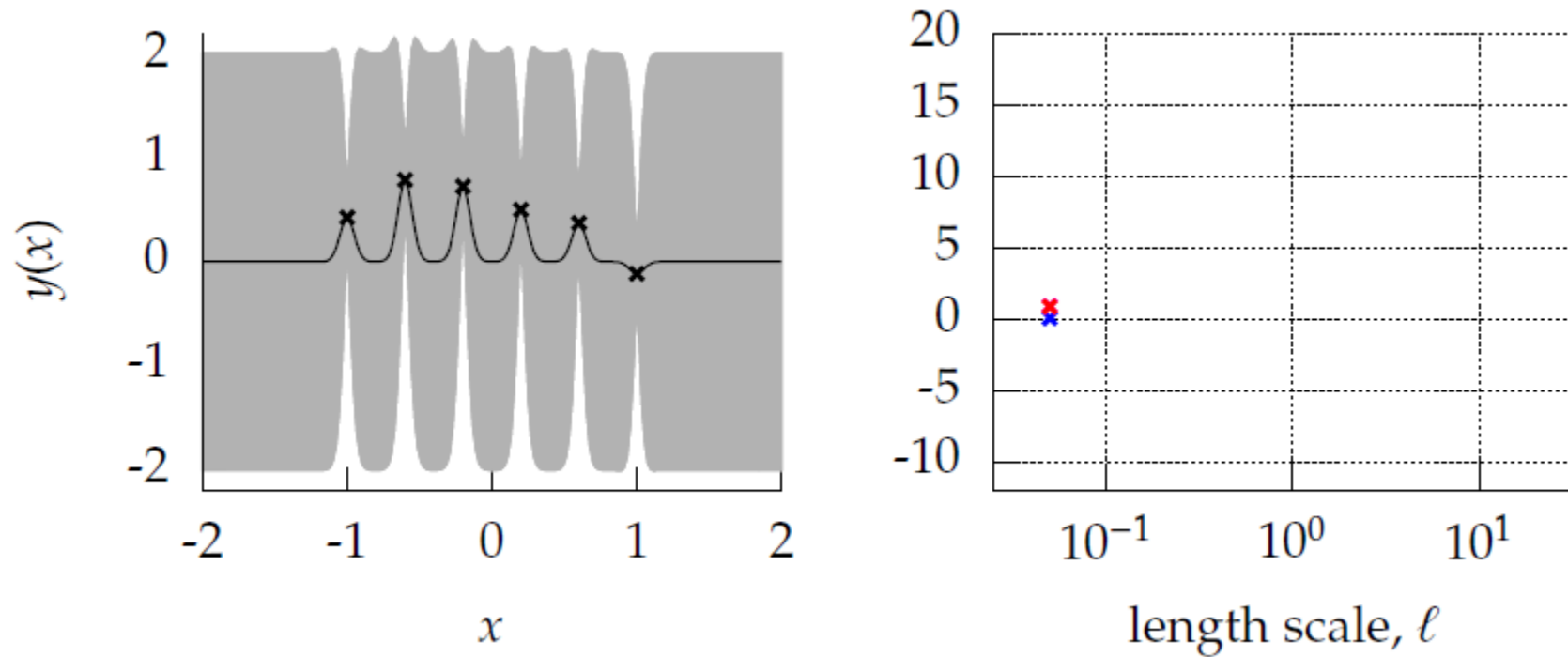
- Noise variance: σ_ϵ^2
- Parameters for the kernel function $k(x_i, x_j)$
 - For example, when $k(x_i, x_j) = \sigma_0^2 \exp \left[-\frac{1}{2} \sum_{d=1}^D \left(\frac{x_d - x'_d}{\lambda_d} \right)^2 \right]$
 - Parameters are $(\sigma_0^2, \lambda_1, \dots, \lambda_D)$

Determination of Hyper-parameters (Training)

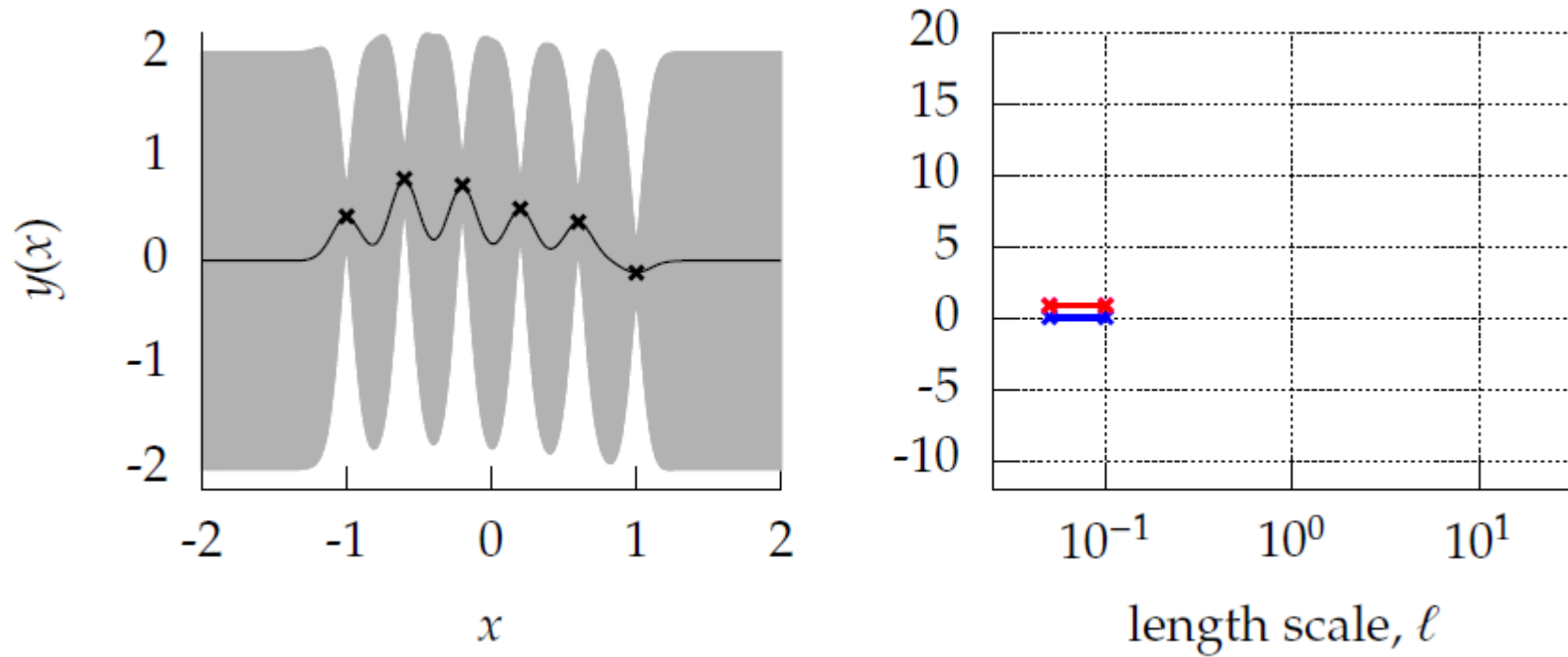
- The hyper-parameters $\boldsymbol{\theta} = (\sigma_\epsilon, \sigma_s, \boldsymbol{\lambda})$ for the noise model and the kernel function are determined as ones maximizing the marginal log-likelihood of the training data $\mathcal{D} = \{(x_i, y_i) | i = 1, \dots, n\}$ as

$$\begin{aligned}\theta^* &= \underset{\boldsymbol{\theta}}{\operatorname{argmax}} \log p(\mathbf{y}_{1:n} | \boldsymbol{\theta}) \\ &= \underset{\boldsymbol{\theta}}{\operatorname{argmax}} \log \int_{\mathbf{f}_{1:n}} (\mathbf{y}_{1:n}, \mathbf{f}_{1:n} | \boldsymbol{\theta}) d\mathbf{f}_{1:n} \\ &= \underset{\boldsymbol{\theta}}{\operatorname{argmax}} \log \int p(\mathbf{y}_{1:n} | \mathbf{f}_{1:n}, \boldsymbol{\theta}) p(\mathbf{f}_{1:n} | \boldsymbol{\theta}) d\mathbf{f}_{1:n} \\ &= \underset{\boldsymbol{\theta}}{\operatorname{argmax}} \left(-\frac{1}{2} \log |(\mathbf{K} + \sigma_\epsilon^2 \mathbf{I})| - \frac{1}{2} (\mathbf{y}_{1:n})^T (\mathbf{K} + \sigma_\epsilon^2 \mathbf{I})^{-1} \mathbf{y}_{1:n} - \frac{n}{2} \log 2\pi \right) \\ &= \underset{\boldsymbol{\theta}}{\operatorname{argmin}} \left(\frac{1}{2} \log |(\mathbf{K} + \sigma_\epsilon^2 \mathbf{I})| + \frac{1}{2} (\mathbf{y}_{1:n})^T (\mathbf{K} + \sigma_\epsilon^2 \mathbf{I})^{-1} \mathbf{y}_{1:n} \right)\end{aligned}$$

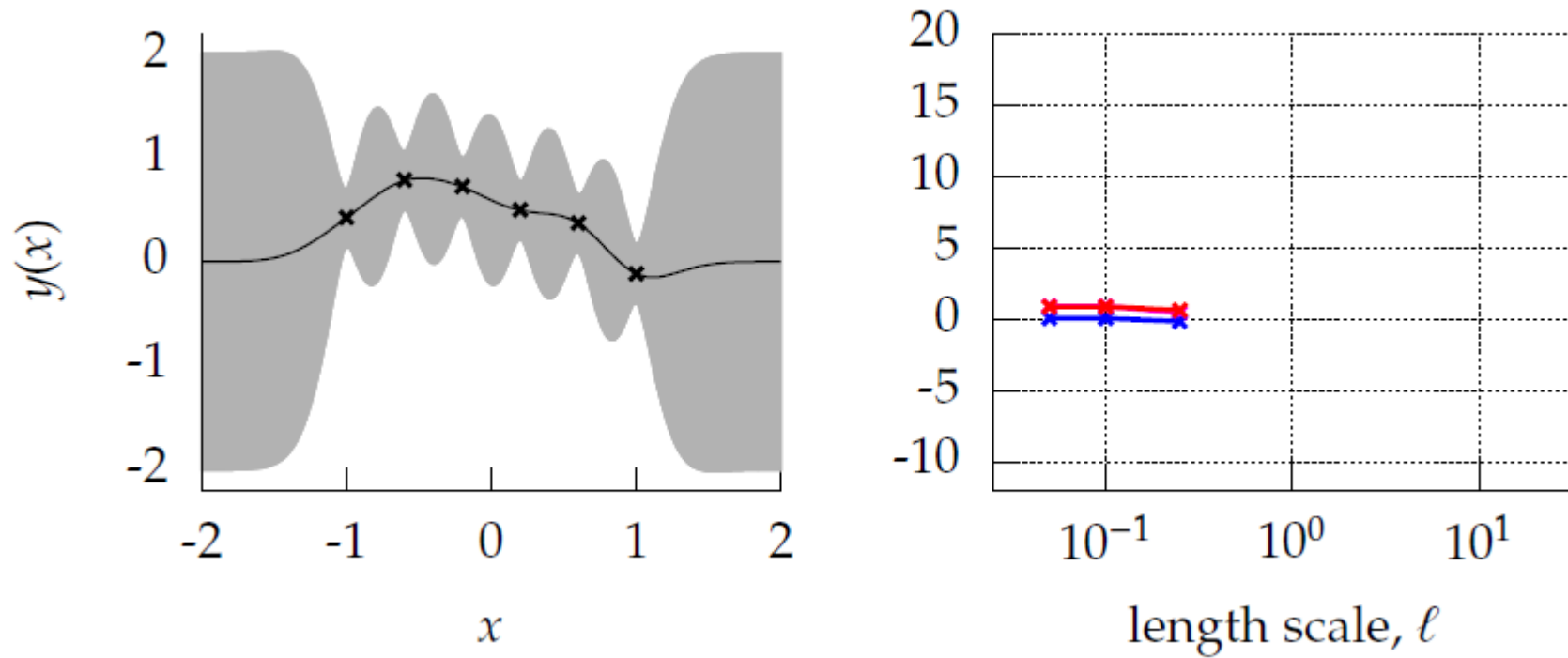
Determination of Hyper-parameters (Training)



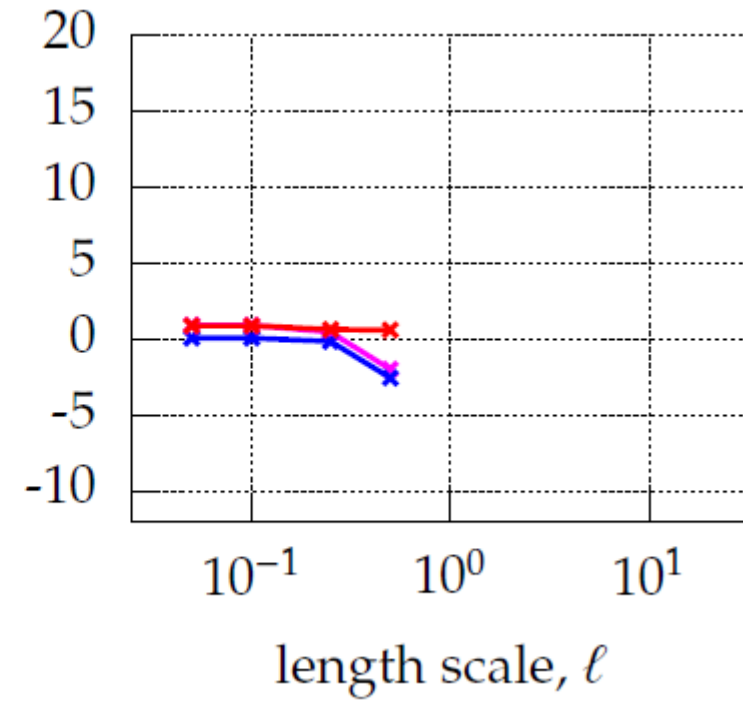
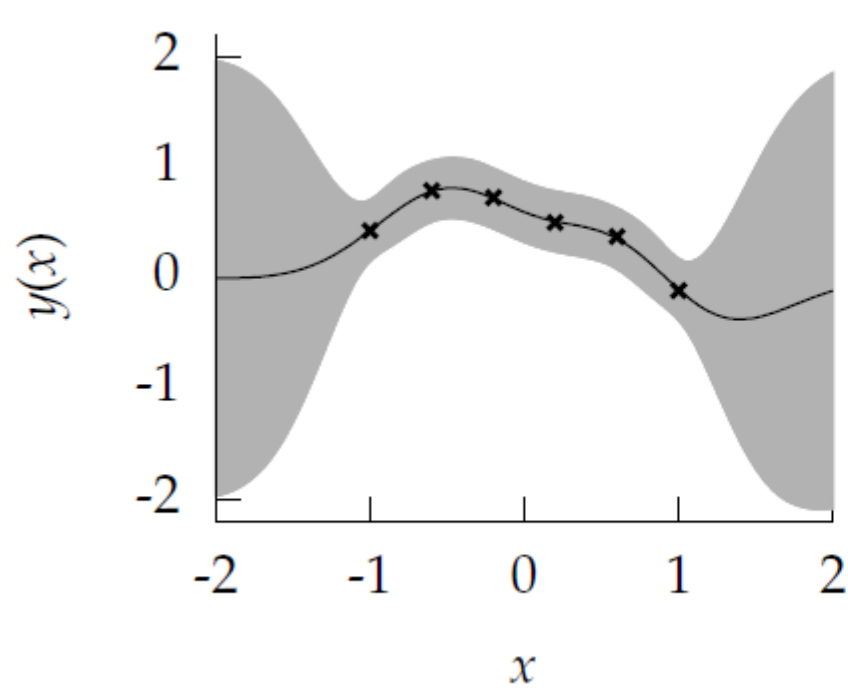
Determination of Hyper-parameters (Training)



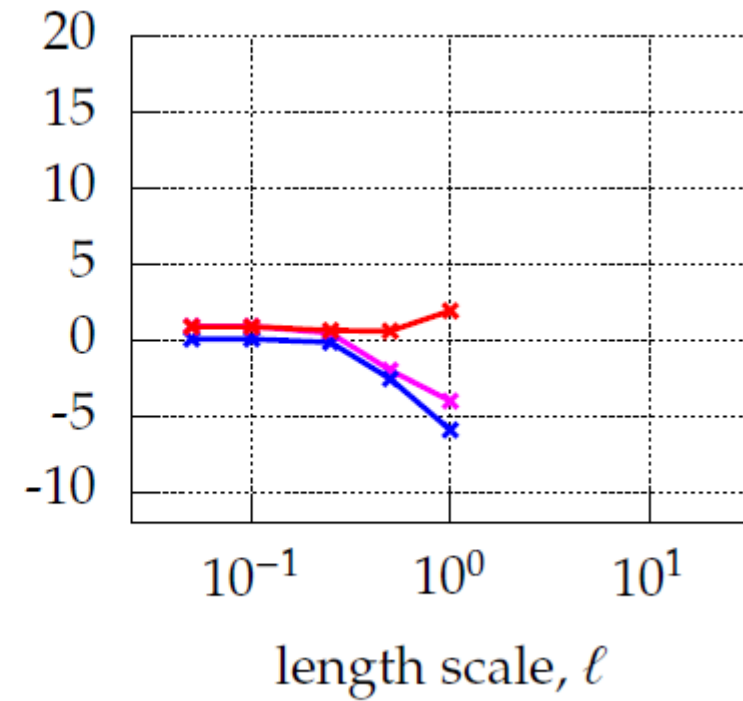
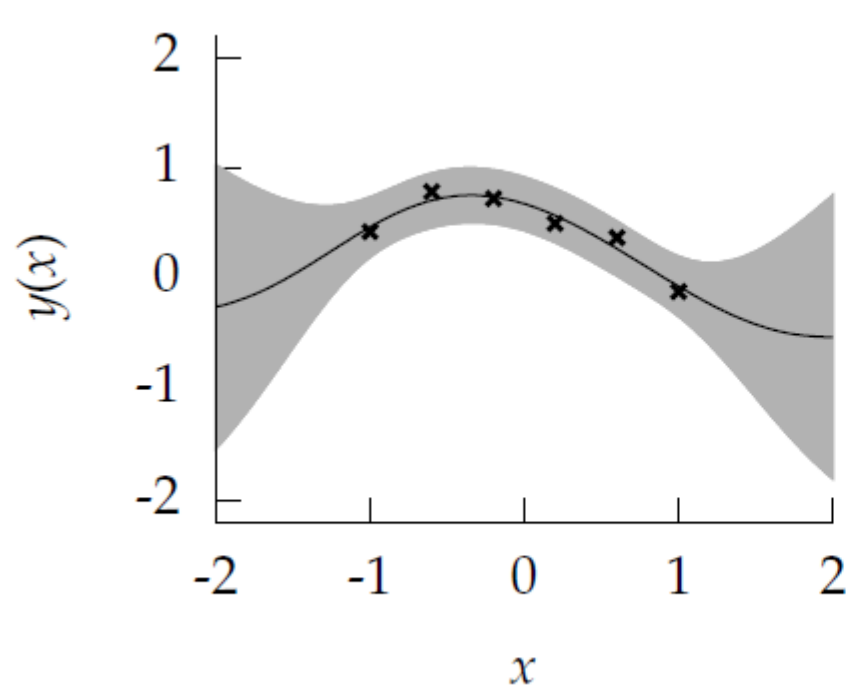
Determination of Hyper-parameters (Training)



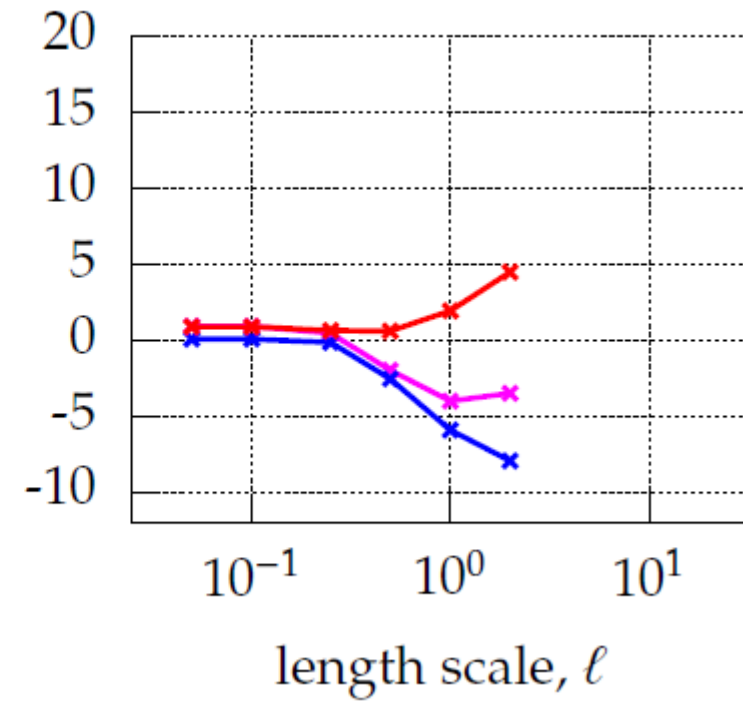
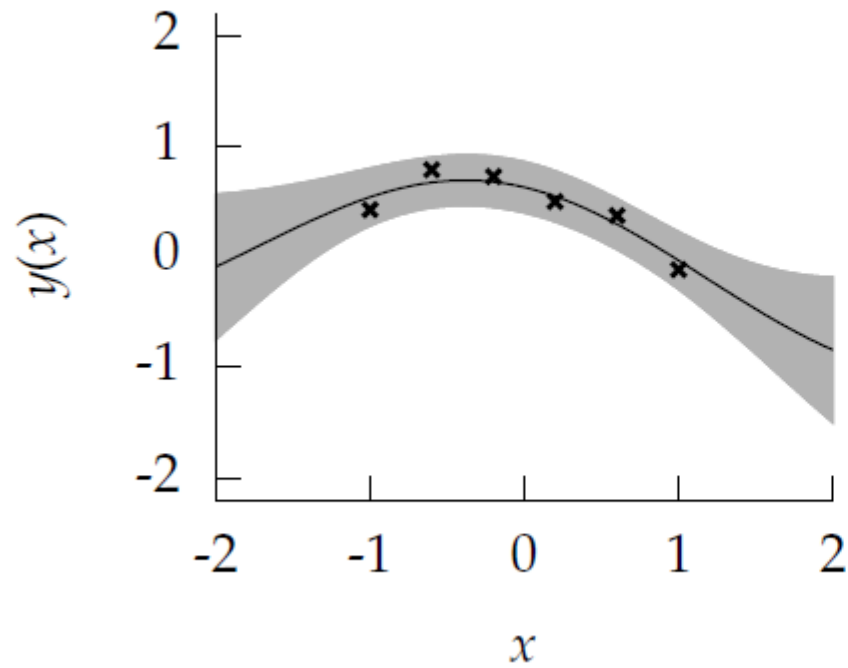
Determination of Hyper-parameters (Training)



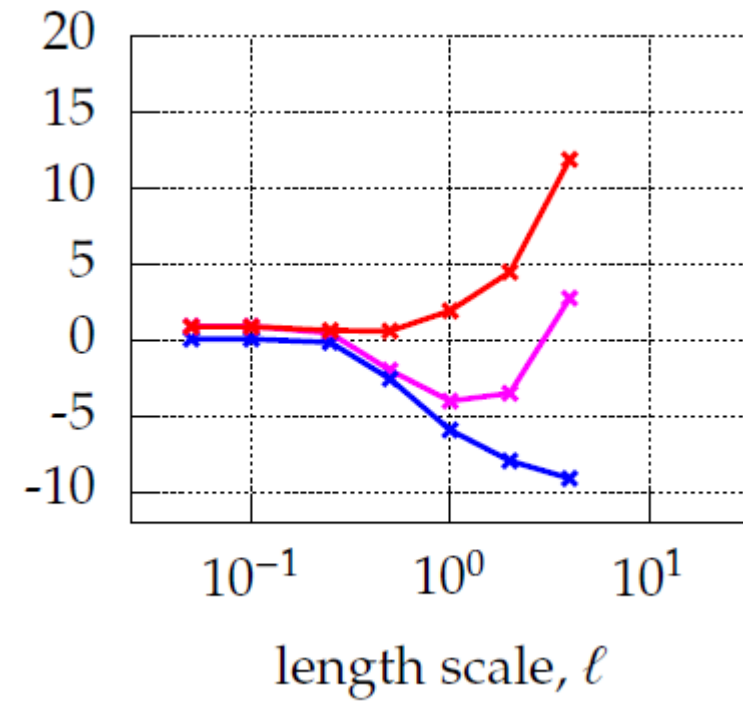
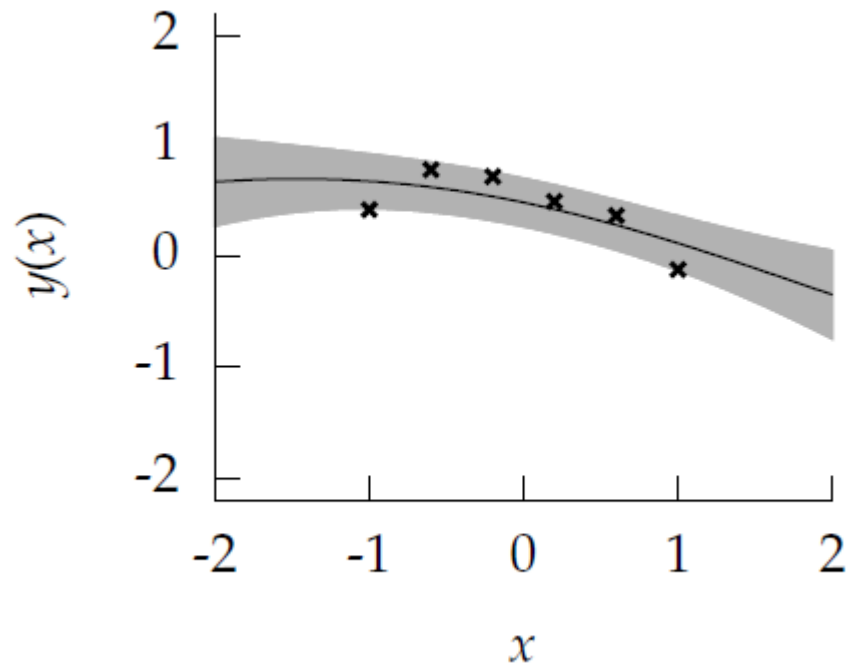
Determination of Hyper-parameters (Training)



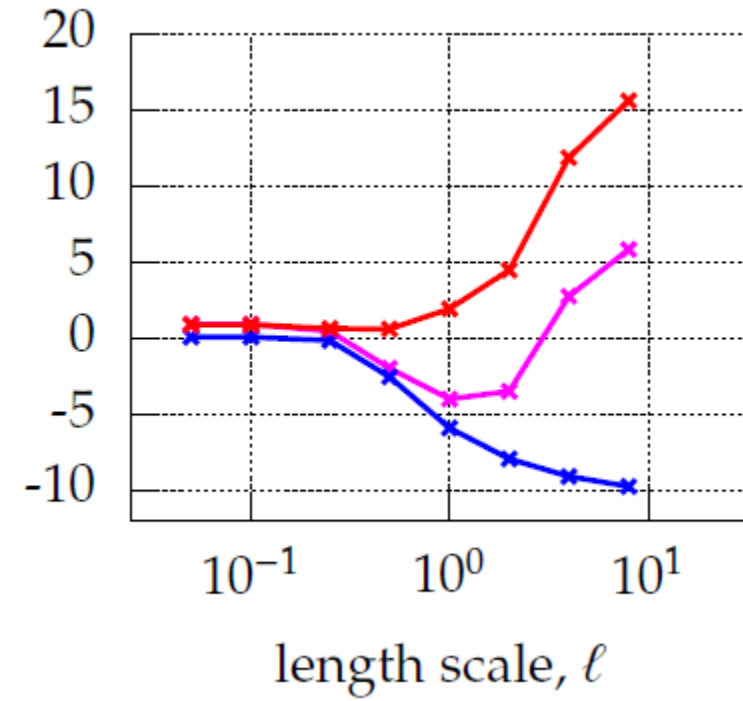
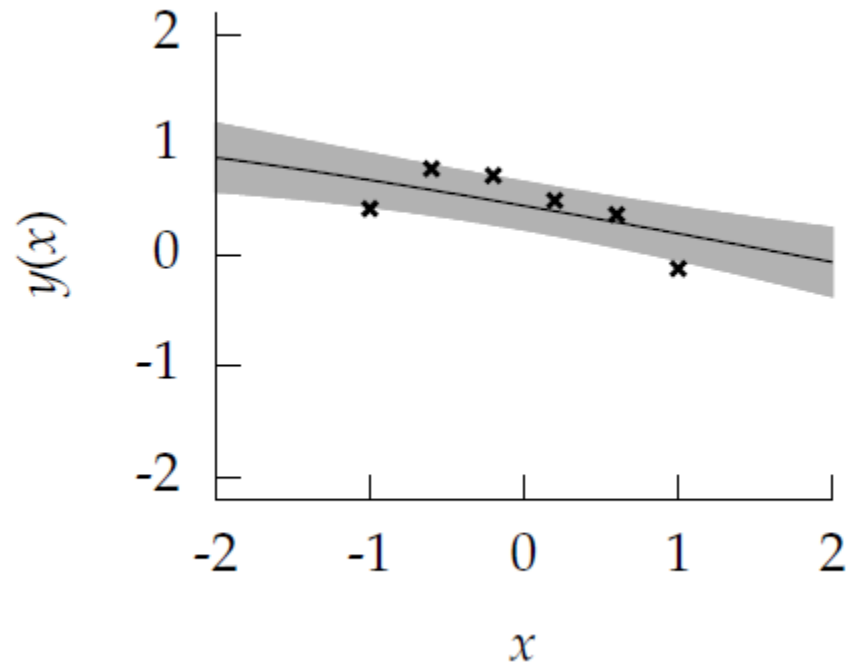
Determination of Hyper-parameters (Training)



Determination of Hyper-parameters (Training)



Determination of Hyper-parameters (Training)



Constructing the joint distribution

- Prior on the function values $\mathbf{f}_{1:n}$ is represented as Gaussian Process (GP)

$$p(\mathbf{f}_{1:n}) = \mathcal{GP}(m(\cdot), k(\cdot, \cdot)): \begin{bmatrix} f_1 \\ \vdots \\ f_n \end{bmatrix} \sim \mathcal{N} \left(\begin{bmatrix} m(x_1) \\ \vdots \\ m(x_n) \end{bmatrix}, \begin{bmatrix} k(x_1, x_1) & \cdots & k(x_1, x_n) \\ \vdots & \ddots & \vdots \\ k(x_n, x_1) & \cdots & k(x_n, x_n) \end{bmatrix} \right)$$

- $m(\cdot)$: mean function represents the overall trend of a target function
- $k(\cdot, \cdot)$: kernel function describes overall structure (smoothness) of a target function

$$\text{cov}(f_i, f_j) = \mathbb{E}[(f_i - m(x_i))(f_j - m(x_j))] \approx k(x_i, x_j)$$

- Likelihood is constructed base the assumption on the noise, i.e., i.i.d. Gaussian noise

$$y_i = f_i + \epsilon_i, \quad \epsilon_i \sim \mathcal{N}(0, \sigma_\epsilon^2) \rightarrow (\text{i.i.d.}) : p(\mathbf{y}_{1:n} | \mathbf{f}_{1:n}) = \mathcal{N}(\mathbf{f}_{1:n}, \sigma_\epsilon^2 \mathbf{I})$$

- Joint distribution based on Bayes' rule :

$$p(f, \mathbf{y}^{1:n}) = \int p(f, \mathbf{f}_{1:n}) p(\mathbf{y}^{1:n} | \mathbf{f}_{1:n}) d\mathbf{f}_{1:n} \rightarrow \begin{bmatrix} \mathbf{y}^{1:n} \\ f \end{bmatrix} \sim \mathcal{N} \left(\mathbf{0}, \begin{bmatrix} \mathbf{K} + \sigma_\epsilon^2 \mathbf{I} & \mathbf{k} \\ \mathbf{k}^T & k(x, x) \end{bmatrix} \right)$$

Prediction as conditionalization

- Prior on the function values $\mathbf{f}_{1:n}$ is represented as Gaussian Process (GP)

$$p(\mathbf{f}_{1:n}) = \mathcal{GP}(m(\cdot), k(\cdot, \cdot)): \begin{bmatrix} f_1 \\ \vdots \\ f_n \end{bmatrix} \sim \mathcal{N} \left(\begin{bmatrix} m(x_1) \\ \vdots \\ m(x_n) \end{bmatrix}, \begin{bmatrix} k(x_1, x_1) & \cdots & k(x_1, x_n) \\ \vdots & \ddots & \vdots \\ k(x_n, x_1) & \cdots & k(x_n, x_n) \end{bmatrix} \right)$$

- $m(\cdot)$: mean function represents the overall trend of a target function
- $k(\cdot, \cdot)$: kernel function describes overall structure (smoothness) of a target function

$$\text{cov}(f_i, f_j) = \mathbb{E}[(f_i - m(x_i))(f_j - m(x_j))] \approx k(x_i, x_j)$$

- Likelihood is constructed base the assumption on the noise, i.e., i.i.d. Gaussian noise

$$y_i = f_i + \epsilon_i, \quad \epsilon_i \sim \mathcal{N}(0, \sigma_\epsilon^2) \rightarrow (\text{i.i.d.}) : p(\mathbf{y}_{1:n} | \mathbf{f}_{1:n}) = \mathcal{N}(\mathbf{f}_{1:n}, \sigma_\epsilon^2 \mathbf{I})$$

- Joint distribution based on Bayes' rule :

$$p(f, \mathbf{y}^{1:n}) = \int p(f, \mathbf{f}_{1:n}) p(\mathbf{y}^{1:n} | \mathbf{f}_{1:n}) d\mathbf{f}_{1:n} \rightarrow \begin{bmatrix} \mathbf{y}^{1:n} \\ f \end{bmatrix} \sim \mathcal{N} \left(\mathbf{0}, \begin{bmatrix} \mathbf{K} + \sigma_\epsilon^2 \mathbf{I} & \mathbf{k} \\ \mathbf{k}^T & k(x, x) \end{bmatrix} \right)$$

- **Conditionalization** $\rightarrow$ Posterior distribution on the function value $f = f(x)$ for x given $\mathcal{D} = \{\mathbf{x}_{1:n}, \mathbf{y}_{1:n}\}$

$$p(f | \mathcal{D}) = \mathcal{N}(\mu(x | \mathcal{D}), \sigma^2(x | \mathcal{D})) \rightarrow \begin{array}{l} \text{Mean} : \mu(x | \mathcal{D}) = \mathbf{k}^T (\mathbf{K} + \sigma_\epsilon^2 \mathbf{I})^{-1} \mathbf{y}_{1:n} \\ \text{Variance} : \sigma^2(x | \mathcal{D}) = k(x, x) - \mathbf{k}^T (\mathbf{K} + \sigma_\epsilon^2 \mathbf{I})^{-1} \mathbf{k} \end{array}$$

Prediction as conditionalization

Property 4.

Conditional of a Gaussian random variables are Gaussians, more specifically, if

$$Z = \begin{bmatrix} Y_1 \\ Y_2 \end{bmatrix} \sim N \left(\begin{bmatrix} \mu_1 \\ \mu_2 \end{bmatrix}, \begin{bmatrix} \Sigma_{11} & \Sigma_{12} \\ \Sigma_{21} & \Sigma_{22} \end{bmatrix} \right)$$

where Y_1 is k -dim random variable and Y_2 is an $n - k$ dim random variable, then

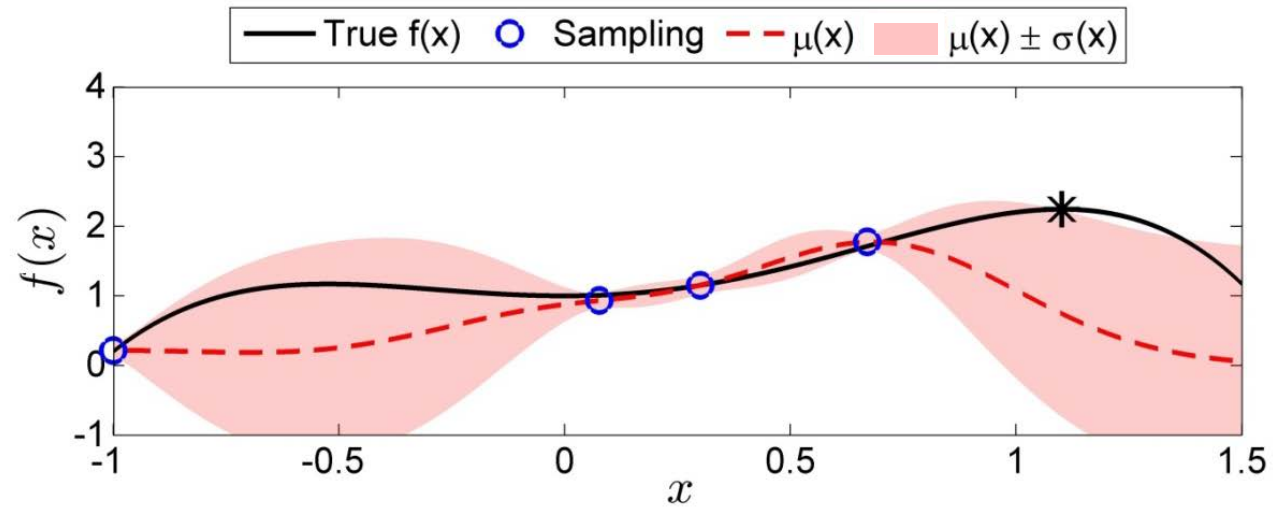
$$\{Y_2 | Y_1 = y\} \sim N(\Sigma_{21}\Sigma_{11}^{-1}(y - \mu_1) + \mu_2, \Sigma_{22} - \Sigma_{21}\Sigma_{11}^{-1}\Sigma_{12})$$

- **Conditionalization** → Posterior distribution on the function value $f = f(x)$ for x given $\mathcal{D} = \{\mathbf{x}_{1:n}, \mathbf{y}_{1:n}\}$

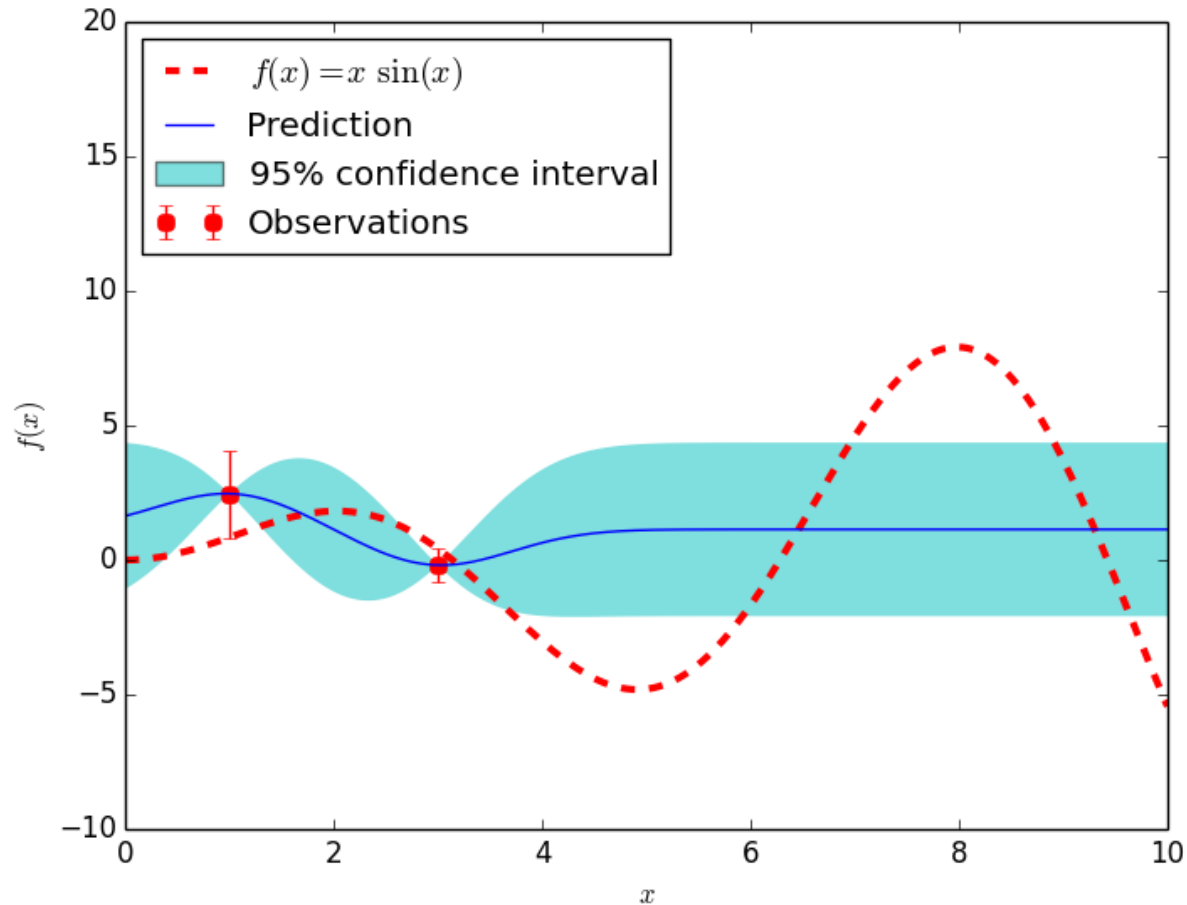
$$p(f|\mathcal{D}) = N(\mu(x|\mathcal{D}), \sigma^2(x|\mathcal{D})) \rightarrow \begin{array}{l} \text{Mean : } \mu(x|\mathcal{D}) = \mathbf{k}^T (\mathbf{K} + \sigma_\epsilon^2 \mathbf{I})^{-1} \mathbf{y}_{1:n} \\ \text{Variance : } \sigma^2(x|\mathcal{D}) = k(x, x) - \mathbf{k}^T (\mathbf{K} + \sigma_\epsilon^2 \mathbf{I})^{-1} \mathbf{k} \end{array}$$

Gaussian process (Step by Step)

$$p(f|\mathcal{D}) = \mathcal{N}(\mu(x|\mathcal{D}), \sigma^2(x|\mathcal{D}))$$



Gaussian process (Step by Step)



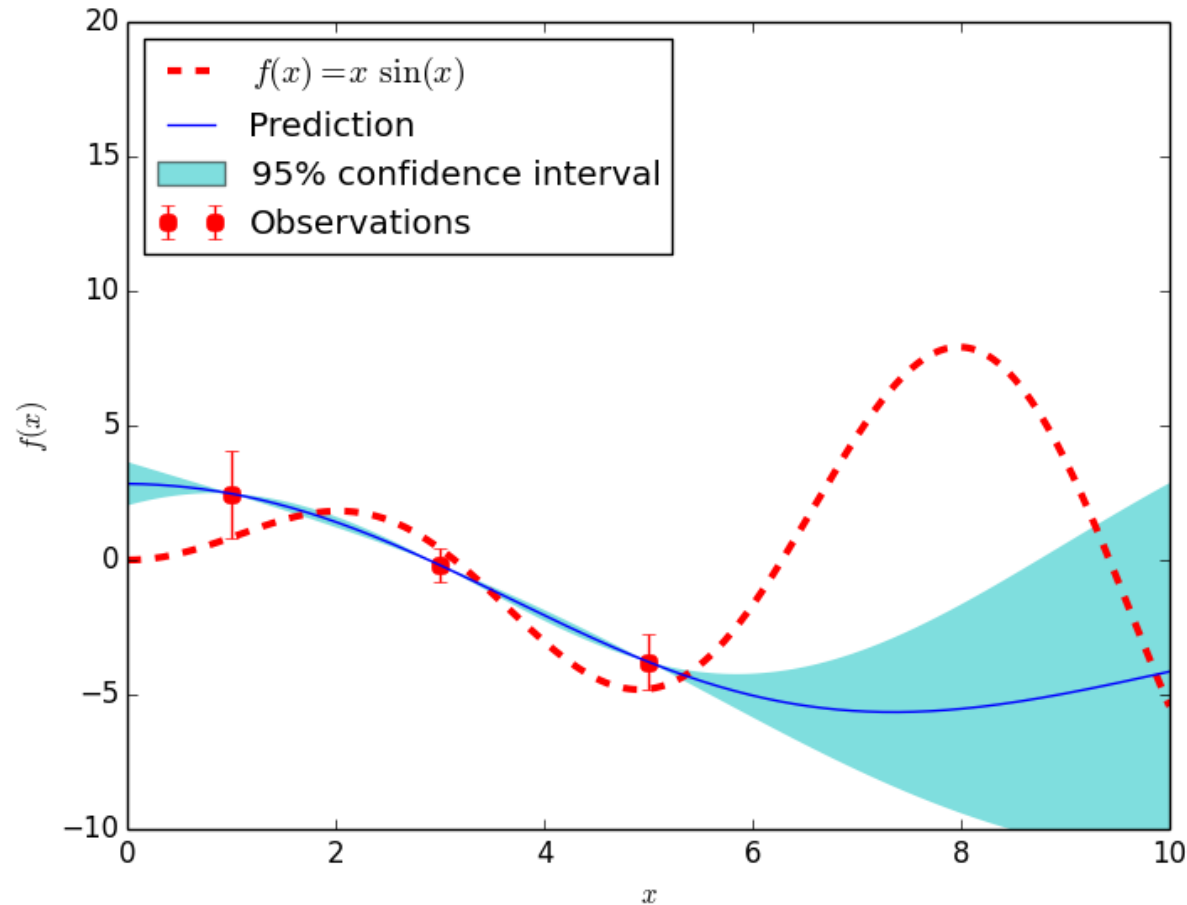
True $f(x) = x \sin(x)$

Sampled with error:

$$y = x \sin(x) + \epsilon, \epsilon \sim N(0, 1^2)$$

$$n = 2, \quad \begin{bmatrix} y^1 \\ y^2 \\ f(\mathbf{x}) \end{bmatrix} \sim N \left(\mathbf{0}, \begin{bmatrix} & & \mathbf{k} \\ & \mathbf{k}^T & \\ \mathbf{k} & & k(\mathbf{x}, \mathbf{x}) \end{bmatrix} \right)$$

Gaussian process (Step by Step)



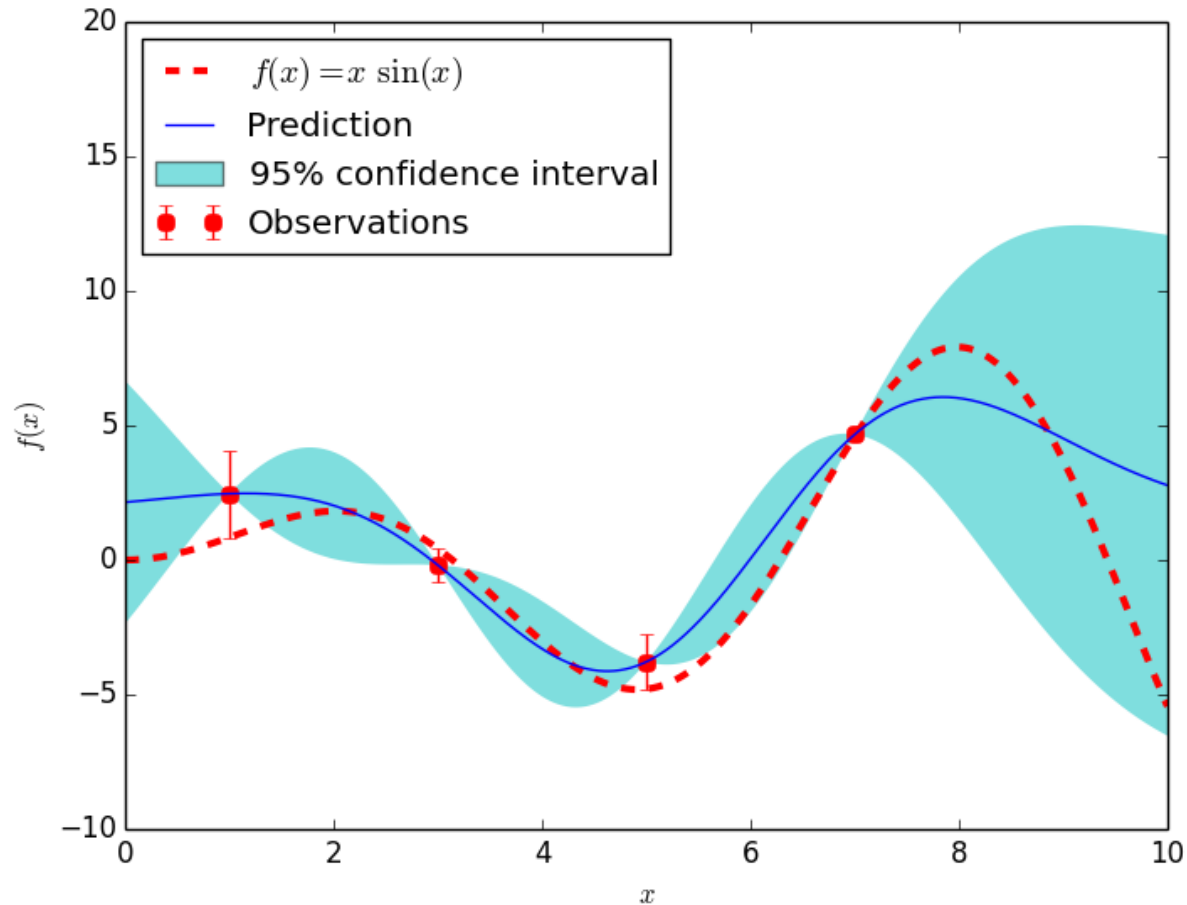
True $f(x) = x \sin(x)$

Sampled with error:

$$y = x \sin(x) + \epsilon, \epsilon \sim N(0, 1^2)$$

$$n = 3, \quad \begin{bmatrix} y^1 \\ y^2 \\ y^3 \\ f(\mathbf{x}) \end{bmatrix} \sim N \left(\mathbf{0}, \begin{bmatrix} & & & \mathbf{k} \\ & & & \\ & & & \\ \mathbf{k}^T & & & k(\mathbf{x}, \mathbf{x}) \end{bmatrix} \right)$$

Gaussian process (Step by Step)



True $f(x) = x \sin(x)$

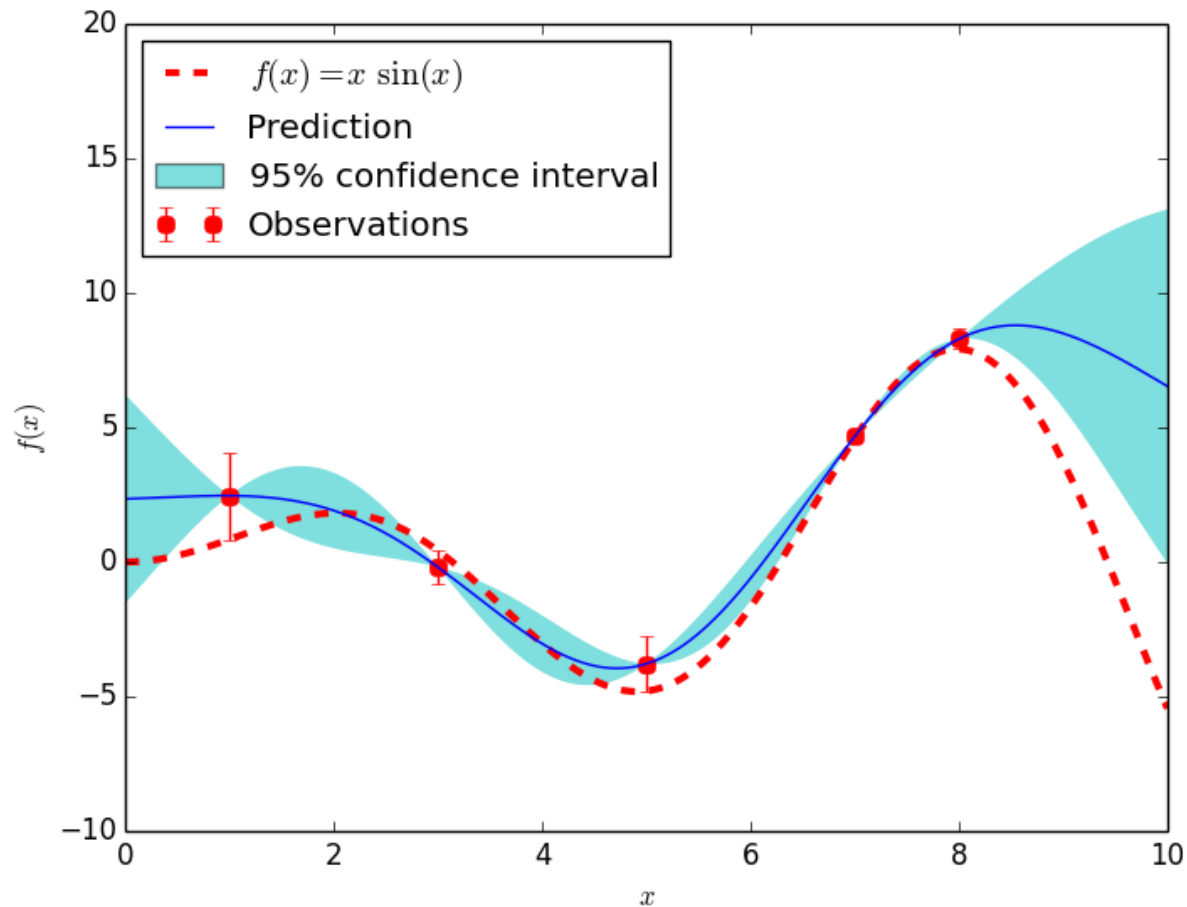
Sampled with error:

$$y = x \sin(x) + \epsilon, \epsilon \sim N(0, 1^2)$$

$n = 4,$

$$\begin{bmatrix} y^1 \\ y^2 \\ y^3 \\ y^4 \\ f(x) \end{bmatrix} \sim N \left(\mathbf{0}, \begin{bmatrix} & & & & \\ & & & & \mathbf{k} \\ & & & & \\ \mathbf{k}^T & & & & \\ & & & & k(x, x) \end{bmatrix} \right)$$

Gaussian process (Step by Step)



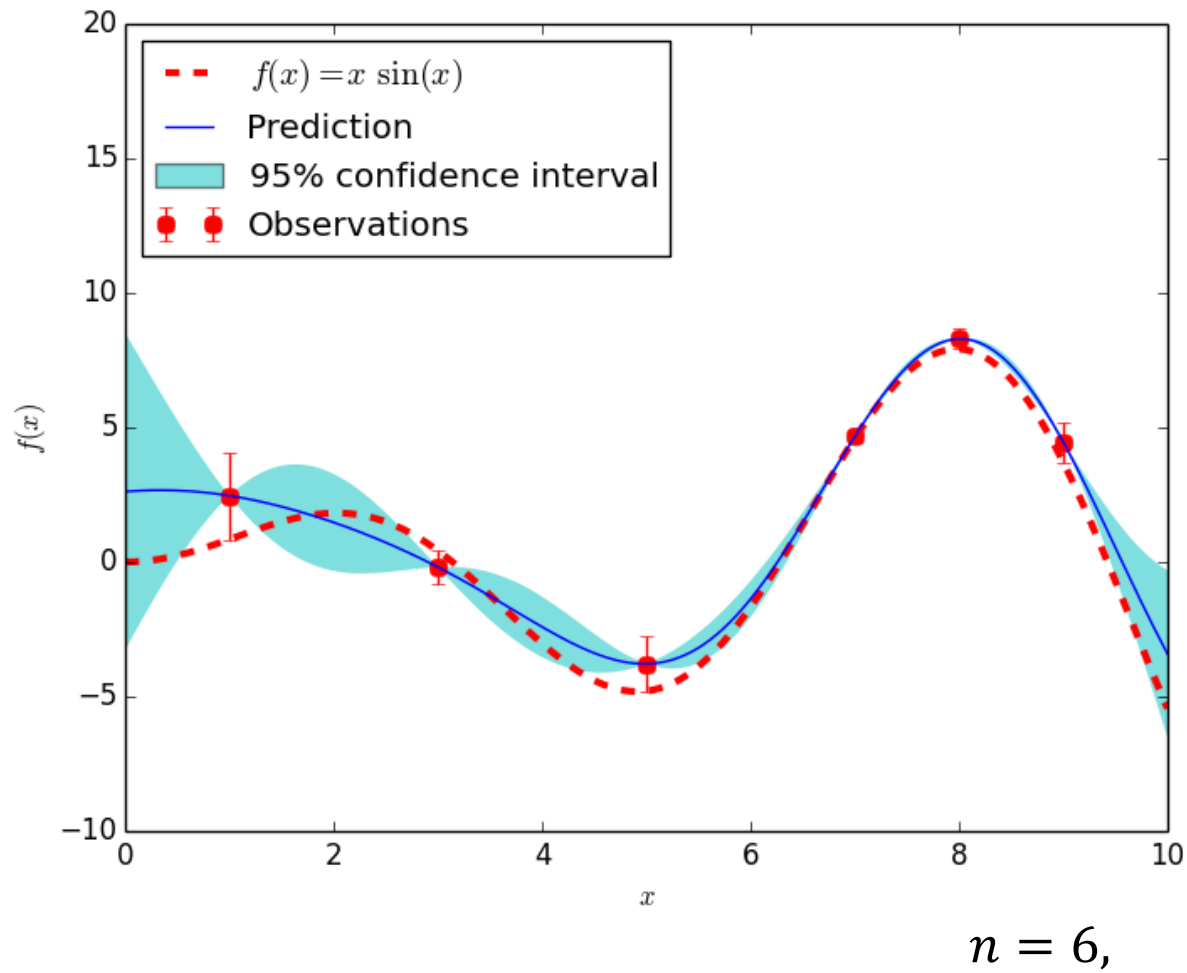
True $f(x) = x \sin(x)$

Sampled with error:

$$y = x \sin(x) + \epsilon, \epsilon \sim N(0, 1^2)$$

$$\begin{bmatrix} y^1 \\ y^2 \\ y^3 \\ y^4 \\ y^5 \\ f(\mathbf{x}) \end{bmatrix} \sim N \left(\begin{bmatrix} \mathbf{0}, \\ \mathbf{k}^T \end{bmatrix} \begin{bmatrix} \mathbf{k} \\ k(\mathbf{x}, \mathbf{x}) \end{bmatrix} \right)$$

Gaussian process (Step by Step)



True $f(x) = x \sin(x)$

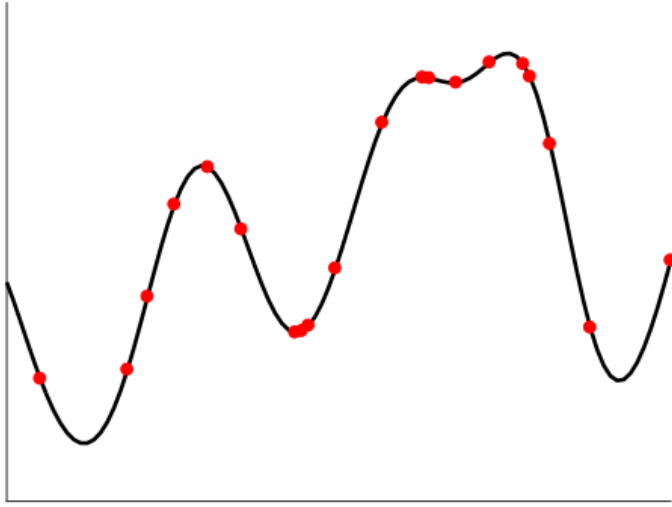
Sampled with error:

$$y = x \sin(x) + \epsilon, \epsilon \sim N(0, 1^2)$$

$$\begin{bmatrix} y^1 \\ y^2 \\ y^3 \\ y^4 \\ y^5 \\ y^6 \\ f(\mathbf{x}) \end{bmatrix} \sim N \left(\mathbf{0}, \begin{bmatrix} & & & & & \\ & & & & & \\ & & & & & \\ & & & & & \\ & & & & & \\ & & & & & \\ \mathbf{k}^T & & & & & k(\mathbf{x}, \mathbf{x}) \end{bmatrix} \right)$$

Multi-output Gaussian Process Regression

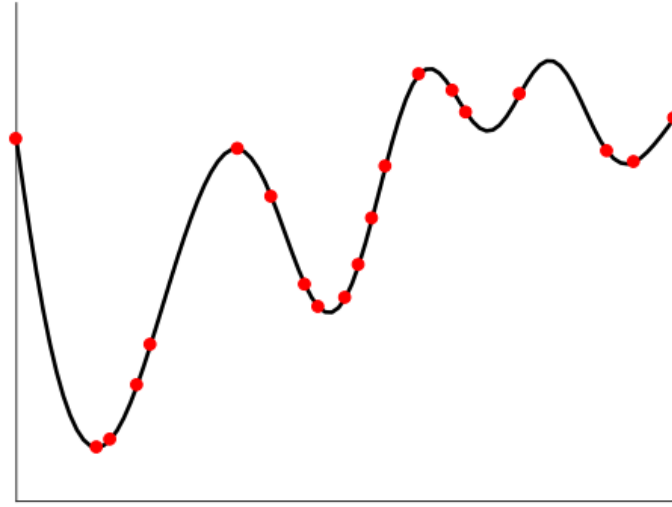
Intrinsic Coregionalization Model (ICM)



$$f_1(\mathbf{X}) \sim \mathcal{GP}(0, k_1(\mathbf{X}, \mathbf{X}'))$$

$$\mathcal{D}_1 = \left\{ \left(x_{1,i}, f_1(x_{1,i}) \right) \mid i = 1, \dots, N_1 \right\}$$

$$\mathbf{f}_1 \sim \mathcal{N}(\mathbf{0}, \mathbf{K}_1)$$



$$f_2(\mathbf{X}) \sim \mathcal{GP}(0, k_2(\mathbf{X}, \mathbf{X}'))$$

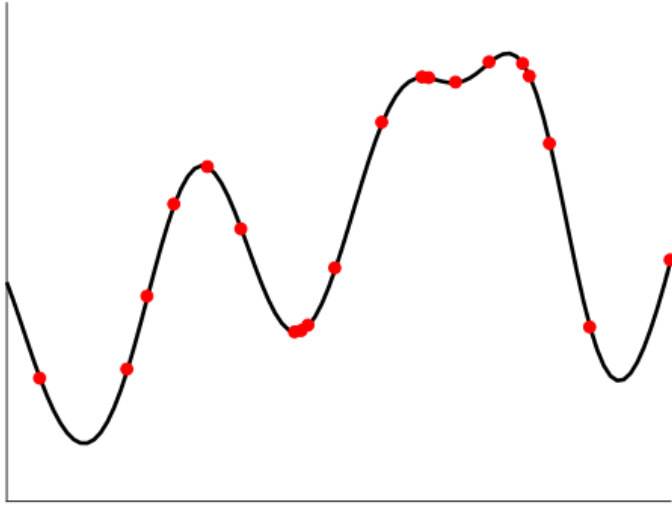
$$\mathcal{D}_2 = \left\{ \left(x_{2,i}, f_2(x_{2,i}) \right) \mid i = 1, \dots, N_2 \right\}$$

$$\mathbf{f}_2 \sim \mathcal{N}(\mathbf{0}, \mathbf{K}_2)$$

$$\begin{bmatrix} \mathbf{f}_1 \\ \mathbf{f}_2 \end{bmatrix} \sim \mathcal{N} \left(\begin{bmatrix} \mathbf{0} \\ \mathbf{0} \end{bmatrix}, \begin{bmatrix} \mathbf{K}_1 & \mathbf{0} \\ \mathbf{0} & \mathbf{K}_2 \end{bmatrix} \right)$$

$\mathbf{f} \qquad \mathbf{0} \qquad \mathbf{K}_{\mathbf{f},\mathbf{f}}$

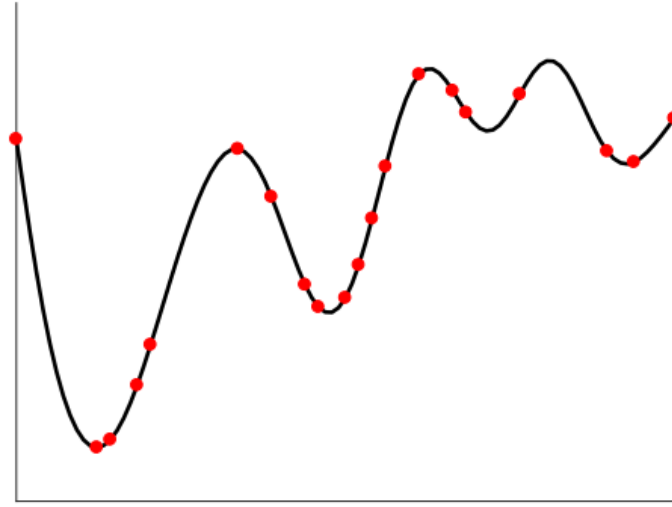
Intrinsic Coregionalization Model (ICM)



$$f_1(\mathbf{X}) \sim \mathcal{GP}(0, k_1(\mathbf{X}, \mathbf{X}'))$$

$$\mathcal{D}_1 = \left\{ \left(x_{1,i}, f_1(x_{1,i}) \right) \mid i = 1, \dots, N_1 \right\}$$

$$\mathbf{f}_1 \sim \mathcal{N}(\mathbf{0}, \mathbf{K}_1)$$



$$f_2(\mathbf{X}) \sim \mathcal{GP}(0, k_2(\mathbf{X}, \mathbf{X}'))$$

$$\mathcal{D}_2 = \left\{ \left(x_{2,i}, f_2(x_{2,i}) \right) \mid i = 1, \dots, N_2 \right\}$$

$$\mathbf{f}_2 \sim \mathcal{N}(\mathbf{0}, \mathbf{K}_2)$$

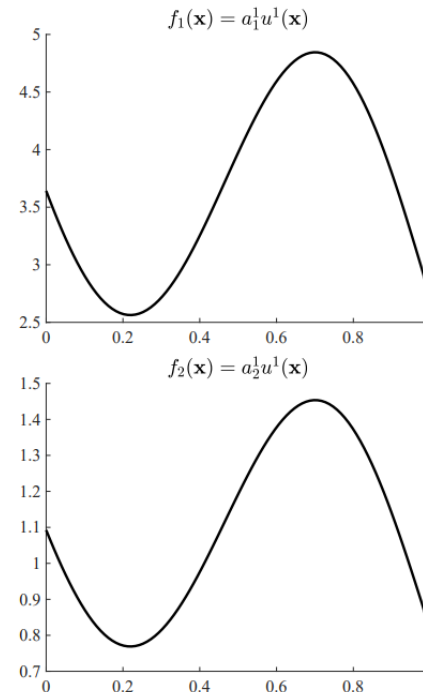
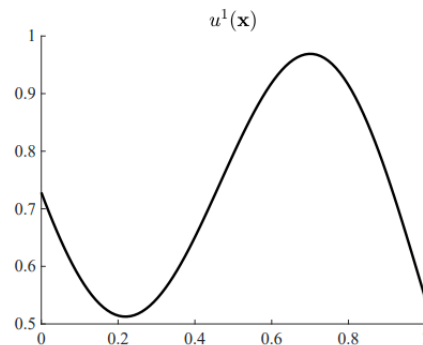
$$\begin{array}{c} \begin{bmatrix} \mathbf{f}_1 \\ \mathbf{f}_2 \end{bmatrix} \sim \mathcal{N} \left(\begin{bmatrix} \mathbf{0} \\ \mathbf{0} \end{bmatrix}, \begin{bmatrix} \mathbf{K}_1 & ? \\ ? & \mathbf{K}_1 \end{bmatrix} \right) \\ \mathbf{f} \qquad \qquad \mathbf{0} \qquad \mathbf{K}_{\mathbf{f},\mathbf{f}} \end{array}$$

We need to build a cross-covariance function $\text{cov}[f_1(\mathbf{X}), f_2(\mathbf{X}')] such that $\mathbf{K}_{\mathbf{f},\mathbf{f}}$ is PSD$

ICM: two outputs and one latent sample

- Consider two outputs $f_1(x)$ and $f_2(x)$ with $x \in \mathbb{R}^p$
- We assume the following generative model for the outputs
 - Sample from a GP $u(x) \sim \mathcal{GP}(0, k(x, x'))$ to obtain $u^1(x)$
 - Obtain $f_1(x)$ and $f_2(x)$ by linearly transforming $u^1(x)$

$$f_1(x) = a_1^1 u^1(x)$$
$$f_2(x) = a_2^1 u^1(x)$$



ICM: two outputs and one latent sample

- For a fixed value of x , we can group $f_1(x)$ and $f_2(x)$ in a vector $\mathbf{f}(x)$

$$\mathbf{f}(x) = \begin{bmatrix} f_1(x) \\ f_2(x) \end{bmatrix}$$

which is referred to as a vector-valued function

- The covariance for $\mathbf{f}(x)$ is computed as

$$\text{cov}[\mathbf{f}(x), \mathbf{f}(x')] = \mathbb{E}[\mathbf{f}(x)\mathbf{f}(x')^T] - \mathbb{E}[\mathbf{f}(x)]\mathbb{E}[\mathbf{f}(x')]^T$$

ICM: two outputs and one latent sample

$$\mathbf{f}(x) = \begin{bmatrix} f_1(x) \\ f_2(x) \end{bmatrix}, \quad \begin{aligned} f_1(x) &= a_1^1 u^1(x) \\ f_2(x) &= a_2^1 u^1(x) \end{aligned}$$

$$\text{cov}[\mathbf{f}(x), \mathbf{f}(x')] = \mathbb{E}[\mathbf{f}(x)\mathbf{f}(x')^T] - \mathbb{E}[\mathbf{f}(x)]\mathbb{E}[\mathbf{f}(x')]^T$$

$$\mathbb{E}[\mathbf{f}(x)\mathbf{f}(x')^T] = \mathbb{E} \left[\begin{bmatrix} f_1(x) \\ f_2(x) \end{bmatrix} [f_1(x'), f_2(x')] \right] = \begin{bmatrix} \mathbb{E}[f_1(x)f_1(x')] & \mathbb{E}[f_1(x)f_2(x')] \\ \mathbb{E}[f_2(x)f_1(x')] & \mathbb{E}[f_2(x)f_2(x')] \end{bmatrix}$$

$$\mathbb{E}[f_1(x)f_1(x')] = \mathbb{E}[a_1^1 u^1(x) a_1^1 u^1(x')] = (a_1^1)^2 \mathbb{E}[u^1(x)u^1(x')]$$

$$\mathbb{E}[f_1(x)f_2(x')] = \mathbb{E}[a_1^1 u^1(x) a_2^1 u^1(x')] = a_1^1 a_2^1 \mathbb{E}[u^1(x)u^1(x')]$$

$$\mathbb{E}[f_2(x)f_1(x')] = \mathbb{E}[a_2^1 u^1(x) a_1^1 u^1(x')] = a_2^1 a_1^1 \mathbb{E}[u^1(x)u^1(x')]$$

$$\mathbb{E}[f_2(x)f_2(x')] = \mathbb{E}[a_2^1 u^1(x) a_2^1 u^1(x')] = (a_2^1)^2 \mathbb{E}[u^1(x)u^1(x')]$$

$$\begin{aligned} \mathbb{E}[\mathbf{f}(x)\mathbf{f}(x')^T] &= \begin{bmatrix} (a_1^1)^2 \mathbb{E}[u^1(x)u^1(x')] & a_1^1 a_2^1 \mathbb{E}[u^1(x)u^1(x')] \\ a_2^1 a_1^1 \mathbb{E}[u^1(x)u^1(x')] & (a_2^1)^2 \mathbb{E}[u^1(x)u^1(x')] \end{bmatrix} \\ &= \begin{bmatrix} (a_1^1)^2 & a_1^1 a_2^1 \\ a_2^1 a_1^1 & (a_2^1)^2 \end{bmatrix} \mathbb{E}[u^1(x)u^1(x')] \end{aligned}$$

ICM: two outputs and one latent sample

$$\mathbf{f}(x) = \begin{bmatrix} f_1(x) \\ f_2(x) \end{bmatrix}, \quad \begin{aligned} f_1(x) &= a_1^1 u^1(x) \\ f_2(x) &= a_2^1 u^1(x) \end{aligned}$$

$$\text{cov}[\mathbf{f}(x), \mathbf{f}(x')] = \mathbb{E}[\mathbf{f}(x)\mathbf{f}(x')^T] - \mathbb{E}[\mathbf{f}(x)]\mathbb{E}[\mathbf{f}(x')]^T$$

$$\mathbb{E}[\mathbf{f}(x)] = \mathbb{E} \begin{bmatrix} f_1(x) \\ f_2(x) \end{bmatrix} = \mathbb{E} \begin{bmatrix} a_1^1 u^1(x) \\ a_2^1 u^1(x) \end{bmatrix} = \begin{bmatrix} a_1^1 \\ a_2^1 \end{bmatrix} \mathbb{E}[u^1(x)]$$

$$\begin{aligned} \mathbb{E}[\mathbf{f}(x)]\mathbb{E}[\mathbf{f}(x')]^T &= \begin{bmatrix} a_1^1 \\ a_2^1 \end{bmatrix} \mathbb{E}[u^1(x)][a_1^1, a_2^1] \mathbb{E}[u^1(x')] \\ &= \begin{bmatrix} (a_1^1)^2 & a_1^1 a_2^1 \\ a_2^1 a_1^1 & (a_2^1)^2 \end{bmatrix} \mathbb{E}[u^1(x)]\mathbb{E}[u^1(x')] \end{aligned}$$

ICM: two outputs and one latent sample

$$\mathbb{E}[\mathbf{f}(x)\mathbf{f}(x')^T] = \begin{bmatrix} (a_1^1)^2 & a_1^1 a_2^1 \\ a_2^1 a_1^1 & (a_2^1)^2 \end{bmatrix} \mathbb{E}[u^1(x)u^1(x')]$$

$$\mathbb{E}[\mathbf{f}(x)]\mathbb{E}[\mathbf{f}(x')]^T = \begin{bmatrix} (a_1^1)^2 & a_1^1 a_2^1 \\ a_2^1 a_1^1 & (a_2^1)^2 \end{bmatrix} \mathbb{E}[u^1(x)]\mathbb{E}[u^1(x')]$$

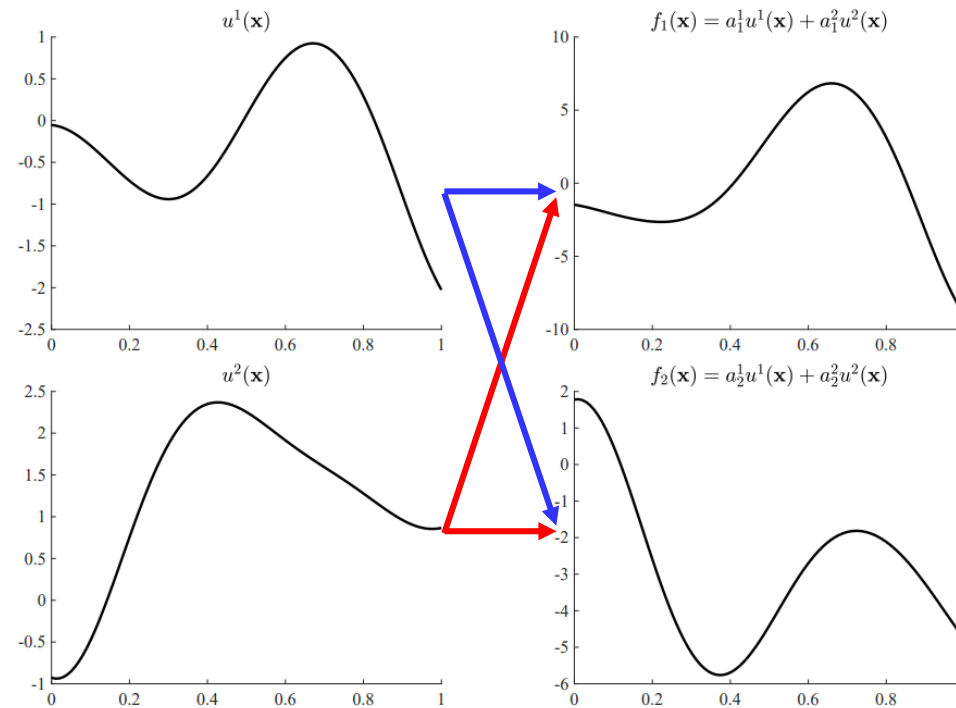
- Then,

$$\begin{aligned} \text{cov}[\mathbf{f}(x), \mathbf{f}(x')] &= \mathbb{E}[\mathbf{f}(x)\mathbf{f}(x')^T] - \mathbb{E}[\mathbf{f}(x)]\mathbb{E}[\mathbf{f}(x')]^T \\ &= \begin{bmatrix} (a_1^1)^2 & a_1^1 a_2^1 \\ a_2^1 a_1^1 & (a_2^1)^2 \end{bmatrix} \mathbb{E}[u^1(x)u^1(x')] - \begin{bmatrix} (a_1^1)^2 & a_1^1 a_2^1 \\ a_2^1 a_1^1 & (a_2^1)^2 \end{bmatrix} \mathbb{E}[u^1(x)]\mathbb{E}[u^1(x')] \\ &= \begin{bmatrix} (a_1^1)^2 & a_1^1 a_2^1 \\ a_2^1 a_1^1 & (a_2^1)^2 \end{bmatrix} (\mathbb{E}[u^1(x)u^1(x')] - \mathbb{E}[u^1(x)]\mathbb{E}[u^1(x')]) \\ &= \mathbf{a}\mathbf{a}^T k(x, x') \quad \text{where } \mathbf{a} = [a_1^1, a_2^1]^T \\ &= \mathbf{B}k(x, x') = \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix} k(x, x') \quad \text{where } \mathbf{B} \text{ has rank one} \end{aligned}$$

ICM: two outputs and two latent sample

- Consider two outputs $f_1(x)$ and $f_2(x)$ with $x \in \mathbb{R}^p$
- We assume the following generative model for the outputs
 - Sample from a GP $u(x) \sim \mathcal{GP}(0, k(x, x'))$ to obtain $u^1(x)$ and $u^2(x)$
 - Obtain $f_1(x)$ and $f_2(x)$ by linearly transforming $u^1(x)$ and $u^2(x)$

$$f_1(x) = a_1^1 u^1(x) + a_1^2 u^2(x)$$
$$f_2(x) = a_2^1 u^1(x) + a_2^2 u^2(x)$$



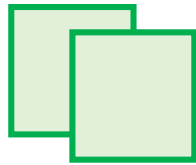
ICM: two outputs and two latent sample

$$\mathbf{f}(x) = \begin{bmatrix} f_1(x) \\ f_2(x) \end{bmatrix} = \begin{bmatrix} a_1^1 u^1(x) + a_1^2 u^2(x) \\ a_2^1 u^1(x) + a_2^2 u^2(x) \end{bmatrix} = \mathbf{a}^1 u^1(x) + \mathbf{a}^2 u^2(x)$$

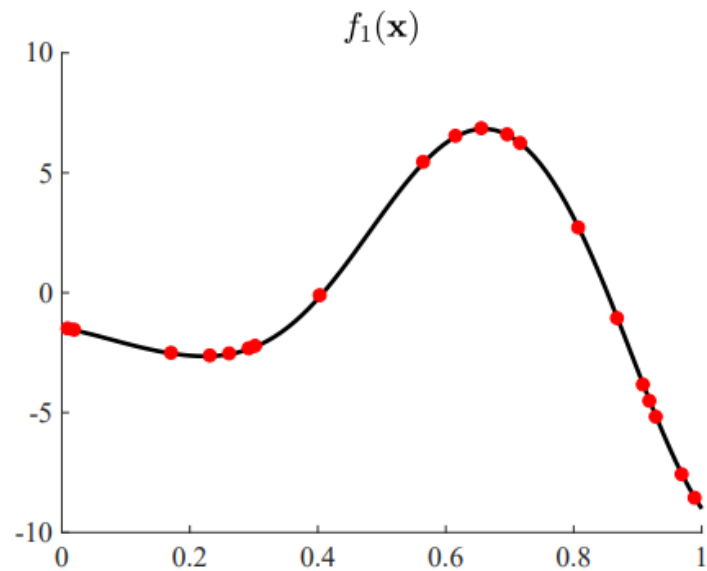
where $\mathbf{a}^1 = [a_1^1, a_2^1]^T$, and $\mathbf{a}^2 = [a_1^2, a_2^2]^T$.

$$\begin{aligned} \text{cov}[\mathbf{f}(x), \mathbf{f}(x')] &= \mathbf{a}^1 (\mathbf{a}^1)^T \text{cov}[u^1(x), u^1(x')] + \mathbf{a}^2 (\mathbf{a}^2)^T \text{cov}[u^2(x), u^2(x')] \\ &= \mathbf{a}^1 (\mathbf{a}^1)^T k(x, x') + \mathbf{a}^2 (\mathbf{a}^2)^T k(x, x') \\ &= \left[\mathbf{a}^1 (\mathbf{a}^1)^T + \mathbf{a}^2 (\mathbf{a}^2)^T \right] k(x, x') \\ &= \mathbf{B} k(x, x') = \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix} k(x, x') \end{aligned}$$

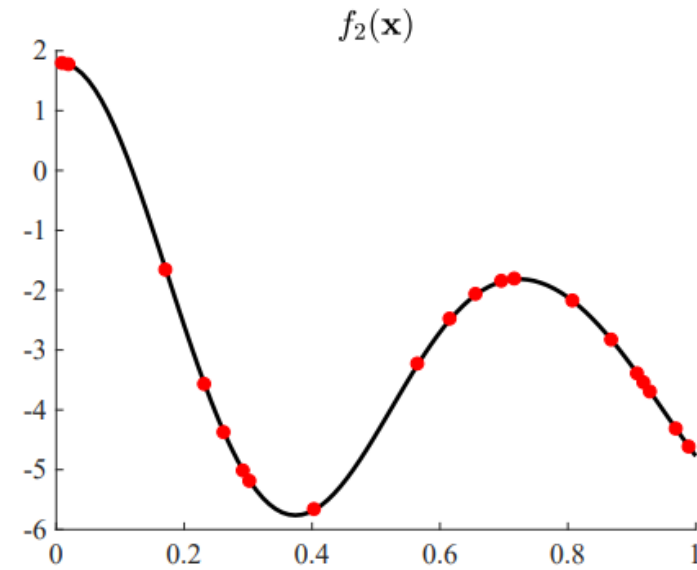
where $\mathbf{B} = \left[\mathbf{a}^1 (\mathbf{a}^1)^T + \mathbf{a}^2 (\mathbf{a}^2)^T \right]$ has rank two.



ICM: two outputs and two latent sample



$$\mathcal{D}_1 = \{(x_i, f_1(x_i)) | i = 1, \dots, N\}$$



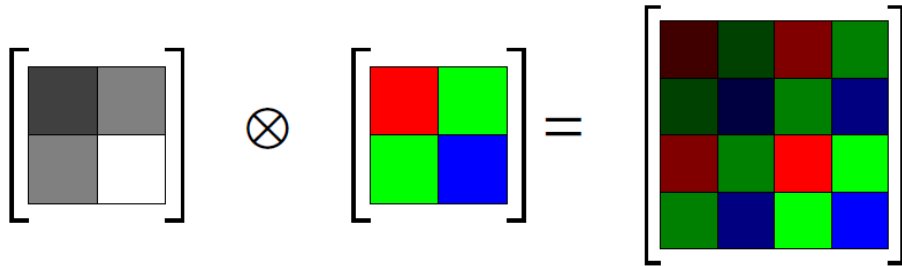
$$\mathcal{D}_2 = \{(x_i, f_2(x_i)) | i = 1, \dots, N\}$$

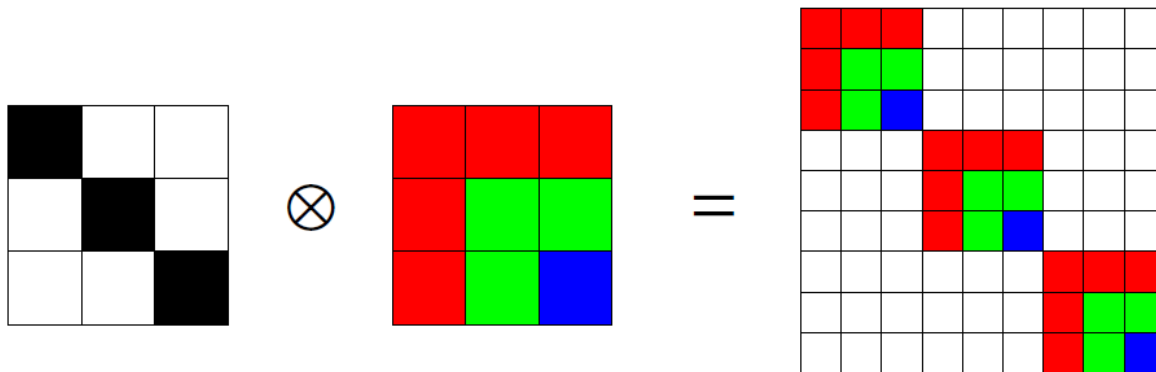
$$\begin{bmatrix} \mathbf{f}_1 \\ \mathbf{f}_2 \end{bmatrix} = \begin{bmatrix} f_1(x_1) \\ \vdots \\ f_1(x_N) \\ f_2(x_1) \\ \vdots \\ f_2(x_N) \end{bmatrix} \sim N \left(\begin{bmatrix} \mathbf{0} \\ \mathbf{0} \end{bmatrix}, \begin{bmatrix} b_{11}\mathbf{K} & b_{12}\mathbf{K} \\ b_{21}\mathbf{K} & b_{22}\mathbf{K} \end{bmatrix} \right) = N \left(\begin{bmatrix} \mathbf{0} \\ \mathbf{0} \end{bmatrix}, \mathbf{B} \otimes \mathbf{K} \right)$$

The matrix $\mathbf{K} \in \mathbb{R}^{N \times N}$ whose (i, j) element is $k(x_i, x_j)$

Kronecker Product

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \otimes \mathbf{K} = \begin{bmatrix} a\mathbf{K} & b\mathbf{K} \\ c\mathbf{K} & d\mathbf{K} \end{bmatrix}$$





ICM: general case

- Let D be a dimension of output, and R be a dimension of latent samples. Consider a set of functions $\{f_d(x)\}_{d=1}^D$.

$$f_1(x) = a_1^1 u^1(x) + a_1^2 u^2(x) + \cdots + a_1^R u^R(x)$$

$$\vdots$$

$$f_d(x) = a_d^1 u^1(x) + a_d^2 u^2(x) + \cdots + a_d^R u^R(x)$$

$$\vdots$$

$$f_D(x) = a_D^1 u^1(x) + a_D^2 u^2(x) + \cdots + a_D^R u^R(x)$$

$$f_d(x) = \sum_{i=1}^R a_d^i u^i(x)$$

where $u^i(x)$ are sampled independently from the same GP whose covariance function is $k(x, x')$.

- For $\mathbf{f}(x) = [f_1(x), \cdots, f_D(x)]^T$, the covariance $\text{cov}[\mathbf{f}(x), \mathbf{f}(x')]$ is given as

$$\text{cov}[\mathbf{f}(x), \mathbf{f}(x')] = \mathbf{A}\mathbf{A}^T k(x, x') = \mathbf{B}k(x, x')$$

where $\mathbf{A} = [\mathbf{a}^1, \mathbf{a}^2, \dots, \mathbf{a}^R]$. The rank of $\mathbf{B} \in \mathbb{R}^{R \times R}$ is R .

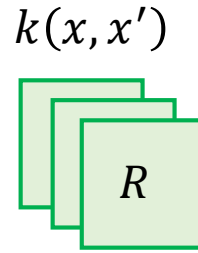
ICM: general case

$$\begin{bmatrix} \mathbf{f}_1 \\ \mathbf{f}_2 \\ \vdots \\ \mathbf{f}_D \end{bmatrix} = \begin{bmatrix} f_1(x_1) \\ \vdots \\ f_1(x_N) \\ f_2(x_1) \\ \vdots \\ f_2(x_N) \\ \vdots \\ f_D(x_1) \\ \vdots \\ f_D(x_N) \end{bmatrix} \sim \mathcal{N} \left(\begin{bmatrix} \mathbf{0} \\ \mathbf{0} \\ \vdots \\ \mathbf{0} \end{bmatrix}, \begin{bmatrix} b_{11}\mathbf{K} & b_{12}\mathbf{K} & \cdots & b_{1N}\mathbf{K} \\ b_{21}\mathbf{K} & b_{22}\mathbf{K} & \cdots & b_{2N}\mathbf{K} \\ \vdots & \vdots & \ddots & \vdots \\ b_{N1}\mathbf{K} & b_{N2}\mathbf{K} & \cdots & b_{NN}\mathbf{K} \end{bmatrix} \right) = \mathcal{N} \left(\begin{bmatrix} \mathbf{0} \\ \mathbf{0} \\ \vdots \\ \mathbf{0} \end{bmatrix}, \mathbf{B} \otimes \mathbf{K} \right)$$

- The matrix $\mathbf{B} \in \mathbb{R}^{R \times R}$ has rank R
- The matrix $\mathbf{K} \in \mathbb{R}^{N \times N}$ whose (i, j) element is $k(x_i, x_j)$
- $\mathbf{B} \otimes \mathbf{K} \in \mathbb{R}^{NR \times NR}$

Semiparametric Latent Factor Model (SLFM)

- ICM uses R samples $u^i(x)$ $i = 1, \dots, R$ from the same GP having the same covariance function $k(x, x')$



- SLFM uses Q samples from $u_q(x)$ processes with different covariance function $k_q(x, x')$

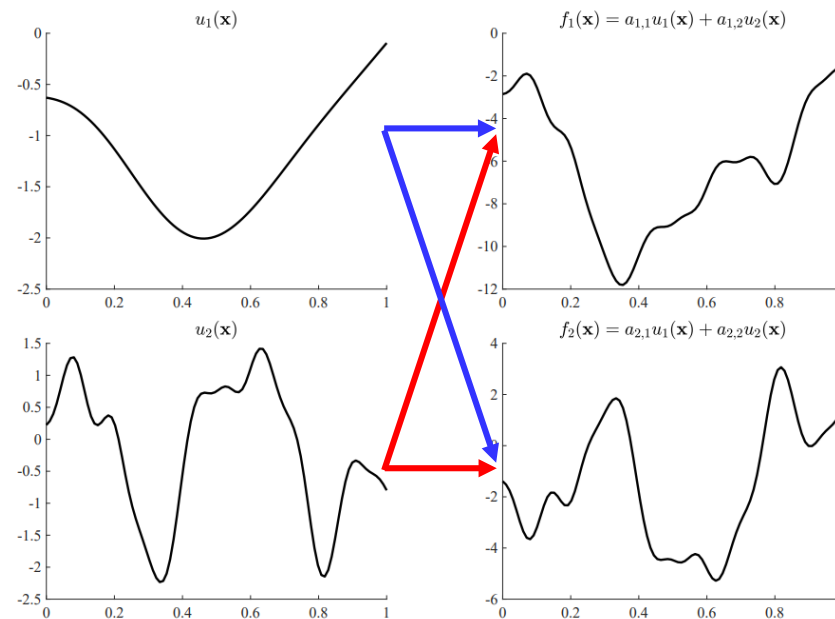


SLFM: two latent factors

- Consider two outputs $f_1(x)$ and $f_2(x)$ with $x \in \mathbb{R}^p$
- We assume the following generative model for the outputs (Assume $Q = 2$)
 - Sample from a GP $u_1(x) \sim \mathcal{GP}(0, k_1(x, x'))$ to obtain $u_1(x)$
 - Sample from a GP $u_2(x) \sim \mathcal{GP}(0, k_2(x, x'))$ to obtain $u_2(x)$
 - Obtain $f_1(x)$ and $f_2(x)$ by linearly transforming $u_1(x)$ and $u_2(x)$

$$f_1(x) = a_{1,1}u_1(x) + a_{1,2}u_2(x)$$

$$f_2(x) = a_{2,1}u_1(x) + a_{2,2}u_2(x)$$



SLFM: two latent factors

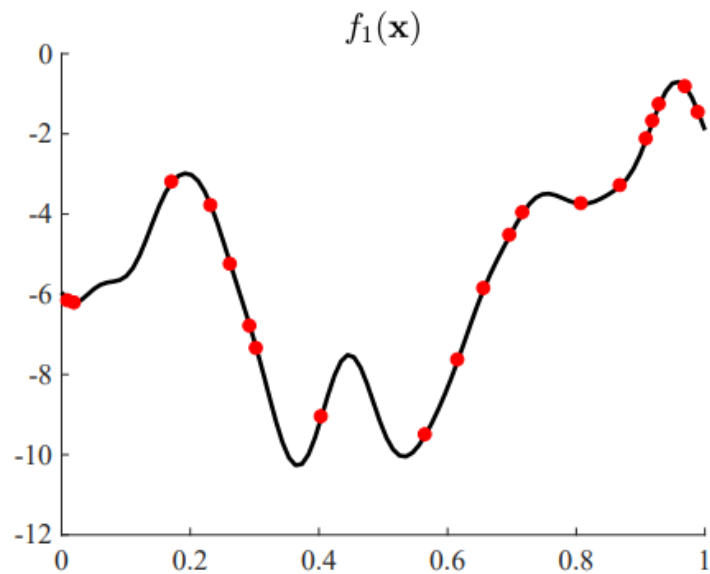
$$\mathbf{f}(x) = \begin{bmatrix} f_1(x) \\ f_2(x) \end{bmatrix} = \begin{bmatrix} a_{1,1}u_1(x) + a_{1,2}u_2(x) \\ a_{2,1}u_1(x) + a_{2,2}u_2(x) \end{bmatrix} = \mathbf{a}_1 u_1(x) + \mathbf{a}_2 u_2(x)$$

where $\mathbf{a}_1 = [a_{1,1}, a_{2,1}]^T$, and $\mathbf{a}_2 = [a_{1,2}, a_{2,2}]^T$.

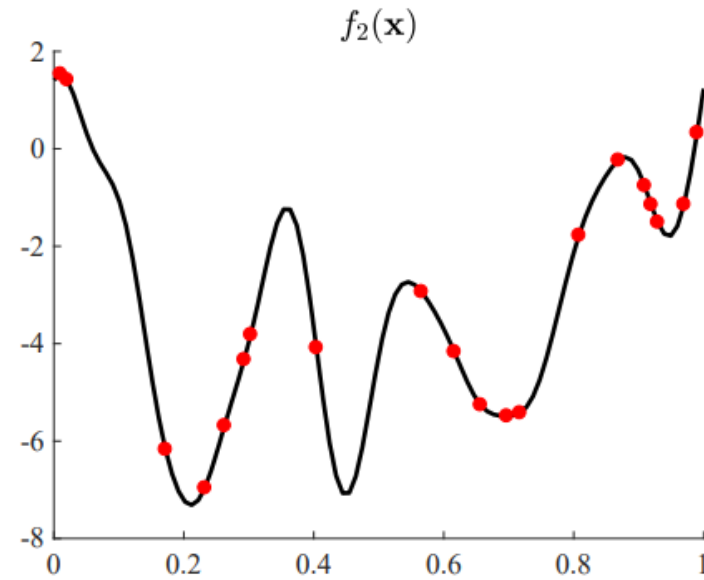
$$\begin{aligned} \text{cov}[\mathbf{f}(x), \mathbf{f}(x')] &= \mathbf{a}_1 (\mathbf{a}_1)^T \text{cov}[u_1(x), u_1(x')] + \mathbf{a}_2 (\mathbf{a}_2)^T \text{cov}[u_2(x), u_2(x')] \\ &= \mathbf{a}_1 (\mathbf{a}_1)^T k_1(x, x') + \mathbf{a}_2 (\mathbf{a}_2)^T k_2(x, x') \\ &= \mathbf{B}_1 k_1(x, x') + \mathbf{B}_2 k_2(x, x') \end{aligned}$$

where $\mathbf{B}_1 = \mathbf{a}_1 (\mathbf{a}_1)^T$ and $\mathbf{B}_2 = \mathbf{a}_2 (\mathbf{a}_2)^T$ has rank one.

Semiparametric Latent Factor Model (SLFM)



$$\mathcal{D}_1 = \{(x_i, f_1(x_i)) | i = 1, \dots, N\}$$



$$\mathcal{D}_2 = \{(x_i, f_2(x_i)) | i = 1, \dots, N\}$$

$$\begin{bmatrix} \mathbf{f}_1 \\ \mathbf{f}_2 \end{bmatrix} = \begin{bmatrix} f_1(x_1) \\ \vdots \\ f_1(x_N) \\ f_2(x_1) \\ \vdots \\ f_2(x_N) \end{bmatrix} \sim \mathcal{N} \left(\begin{bmatrix} \mathbf{0} \\ \mathbf{0} \end{bmatrix}, \mathbf{B}_1 \otimes \mathbf{K}_1 + \mathbf{B}_2 \otimes \mathbf{K}_2 \right)$$

The matrix $\mathbf{K}_1 \in \mathbb{R}^{N \times N}$ whose (i, j) element is $k_1(x_i, x_j)$

The matrix $\mathbf{K}_2 \in \mathbb{R}^{N \times N}$ whose (i, j) element is $k_2(x_i, x_j)$

SLFM: general case

- Let D be a dimension of output, and Q be a dimension of latent samples. Consider a set of functions $\{f_d(x)\}_{d=1}^D$.

$$\begin{aligned} f_1(x) &= a_{1,1}u_1(x) + a_{1,2}u_2(x) + \cdots + a_{1,Q}u_Q(x) \\ &\vdots \\ f_d(x) &= a_{d,1}u_1(x) + a_{d,2}u_2(x) + \cdots + a_{d,Q}u_Q(x) \\ &\vdots \\ f_D(x) &= a_{D,1}u_1(x) + a_{D,2}u_2(x) + \cdots + a_{D,Q}u_Q(x) \\ f_d(x) &= \sum_{q=1}^Q a_{d,q}u_q(x) \end{aligned}$$

where $u_q(x)$ is GP whose covariance function is $k_q(x, x')$.

- For $\mathbf{f}(x) = [f_1(x), \cdots, f_D(x)]^T$, the covariance $\text{cov}[\mathbf{f}(x), \mathbf{f}(x')]$ is given as

$$\text{cov}[\mathbf{f}(x), \mathbf{f}(x')] = \sum_{q=1}^Q \mathbf{A}_q \mathbf{A}_q^T k_q(x, x') = \sum_{q=1}^Q \mathbf{B}_q k_q(x, x')$$

where $\mathbf{A}_q = [a_{1,q}, a_{2,q}, \cdots, a_{D,q}]^T$. The rank of $\mathbf{B}_q \in \mathbb{R}^{D \times D}$ is 1.

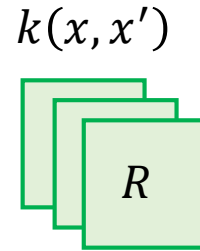
SLFM: general case

$$\begin{bmatrix} \mathbf{f}_1 \\ \mathbf{f}_2 \\ \vdots \\ \mathbf{f}_D \end{bmatrix} = \begin{bmatrix} f_1(x_1) \\ \vdots \\ f_1(x_N) \\ f_2(x_1) \\ \vdots \\ f_2(x_N) \\ \vdots \\ f_D(x_1) \\ \vdots \\ f_D(x_N) \end{bmatrix} \sim \mathcal{N} \left(\begin{bmatrix} \mathbf{0} \\ \mathbf{0} \\ \vdots \\ \mathbf{0} \end{bmatrix}, \sum_{q=1}^Q \mathbf{B}_q \otimes \mathbf{K}_q \right)$$

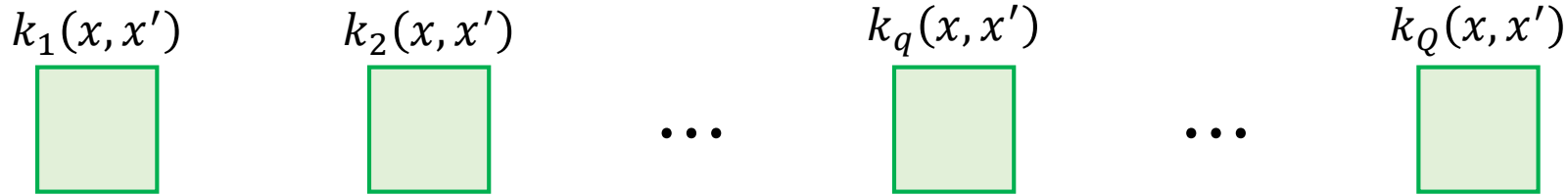
- The matrix $\mathbf{B}_q \in \mathbb{R}^{D \times D}$ has rank 1
- The matrix $\mathbf{K}_q \in \mathbb{R}^{N \times N}$ whose (i, j) element is $k_q(x_i, x_j)$
- $\mathbf{B}_q \otimes \mathbf{K}_q \in \mathbb{R}^{ND \times ND}$

Linear Model of Coregionalization (LMC)

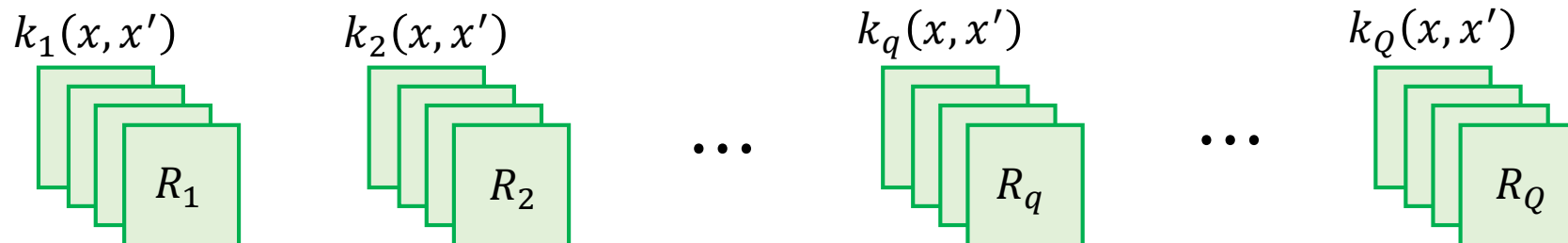
- ICM uses R samples $u^i(x)$ $i = 1, \dots, R$ from the same GP having the same covariance function $k(x, x')$



- SLFM uses Q samples from $u_q(x)$ processes with different covariance function $k_q(x, x')$



- Linear model of coregionalization (LMC) generalizes the ICM and SLFM allowing several independent samples from GPs with different covariances.



$$\mathbf{B}_q \in \mathbb{R}^{D \times D} \text{ has rank } R_q$$

Linear Model of Coregionalization (LMC)

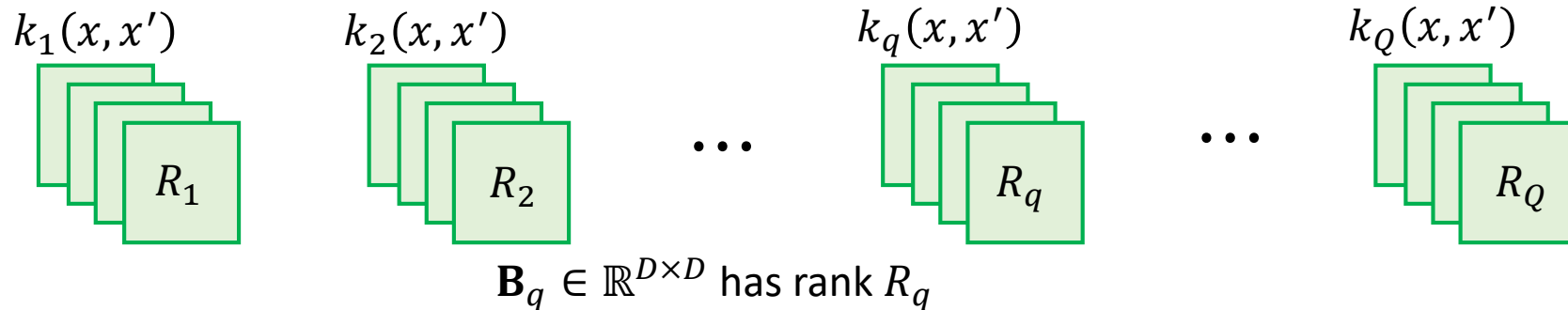
- Linear model of coregionalization (LMC) generalizes the ICM and SLFM allowing several independent samples from GPs with different covariances.
- Consider a set of functions $\{f_d(x)\}_{d=1}^D$.
- In the LMC

$$f_d(x) = \sum_{q=1}^Q \sum_{i=1}^{R_q} a_{d,q}^i u_q^i(x)$$

d : output index
 q : GP index sharing $k_q(x, x')$
 i : i -th sample $u_q^i(x)$ from q -th GP

where $u_q^i(x)$ for $i = 1, \dots, R_q$ are samples from GP whose covariance function is $k_q(x, x')$

$$\text{cov}[u_q^i(x), u_{q'}^{i'}(x)] = k_q(x, x') \text{ if } i = i' \text{ and } q = q'$$



Linear Model of Coregionalization (LMC)

- **SLMC**

- For $\mathbf{f}(x) = [f_1(x), \dots, f_D(x)]^T$, the covariance $\text{cov}[\mathbf{f}(x), \mathbf{f}(x')]$ is given as

$$\text{cov}[\mathbf{f}(x), \mathbf{f}(x')] = \sum_{q=1}^Q \mathbf{A}_q \mathbf{A}_q^T k_q(x, x') = \sum_{q=1}^Q \mathbf{B}_q k_q(x, x')$$

where $\mathbf{A}_q = \mathbf{a}_q = [a_{1,q}, a_{2,q}, \dots, a_{D,q}]^T$.

- The rank of $\mathbf{B}_q \in \mathbb{R}^{D \times D}$ is **1**.

- **LMC**

- For $\mathbf{f}(x) = [f_1(x), \dots, f_D(x)]^T$, the covariance $\text{cov}[\mathbf{f}(x), \mathbf{f}(x')]$ is given as

$$\text{cov}[\mathbf{f}(x), \mathbf{f}(x')] = \sum_{q=1}^Q \mathbf{A}_q \mathbf{A}_q^T k_q(x, x') = \sum_{q=1}^Q \mathbf{B}_q k_q(x, x')$$

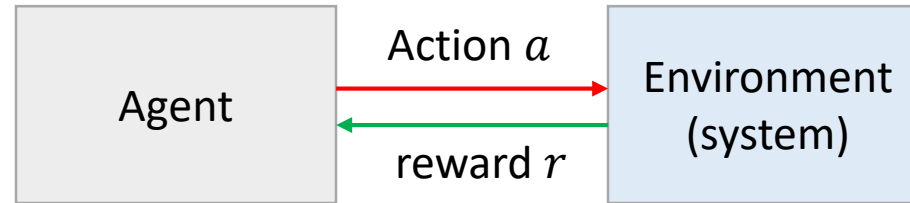
where $\mathbf{A}_q = [\mathbf{a}_q^1, \mathbf{a}_q^2, \dots, \mathbf{a}_q^{R_q}]$ where $\mathbf{a}_q^i = [a_{1,q}^i, a_{2,q}^i, \dots, a_{D,q}^i]^T$.

- The rank of $\mathbf{B}_q \in \mathbb{R}^{D \times D}$ is R_q (sample $u_q^i(x)$ for $i = 1, \dots, R_q$ from GP with $k_q(x, x')$).

Bandit problem

Introduction

- Learning by taking action: Take action, observe reward, infer environment(system) behavior and take a better action



- Two types of feedbacks can be used to infer environment (system)
- **Evaluative feedback** – how good the action taken is
 - ✓ Basis of reinforcement learning
 - ✓ Require extensive search among the alternative actions
- **Instructive feedback** – indicates the correct action to take
 - ✓ Basis of supervised learning
 - ✓ Independent of the action taken → whatever action taken, the right one should have been taken will be revealed.
 - ✓ Search over a parameter space
- Before discussing Reinforcement learning, we first study the evaluative aspect of reinforcement learning in a simplified setting.

Evaluation vs Instructive feedback



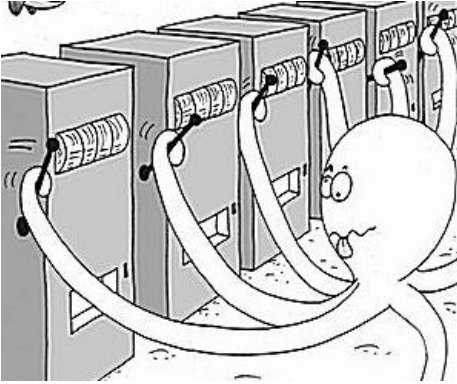
- When I pulled No. 3,
- **Evaluative feedback** says “you earn 1\$”
 - ✓ Correctness is a **relative property** of actions that can be determined only by trying them all and comparing their rewards
- **Instructive feedback** says “The best machine is No.2”
 - ✓ No need to try variety of actions
 - ✓ Condensed information
 - ✓ Regardless of the action chosen

From Bandit to Reinforcement Learning

		Model is unknown	
		No state dynamic	State dynamic (action affect state transition and future reward)
Optimum action	Single state (non-associative)	Bandit	-
Optimum policy	Multiple state (associative)	Contextual Bandit	Reinforcement learning

- Contextual Bandit (associative search task) is intermediate between the n -armed bandit problem and full reinforcement learning problem

An n –armed Bandit Problem



- There are n machines
- Each machine i returns a reward $r \sim P(\theta_i)$: θ_i is unknown
- $a_t \in \{1, \dots, n\}$: the choice of machine at time t
- r_t : the reward at time t
- Policy π maps all the history to new action:

$$\pi: [(a_1, r_1), (a_2, r_2), \dots, (a_{t-1}, r_{t-1})] \rightarrow a_t$$

Find the optimal policy π^* that maximizes $\mathbb{E}[\sum_{t=1}^T r_t]$ or $\mathbb{E}[r_T]$

Acquiring new information
(exploration)

trade-off

capitalizing on the information
available so far
(exploitation)

Internet add : Advertising showing strategy for users

Finance : Portfolio optimization under unknown return profiles (risk vs mean profit)

Health care : Choosing the best treatment among alternatives

Internet shopping : Choosing the optimum price (sales v.s. profits)

Experiment design : Sequential experimental design (or sequential simulation parameter)

An n –armed Bandit Problem

The value of the action $Q^*(a)$:

The true value of action a is defined as the mean reward received when that action is selected

$$Q^*(a) = \mathbb{E}[r|a]$$

At any time there will be at least one action whose **value of the action** is the largest

- **Select greedy action (Exploiting)**: the action with the largest value of action
- **Select non-greedy action (Exploring)**: the action enabling you to improve your estimate of the non-greedy actions' value

Exploiting

Go to a company to make money
right after undergraduate



Exploring

Go to a graduate school to explore my
intellectual capability?



How to balance between exploitation and exploration ?

Action-Value method

- Action-value method estimates the value of an action and uses this to select the action.
- Natural way to estimate the *action-value* is averaging the rewards actually received when the action was selected.
- If at time t , **action a has been chosen k_a times prior to t** , yielding rewards, $r_1, r_2, \dots, r_{k_a}$, the estimated action value $Q_t(a)$ for a at time t is defined as

$$Q_t(a) = \frac{r_1 + r_2 + \dots + r_{k_a}}{k_a}$$

If $k_a = 0$, $Q_t(a) = 0$
If $k_a = \infty$, $Q_t(a) \rightarrow Q^*(a)$

- Greedy action selection rule is

$$a_t = \operatorname{argmax}_a Q_t(a)$$

- Always exploits current knowledge to maximize immediate reward, that is, it spends no time at all sampling apparently inferior actions to see if they might really be better.

Action-Value method

- If at t -th play, action a has been chosen k_a times prior to t , yielding rewards, $r_1, r_2, \dots, r_{k_a}$, the estimated action value for a is defined as

$$Q_t(a) = \frac{r_1 + r_2 + \dots + r_{k_a}}{k_a}$$

If $k_a = 0$, $Q_t(a) = 0$

If $k_a = \infty$, $Q_t(a) \rightarrow Q^*(a)$

greedy action selection rule

$$a_t = \underset{a}{\operatorname{argmax}} Q_t(a)$$

- ✓ Always exploits current knowledge to maximize immediate reward
- ✓ spends no time at all sampling apparently inferior actions to see if they might really be better

ϵ - *greedy* action selection rule

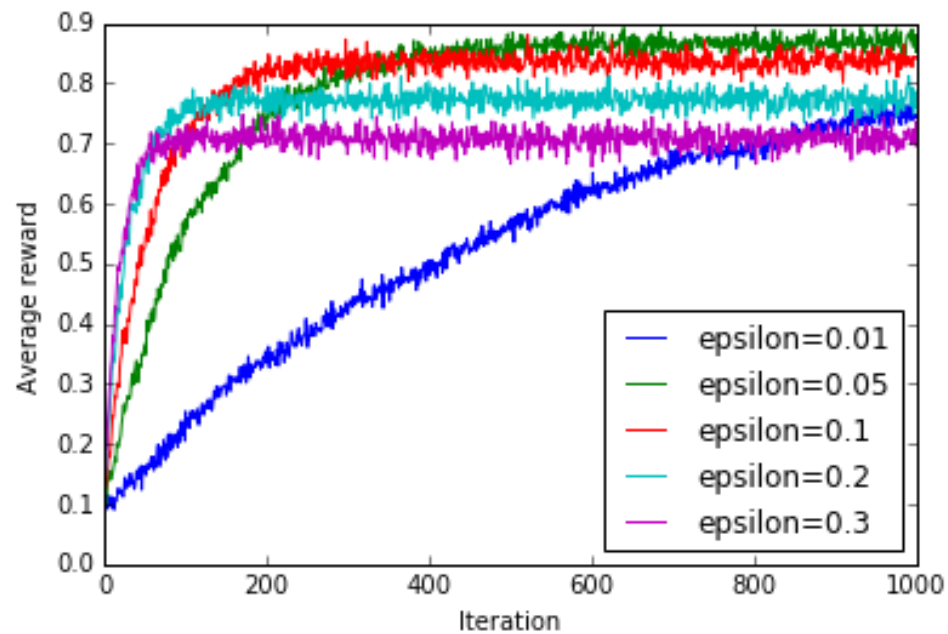
$$\pi(a) = \begin{cases} 1 - \epsilon + \epsilon/|A(s)| & \text{If } a = a^* \\ \epsilon/|A(s)| & \text{If } a \neq a^* \end{cases}$$

$$\left(1 - \epsilon + \frac{\epsilon}{|A(s)|}\right) \times 1 + \frac{\epsilon}{|A(s)|} (|A(s)| - 1) = 1$$

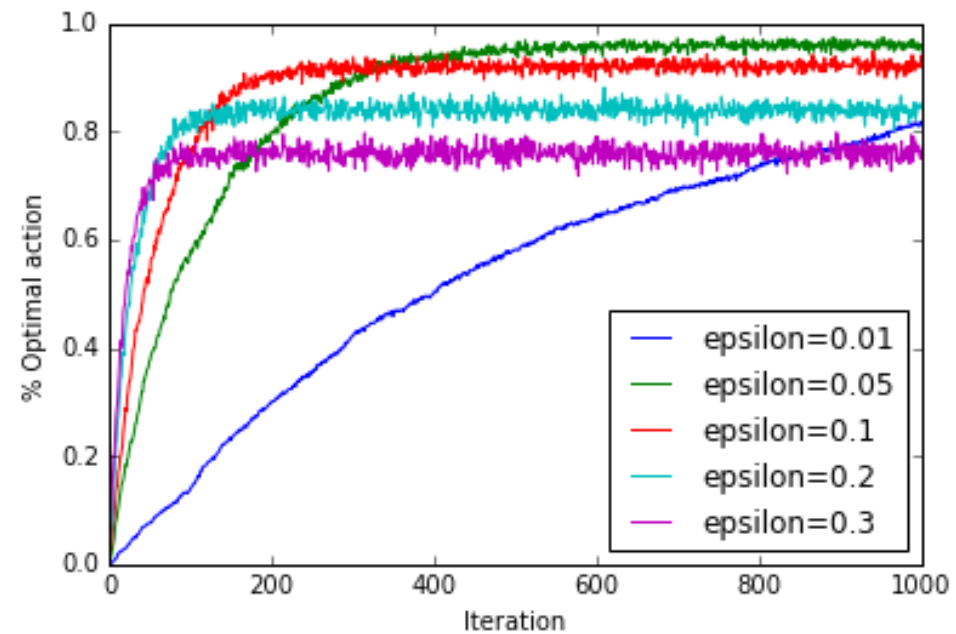
- ✓ In the limit as the number of plays increases, every a will be sampled an infinite number of times, guaranteeing $k_a \rightarrow \infty$ thus $Q_t(a) \rightarrow Q^*(a)$
- ✓ The probability of selecting the optimum action converges to a value greater than $1 - \epsilon$

Action-Value method

Average reward $\mathbb{E}[r_t]$

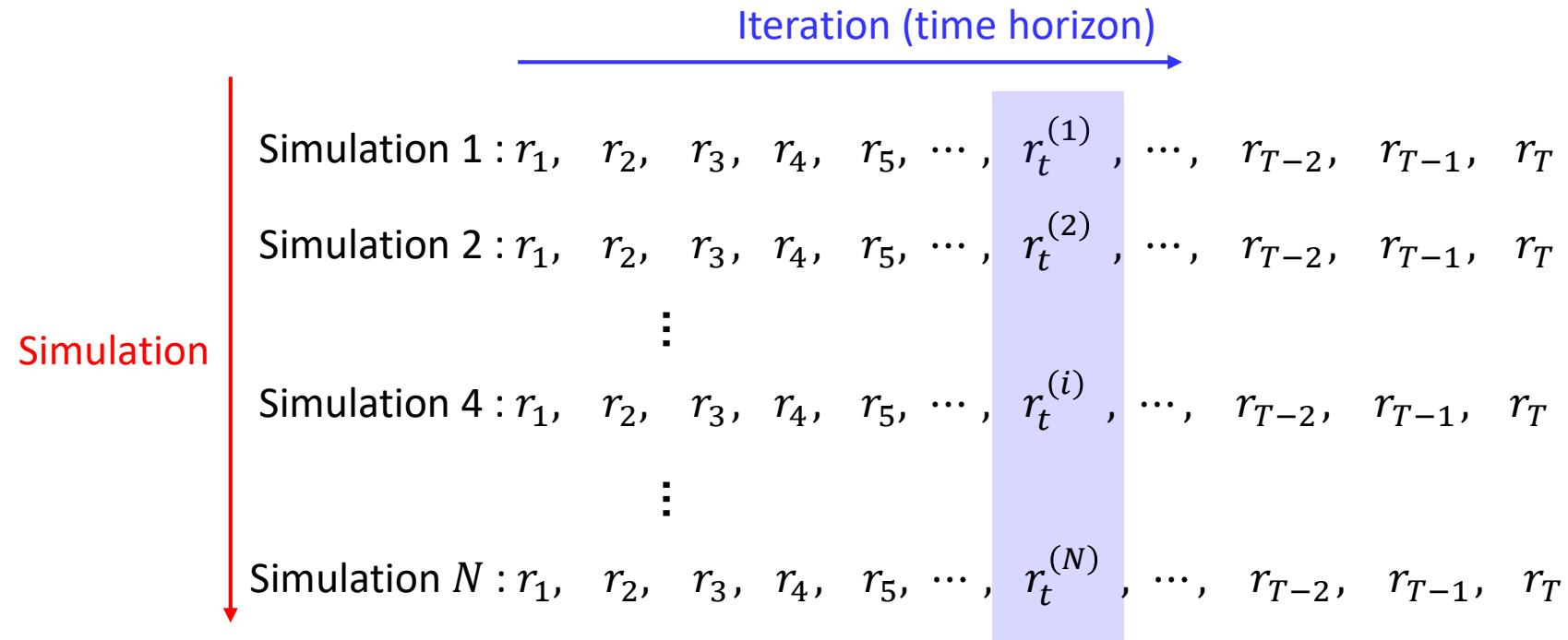


% of selecting a^*



Action-Value method

- Obtaining the performance curve



$$\mathbb{E}[r_t] = \frac{1}{N} \sum_{i=1}^N r_t^{(i)}$$

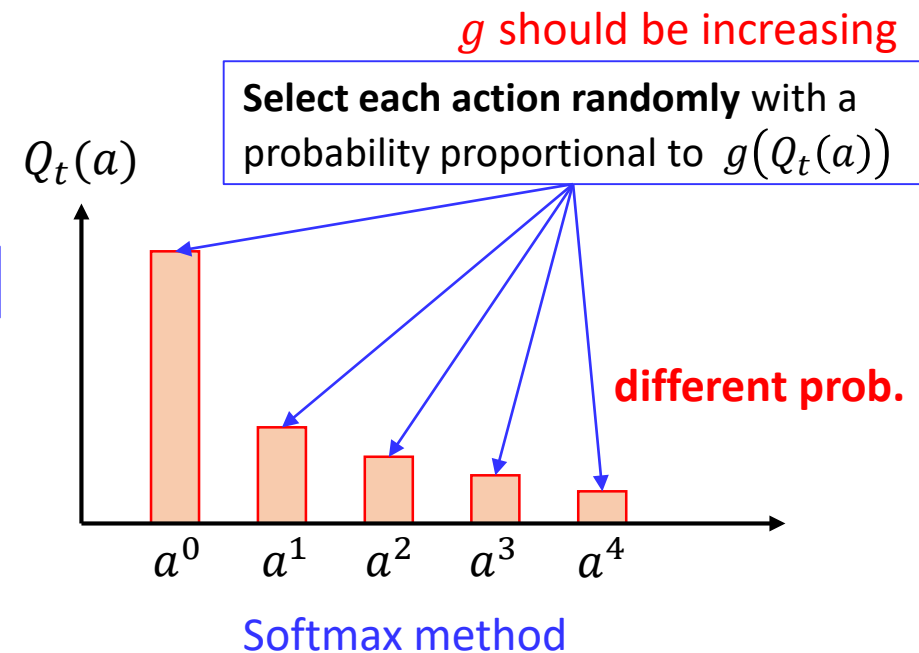
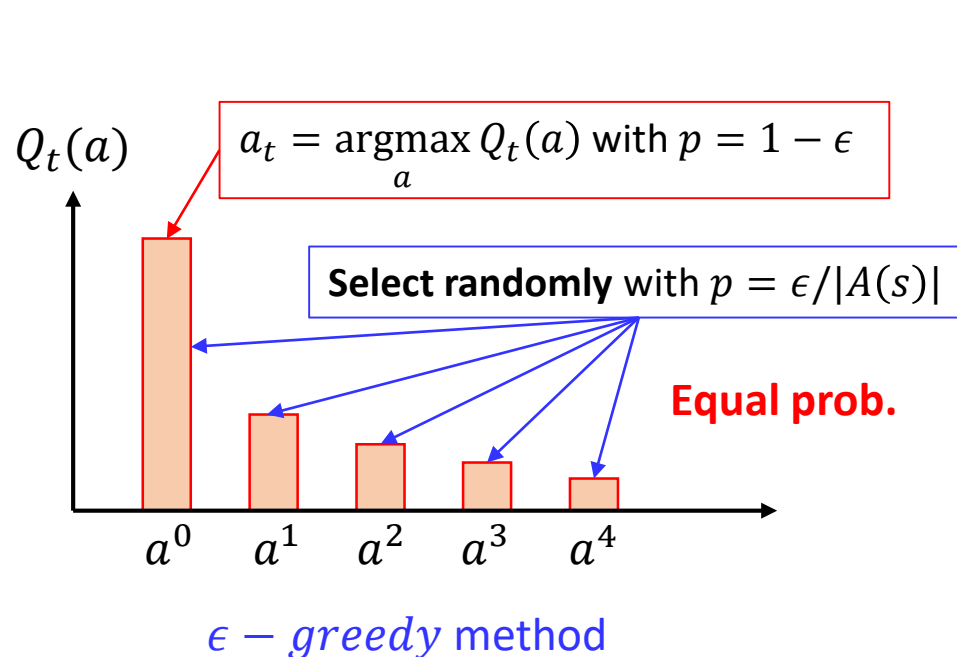
$$\% \text{ of } a^* = \frac{\sum_{i=1}^N \mathbf{1}\{a_t^i = a^*\}}{N}$$

Action-Value method

- Effectiveness of $\epsilon - greedy$ method depends on the task
 - If the reward variance is large, it would take more exploration to find the optimal action.
 - If the reward variance is 0 (deterministic), the $\epsilon - greedy$ method would know the true value of each action after trying it once. After finding the true reward, it is best to seek to exploit.
 - If the bandit task is non-stationary, exploration needs to continue (even in the deterministic case) to find a better action than to choose the current best action greedily.

Softmax Action Selection

- Disadvantage in ϵ – *greedy* action selection:
 - When it explores, it chooses equally among all actions. That is, it is as likely to choose the worst-appearing action as it is to choose the next-to best action
- Solution:
 - Vary the action probabilities as a graded function of estimated value
 - The greedy action is still given the highest selection probability, but all the others are ranked and weighted according to their values estimates



Softmax Action Selection

- **Disadvantage in ϵ – *greedy* action selection:**
 - When it explores, it chooses equally among all actions. That is, it is as likely to choose the worst-appearing action as it is to choose the next-to best action
- **Solution:**
 - Vary the action probabilities as a graded function of estimated value
 - The greedy action is still given the highest selection probability, but all the others are ranked and weighted according to their values estimates

Softmax action selection rule

It chooses action a on the t -th play with probability

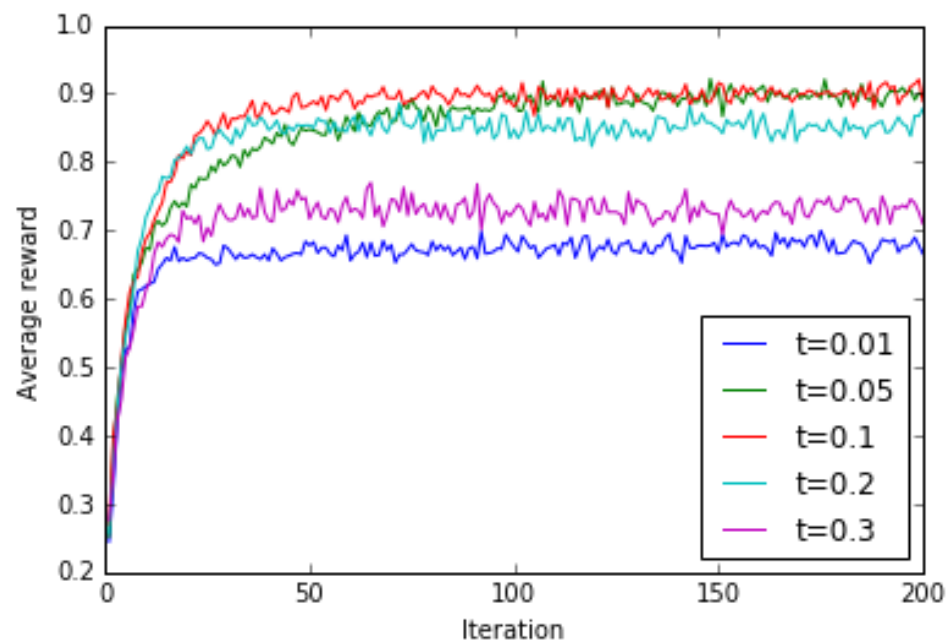
$$\frac{e^{Q_t(a)/\tau}}{\sum_{b=1}^n e^{Q_t(b)/\tau}}$$

- ✓ τ is a positive parameter called the temperature:
 - high $\tau \rightarrow$ all actions are equiprobable
 - low $\tau \rightarrow$ greater difference in selection probability for action
 - When $\tau = 0$, softmax selection rule become greedy one

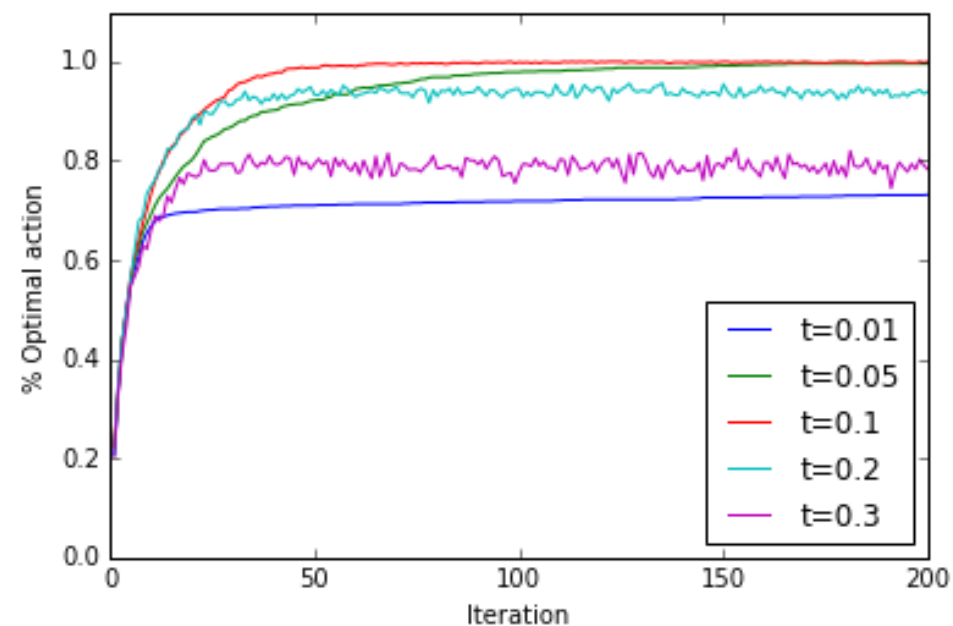
Whether softmax action selection or greedy action selection rule is better is unclear and may depend on the task and on human factors (i.e., setting τ and ϵ)

Softmax Action Selection

Average reward $\mathbb{E}[r_t]$



% of selecting a^*



Evaluation vs Instructive feedback



- When I pulled No. 3,
- **Evaluative feedback** says “you earn 1\$”
 - ✓ Correctness is a **relative property** of actions that can be determined only by trying them all and comparing their rewards
- **Instructive feedback** says “The best machine is No.2”
 - ✓ No need to try variety of actions
 - ✓ Condensed information
 - ✓ Regardless of the action chosen

Incremental Implementation

- At t -th play, action a has been chosen k_a times prior to t , yielding rewards, $r_1, r_2, \dots, r_{k_a}$, **the estimated action value** for a is defined as

$$Q_t(a) = \frac{r_1 + r_2 + \dots + r_{k_a}}{k_a}$$

- Issues in estimating the action value:**

- It needs to store the reward history $r_1 + r_2 + \dots + r_{k_a}$ for all different actions
- Memory and computational requirements grow over time without bound

- Incremental Update formula**

$$\begin{aligned} Q_{k+1} &= \frac{1}{k+1} \sum_{i=1}^{k+1} r_i = \frac{1}{k+1} \left(r_{k+1} + \sum_{i=1}^k r_i \right) \\ &= \frac{1}{k+1} (r_{k+1} + kQ_k + Q_k - Q_k) \\ &= \frac{1}{k+1} (r_{k+1} + (k+1)Q_k - Q_k) = Q_k + \frac{1}{k+1} [r_{k+1} - Q_k] \end{aligned}$$

- This implementation requires memory only for Q_k and k , and only the small computation

Step-size parameter and non-stationary

Incremental Update formula for action value

$$Q_{k+1} = Q_k + \frac{1}{k+1} [r_{k+1} - Q_k]$$

$$NewEstimate \leftarrow OldEstimate + \underset{\alpha_k}{StepSize} [Target - OldEstimate]$$

- When the distribution of the reward is non-stationary, it is natural to weight recent rewards more heavily than long-past ones
- One of the most popular ways of doing this is to use **a constant step-size parameter**

$$Q_{k+1} = Q_k + \alpha [r_{k+1} - Q_k]$$

$$\begin{aligned} Q_k &= Q_{k-1} + \alpha[r_k - Q_{k-1}] = \alpha r_k + (1 - \alpha)Q_{k-1} \\ &= \alpha r_k + (1 - \alpha)[\alpha r_{k-1} + (1 - \alpha)Q_{k-2}] \\ &= \alpha r_k + (1 - \alpha)\alpha r_{k-1} + (1 - \alpha)^2 Q_{k-2} \\ &= \alpha r_k + (1 - \alpha)\alpha r_{k-1} + (1 - \alpha)^2 \alpha r_{k-2} + \cdots + (1 - \alpha)^{k-1} \alpha r_1 + (1 - \alpha)^k Q_0 \\ &= (1 - \alpha)^k Q_0 + \sum_{i=1}^k \alpha (1 - \alpha)^{k-i} r_i \end{aligned}$$

- Reward $\alpha(1 - \alpha)^{k-i}$ given to i th reward r_i is smaller when $k - i$ is larger (i.e., older reward is weighted less)

The estimate of Q_k varies more in response to the most recently receive rewards!

Optimistic initial values

- In Estimating the action value $Q_{k+1} = Q_k + \alpha[r_{k+1} - Q_k]$, the initial value Q_0 should be specified by user.
- the initial value Q_0 can be used as **prior knowledge** about what level of rewards can be expected.
- Optimistic initial value encourages exploration
 - ✓ When the reward corresponding the current action is lower than the initial, the method will explore (change another action) to find a better reward.

$$Q_k = (1 - \alpha)^k Q_0 + \sum_{i=1}^k \alpha(1 - \alpha)^{k-i} r_i$$

Reinforcement Comparison

- Actions followed by large rewards should be made more likely to recur.
 - Actions followed by small rewards should be made less likely to recur.
 - If an action is taken and the environment returns a reward of 5, Is it good or bad?
- To make such a judgment, one must compare the reward with some standard or reference level, called the *reference reward*.

Reinforcement Comparison

Reinforcement Comparison algorithm

- $p_t(a)$: The preference for each action at time t (not an actual action value)
- Action determination rule (soft max) :

$$\pi_t(a) = \frac{e^{p_t(a)/\tau}}{\sum_{b=1}^n e^{p_t(b)/\tau}}$$

- After selecting action and observing reward, the preference $p_t(a)$ is updated :

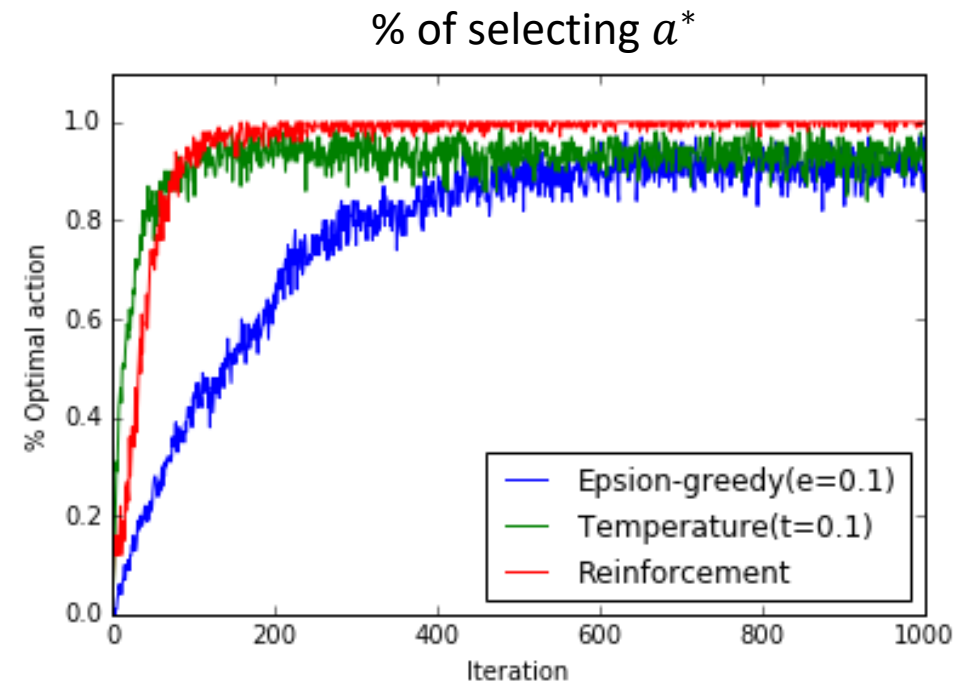
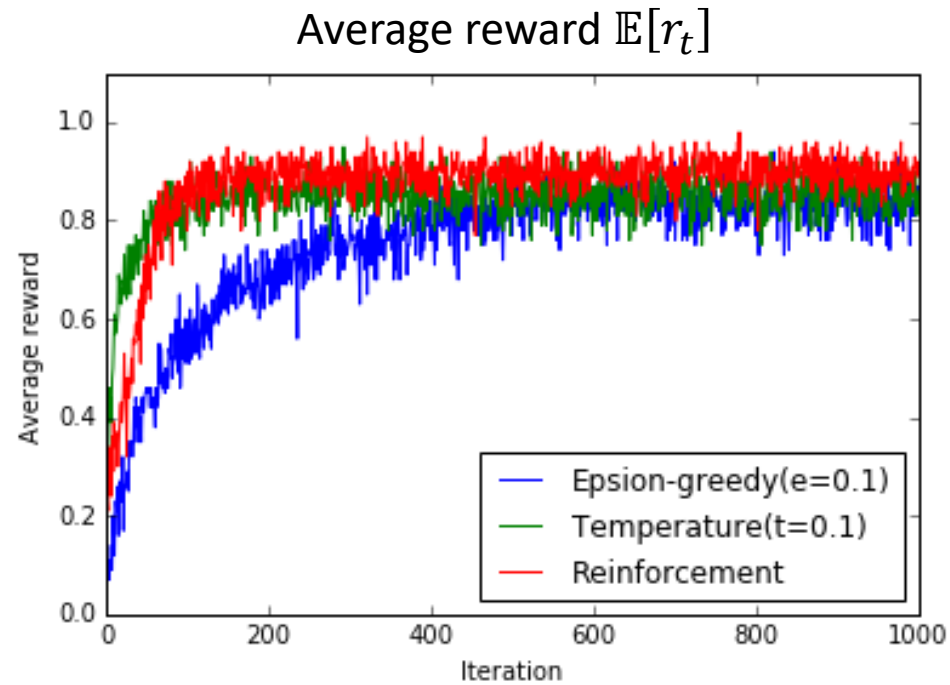
$$p_{t+1}(a_t) = p_t(a_t) + \beta[r_t - \bar{r}_t]$$

- ✓ High rewards should increase the probability of reselecting the action taken
- ✓ β is a positive step-size parameter
- ✓ The reference reward $\bar{r}_t$ (i.e., average reward) is updated using all recently received rewards whichever actions were taken:

$$\bar{r}_{t+1} = \bar{r}_t + \alpha[r_t - \bar{r}_t]$$

Reinforcement comparison method can be very effective sometimes outperforming $\epsilon - greedy$ method

Reinforcement Comparison



Pursuit method

Pursuit method maintain

- $Q_t(a)$: action–value estimate
- $\pi_t(a)$: action preferences, the probability of selecting action a

- The estimation of the action value $Q_t(a)$ is **used to select the greedy action**

$$a_{t+1}^* = \operatorname{argmax}_a Q_{t+1}(a)$$

- **Action preferences continually pursuit the action that is greedy.** After each play, the probabilities are incremented toward 1 so as to make the greedy action more likely to be selected

$$\pi_{t+1}(a_{t+1}^*) = \pi_t(a_{t+1}^*) + \beta[1 - \pi_t(a_{t+1}^*)]$$

- Probability of selecting other actions are decremented toward zero:

$$\pi_{t+1}(a) = \pi_t(a) + \beta[0 - \pi_t(a)]$$

- The next action is selected according to the policy

$$a_{t+1} \sim \pi_{t+1}(a)$$

Pursuit method

Pursuit method maintain

- $Q_t(a)$: action–value estimate
- $\pi_t(a)$: action preferences, the probability of selecting action a

- The estimation of the action value $Q_t(a)$ is **used to select the greedy action**

$$a_{t+1}^* = \operatorname{argmax}_a Q_{t+1}(a)$$

- **Action preferences continually pursuit the action that is greedy.** After each play, the probabilities are incremented toward 1 so as to make the greedy action more likely to be selected

$$\pi_{t+1}(a_{t+1}^*) = \pi_t(a_{t+1}^*) + \beta[1 - \pi_t(a_{t+1}^*)]$$

- Probability of selecting other actions are decremented toward zero:

$$\pi_{t+1}(a) = \pi_t(a) + \beta[0 - \pi_t(a)]$$

- The next action is selected according to the policy

$$a_{t+1} \sim \pi_{t+1}(a)$$

$$Q_t(a) \rightarrow \pi_t(a) \rightarrow a_{t+1} + r_{t+1} \rightarrow Q_{t+1}(a)$$

Upper Confidence Bound (UCB method)

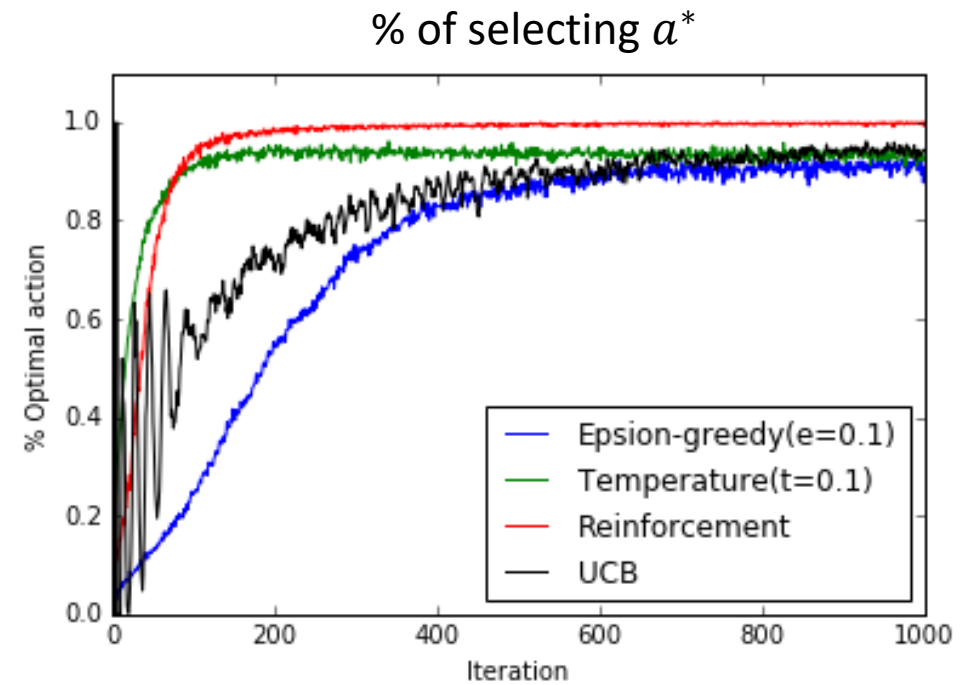
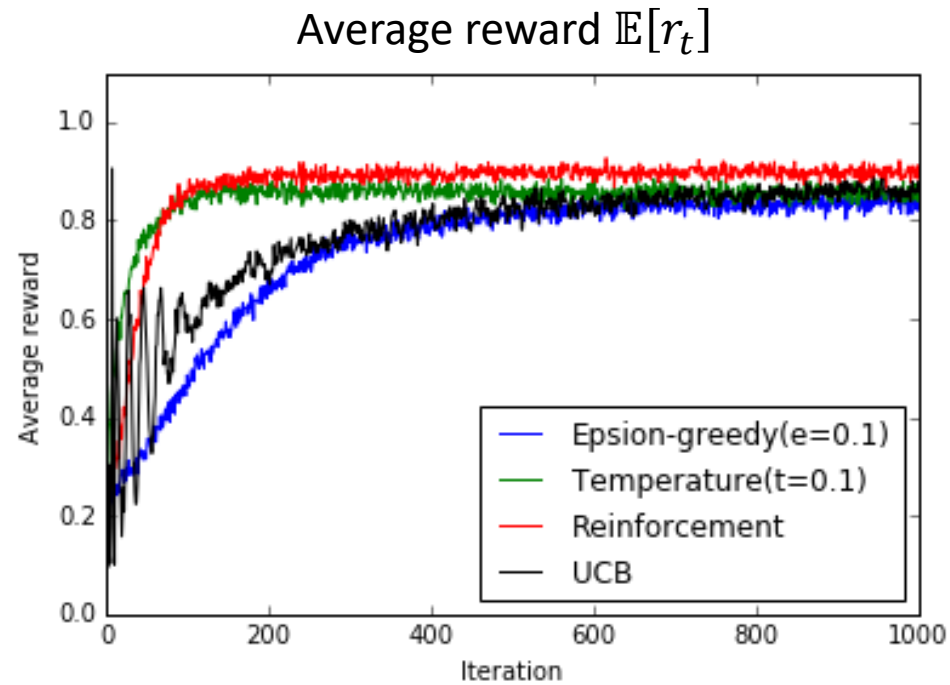
- Use confidence intervals, or some other measure of uncertainty, to encourage exploration of particular actions is sound and appealing

- **Upper Confidence Bound (Cesa-Bianchi and Fisher, 2002)**

$$a_t = \operatorname{argmax}_i \left(\mu_i + \sqrt{\frac{2 \ln t}{n_i}} \right)$$

t : the total counts
 n_i : number of pulling for arm i

Upper Confidence Bound (UCB method)



Bayesian view on Bandit Problem (MDP formulation)

- The posterior of the success probability given w_t winnings and l_t loss:

$$\theta_t | w_t, l_t \sim \text{Beta}(\theta | \alpha = 1 + w_t, \beta = 1 + l_t)$$

- The mean probability of success:

$$\rho_i = \int_0^1 \theta \text{Beta}(\theta | \alpha = 1 + w_i, \beta = 1 + l_i) d\theta = \frac{w_i + 1}{w_i + l_i + 2}$$



w

data

MLE

$$\theta = \frac{1}{1}$$

Bayesian

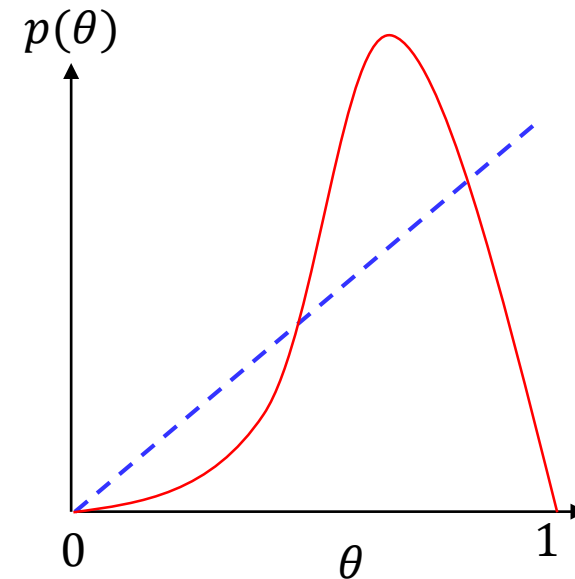
$$\theta | \text{data} \sim \text{Beta}(2, 1)$$



w, w, w, l, w

$$\theta = \frac{4}{5}$$

$$\theta | \text{data} \sim \text{Beta}(5, 2)$$



Bayesian optimization

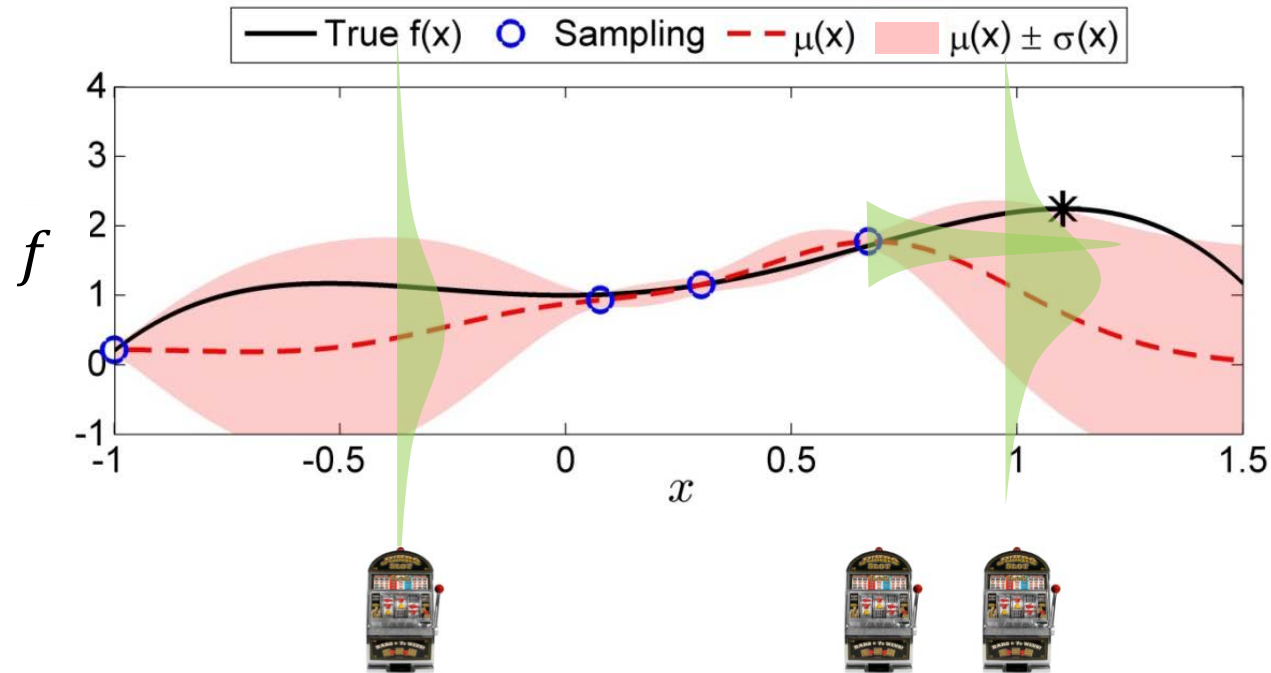
- A Tutorial on Bayesian Optimization of Expensive Cost Functions, with Application to Active User Modeling and Hierarchical Reinforcement Learning (Eric Brochu, Vlad M. Cora and Nando de Freitas)

Motivation

∞ — armed Bandit Problem

How to solve?

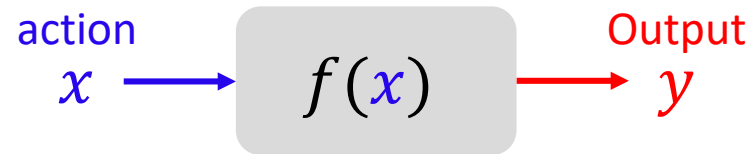
- Learn target function (exploration)
- Improve target value (exploitation)



Which machine should be selected?

Bayesian optimization

- **Bayesian Optimization (BO)** is a method to maximize (or minimize) a target value using measurement data from the **unknown target system**.
- BO is composed of three iterative steps:
 - 1) Learning : construct a probabilistic model function for a target system
 - 2) Optimization : select the next trial input to improve a target
 - 3) Observation : measure the output of a target system



- Policy π maps all the history to new action:

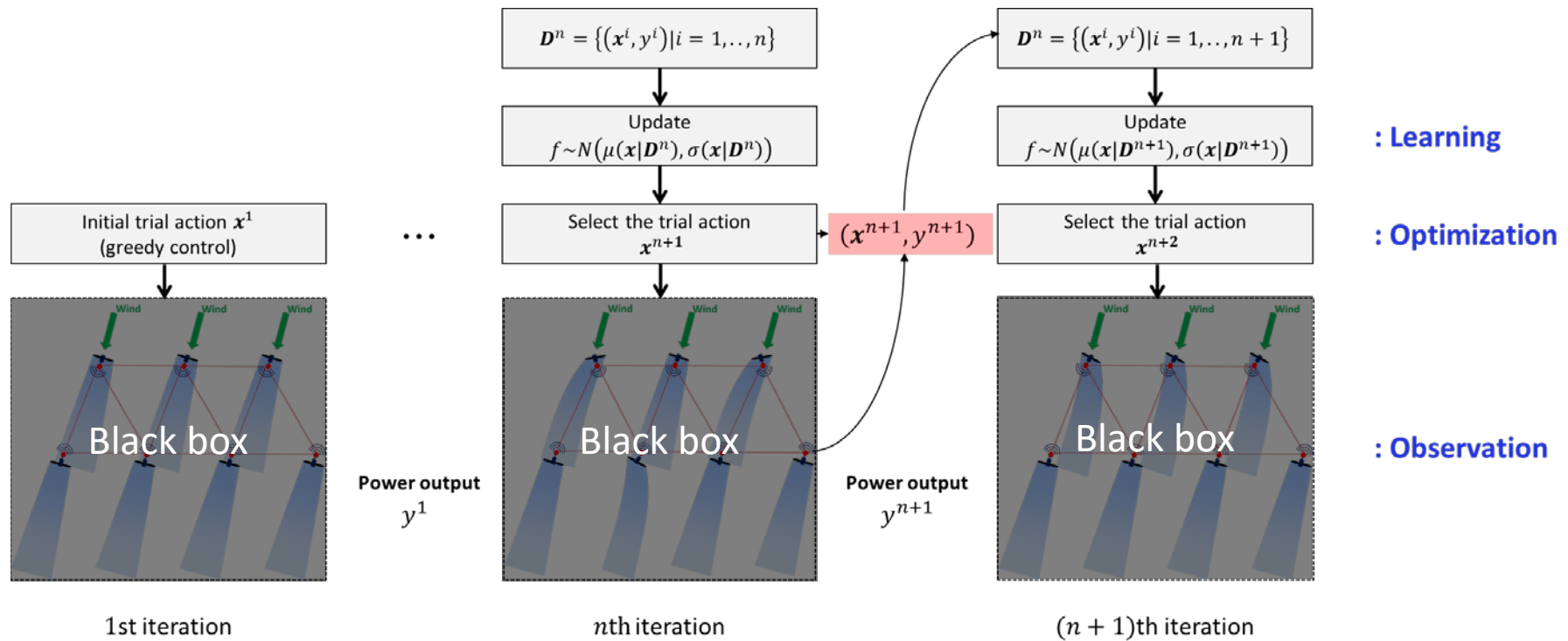
$$\pi: [(x^1, y^1), (x^2, y^2), \dots, (x^{n-1}, y^{n-1})] \rightarrow x^n$$

- Find the optimal policy π^* that maximizes $\mathbb{E}[\sum_{t=1}^T y^t]$ or $\mathbb{E}[y^T]$

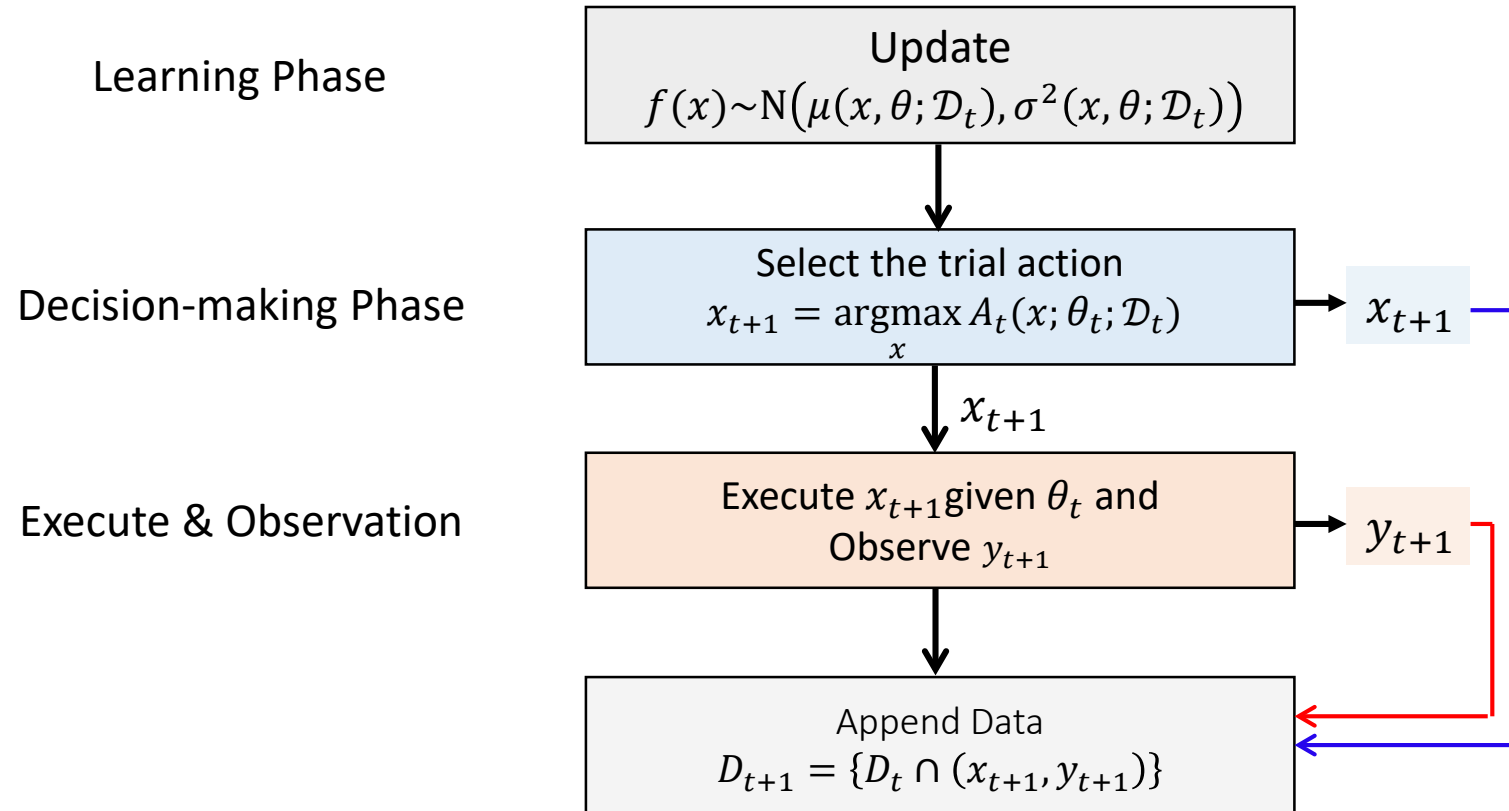
$$x^* = \operatorname{argmax}_x f(x) \equiv \mathbb{E}[y^T]$$

Bayesian optimization

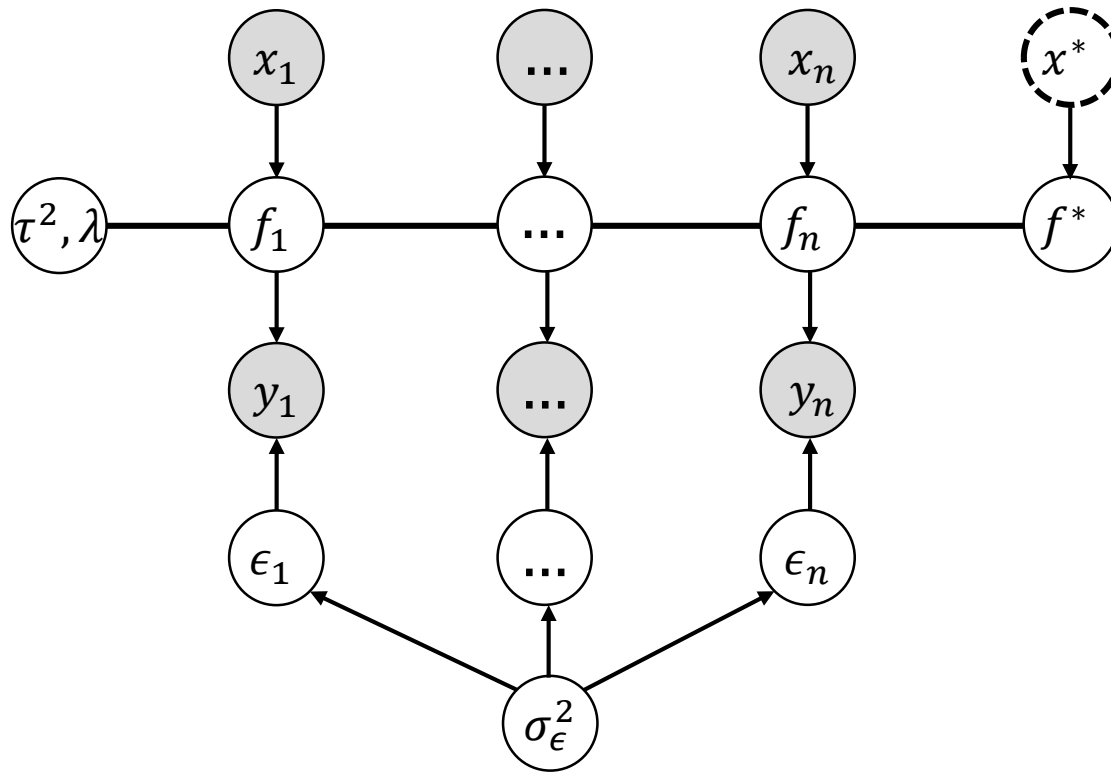
- **Bayesian Optimization (BO)** is a method to maximize (or minimize) a target value using measurement data from the **unknown target system**.



Bayesian optimization process



Gaussian process



Input

$$\mathbf{x}_{1:n} = \{x_1, \dots, x_n\}$$

True function values

$$\mathbf{f}_{1:n} = \{f_1, \dots, f_n\}$$

Observation

$$\mathbf{y}_{1:n} = \{y_1, \dots, y_n\}$$

Noise

$$\boldsymbol{\epsilon} = \{\epsilon_1, \dots, \epsilon_n\}$$

Bayesian optimization

Learning phase : Gaussian Process (GP) regression

Construct the distribution on unknown target value $f = f(x)$ corresponding x

1. Given the data at n -th iteration

Inputs

$$\mathbf{x}_{1:n} = \{x_1, \dots, x_i, \dots, x_n\}$$

Latent function values

$$\mathbf{f}_{1:n} = \{f_1, \dots, f_i, \dots, f_n\}$$

$$y_i = f_i + \epsilon_i$$

Observations

$$\mathbf{y}_{1:n} = \{y_1, \dots, y_i, \dots, y_n\}$$

Bayesian optimization

Learning phase : Gaussian Process (GP) regression

Construct the distribution on unknown target value $f = f(x)$ corresponding x

1. Given the data at n -th iteration

Inputs	Latent function values	Observations
$\mathbf{x}_{1:n} = \{x_1, \dots, x_i, \dots, x_n\}$	$\mathbf{f}_{1:n} = \{f_1, \dots, f_i, \dots, f_n\}$	$\mathbf{y}_{1:n} = \{y_1, \dots, y_i, \dots, y_n\}$

$$y_i = f_i + \epsilon_i$$

2. Prior on the function values $\mathbf{f}^{1:n}$ is represented as Gaussian Process (GP)

$$p(\mathbf{f}_{1:n}) = \mathcal{GP}(m(\cdot), k(\cdot, \cdot)) : \begin{bmatrix} f_1 \\ \vdots \\ f_n \end{bmatrix} \sim \mathcal{N} \left(\begin{bmatrix} m(x_1) \\ \vdots \\ m(x_n) \end{bmatrix}, \begin{bmatrix} k(x_1, x_1) & \cdots & k(x_1, x_n) \\ \vdots & \ddots & \vdots \\ k(x_n, x_1) & \cdots & k(x_n, x_n) \end{bmatrix} \right)$$

$m(\cdot)$: mean function
 $k(\cdot, \cdot)$: kernel function

$$m(\mathbf{x}) = \mathbf{0} \quad k(x, x') = \gamma \exp \left(-\frac{1}{2} (x - x')^T \text{diag}(\boldsymbol{\lambda})^{-2} (x - x') \right)$$

$$\text{cov}(f_i, f_j) = \mathbb{E}[(f_i - m(x_i))(f_j - m(x_j))] \approx k(x_i, x_j)$$

$$\text{Exponential Square kernel function: } k(x_i, x_j) = \gamma \exp \left(-\frac{1}{2} (x_i - x_j)^T \text{diag}(\boldsymbol{\lambda})^{-2} (x_i - x_j) \right)$$

Kernel function

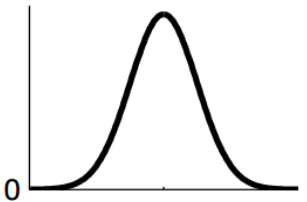
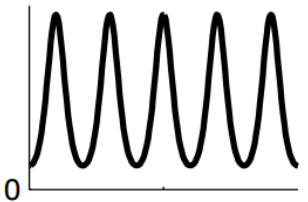
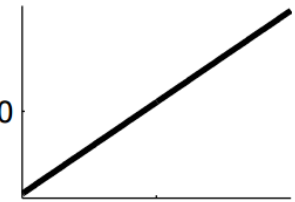
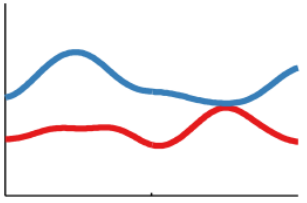
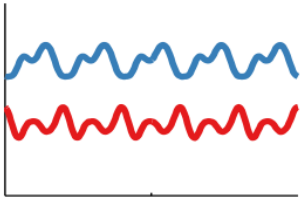
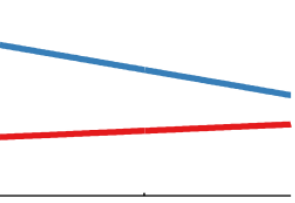
Kernel name:	Squared-exp (SE)	Periodic (Per)	Linear (Lin)
$k(x, x') =$	$\sigma_f^2 \exp\left(-\frac{(x-x')^2}{2\ell^2}\right)$	$\sigma_f^2 \exp\left(-\frac{2}{\ell^2} \sin^2\left(\pi \frac{x-x'}{p}\right)\right)$	$\sigma_f^2 (x - c)(x' - c)$
Plot of $k(x, x')$:			
	$x - x'$ ↓	$x - x'$ ↓	x (with $x' = 1$) ↓
Functions $f(x)$ sampled from GP prior:			
Type of structure:	local variation	repeating structure	linear functions

Figure from <http://www.cs.toronto.edu/~duvenaud/>

Bayesian optimization

Learning phase : Gaussian Process (GP) regression

Construct the distribution on unknown target value $f = f(x)$ corresponding x

1. Given the data at n -th iteration

Inputs	Latent function values	Observations
$\mathbf{x}_{1:n} = \{x_1, \dots, x_i, \dots, x_n\}$	$\mathbf{f}_{1:n} = \{f_1, \dots, f_i, \dots, f_n\}$	$\mathbf{y}_{1:n} = \{y_1, \dots, y_i, \dots, y_n\}$

$$y_i = f_i + \epsilon_i$$

2. Prior on the function values $\mathbf{f}^{1:n}$ is represented as Gaussian Process (GP)

$$p(\mathbf{f}_{1:n}) = \mathcal{GP}(m(\cdot), k(\cdot, \cdot)) : \begin{bmatrix} f_1 \\ \vdots \\ f_n \end{bmatrix} \sim \mathcal{N} \left(\begin{bmatrix} m(x_1) \\ \vdots \\ m(x_n) \end{bmatrix}, \begin{bmatrix} k(x_1, x_1) & \dots & k(x_1, x_n) \\ \vdots & \ddots & \vdots \\ k(x_n, x_1) & \dots & k(x_n, x_n) \end{bmatrix} \right)$$

$m(\cdot)$: mean function
 $k(\cdot, \cdot)$: kernel function

$$m(\mathbf{x}) = \mathbf{0} \quad k(x, x') = \gamma \exp \left(-\frac{1}{2} (x - x')^T \text{diag}(\boldsymbol{\lambda})^{-2} (x - x') \right)$$

3. Likelihood is constructed base the assumption on the noise, i.e., i.i.d. Gaussian noise

$$p(\mathbf{y}_{1:n} | \mathbf{f}_{1:n}) = \mathcal{N}(\mathbf{f}_{1:n}, \sigma_\epsilon^2 \mathbf{I})$$

The hyper-parameters $\boldsymbol{\theta} = (\sigma_\epsilon, \sigma_s, \boldsymbol{\lambda})$ for the noise model and the kernel function are determined as ones maximizing the marginal log-likelihood of the training data $\mathcal{D} = \{(\mathbf{x}^i, y^i) | i = 1, \dots, n\}$ as

$$\begin{aligned} \boldsymbol{\theta}^* &= \underset{\boldsymbol{\theta}}{\operatorname{argmax}} \log p(\mathbf{y}_{1:n} | \boldsymbol{\theta}) \\ &= \underset{\boldsymbol{\theta}}{\operatorname{argmax}} \left(-\frac{1}{2} (\mathbf{y}_{1:n})^T (\mathbf{K} + \sigma_\epsilon^2 \mathbf{I})^{-1} \mathbf{y}_{1:n} - \frac{1}{2} \log |\mathbf{K} + \sigma_\epsilon^2 \mathbf{I}| - \frac{n}{2} \log 2\pi \right) \end{aligned}$$

Bayesian optimization

Learning phase : Gaussian Process (GP) regression

Construct the distribution on unknown target value $f = f(x)$ corresponding x

1. Given the data at n -th iteration

Inputs	Latent function values	Observations
$\mathbf{x}_{1:n} = \{x_1, \dots, x_i, \dots, x_n\}$	$\mathbf{f}_{1:n} = \{f_1, \dots, f_i, \dots, f_n\}$	$\mathbf{y}_{1:n} = \{y_1, \dots, y_i, \dots, y_n\}$

$$y_i = f_i + \epsilon_i$$

2. Prior on the function values $\mathbf{f}^{1:n}$ is represented as Gaussian Process (GP)

$$p(\mathbf{f}_{1:n}) = \mathcal{GP}(m(\cdot), k(\cdot, \cdot)) : \begin{bmatrix} f_1 \\ \vdots \\ f_n \end{bmatrix} \sim N \left(\begin{bmatrix} m(x_1) \\ \vdots \\ m(x_n) \end{bmatrix}, \begin{bmatrix} k(x_1, x_1) & \dots & k(x_1, x_n) \\ \vdots & \ddots & \vdots \\ k(x_n, x_1) & \dots & k(x_n, x_n) \end{bmatrix} \right)$$

$m(\cdot)$: mean function
 $k(\cdot, \cdot)$: kernel function

$$m(\mathbf{x}) = \mathbf{0} \quad k(x, x') = \gamma \exp \left(-\frac{1}{2} (x - x')^T \text{diag}(\boldsymbol{\lambda})^{-2} (x - x') \right)$$

3. Likelihood is constructed base the assumption on the noise, i.e., i.i.d. Gaussian noise

$$p(\mathbf{y}_{1:n} | \mathbf{f}_{1:n}) = N(\mathbf{f}_{1:n}, \sigma_\epsilon^2 \mathbf{I})$$

4. Joint distribution based on Bayes' rule :

$$p(f, \mathbf{y}_{1:n}) = \int p(f, \mathbf{f}_{1:n}) p(\mathbf{y}_{1:n} | \mathbf{f}_{1:n}) d\mathbf{f}_{1:n} \quad \rightarrow \quad \begin{bmatrix} \mathbf{y}_{1:n} \\ f \end{bmatrix} \sim N \left(\mathbf{0}, \begin{bmatrix} \mathbf{K} + \sigma_\epsilon^2 \mathbf{I} & \mathbf{k} \\ \mathbf{k}^T & k(\mathbf{x}, \mathbf{x}) \end{bmatrix} \right)$$

Bayesian optimization

Learning phase : Gaussian Process (GP) regression

Construct the distribution on unknown target value $f = f(x)$ corresponding x

Property 4.

Conditional of a Gaussian random variables are Gaussians, more specifically, if

$$Z = \begin{bmatrix} Y_1 \\ Y_2 \end{bmatrix} \sim N \left(\begin{bmatrix} \mu_1 \\ \mu_2 \end{bmatrix}, \begin{bmatrix} \Sigma_{11} & \Sigma_{12} \\ \Sigma_{21} & \Sigma_{22} \end{bmatrix} \right)$$

where Y_1 is k -dim random variable and Y_2 is an $n - k$ dim random variable, then

$$\{Y_2 | Y_1 = y\} \sim N(\Sigma_{21}\Sigma_{11}^{-1}(y - \mu_1) + \mu_2, \Sigma_{22} - \Sigma_{21}\Sigma_{11}^{-1}\Sigma_{12})$$

4. Joint distribution based on Bayes' rule :

$$p(f, \mathbf{y}_{1:n}) = \int p(f, \mathbf{f}_{1:n}) p(\mathbf{y}_{1:n} | \mathbf{f}_{1:n}) d\mathbf{f}_{1:n} \quad \rightarrow \quad \begin{bmatrix} \mathbf{y}_{1:n} \\ f \end{bmatrix} \sim N \left(\mathbf{0}, \begin{bmatrix} \mathbf{K} + \sigma_\epsilon^2 \mathbf{I} & \mathbf{k} \\ \mathbf{k}^T & k(x, x) \end{bmatrix} \right)$$

5. Conditionalization $\rightarrow$ Posterior distribution on the function value $f = f(x)$ for x given $\mathcal{D} = \{\mathbf{x}_{1:n}, \mathbf{y}_{1:n}\}$

$$p(f | \mathcal{D}) = N(\mu(x | \mathcal{D}), \sigma^2(x | \mathcal{D}))$$

Mean : $\mu(x | \mathcal{D}) = \mathbf{k}^T (\mathbf{K} + \sigma_\epsilon^2 \mathbf{I})^{-1} \mathbf{y}_{1:n}$

Variance : $\sigma^2(x | \mathcal{D}) = k(x, x) - \mathbf{k}^T (\mathbf{K} + \sigma_\epsilon^2 \mathbf{I})^{-1} \mathbf{k}$

Bayesian optimization

Learning phase : Gaussian Process (GP) regression

Construct the distribution on unknown target value $f = f(x)$ corresponding x

1. Given the data at n -th iteration

Inputs	Latent function values	Observations
$\mathbf{x}_{1:n} = \{x_1, \dots, x_i, \dots, x_n\}$	$\mathbf{f}_{1:n} = \{f_1, \dots, f_i, \dots, f_n\}$	$\mathbf{y}_{1:n} = \{y_1, \dots, y_i, \dots, y_n\}$

$y_i = f_i + \epsilon_i$

2. Prior on the function values $\mathbf{f}^{1:n}$ is represented as Gaussian Process (GP)

$$p(\mathbf{f}_{1:n}) = \mathcal{GP}(m(\cdot), k(\cdot, \cdot)) : \begin{bmatrix} f_1 \\ \vdots \\ f_n \end{bmatrix} \sim N \left(\begin{bmatrix} m(x_1) \\ \vdots \\ m(x_n) \end{bmatrix}, \begin{bmatrix} k(x_1, x_1) & \dots & k(x_1, x_n) \\ \vdots & \ddots & \vdots \\ k(x_n, x_1) & \dots & k(x_n, x_n) \end{bmatrix} \right)$$

$m(\cdot)$: mean function
 $k(\cdot, \cdot)$: kernel function

$$m(\mathbf{x}) = \mathbf{0} \quad k(x, x') = \gamma \exp \left(-\frac{1}{2} (x - x')^T \text{diag}(\boldsymbol{\lambda})^{-2} (x - x') \right)$$

3. Likelihood is constructed base the assumption on the noise, i.e., i.i.d. Gaussian noise

$$p(\mathbf{y}_{1:n} | \mathbf{f}_{1:n}) = N(\mathbf{f}_{1:n}, \sigma_\epsilon^2 \mathbf{I})$$

4. Joint distribution based on Bayes' rule :

$$p(f, \mathbf{y}_{1:n}) = \int p(f, \mathbf{f}_{1:n}) p(\mathbf{y}_{1:n} | \mathbf{f}_{1:n}) d\mathbf{f}_{1:n} \quad \Rightarrow \quad \begin{bmatrix} \mathbf{y}_{1:n} \\ f \end{bmatrix} \sim N \left(\mathbf{0}, \begin{bmatrix} \mathbf{K} + \sigma_\epsilon^2 \mathbf{I} & \mathbf{k} \\ \mathbf{k}^T & k(x, x) \end{bmatrix} \right)$$

5. Conditionalization $\rightarrow$ Posterior distribution on the function value $f = f(x)$ for x given $\mathcal{D} = \{\mathbf{x}_{1:n}, \mathbf{y}_{1:n}\}$

$$p(f | \mathcal{D}) = N(\mu(x | \mathcal{D}), \sigma^2(x | \mathcal{D}))$$

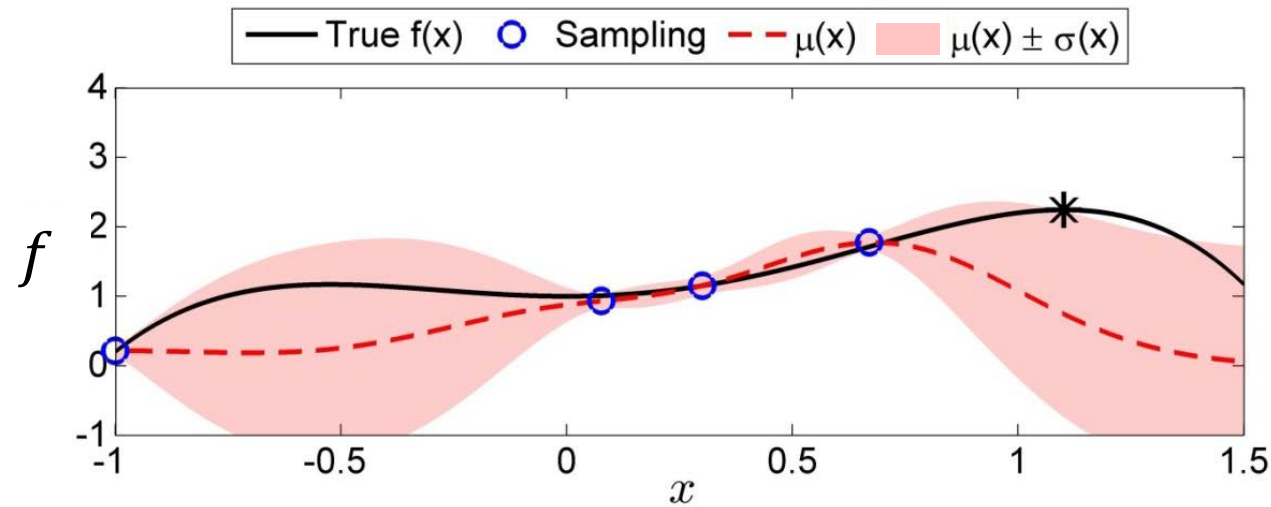
Mean : $\mu(x | \mathcal{D}) = \mathbf{k}^T (\mathbf{K} + \sigma_\epsilon^2 \mathbf{I})^{-1} \mathbf{y}_{1:n}$
Variance : $\sigma^2(x | \mathcal{D}) = k(x, x) - \mathbf{k}^T (\mathbf{K} + \sigma_\epsilon^2 \mathbf{I})^{-1} \mathbf{k}$

Bayesian optimization

Learning phase : Gaussian Process (GP) regression

Construct the distribution on unknown target value $f = f(x)$ corresponding x

$$p(f|\mathcal{D}) = \mathcal{N}(\mu(x|\mathcal{D}), \sigma^2(x|\mathcal{D}))$$

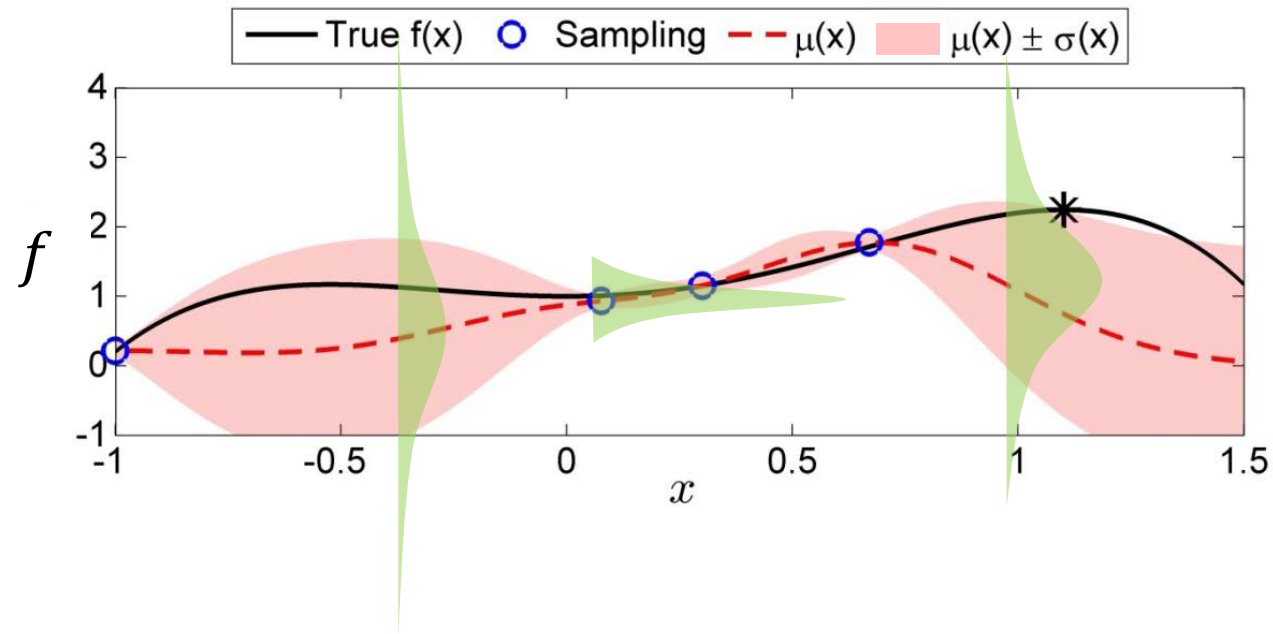


Bayesian optimization

Learning phase : Gaussian Process (GP) regression

Construct the distribution on unknown target value $f = f(x)$ corresponding x

$$p(f|\mathcal{D}) = \mathcal{N}(\mu(x|\mathcal{D}), \sigma^2(x|\mathcal{D}))$$

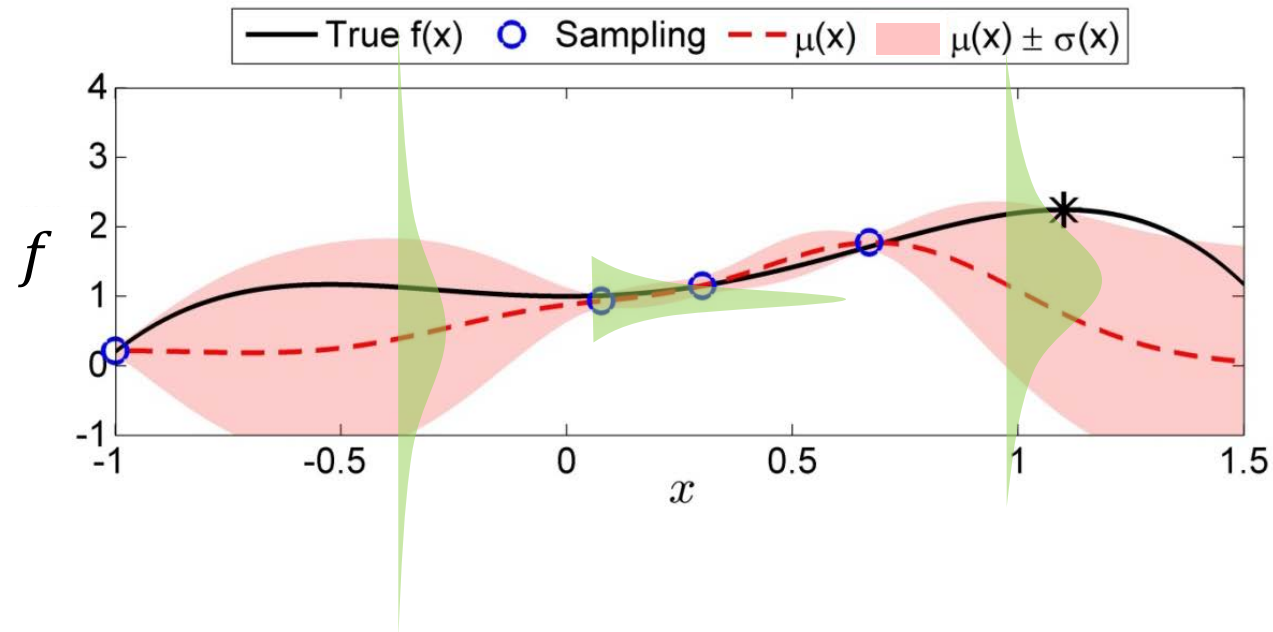


Bayesian optimization

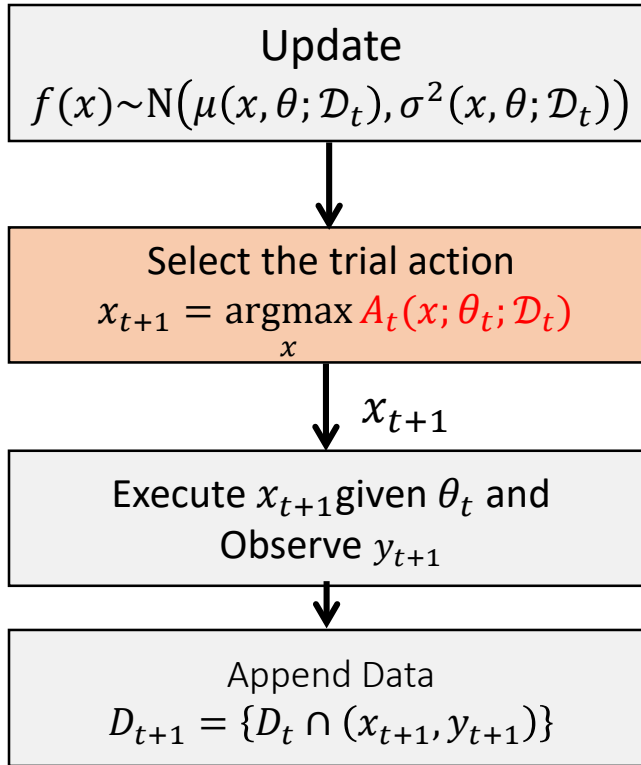
Optimization (Sampling) phase

How to select the next input?

- Learn target function (exploration)
- Improve target value (exploitation)



Acquisition function



- The role of the acquisition function $A_t(x; \theta_t; \mathcal{D}_t)$ is to guide the search for the optimum.
- Typically, acquisition functions $A_t(x; \theta_t; \mathcal{D}_t)$ are defined such that high acquisition corresponds to potentially high values of the objective function, whether because
 - the prediction is high (Exploitation)
 - the uncertainty is great (Exploration)
 - both

Acquisition function types

- Probability Improvement
- Expected Improvement
- Upper Confidence Bound
- Information Theoretic Approaches
- Sampling Based Approaches

Acquisition function: Probability Improvement (PI)

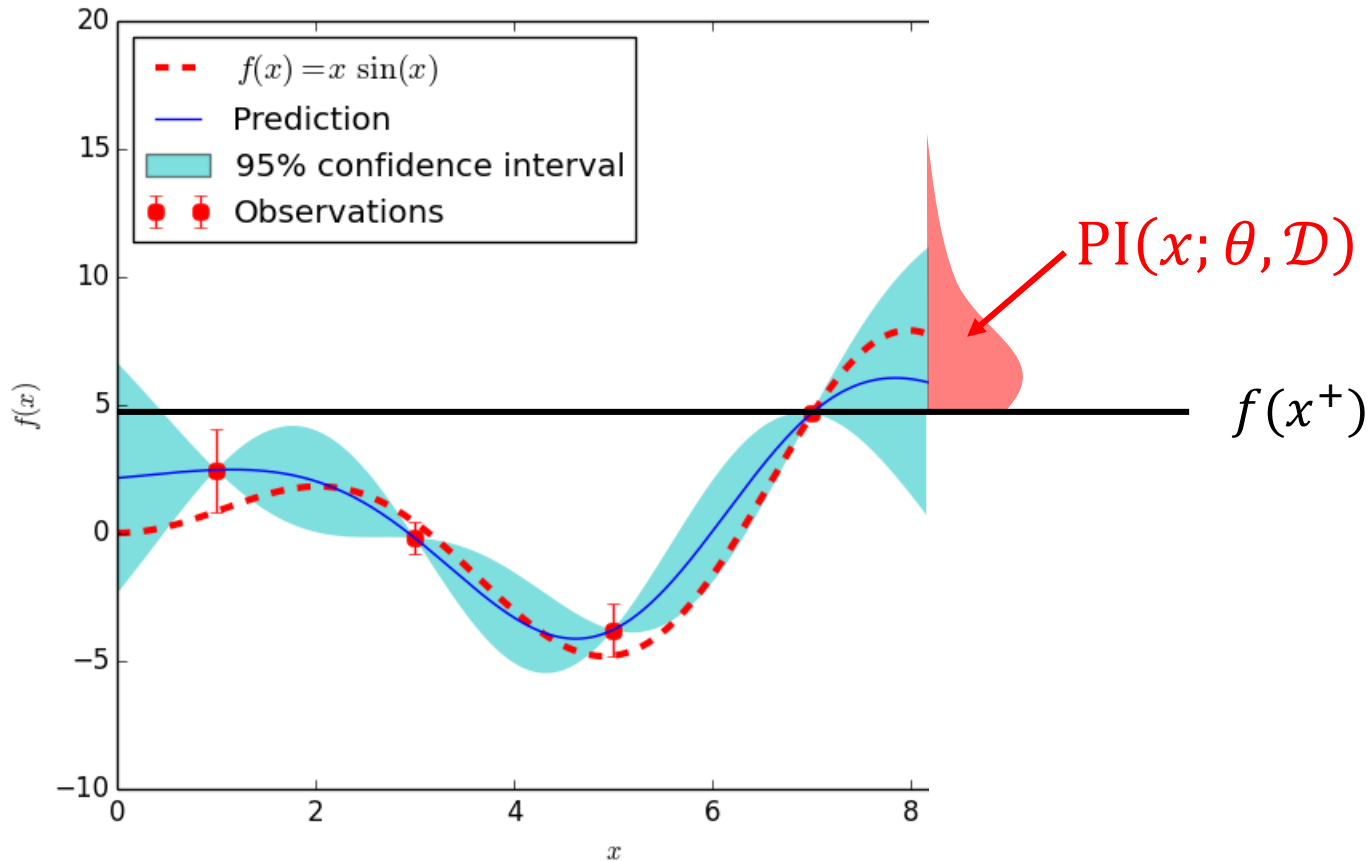
- **Probability of improvement (PI)** is defined as

$$\begin{aligned}\text{PI}(x; \theta, \mathcal{D}) &= p(f(x) \geq f(x^+)) \\ &= \Phi\left(\frac{\mu(x; \theta, \mathcal{D}) - f(x^+)}{\sigma(x; \theta, \mathcal{D})}\right)\end{aligned}$$

- $f(x^+)$ with $x^+ = \max_{x_i \in x_{1:t}} f(x_i)$ is the current estimated maximum
- $f(x) \sim \mathcal{N}(\mu(x; \theta, \mathcal{D}), \sigma(x; \theta, \mathcal{D})^2)$
- $\Phi(\cdot)$ is the normal cumulative distribution function

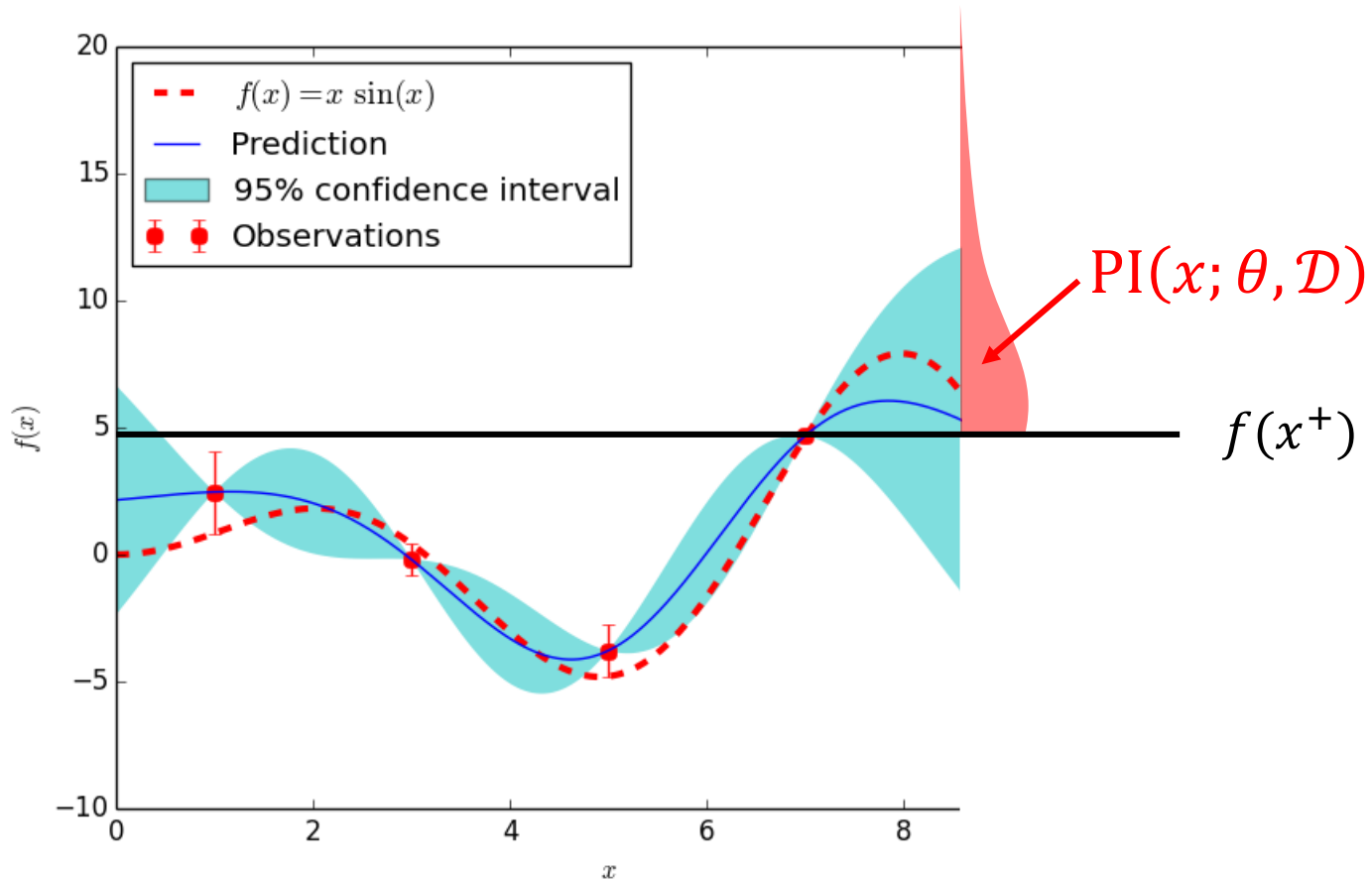
Acquisition function: Probability Improvement (PI)

- How to compute $\text{PI}(x; \theta, \mathcal{D})$?



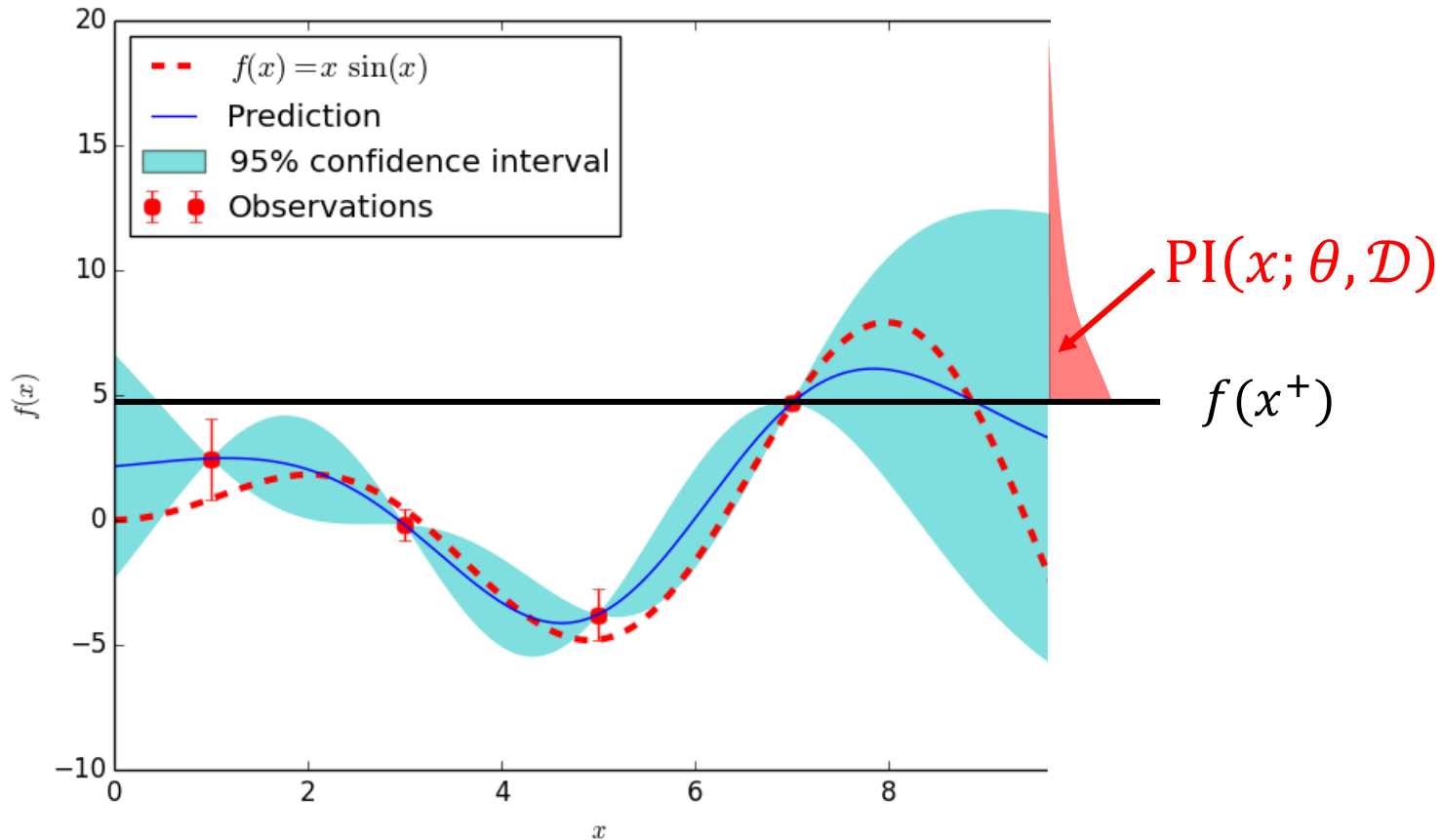
Acquisition function: Probability Improvement (PI)

- How to compute $\text{PI}(x; \theta, \mathcal{D})$?



Acquisition function: Probability Improvement (PI)

- How to compute $\text{PI}(x; \theta, \mathcal{D})$?



Acquisition function: Probability Improvement (PI)

- **Negative aspects of PI**

- The formulation is pure exploitation

- Points that have a high probability of being infinitesimally greater than $f(x^+)$ will be drawn over points that offer larger gains but less certainty.
 - A modification is to add **a trade-off parameter $\xi \geq 0$** :

$$\begin{aligned} \text{PI}(x; \theta, \mathcal{D}) &= p(f(x) \geq f(x^+) + \xi) \\ &= \Phi \left(\frac{\mu(x; \theta, \mathcal{D}) - f(x^+) - \xi}{\sigma(x; \theta, \mathcal{D})} \right) \end{aligned}$$

- The exact choice of ξ is left to the user
 - It is recommended to schedule ξ ,
 - ✓ Large in the early phase to promote exploration
 - ✓ Slowly decreases to zero to settle down on the observed maximum
 - PI is extremely sensitive to the choice of the target.
 - ✓ If the desired improvement is too small, the search will be highly local
 - ✓ On the other hand, if ξ is set too high, the search will be excessively global

Acquisition function: Expected Improvement (EI)

- Improvement function is defined as

$$I(x; \theta, \mathcal{D}) = \max\{0, f(x) - f(x^+)\}$$

- ✓ $I(x; \theta, \mathcal{D})$ is positive when the prediction is higher than the best value known thus far. Otherwise, $I(x; \theta, \mathcal{D})$ is set to zero.

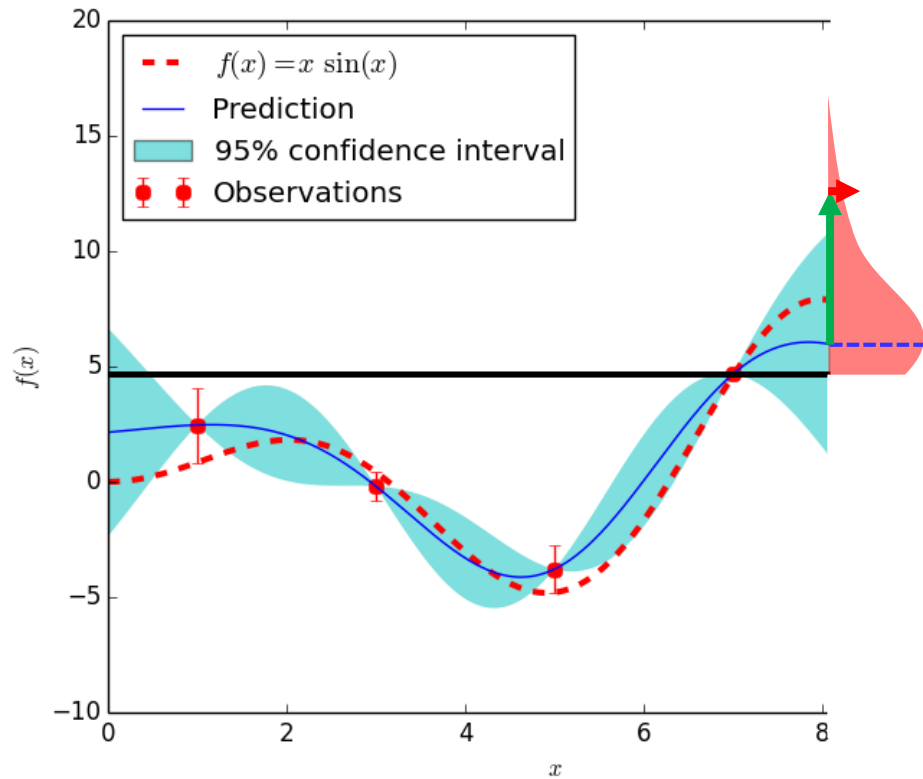
- The expected improvement (EI):

$$\begin{aligned} \text{EI}(x; \theta, \mathcal{D}) &= \mathbb{E}[\max\{0, f(x) - f(x^+)\} | \mathcal{D}] \\ &= \int_{f(x)=f(x^+)}^{f(x)=\infty} (f(x) - f(x^+)) p(f(x) | \mathcal{D}) \\ &= \int_{I=0}^{I=\infty} I \frac{1}{\sqrt{2\pi}\sigma(x)} \exp\left(-\frac{(\mu(x) - f(x^+) - I)^2}{2\sigma^2(x)}\right) dI \\ &= \sigma(x) \left[\frac{\mu(x) - f(x^+)}{\sigma(x)} \Phi\left(\frac{\mu(x) - f(x^+)}{\sigma(x)}\right) + \phi\left(\frac{\mu(x) - f(x^+)}{\sigma(x)}\right) \right] \end{aligned}$$

- ✓ Φ is the cumulative distribution function (CDF) for standard normal R.V.
- ✓ ϕ is the probability distribution function (PDF) for standard normal R.V.

Acquisition function: Expected Improvement (EI)

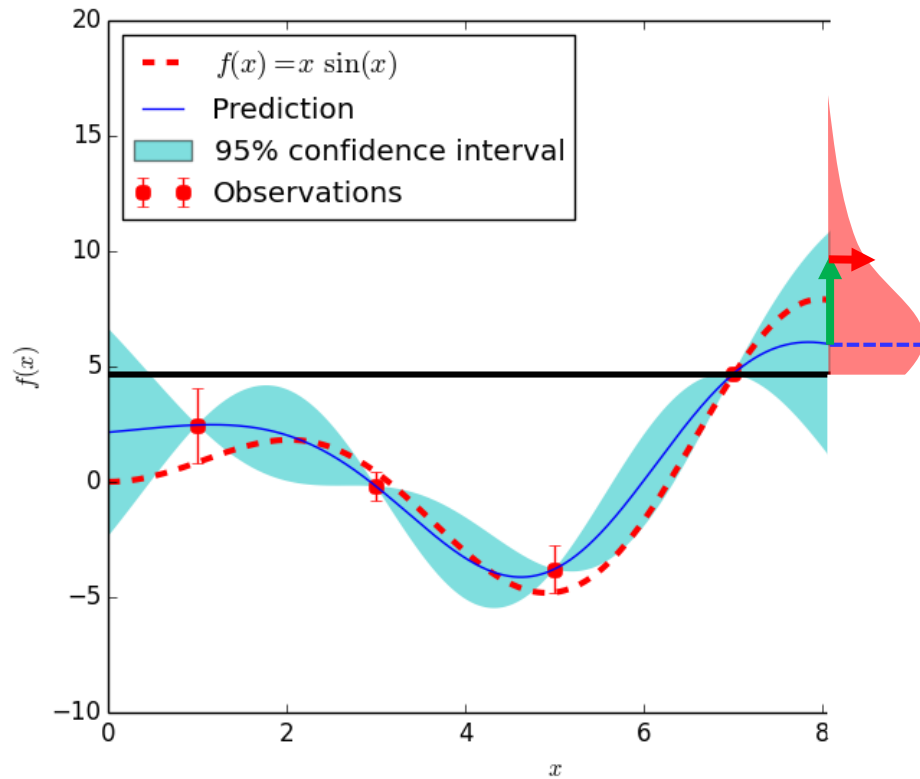
- How to compute $\mathbb{E}[\max\{0, f(x) - f(x^+)\} | \mathcal{D}]$?



$$(f(x) - f(x^+))p(f(x)|\mathcal{D})$$

Acquisition function: Expected Improvement (EI)

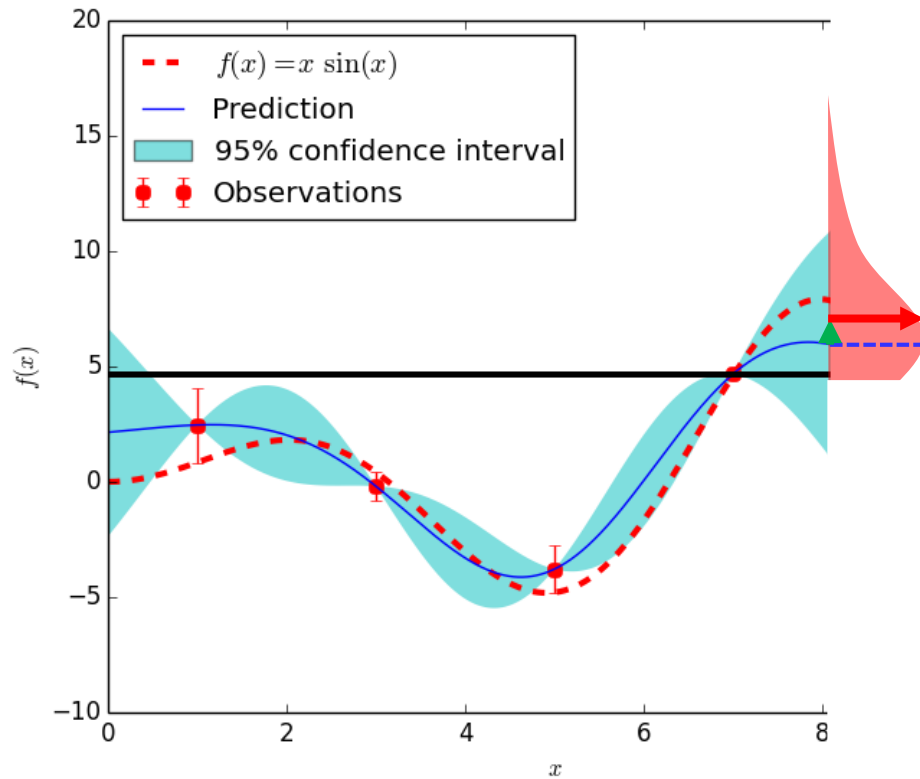
- How to compute $\mathbb{E}[\max\{0, f(x) - f(x^+)\} | \mathcal{D}]$?



$$(f(x) - f(x^+))p(f(x)|\mathcal{D})$$
$$(f(x) - f(x^+))p(f(x)|\mathcal{D})$$

Acquisition function: Expected Improvement (EI)

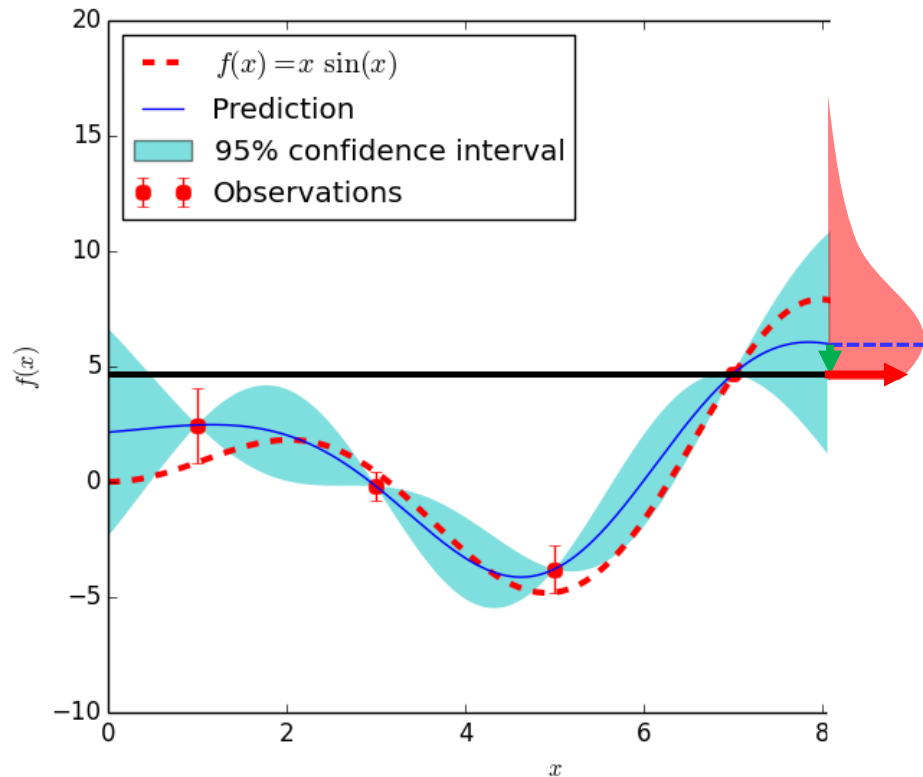
- How to compute $\mathbb{E}[\max\{0, f(x) - f(x^+)\} | \mathcal{D}]$?



$$\begin{aligned} & (f(x) - f(x^+))p(f(x)|\mathcal{D}) \\ & (f(x) - f(x^+))p(f(x)|\mathcal{D}) \\ & (f(x) - f(x^+))p(f(x)|\mathcal{D}) \end{aligned}$$

Acquisition function: Expected Improvement (EI)

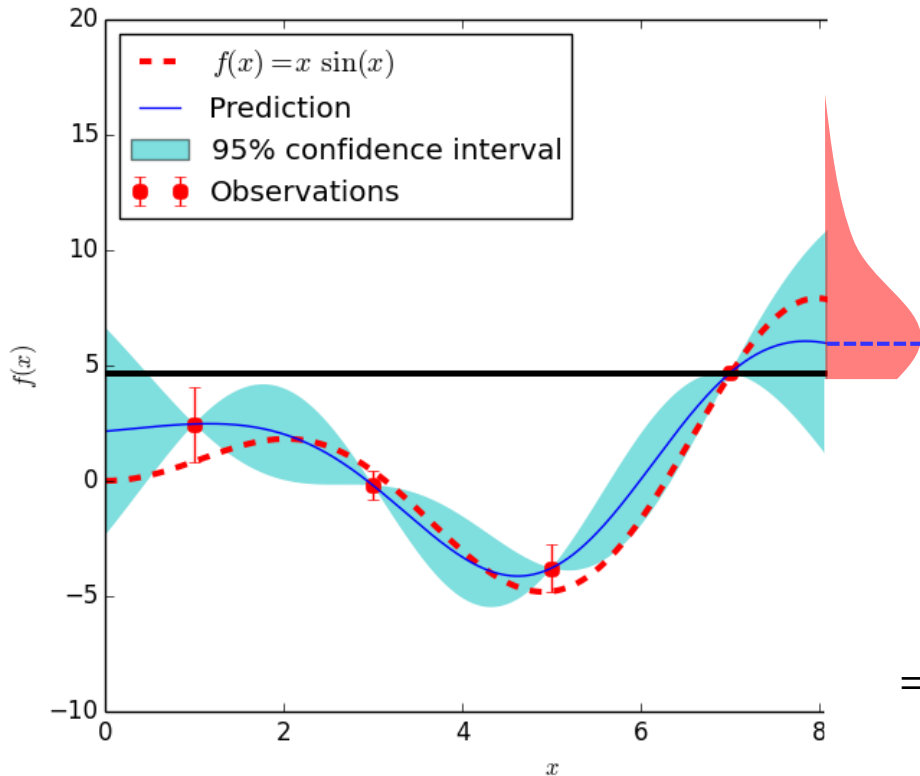
- How to compute $\mathbb{E}[\max\{0, f(x) - f(x^+)\} | \mathcal{D}]$?



$$\begin{aligned} & (f(x) - f(x^+))p(f(x)|\mathcal{D}) \\ & (f(x) - f(x^+))p(f(x)|\mathcal{D}) \\ & (f(x) - f(x^+))p(f(x)|\mathcal{D}) \\ & (f(x) - f(x^+))p(f(x)|\mathcal{D}) \end{aligned}$$

Acquisition function: Expected Improvement (EI)

- How to compute $\mathbb{E}[\max\{0, f(x) - f(x^+)\} | \mathcal{D}]$?



$$\begin{aligned} & (f(x) - f(x^+))p(f(x)|\mathcal{D}) + \\ & (f(x) - f(x^+))p(f(x)|\mathcal{D}) + \\ & (f(x) - f(x^+))p(f(x)|\mathcal{D}) + \\ & (f(x) - f(x^+))p(f(x)|\mathcal{D}) \end{aligned}$$

||

$$\int_{f(x)=f^+}^{f(x)=\infty} (f(x) - f(x^+))p(f(x)|\mathcal{D})$$

$$= \int_{I=0}^{I=\infty} I \frac{1}{\sqrt{2\pi}\sigma(x)} \exp\left(-\frac{(\mu(x) - f(x^+) - I)^2}{2\sigma^2(x)}\right) dI$$

$$= \sigma(x) \left[\frac{\mu(x) - f(x^+)}{\sigma(x)} \Phi\left(\frac{\mu(x) - f(x^+)}{\sigma(x)}\right) + \phi\left(\frac{\mu(x) - f(x^+)}{\sigma(x)}\right) \right]$$

- ✓ Φ is the cumulative distribution function (CDF) for standard normal R.V.
- ✓ ϕ is the probability distribution function (PDF) for standard normal R.V.

Acquisition function: Expected Improvement (EI)

- EI enables us to balance the trade-off of exploiting and exploring.
 - When exploring, we should choose points where the surrogate variance is large.
 - When exploiting, we should choose points where the surrogate mean is high.
- It is highly desirable for our purposes to express EI in a generalized form which controls the trade-off between global search and local optimization (exploration/exploitation).
- Can use additional parameter ξ to further balance between exploration/exploitation

$$\text{EI}(x; \theta, \mathcal{D}) = \begin{cases} (\mu(x) - f(x^+) - \xi)\Phi(Z) + \sigma(x)\phi(Z) & \text{if } \sigma(x) > 0 \\ 0 & \text{if } \sigma(x) = 0 \end{cases}$$

where

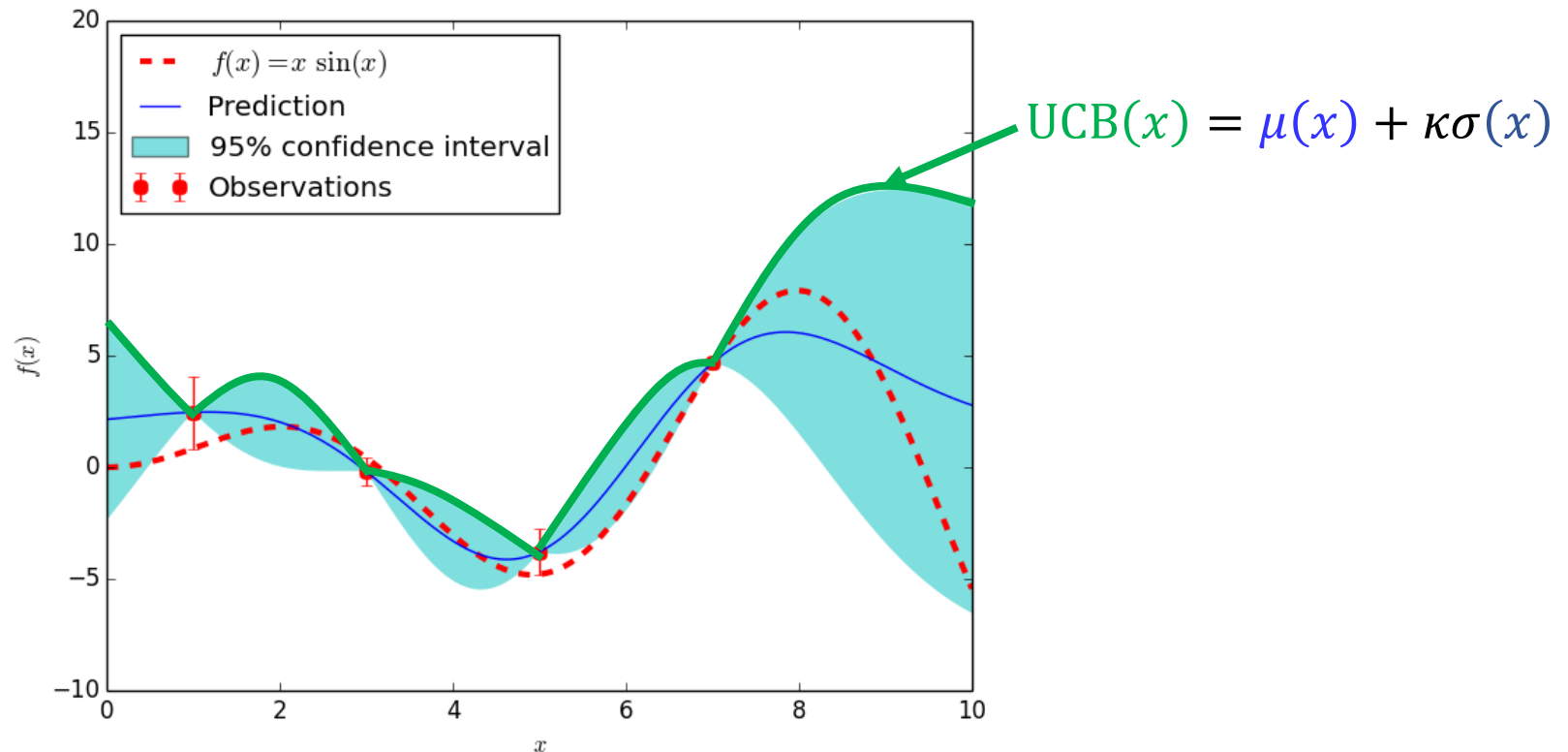
$$Z = \begin{cases} \frac{\mu(x) - f(x^+) - \xi}{\sigma(x)} & \text{if } \sigma(x) > 0 \\ 0 & \text{if } \sigma(x) = 0 \end{cases}$$

Acquisition function: Upper Confidence Bound (UCB)

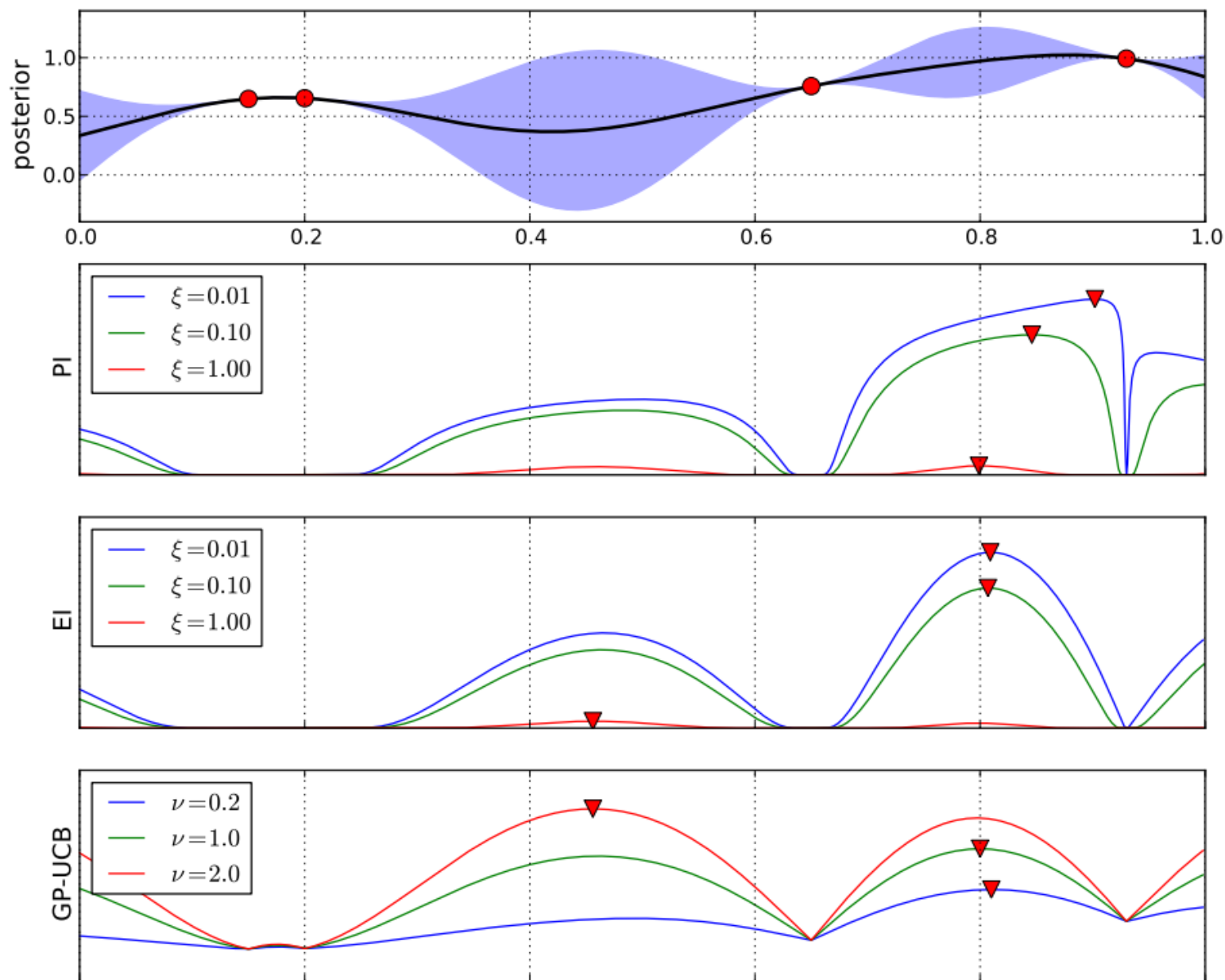
- The **upper confidence bound (UCB)** is defined

$$\text{UCB}(x) = \mu(x) + \kappa\sigma(x)$$

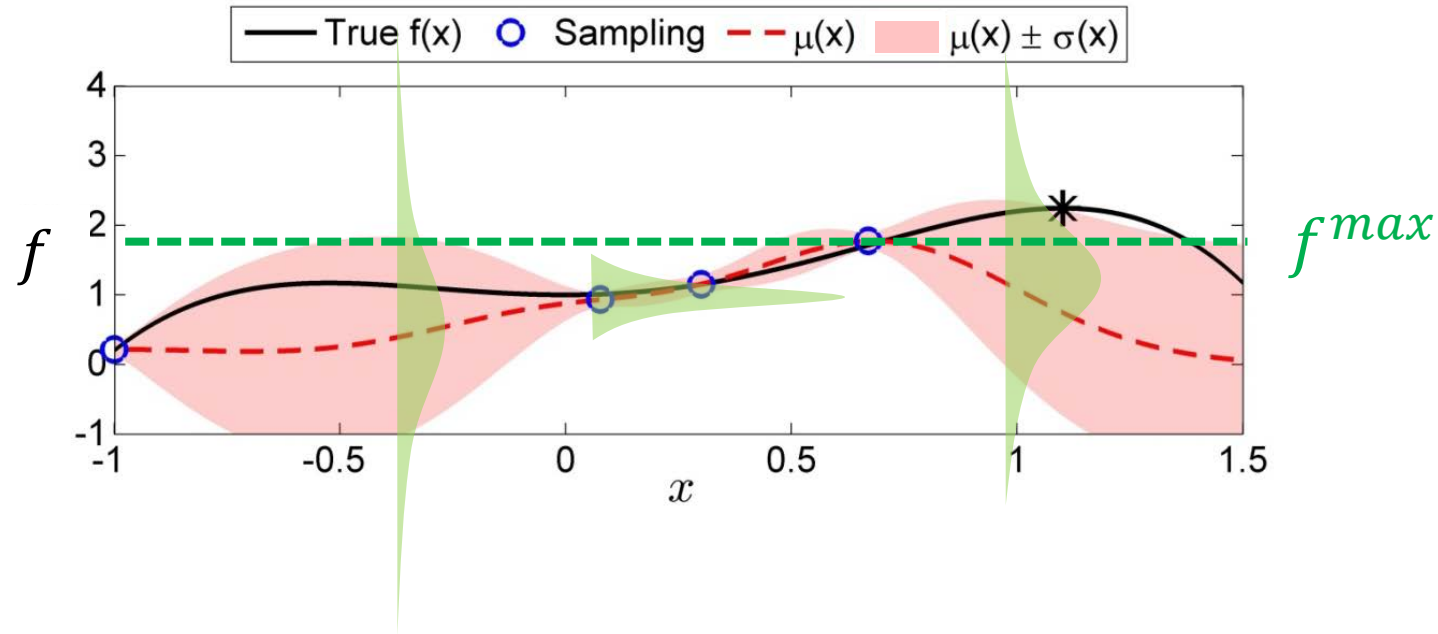
- ✓ By maximizing $\mu(x)$ in UCB, one can exploit the surrogate function
- ✓ By maximizing $\sigma(x)$ in UCB, one can explore the surrogate function
- ✓ Thus, the parameter κ control between the exploration and exploitation



Acquisition function comparison



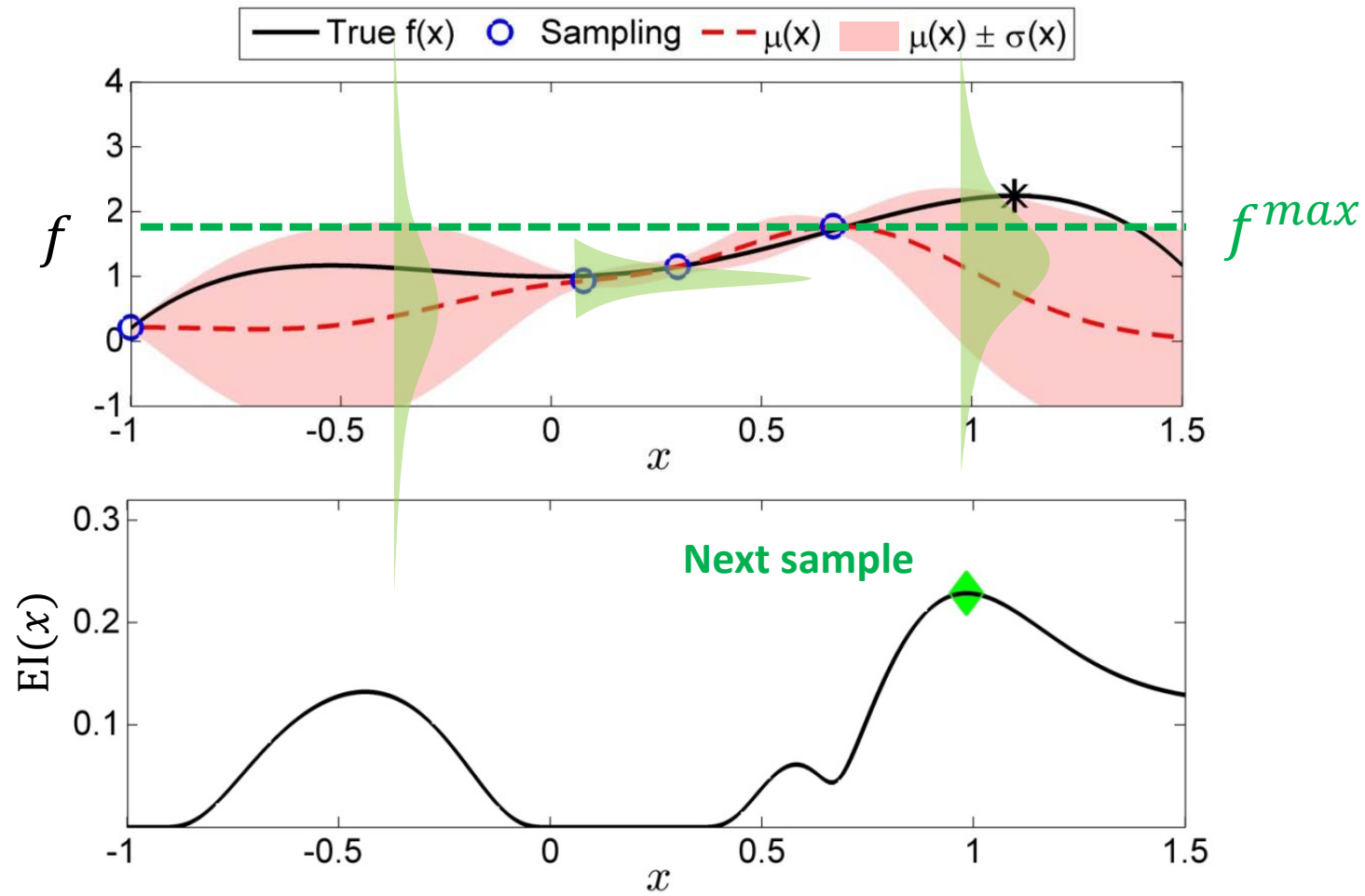
Bayesian optimization with EI function example



- Next sampling point $x^{(n+1)}$ is determined by solving

$$x^{(n+1)} = \operatorname{argmax}_x \operatorname{EI}(x) \triangleq \operatorname{E}[\max\{0, f - f^{max}\} | \mathcal{D}]$$

Bayesian optimization with EI function example

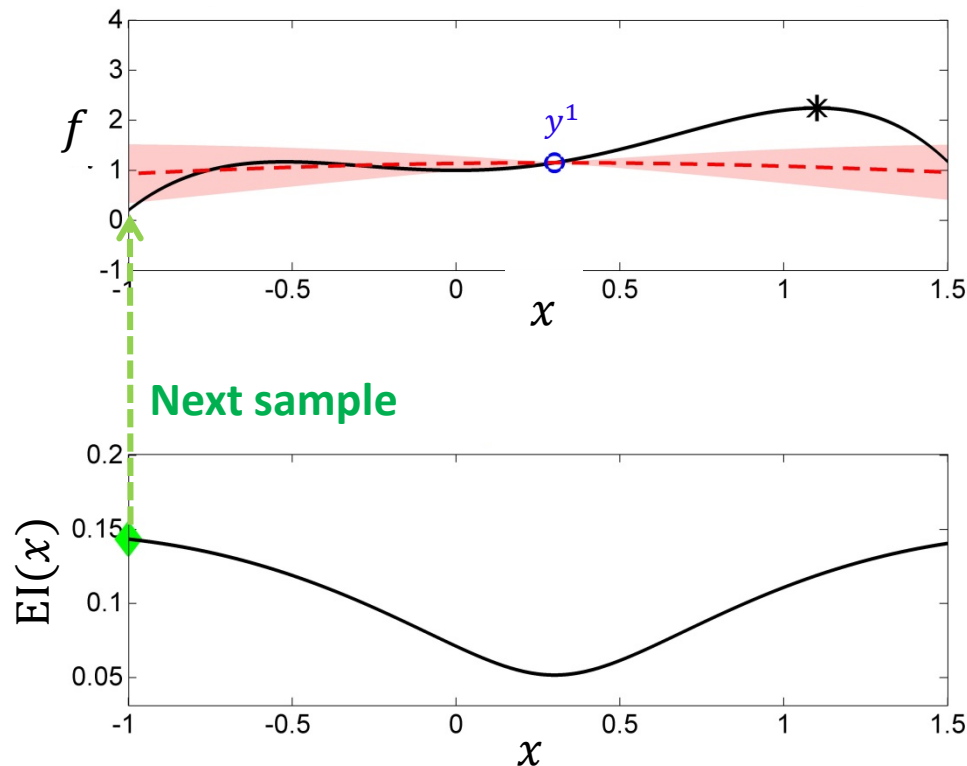


Bayesian optimization with EI function example

Illustrative example

$$\begin{aligned} & \underset{x}{\text{maximize}} \quad f(x) = -1.3x^4 + 1x^3 + 1.5x^2 + 1 + \epsilon \\ & \text{subject to} \quad -1 \leq x \leq 1.5 \quad \epsilon \sim \mathcal{N}(0, 0.01^2) \end{aligned}$$

$$y^{(1)} \begin{bmatrix} 1.15 \\ f \end{bmatrix} \sim \mathcal{N} \left(\mathbf{0}, \begin{bmatrix} \mathbf{K} + \sigma_\epsilon^2 \mathbf{I} & \mathbf{k} \\ \mathbf{k}^T & k(x, x) \end{bmatrix} \right)$$



Construct probability distribution on the unknown function value

$$p(f|\mathcal{D}) = \mathcal{N}(\mu(x|\mathcal{D}), \sigma^2(x|\mathcal{D}))$$

$$\mu(x|\mathcal{D}) = \mathbf{k}^T (\mathbf{K} + \sigma_\epsilon^2 \mathbf{I})^{-1} \mathbf{y}_{1:n}$$

$$\sigma^2(x|\mathcal{D}) = k(x, x) - \mathbf{k}^T (\mathbf{K} + \sigma_\epsilon^2 \mathbf{I})^{-1} \mathbf{k}$$

Select the next trial action that maximizes

$$x^{(2)} = \underset{x}{\operatorname{argmax}} EI(x) \triangleq \mathbb{E}[\max\{0, f - f^{\max}\} | \mathcal{D}]$$

$$x^{(2)} = -1.00$$

Query the function value

$$y^{(2)} = f(x^{(2)}) + \epsilon, \epsilon \sim \mathcal{N}(0, 0.01^2)$$

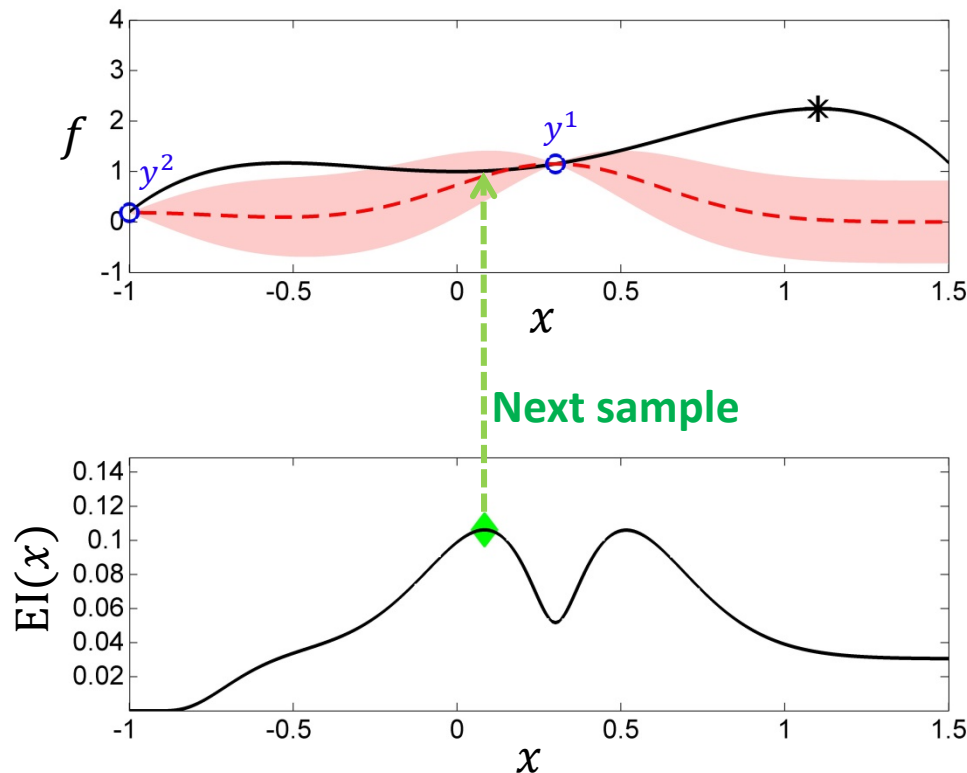
$$y^{(2)} = 0.18$$

Bayesian optimization with EI function example

Illustrative example

$$\begin{aligned} &\underset{x}{\text{maximize}} \quad f(x) = -1.3x^4 + 1x^3 + 1.5x^2 + 1 + \epsilon \\ &\text{subject to} \quad -1 \leq x \leq 1.5 \quad \epsilon \sim \mathcal{N}(0, 0.01^2) \end{aligned}$$

$$\begin{matrix} y^{(1)} \\ y^{(2)} \\ f \end{matrix} \begin{bmatrix} 1.15 \\ 0.18 \\ f \end{bmatrix} \sim \mathcal{N} \left(\mathbf{0}, \begin{bmatrix} \mathbf{K} + \sigma_\epsilon^2 \mathbf{I} & \mathbf{k} \\ \mathbf{k}^T & k(x, x) \end{bmatrix} \right)$$



Construct probability distribution on the unknown function value

$$p(f|\mathcal{D}) = \mathcal{N}(\mu(x|\mathcal{D}), \sigma^2(x|\mathcal{D}))$$

$$\mu(x|\mathcal{D}) = \mathbf{k}^T (\mathbf{K} + \sigma_\epsilon^2 \mathbf{I})^{-1} \mathbf{y}_{1:n}$$

$$\sigma^2(x|\mathcal{D}) = k(x, x) - \mathbf{k}^T (\mathbf{K} + \sigma_\epsilon^2 \mathbf{I})^{-1} \mathbf{k}$$

Select the next trial action that maximizes

$$x^{(3)} = \underset{x}{\operatorname{argmax}} EI(x) \triangleq \mathbb{E}[\max\{0, f - f^{\max}\} | \mathcal{D}]$$

$$x^{(3)} = 0.03$$

Query the function value

$$y^{(3)} = f(x^{(3)}) + \epsilon, \epsilon \sim \mathcal{N}(0, 0.01^2)$$

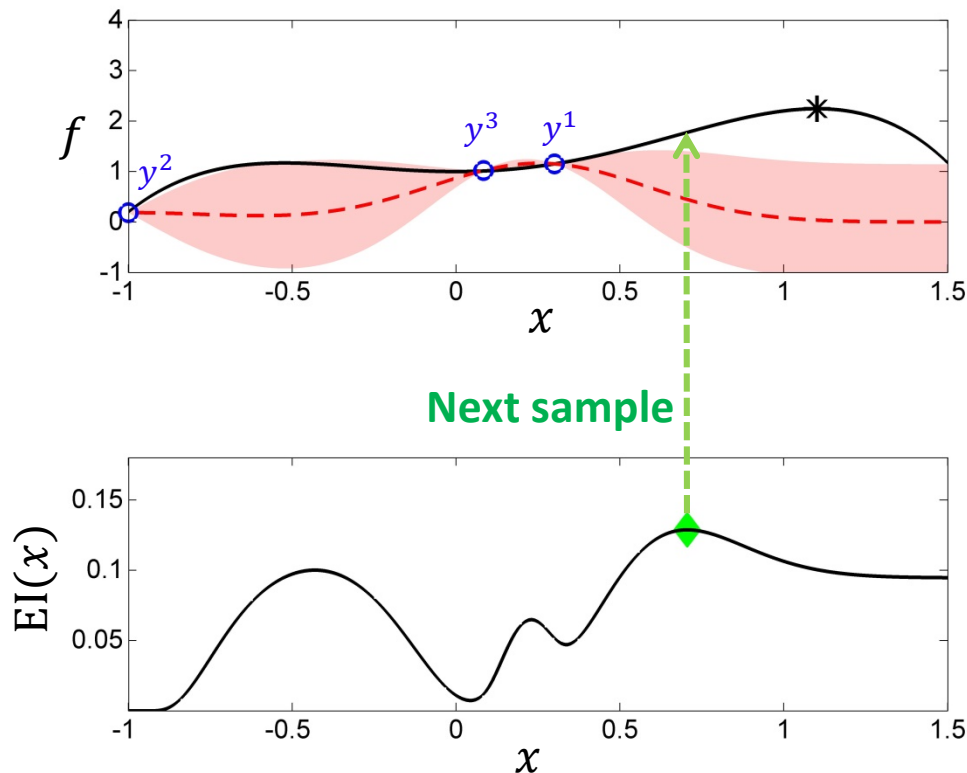
$$y^{(3)} = 1.02$$

Bayesian optimization with EI function example

Illustrative example

$$\begin{aligned} & \underset{x}{\text{maximize}} \quad f(x) = -1.3x^4 + 1x^3 + 1.5x^2 + 1 + \epsilon \\ & \text{subject to} \quad -1 \leq x \leq 1.5 \quad \epsilon \sim \mathcal{N}(0, 0.01^2) \end{aligned}$$

$$\begin{matrix} y^{(1)} \\ y^{(2)} \\ y^{(3)} \\ f \end{matrix} \begin{bmatrix} 1.15 \\ 0.18 \\ 1.02 \\ f \end{bmatrix} \sim \mathcal{N} \left(\mathbf{0}, \begin{bmatrix} \mathbf{K} + \sigma_\epsilon^2 \mathbf{I} & \mathbf{k} \\ \mathbf{k}^T & k(x, x) \end{bmatrix} \right)$$



Construct probability distribution on the unknown function value

$$p(f|\mathcal{D}) = \mathcal{N}(\mu(x|\mathcal{D}), \sigma^2(x|\mathcal{D}))$$

$$\mu(x|\mathcal{D}) = \mathbf{k}^T (\mathbf{K} + \sigma_\epsilon^2 \mathbf{I})^{-1} \mathbf{y}_{1:n}$$

$$\sigma^2(x|\mathcal{D}) = k(x, x) - \mathbf{k}^T (\mathbf{K} + \sigma_\epsilon^2 \mathbf{I})^{-1} \mathbf{k}$$

Select the next trial action that maximizes

$$x^{(4)} = \underset{x}{\operatorname{argmax}} \operatorname{EI}(x) \triangleq \mathbb{E}[\max\{0, f - f^{\max}\} | \mathcal{D}]$$

$$x^{(4)} = 0.70$$

Query the function value

$$y^{(4)} = f(x^{(4)}) + \epsilon, \epsilon \sim \mathcal{N}(0, 0.01^2)$$

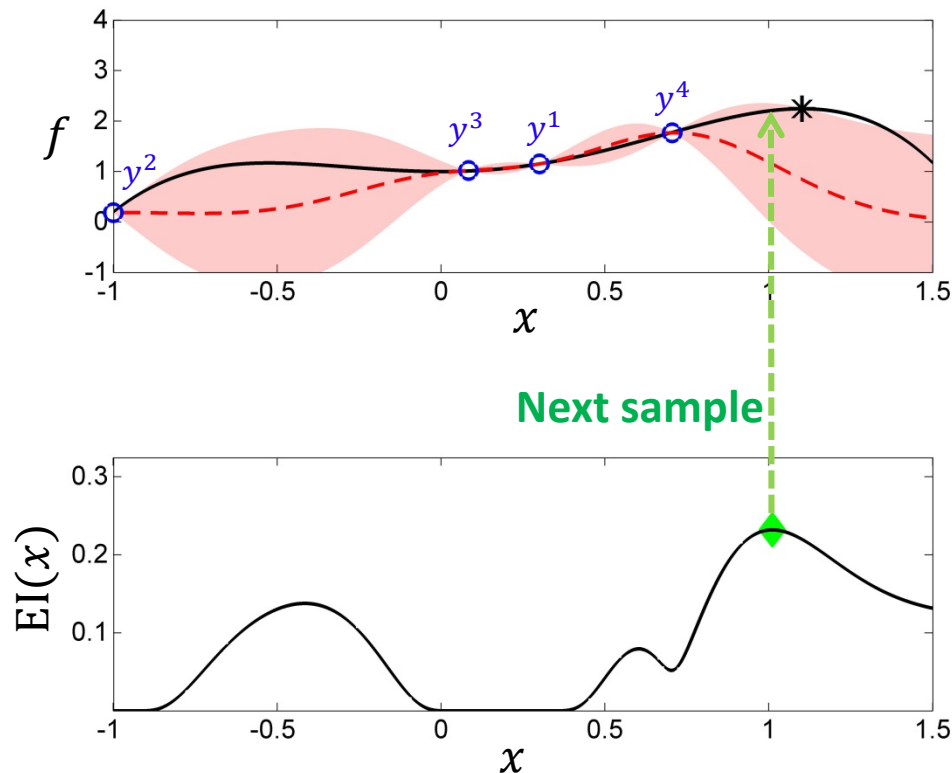
$$y^{(4)} = 1.76$$

Bayesian optimization with EI function example

Illustrative example

$$\begin{aligned} & \underset{x}{\text{maximize}} \quad f(x) = -1.3x^4 + 1x^3 + 1.5x^2 + 1 + \epsilon \\ & \text{subject to} \quad -1 \leq x \leq 1.5 \quad \epsilon \sim \mathcal{N}(0, 0.01^2) \end{aligned}$$

$$\begin{matrix} y^{(1)} \\ y^{(2)} \\ y^{(3)} \\ y^{(4)} \\ f \end{matrix} \begin{bmatrix} 1.15 \\ 0.18 \\ 1.02 \\ 1.76 \\ f \end{bmatrix} \sim \mathcal{N} \left(\mathbf{0}, \begin{bmatrix} \mathbf{K} + \sigma_\epsilon^2 \mathbf{I} & \mathbf{k} \\ \mathbf{k}^T & k(x, x) \end{bmatrix} \right)$$



Construct probability distribution on the unknown function value

$$p(f|\mathcal{D}) = \mathcal{N}(\mu(x|\mathcal{D}), \sigma^2(x|\mathcal{D}))$$

$$\mu(x|\mathcal{D}) = \mathbf{k}^T (\mathbf{K} + \sigma_\epsilon^2 \mathbf{I})^{-1} \mathbf{y}_{1:n}$$

$$\sigma^2(x|\mathcal{D}) = k(x, x) - \mathbf{k}^T (\mathbf{K} + \sigma_\epsilon^2 \mathbf{I})^{-1} \mathbf{k}$$

Select the next trial action that maximizes

$$x^{(5)} = \underset{x}{\operatorname{argmax}} EI(x) \triangleq \mathbb{E}[\max\{0, f - f^{\max}\} | \mathcal{D}]$$

$$x^{(5)} = 1.01$$

Query the function value

$$y^{(5)} = f(x^{(5)}) + \epsilon, \epsilon \sim \mathcal{N}(0, 0.01^2)$$

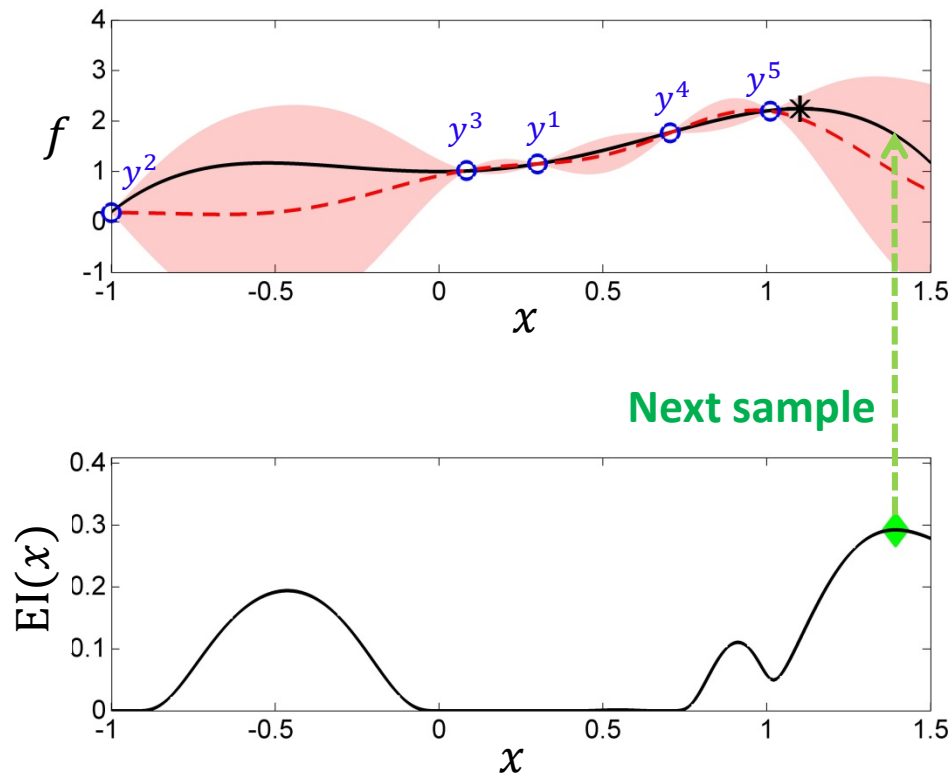
$$y^{(5)} = 2.19$$

Bayesian optimization with EI function example

Illustrative example

$$\begin{aligned} &\underset{x}{\text{maximize}} \quad f(x) = -1.3x^4 + 1x^3 + 1.5x^2 + 1 + \epsilon \\ &\text{subject to} \quad -1 \leq x \leq 1.5 \quad \epsilon \sim \mathcal{N}(0, 0.01^2) \end{aligned}$$

$$\begin{bmatrix} y^{(1)} \\ y^{(2)} \\ y^{(3)} \\ y^{(4)} \\ y^{(5)} \\ f \end{bmatrix} \sim \mathcal{N} \left(\mathbf{0}, \begin{bmatrix} \mathbf{K} + \sigma_\epsilon^2 \mathbf{I} & \mathbf{k} \\ \mathbf{k}^T & k(x, x) \end{bmatrix} \right)$$



Construct probability distribution on the unknown function value

$$p(f|\mathcal{D}) = \mathcal{N}(\mu(x|\mathcal{D}), \sigma^2(x|\mathcal{D}))$$

$$\mu(x|\mathcal{D}) = \mathbf{k}^T (\mathbf{K} + \sigma_\epsilon^2 \mathbf{I})^{-1} \mathbf{y}_{1:n}$$

$$\sigma^2(x|\mathcal{D}) = k(x, x) - \mathbf{k}^T (\mathbf{K} + \sigma_\epsilon^2 \mathbf{I})^{-1} \mathbf{k}$$

Select the next trial action that maximizes

$$x^{(6)} = \underset{x}{\operatorname{argmax}} EI(x) \triangleq \mathbb{E}[\max\{0, f - f^{\max}\} | \mathcal{D}]$$

$$x^{(6)} = 1.39$$

Query the function value

$$y^{(6)} = f(x^{(6)}) + \epsilon, \epsilon \sim \mathcal{N}(0, 0.01^2)$$

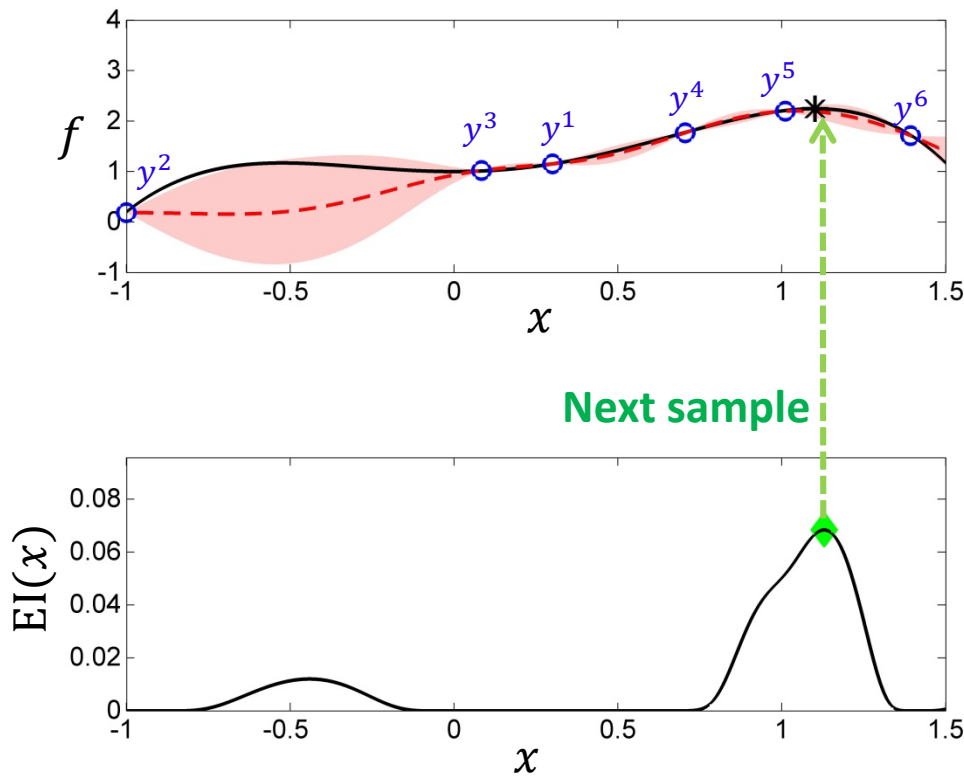
$$y^{(6)} = 1.70$$

Bayesian optimization with EI function example

Illustrative example

$$\begin{aligned} & \underset{x}{\text{maximize}} \quad f(x) = -1.3x^4 + 1x^3 + 1.5x^2 + 1 + \epsilon \\ & \text{subject to} \quad -1 \leq x \leq 1.5 \quad \epsilon \sim \mathcal{N}(0, 0.01^2) \end{aligned}$$

$$\begin{bmatrix} y^{(1)} \\ y^{(2)} \\ y^{(3)} \\ y^{(4)} \\ y^{(5)} \\ y^{(6)} \\ f \end{bmatrix} \sim \mathcal{N} \left(\mathbf{0}, \begin{bmatrix} \mathbf{K} + \sigma_\epsilon^2 \mathbf{I} & \mathbf{k} \\ \mathbf{k}^T & k(x, x) \end{bmatrix} \right)$$



Construct probability distribution on the unknown function value

$$p(f|\mathcal{D}) = \mathcal{N}(\mu(x|\mathcal{D}), \sigma^2(x|\mathcal{D}))$$

$$\mu(x|\mathcal{D}) = \mathbf{k}^T (\mathbf{K} + \sigma_\epsilon^2 \mathbf{I})^{-1} \mathbf{y}_{1:n}$$

$$\sigma^2(x|\mathcal{D}) = k(x, x) - \mathbf{k}^T (\mathbf{K} + \sigma_\epsilon^2 \mathbf{I})^{-1} \mathbf{k}$$

Select the next trial action that maximizes

$$x^{(7)} = \underset{x}{\operatorname{argmax}} EI(x) \triangleq \mathbb{E}[\max\{0, f - f^{\max}\} | \mathcal{D}]$$

$$x^{(7)} = 1.13$$

Query the function value

$$y^{(7)} = f(x^{(7)}) + \epsilon, \epsilon \sim \mathcal{N}(0, 0.01^2)$$

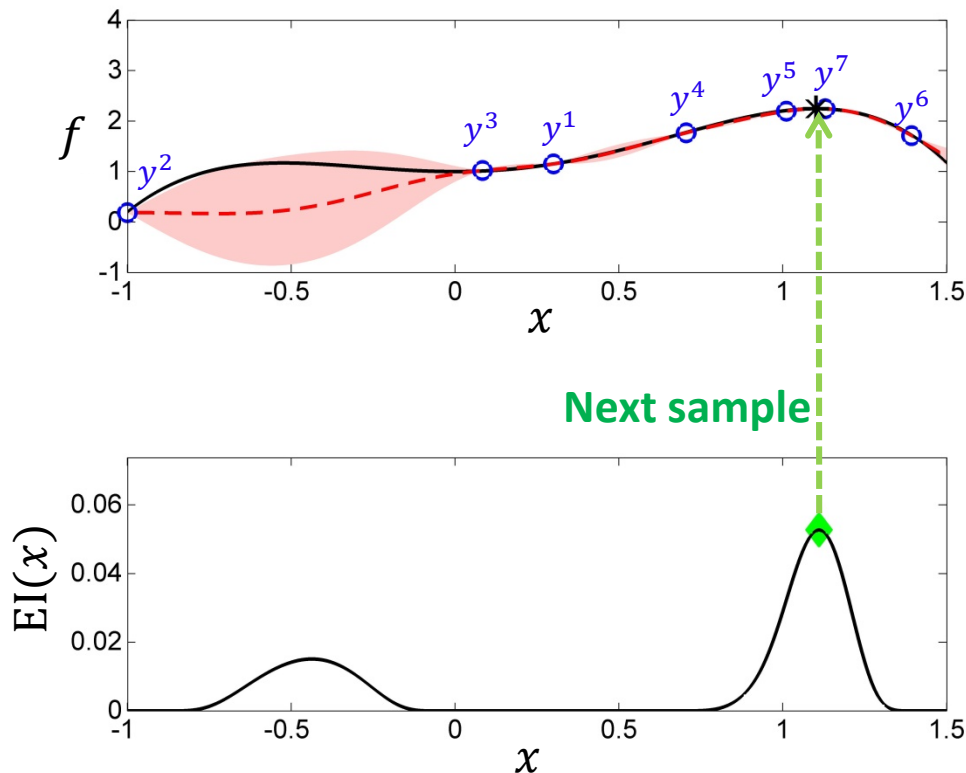
$$y^{(7)} = 2.24$$

Bayesian optimization with EI function example

Illustrative example

$$\begin{aligned} & \underset{x}{\text{maximize}} \quad f(x) = -1.3x^4 + 1x^3 + 1.5x^2 + 1 + \epsilon \\ & \text{subject to} \quad -1 \leq x \leq 1.5 \quad \epsilon \sim N(0, 0.01^2) \end{aligned}$$

$$\begin{bmatrix} y^{(1)} \\ y^{(2)} \\ y^{(3)} \\ y^{(4)} \\ y^{(5)} \\ y^{(6)} \\ y^{(7)} \\ f \end{bmatrix} \sim N \left(\mathbf{0}, \begin{bmatrix} \mathbf{K} + \sigma_\epsilon^2 \mathbf{I} & \mathbf{k} \\ \mathbf{k}^T & k(x, x) \end{bmatrix} \right)$$



Construct probability distribution on the unknown function value

$$p(f|\mathcal{D}) = N(\mu(x|\mathcal{D}), \sigma^2(x|\mathcal{D}))$$

$$\mu(x|\mathcal{D}) = \mathbf{k}^T (\mathbf{K} + \sigma_\epsilon^2 \mathbf{I})^{-1} \mathbf{y}_{1:n}$$

$$\sigma^2(x|\mathcal{D}) = k(x, x) - \mathbf{k}^T (\mathbf{K} + \sigma_\epsilon^2 \mathbf{I})^{-1} \mathbf{k}$$

Select the next trial action that maximizes

$$x^{(8)} = \underset{x}{\operatorname{argmax}} EI(x) \triangleq \mathbb{E}[\max\{0, f - f^{\max}\} | \mathcal{D}]$$

$$x^{(8)} = 1.11$$

Query the function value

$$y^{(8)} = f(x^{(8)}) + \epsilon, \epsilon \sim N(0, 0.01^2)$$

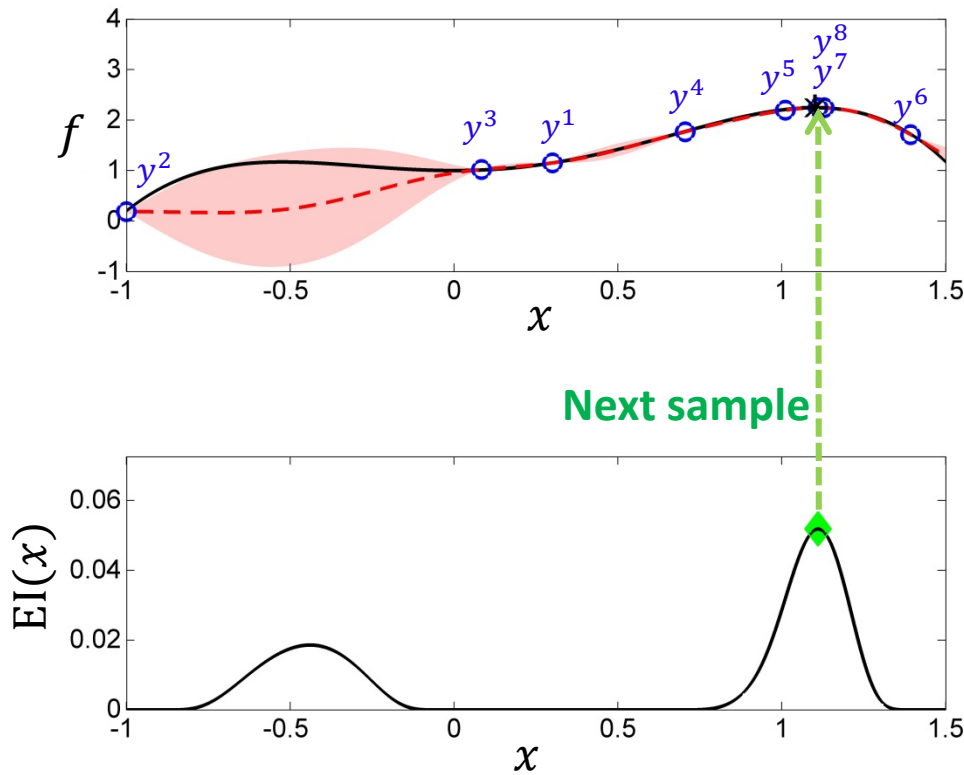
$$y^{(8)} = 2.24$$

Bayesian optimization with EI function example

Illustrative example

$$\begin{aligned} & \underset{x}{\text{maximize}} \quad f(x) = -1.3x^4 + 1x^3 + 1.5x^2 + 1 + \epsilon \\ & \text{subject to} \quad -1 \leq x \leq 1.5 \quad \epsilon \sim \mathcal{N}(0, 0.01^2) \end{aligned}$$

$$\begin{bmatrix} y^{(1)} \\ y^{(2)} \\ y^{(3)} \\ y^{(4)} \\ y^{(5)} \\ y^{(6)} \\ y^{(7)} \\ y^{(8)} \\ f \end{bmatrix} \sim \mathcal{N} \left(\mathbf{0}, \begin{bmatrix} \mathbf{K} + \sigma_\epsilon^2 \mathbf{I} & \mathbf{k} \\ \mathbf{k}^T & k(x, x) \end{bmatrix} \right)$$



Construct probability distribution on the unknown function value

$$p(f|\mathcal{D}) = \mathcal{N}(\mu(x|\mathcal{D}), \sigma^2(x|\mathcal{D}))$$

$$\mu(x|\mathcal{D}) = \mathbf{k}^T (\mathbf{K} + \sigma_\epsilon^2 \mathbf{I})^{-1} \mathbf{y}_{1:n}$$

$$\sigma^2(x|\mathcal{D}) = k(x, x) - \mathbf{k}^T (\mathbf{K} + \sigma_\epsilon^2 \mathbf{I})^{-1} \mathbf{k}$$

Select the next trial action that maximizes

$$x^{(9)} = \underset{x}{\operatorname{argmax}} EI(x) \triangleq \mathbb{E}[\max\{0, f - f^{\max}\} | \mathcal{D}]$$

$$x^{(9)} = 1.11$$

Query the function value

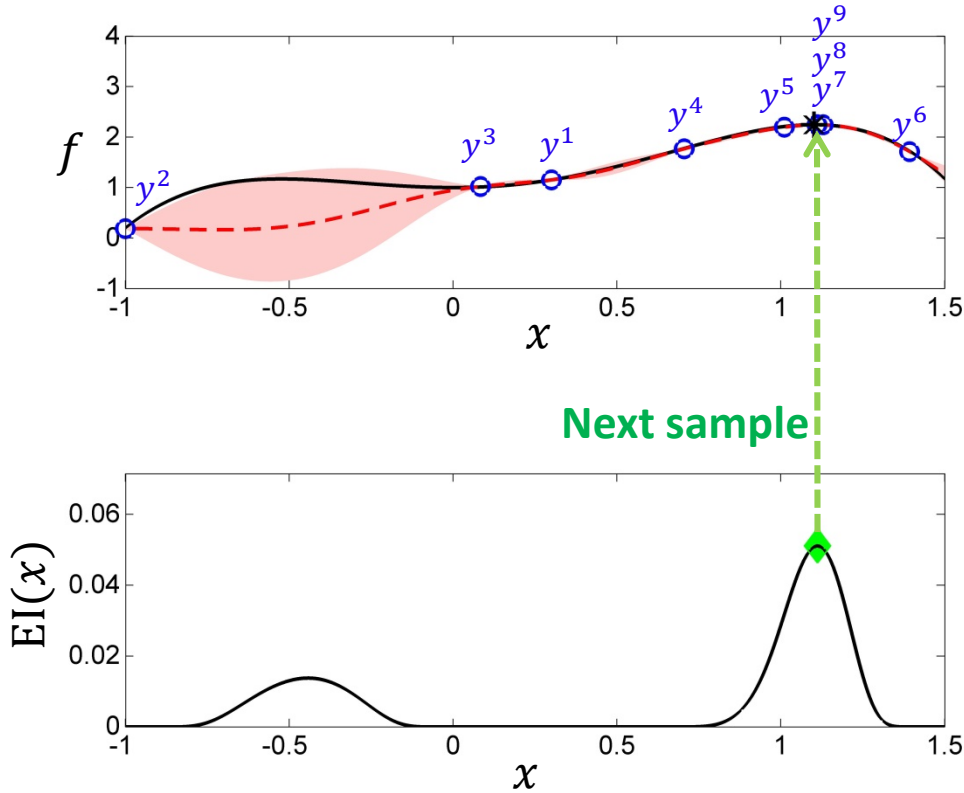
$$y^{(9)} = f(x^{(9)}) + \epsilon, \epsilon \sim \mathcal{N}(0, 0.01^2)$$

$$y^{(9)} = 2.24$$

Bayesian optimization with EI function example

Illustrative example

$$\begin{aligned} & \underset{x}{\text{maximize}} \quad f(x) = -1.3x^4 + 1x^3 + 1.5x^2 + 1 + \epsilon \\ & \text{subject to} \quad -1 \leq x \leq 1.5 \quad \epsilon \sim N(0, 0.01^2) \end{aligned}$$



$$\begin{bmatrix} y^{(1)} \\ y^{(2)} \\ y^{(3)} \\ y^{(4)} \\ y^{(5)} \\ y^{(6)} \\ y^{(7)} \\ y^{(8)} \\ y^{(9)} \\ f \end{bmatrix} \sim N \left(\mathbf{0}, \begin{bmatrix} \mathbf{K} + \sigma_{\epsilon}^2 \mathbf{I} & \mathbf{k} \\ \mathbf{k}^T & k(x, x) \end{bmatrix} \right)$$

Construct probability distribution on the unknown function value

$$p(f|\mathcal{D}) = N(\mu(x|\mathcal{D}), \sigma^2(x|\mathcal{D}))$$

$$\mu(x|\mathcal{D}) = \mathbf{k}^T (\mathbf{K} + \sigma_{\epsilon}^2 \mathbf{I})^{-1} \mathbf{y}_{1:n}$$

$$\sigma^2(x|\mathcal{D}) = k(x, x) - \mathbf{k}^T (\mathbf{K} + \sigma_{\epsilon}^2 \mathbf{I})^{-1} \mathbf{k}$$

Select the next trial action that maximizes

$$x^{(10)} = \underset{x}{\operatorname{argmax}} EI(x) \triangleq \mathbb{E}[\max\{0, f - f^{\max}\} | \mathcal{D}]$$

$$x^{(10)} = 1.11$$

Query the function value

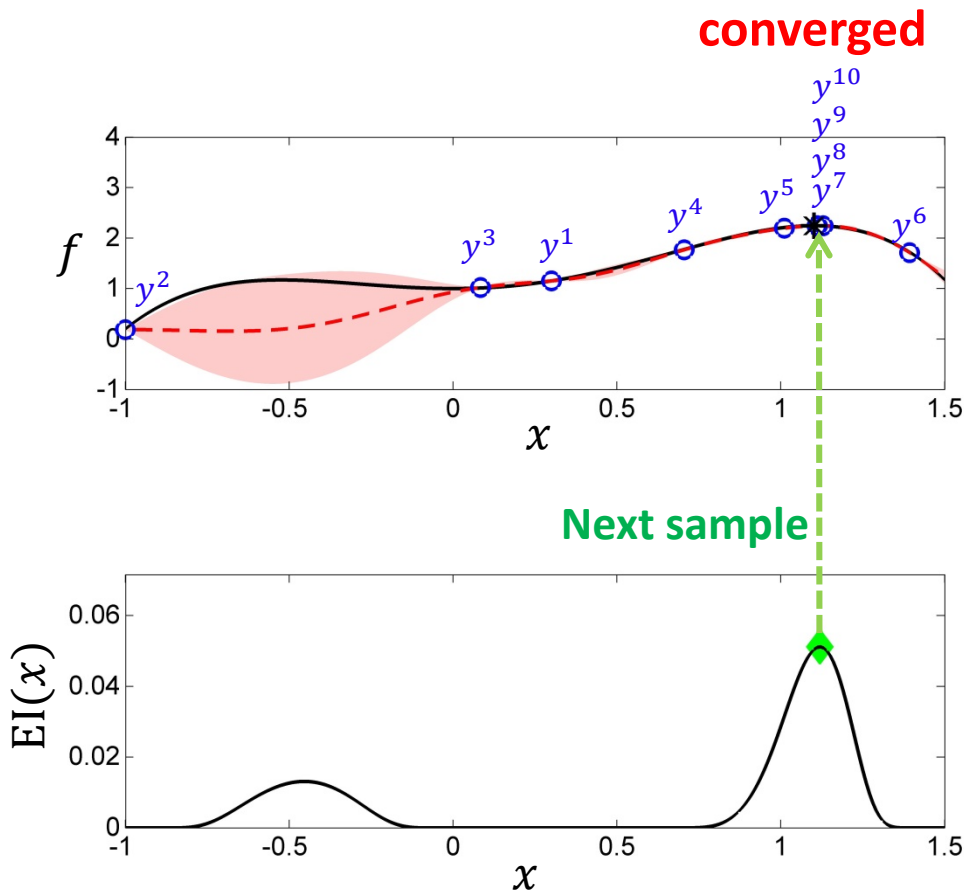
$$y^{(10)} = f(x^{(10)}) + \epsilon, \epsilon \sim N(0, 0.01^2)$$

$$y^{(10)} = 2.24$$

Bayesian optimization with EI function example

Illustrative example

$$\begin{aligned} & \underset{x}{\text{maximize}} \quad f(x) = -1.3x^4 + 1x^3 + 1.5x^2 + 1 + \epsilon \\ & \text{subject to} \quad -1 \leq x \leq 1.5 \quad \epsilon \sim \mathcal{N}(0, 0.01^2) \end{aligned}$$



$$\begin{bmatrix} y^{(1)} \\ y^{(2)} \\ y^{(3)} \\ y^{(4)} \\ y^{(5)} \\ y^{(6)} \\ y^{(7)} \\ y^{(8)} \\ y^{(9)} \\ y^{(10)} \\ f \end{bmatrix} \sim N \left(\mathbf{0}, \begin{bmatrix} \mathbf{K} + \sigma_\epsilon^2 \mathbf{I} & \mathbf{k} \\ \mathbf{k}^T & k(x, x) \end{bmatrix} \right)$$

Construct probability distribution on the unknown function value

$$p(f|\mathcal{D}) = \mathcal{N}(\mu(x|\mathcal{D}), \sigma^2(x|\mathcal{D}))$$

$$\mu(x|\mathcal{D}) = \mathbf{k}^T (\mathbf{K} + \sigma_\epsilon^2 \mathbf{I})^{-1} \mathbf{y}_{1:n}$$

$$\sigma^2(x|\mathcal{D}) = k(x, x) - \mathbf{k}^T (\mathbf{K} + \sigma_\epsilon^2 \mathbf{I})^{-1} \mathbf{k}$$

Select the next trial action that maximizes

$$x^{(11)} = \underset{x}{\operatorname{argmax}} EI(x) \triangleq \mathbb{E}[\max\{0, f - f^{\max}\} | \mathcal{D}]$$

$$x^{(11)} = 1.11$$

Query the function value

$$y^{(11)} = f(x^{(11)}) + \epsilon, \epsilon \sim \mathcal{N}(0, 0.01^2)$$

$$y^{(11)} = 2.24$$

Supplementary issues in BO

- How to optimize the acquisition function?
- What to do with the hyper-parameters of the model?
- How to select points to initialize the model?

How to optimize the acquisition function?

- Many alternatives are available:
 - Gradient descent methods: Conjugate gradient, BFGS, etc.
 - Lipschitz based heuristics: DIRECT
 - Evolutionary algorithms: CMA
- Some of these methods can also be used to directly optimize f when the analytical form of f is unknown but its evaluations are available.

Gradient descent algorithm

- **Batch gradient algorithm**

$$\theta^{(t+1)} = \theta^{(t)} + \alpha \nabla_{\theta} L(\theta^{(t)}, \mathbf{X}, \mathbf{y})$$

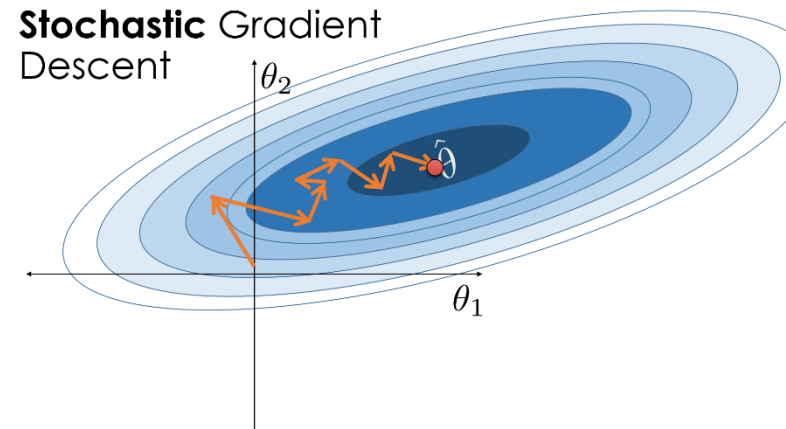
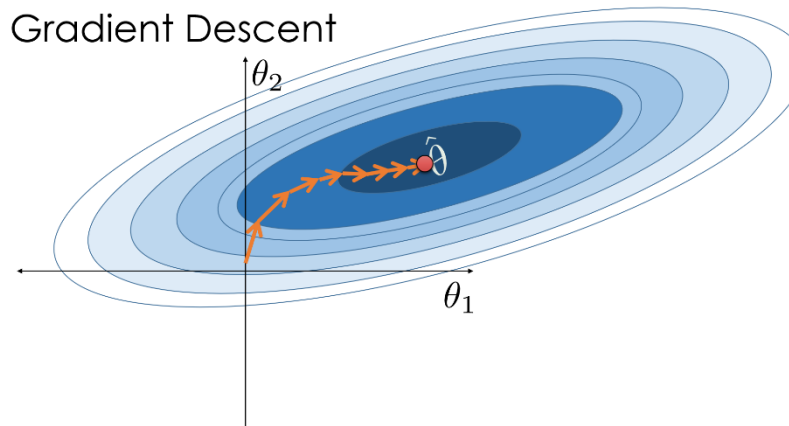
where $\nabla_{\theta} L(\theta^{(t)}, \mathbf{X}, \mathbf{y})$ is the average gradient of the loss function across all points in the training data set

$$\nabla_{\theta} L(\theta^{(t)}, \mathbf{X}, \mathbf{y}) = \frac{1}{n} \sum_{i=1}^n \nabla_{\theta} l(\theta^{(t)}, x_i, y_i)$$

where $l(\theta^{(t)}, x_i, y_i)$ is the loss at a single point in the training set.

- **Stochastic gradient algorithm**

$$\theta^{(t+1)} = \theta^{(t)} + \alpha \nabla_{\theta} l(\theta^{(t)}, x_i, y_i)$$



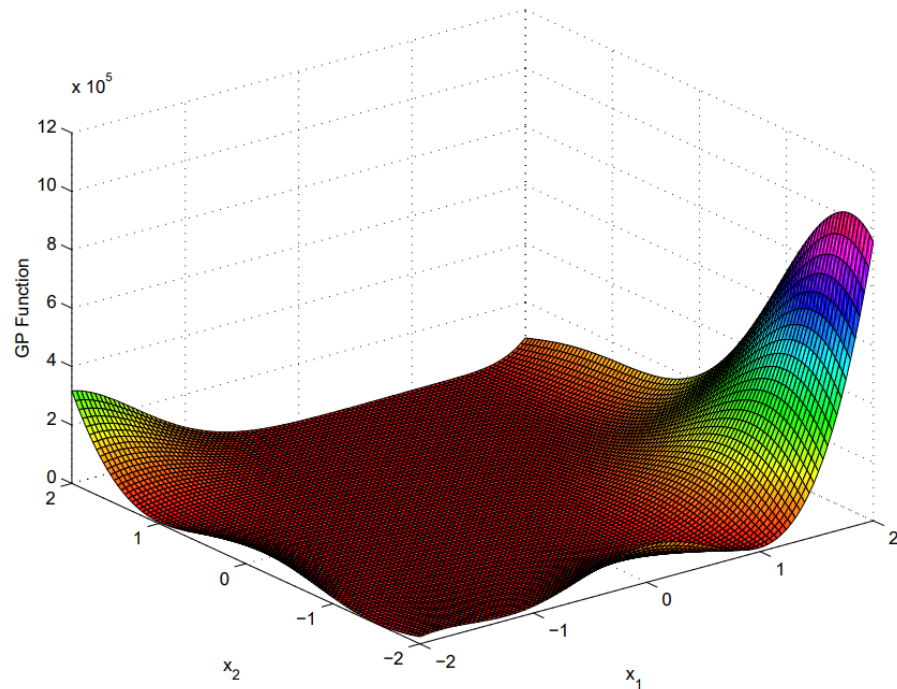
Dividing RECTangles (DIRECT) algorithm

- It was created in order to solve difficult global optimization problems with bound constraints and a real-valued objective function.
- **DIRECT is a sampling algorithm.** That is, it requires no knowledge of the objective function gradient
 - ✓ A global search algorithm like DIRECT can be very useful when the objective function is a "black box" function or simulation.
- Instead, the algorithm samples points in the domain, and uses the information it has obtained to decide where to search next.
- The DIRECT algorithm will globally converge to the minimal value of the objective function
 - ✓ Unfortunately, this global convergence may come at the expense of a large and exhaustive search over the domain.
- The strengths of DIRECT lie in the balanced effort it gives to local and global searches, and the few parameters it requires to run.

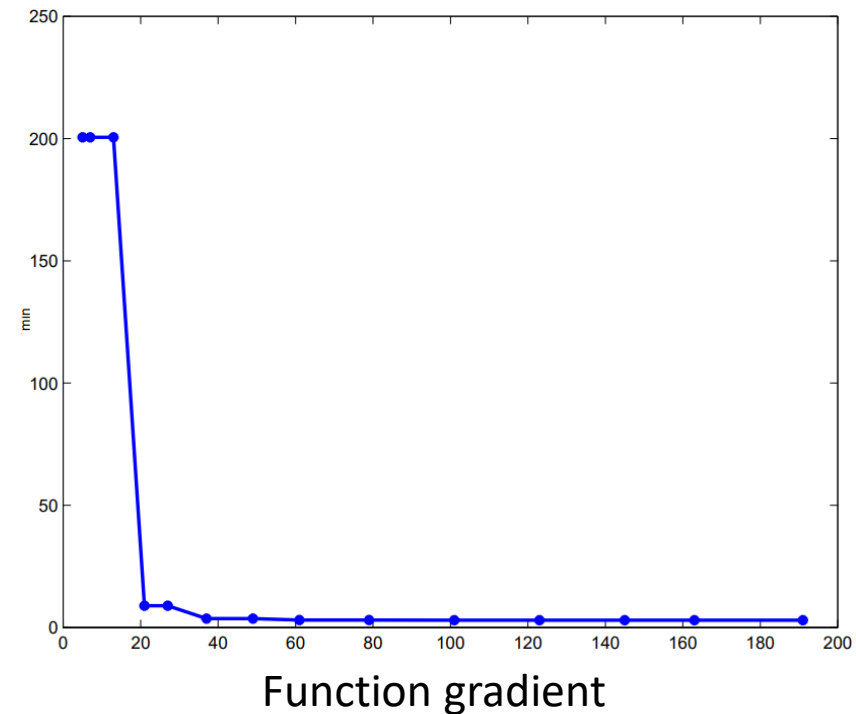
Dividing RECTangles (DIRECT) algorithm

Example

$$f(x_1, x_2) = [1 + (x_1 + x_2 + 1)^2(19 - 14x_1 + 3x_1^2 - 14x_2 + 6x_1x_2 + 3x_2^2)] \\ [30 + (2x_1 - 3x_2)^2(18 - 32x_1 + 12x_1^2 + 48x_2 - 36x_1x_2 + 27x_2^2)]$$



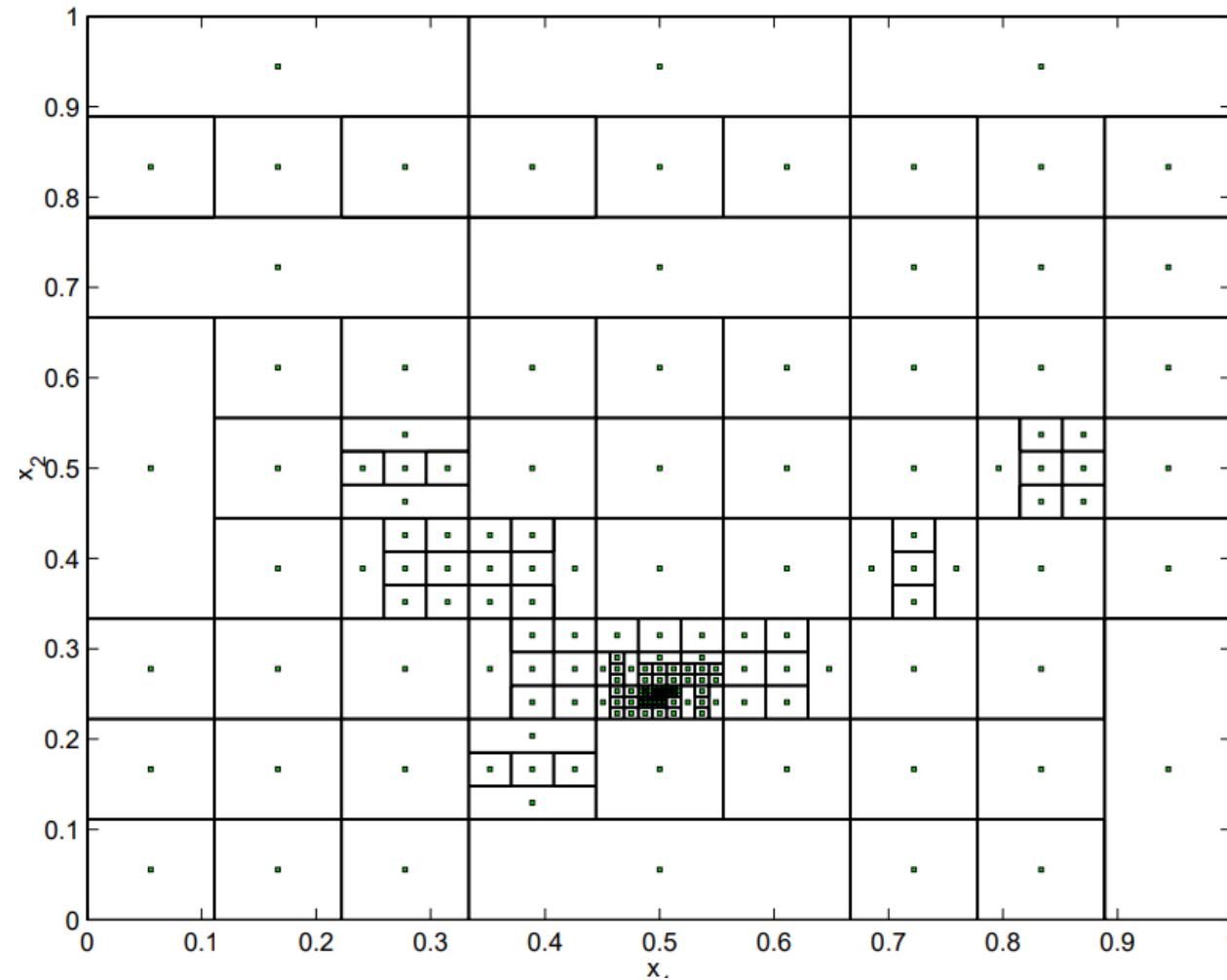
The Goldstein-Price test function.



Dividing RECTangles (DIRECT) algorithm

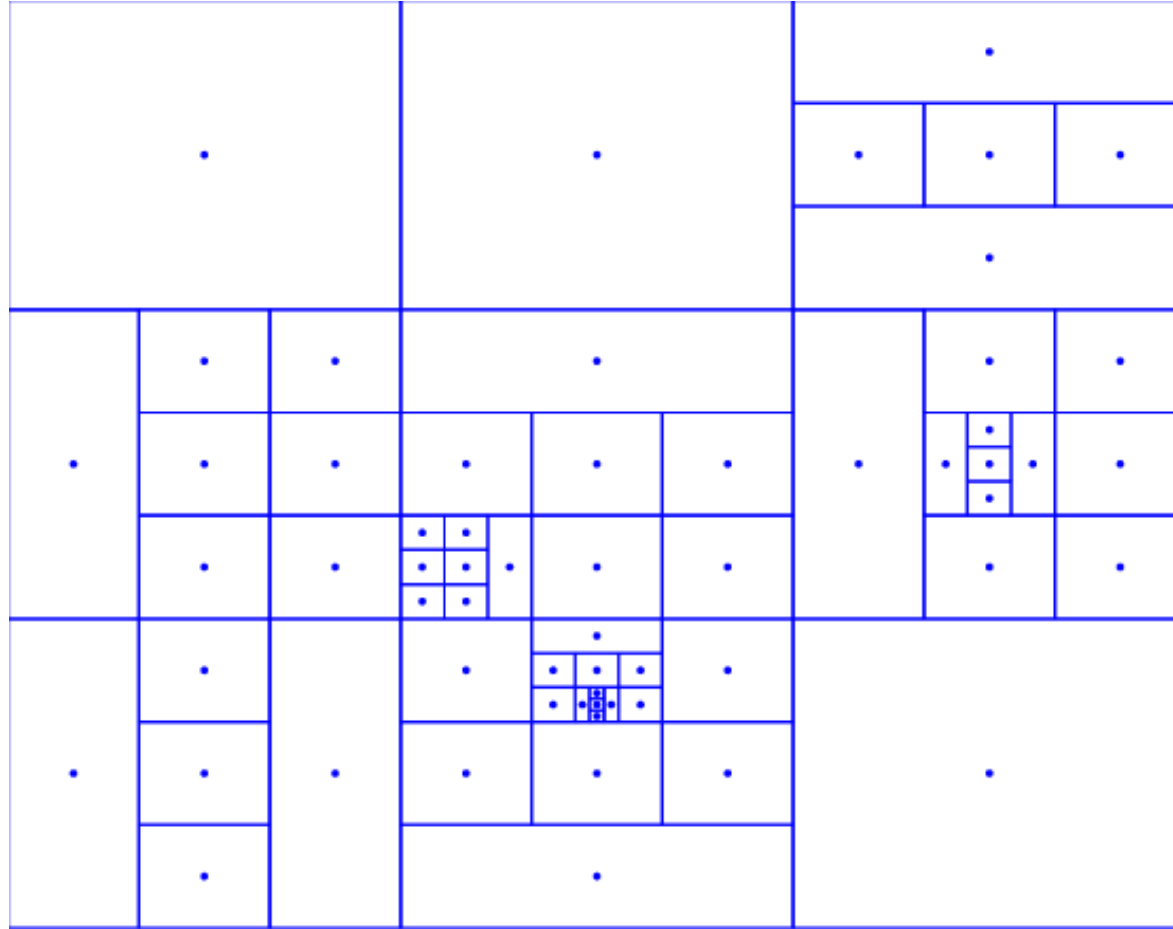


Dividing RECTangles (DIRECT) algorithm



- Finds good solution in general
- Doesn't need gradient
- Not generalization to non-squared domains.

Dividing RECTangles (DIRECT) algorithm



- Finds good solution in general
- Doesn't need gradient
- Not generalization to non-squared domains.

Covariance matrix adaptation

- The **CMA-ES** is a stochastic, or randomized, method for real-parameter (continuous domain) optimization of non-linear, non-convex functions
 - CMA: Covariance Matrix Adaptation
 - ES: Evolutionary Search

Initialize distribution parameters $\theta^{(0)}$

For generation $g = 0, 1, 2, \dots$

Sample λ independent points from distribution $P(\mathbf{x}|\theta^{(g)}) \rightarrow \mathbf{x}_1, \dots, \mathbf{x}_\lambda$

Evaluate the sample $\mathbf{x}_1, \dots, \mathbf{x}_\lambda$ on f

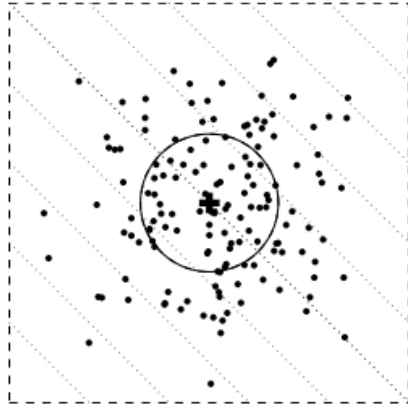
Update parameters $\theta^{(g+1)} = F_\theta(\theta^{(g)}, (\mathbf{x}_1, f(\mathbf{x}_1)), \dots, (\mathbf{x}_\lambda, f(\mathbf{x}_\lambda)))$

break, if termination criterion met

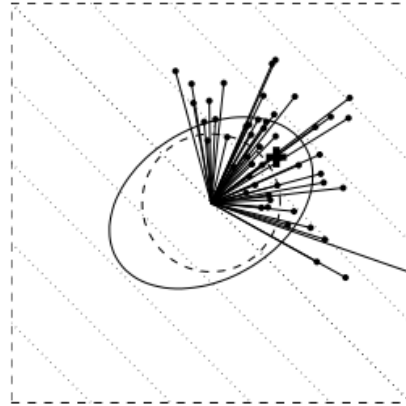
A randomized black box search algorithm

Covariance matrix adaptation

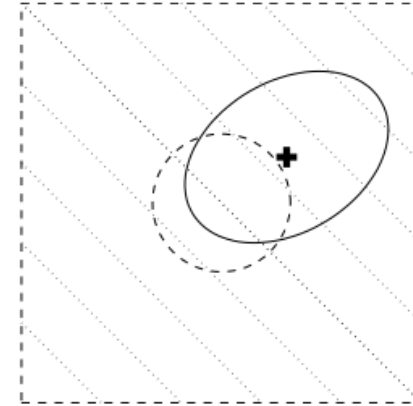
Sampling



Estimation



New distribution



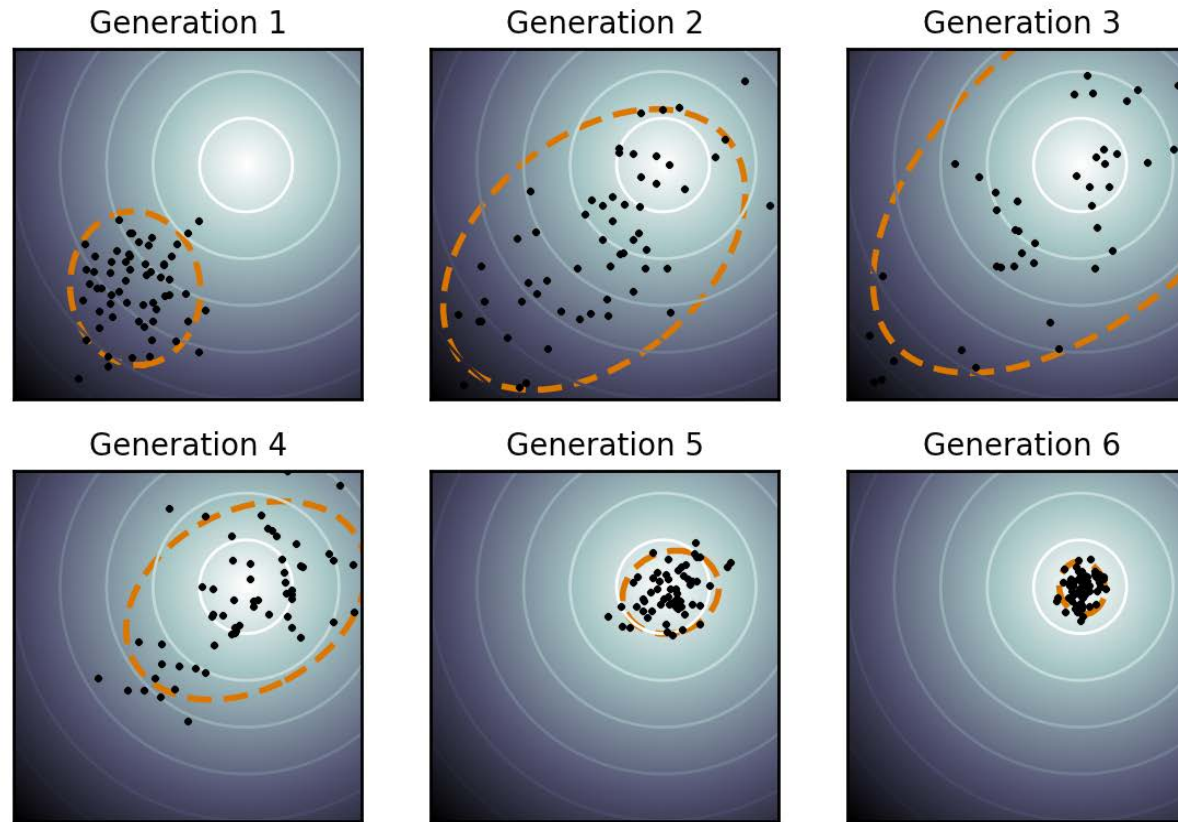
$$\mathbf{C}_{\mu}^{(g+1)}$$

$$\mathbf{x}_k^{(g+1)} \sim \mathbf{m}^{(g)} + \sigma^{(g)} \mathbf{N}(\mathbf{0}, \mathbf{C}^{(g)})$$

$$\mathbf{m}^{(g+1)} = \mathbf{m}^{(g)} + c_m \sum_{i=1}^{\mu} w_i \left(\mathbf{x}_{1:\lambda}^{(g+1)} - \mathbf{m}^{(g)} \right)$$

$$\mathbf{C}_{\lambda}^{(g+1)} = \frac{1}{\lambda} \sum_{i=1}^{\lambda} \left(\mathbf{x}_i^{(g+1)} - \mathbf{m}^{(g)} \right) \left(\mathbf{x}_i^{(g+1)} - \mathbf{m}^{(g)} \right)^{\mathrm{T}}$$

Covariance matrix adaptation



- Sample for a Gaussian with some mean μ and covariance matrix Σ .
- Select the best points and use them to update μ and Σ .
- Sample from the new Gaussian

Contextual Bayesian optimization

Motivation



Suggested for you



Facebook partners with Uber for ride-hailing service via Messenger

Reuters - 1 hour ago

Facebook Inc said on Wednesday it is testing a service that will allow users of its Messenger app, without leaving a conversation or downloading the ride-hailing app.



Eye on safety, California sets rules for self-driving cars

The Seattle Times - 13 days ago

FILE - In this May 13, 2015, file photo, Google's new self-driving prototype car is presented at Google campus in Mountain View, Calif.



The Apple iPhone 5s got a massive price cut, but is it still worth it?

Firstpost - 1 hour ago

Apple launched the iPhone 5s in India back in November 2013. Two years later, its price is Rs. 21,499 officially for the 16GB version, does the iPhone 5s still make a good deal?



Philips Hue: Just Kidding About Blocking Third-Party Bulbs

PC Magazine - 10 hours ago

You win, internet. Following user backlash, Philips has decided not to block third-party lighting platform after all.

[More Technology stories](#)



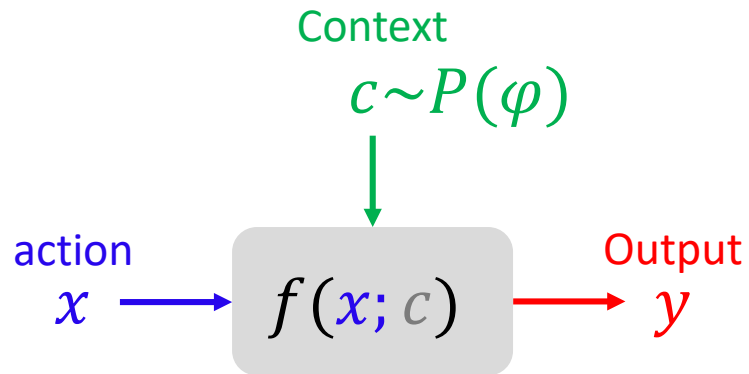
Finance : Portfolio optimization under unknown return profiles given varying economic condition

Health care : Choosing the best treatment among alternatives given different patient information

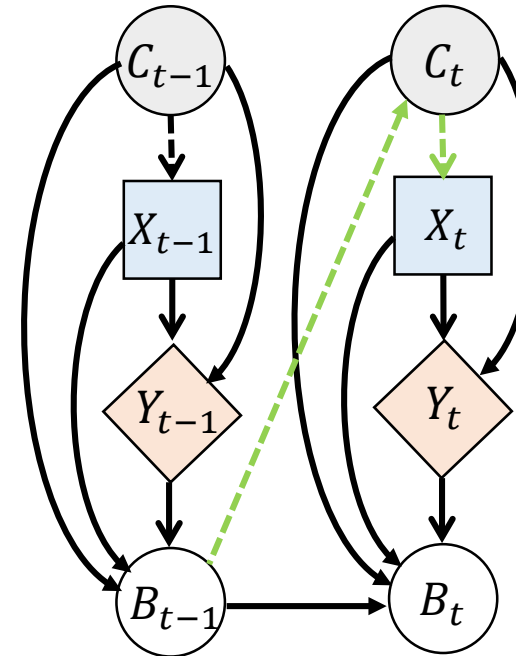
Internet shopping : Choosing the optimum price (sales v.s. profits) given varying season

Experiment design : Sequential experimental design given varying environmental condition

Contextual Bayesian optimization



MDP over belief state



$B_t(f)$: Belief state about unknown function f at t

- Policy π maps all the history to new action:
$$\pi: [(c_1, x_1, y_1), (c_2, x_2, y_2), \dots, (c_{t-1}, x_{t-1}, y_{t-1}), c_t] \rightarrow x_t$$
- Find the optimal policy π^* that maximizes $\mathbb{E}[\sum_{t=1}^T y_t]$ or $\mathbb{E}[y_T]$

$$x^* = \pi^*(c) = \operatorname{argmax}_x f(x; c)$$

From Bandit to Reinforcement Learning

		Model is unknown	
		No state dynamic	State dynamic (action affect state transition and future reward)
Optimum action	Single state (non-associative)	Bandit	-
Optimum policy	Multiple state (associative)	Contextual Bandit	Reinforcement learning

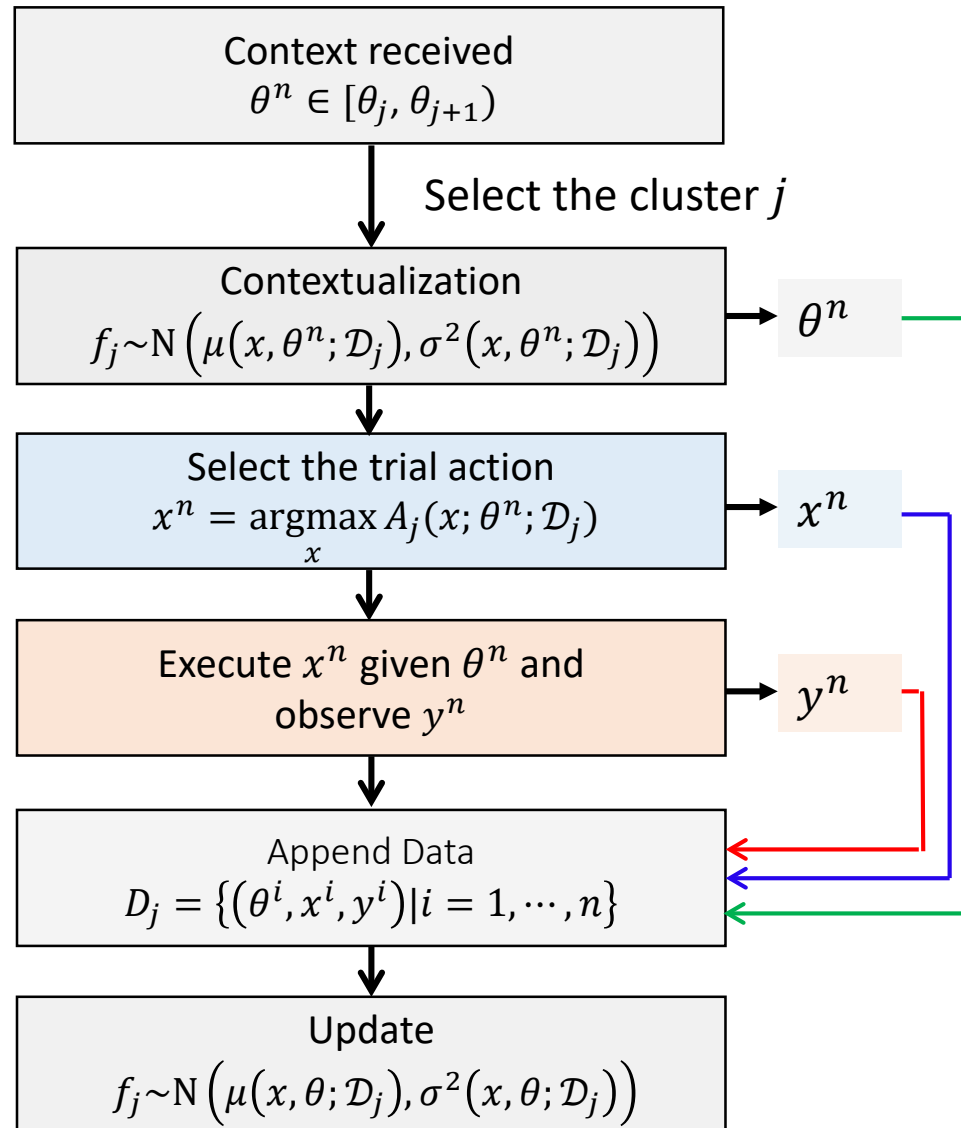
- Associative task involves both
 - ✓ trial-and-error learning in the form of searching for the best actions
 - ✓ association of these actions with the situations in which they are best.
- Contextual Bandit (associative search task) is intermediate between the n -armed bandit problem and full reinforcement learning problem.

Relationship with model based approach

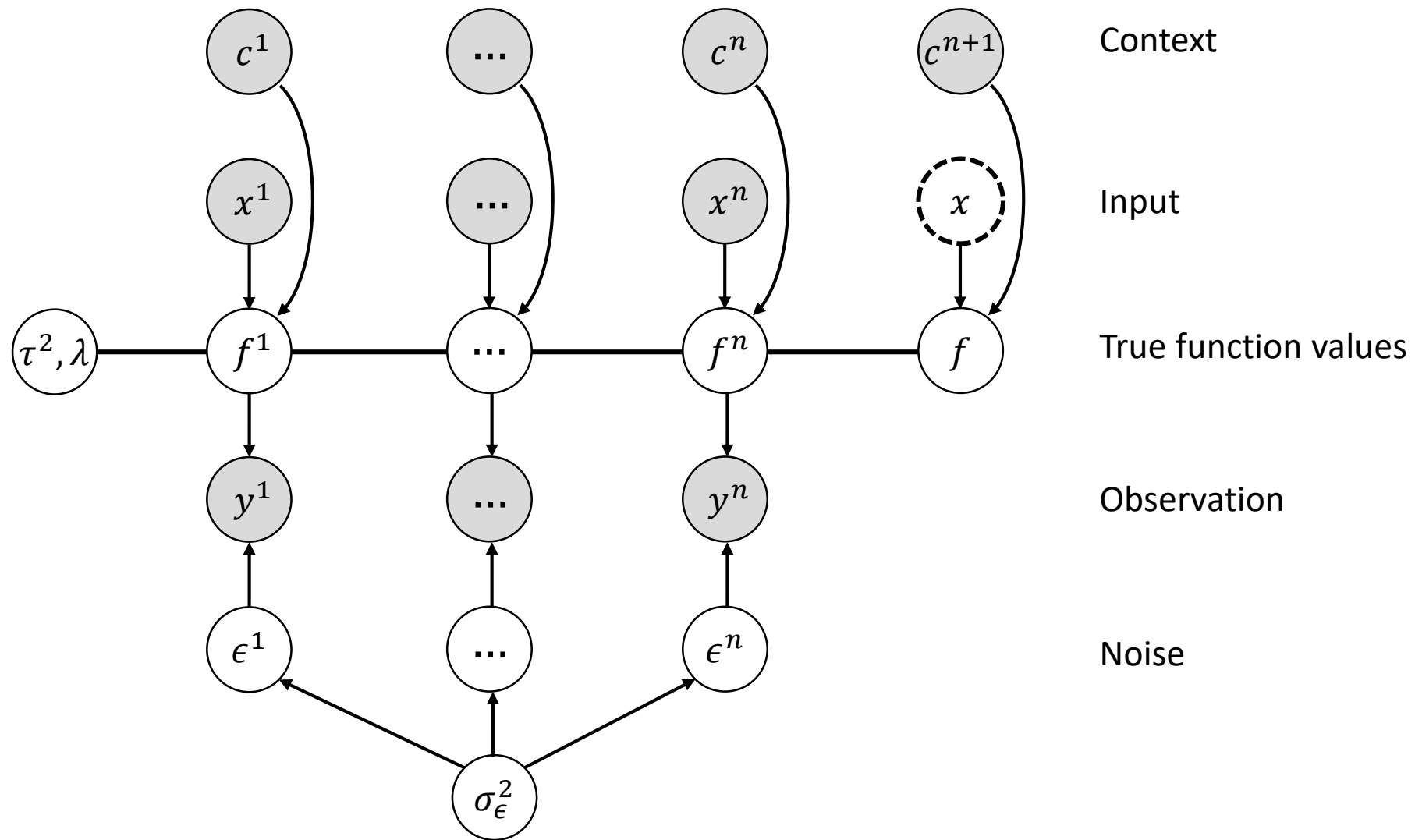
		Only Exploitation	Exploration vs Exploitation
		Model is known	Model is unknown
Optimum action	Single state	Optimization	Bandit
Optimum policy	Multiple state	Dynamic Programing	Contextual Bandit
			Reinforcement learning

- We're going to learn Dynamic Programming approach first, and move to Reinforcement learning.

Contextual Bayesian optimization procedure



Contextual Bayesian optimization



Contextual Bayesian optimization formulation

- The goal of CBO (Contextual Bayesian optimization) is to find the optimum policy π^* that maps the context input $c \in \mathbb{R}^m$ to the best input $x^* \in \mathbb{R}^n$ that maximizes $f(x, c)$ as

$$x^* = \pi^*(c) = \operatorname{argmax}_x f(x, c)$$

- Let's assume we have
 - t selected inputs $x_{1:t} = (x_1, \dots, x_t)$ corresponding to
 - t observed contexts $c_{1:t} = (c_1, \dots, c_t)$, and
 - t resultant observations $y_{1:t} = (y_1, \dots, y_t)$ with $y_i = f(x_i, c_i) + \epsilon_i$ for $i \in \{1, \dots, t\}$.

Contextual Bayesian optimization formulation (learning phase)

- **Gaussian process prior:** We use kernel for both input x and context c as

$$k((x, c), (x', c')) ; \theta = k_X(x, x'; \theta^X) \times k_C(c, c'; \theta^C)$$

Input: $k_X(x, x'; \theta^X) = \tau^X \exp\left(-\frac{1}{2}(x_i - x_j)^T \text{diag}(\lambda^X)^{-2}(x_i - x_j)\right)$

Context: $k_C(c, c'; \theta^C) = \tau^C \exp\left(-\frac{1}{2}(c_i - c_j)^T \text{diag}(\lambda^C)^{-2}(c_i - c_j)\right)$

- **Likelihood:** We assumed that the i th observation comes with i.i.d. Gaussian Noise, i.e., $y_i = f((x_i, c_i)) + \epsilon_i$, with $\epsilon_i \sim N(0, \sigma_\epsilon^2)$.

$$p(\mathbf{y}_{1:t} | \mathbf{f}_{1:t}, \sigma_\epsilon) = N(\mathbf{y}_{1:t} | \mathbf{f}_{1:t}, \sigma_\epsilon^2 \mathbf{I})$$

- **Posterior:** The posterior on target function value corresponding to the pair (c, x) is expressed as

$$p(f | \mathcal{D}_t, (c, x), \theta_t^*) = N(f | \mu(c, x), \sigma^2(c, x))$$

$$\begin{aligned} \mu(c, x) &= \mathbf{k}^T (\mathbf{K} + \sigma_\epsilon^2 \mathbf{I})^{-1} \mathbf{y}_{1:t} \\ \sigma^2(c, x) &= k((c, x), (c, x)) - \mathbf{k}^T (\mathbf{K} + \sigma_\epsilon^2 \mathbf{I})^{-1} \mathbf{k} \end{aligned}$$

Contextual Bayesian optimization formulation (optimization phase)

- In optimization phase, we select the next input by solving the following constrained optimization problem

$$x_{t+1} = \operatorname{argmax}_{x \in \operatorname{TR}(x^c)} A(f(x, c_{t+1}))$$

with $\operatorname{TR}(c_{t+1}) = \{x \mid \|x - x^c\|_2 \leq \rho_t \lambda_t^X / \|\lambda_t^X\|_2\}$.

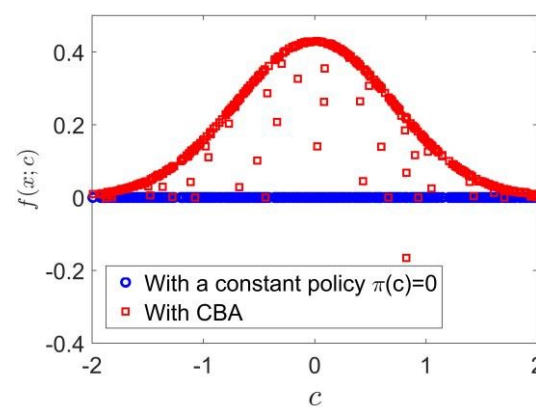
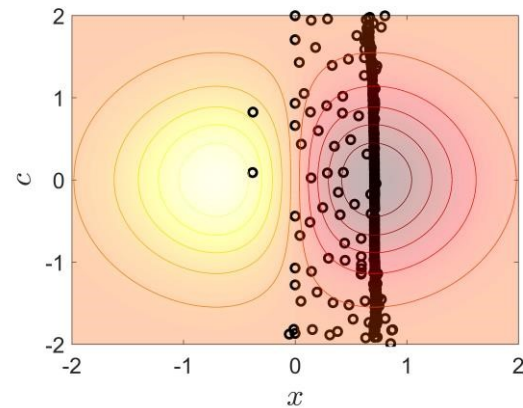
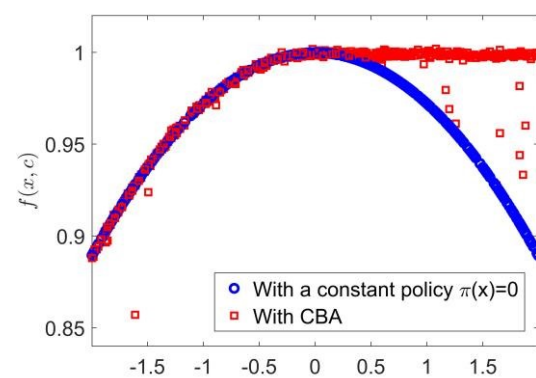
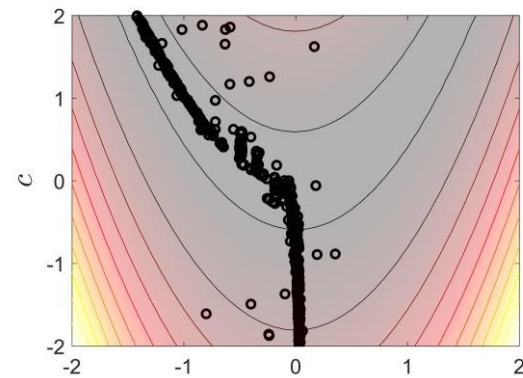
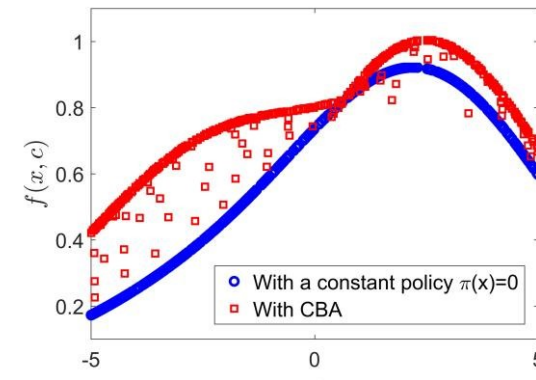
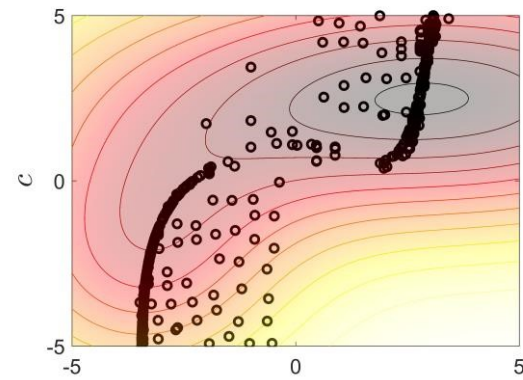
- The following two acquisition function can be used
 - Contextualized Upper Confidence Bound

$$\operatorname{UCB}(x, c_{t+1}) = \mu(x, c_{t+1}) + \kappa \sigma(x, c_{t+1})$$

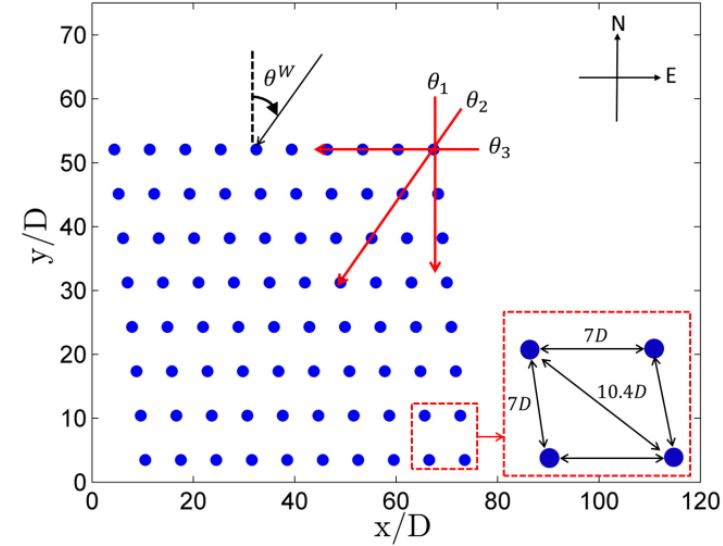
- Contextualized Expected Improvement

$$\operatorname{CEI}(x, c_{t+1}) = \mathbb{E}[\max(f(x, c_{t+1}) - \mu(x^c, c_{t+1}), 0)]$$

Contextual Bayesian optimization simulation



Contextual Bayesian optimization applications



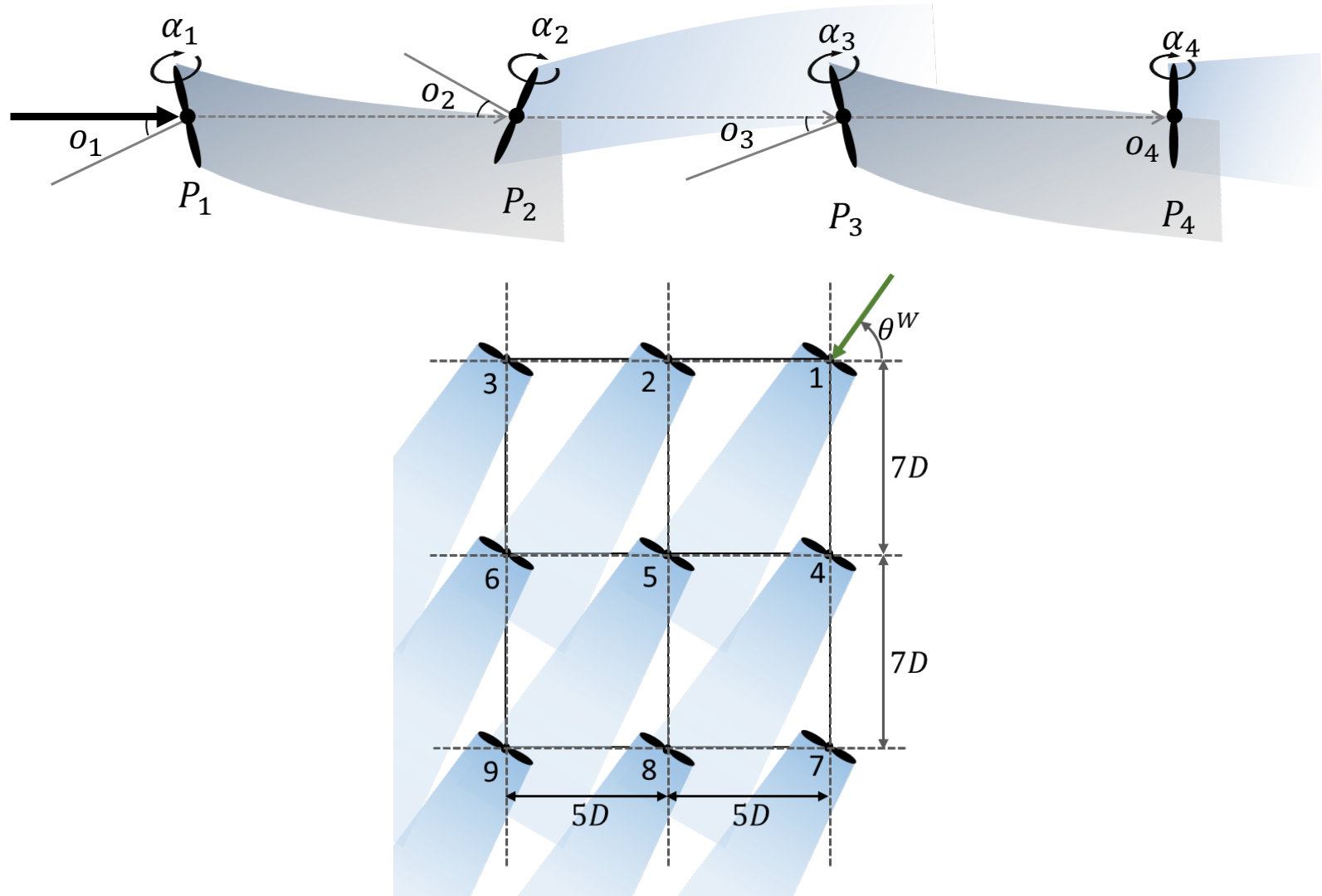
In a wind farm wind direction continuously changing.

What is the optimum control inputs at different wind conditions θ ?

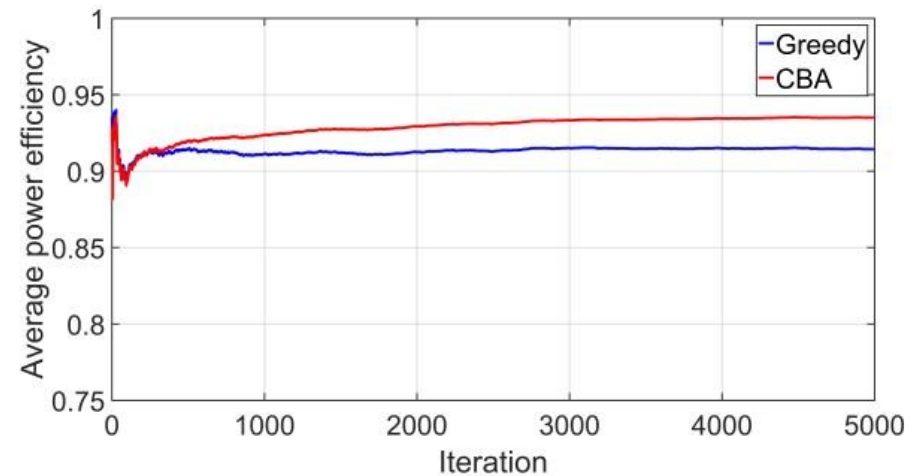
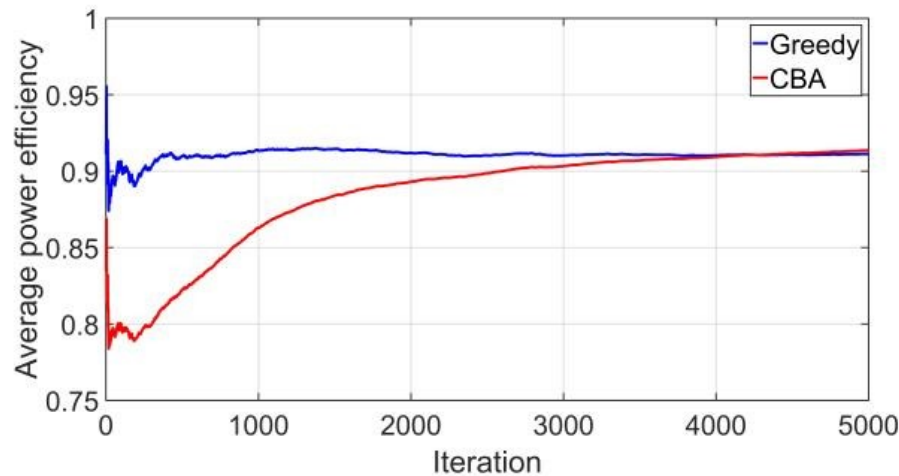
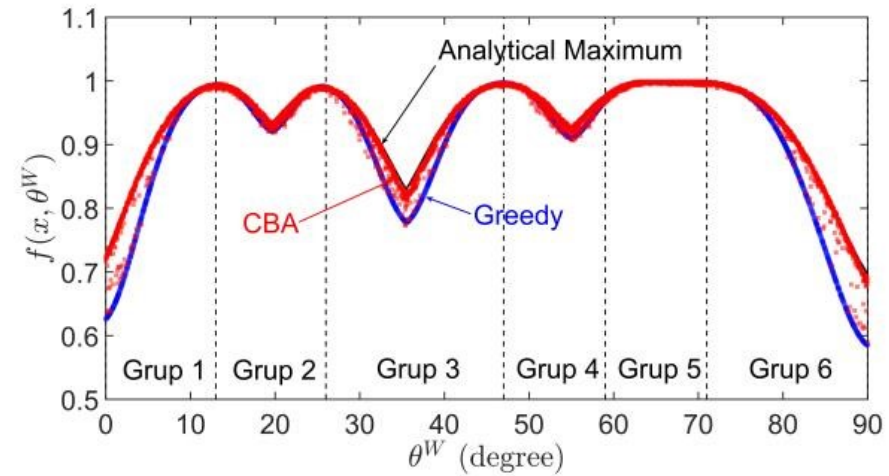
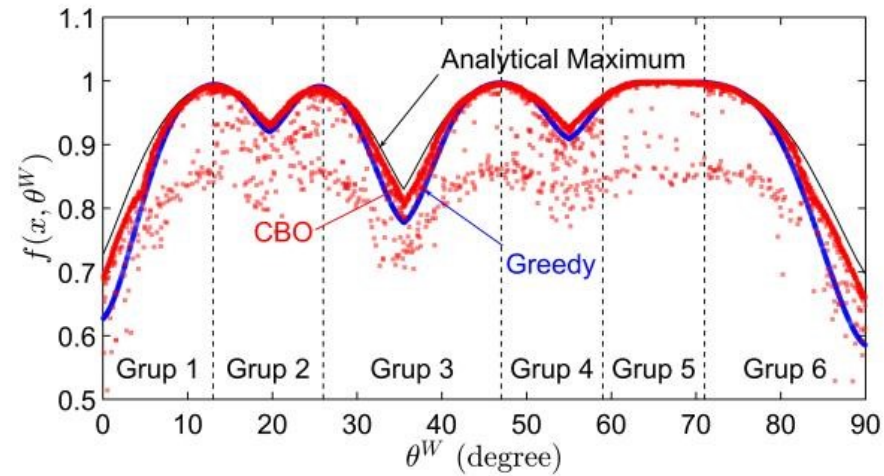
$$\mathbf{x}^* = \operatorname{argmax}_{\mathbf{x}} f(\mathbf{x}; \theta) \triangleq \sum_{i=1}^N P_i(\mathbf{x}; \theta)$$

$\mathbf{x}^* = \pi(\theta)$:Construct the optimum control policy

Contextual Bayesian optimization applications



Contextual Bayesian optimization applications



Introduction to GPflowOpt

GPflowOpt: A Bayesian Optimization Library using TensorFlow

Nicolas Knudde* Joachim van der Herten

Tom Dhaene Ivo Couckuyt[†]

Ghent University – imec

IDLab – Department of Information Technology

{nicolas.knudde, joachim.vanderherten, tom.dhaene, ivo.couckuyt}@ugent.be

- A novel Python framework for Bayesian optimization
- The package is based on the popular GPflow library for Gaussian processes, leveraging the benefits of TensorFlow including
 - automatic differentiation,
 - parallelization,
 - GPU computations for Bayesian optimization.

Introduction

- GPflowOpt is a Bayesian Optimization tool based on Gpflow.
- Gpflow is a tool that, like Gpy, creates Gaussian process models, with the following differences.
 - ✓ Gpflow, one of the machine learning platforms, is based on Tensorflow and allows for faster computation and handling of larger models.
 - ✓ In Gpflow, optimization can be achieved with fewer lines of code compared to Gpy because the computation of gradient are done using Tensorflow
 - ✓ Since Gpy does not have built-in functions for Bayesian optimization computation, GPflowOpt will be used for the practice.
- Specific functions of GPflowOpt:
<https://gpflowopt.readthedocs.io/en/latest/intro.html>

Introduction

- Acquisition functions supported by GPflowOpt

Single-objective	Expected Improvement (EI) (Moćkus, 1975) Lower Confidence Bound (LCB)(Srinivas et al., 2010) Max-Value Entropy Search (MES) (Wang and Jegelka, 2017) Probability of Improvement (PoI) (Kushner, 1964)
Multi-objective	Hypervolume-based PoI (HvPoI) (Couckuyt et al., 2014)
Constraint	Probability of Feasibility (PoF) (Schonlau, 1997)

- Comparison with Other Bo tools

	GPflowOpt	GPyOpt	Spearmint	BayesOpt	RoBO
Language	Python	Python	Python	C++	Python
Auto differentiation	✓	✗	✗	✗	✗
Multi-objective	✓	✗	✗	✗	✗
Code coverage	99%	56%	–	–	46%
Batch BO	✗	✓	✗	✗	✗
GPU support	✓	✗	✗	✗	✗

How to install GPflowOpt?

- Step1. Install git

<https://git-scm.com/download/win>

- Step2. Enter the following command

```
pip install --user tensorflow
```

```
pip install --user gpflowopt
```

```
pip install --user pip==18.1
```

```
pip install --user git+https://github.com/GPflow/GPflowOpt.git --process-dependency-link
```

GpflowOpt Examples: 1. Bayesian optimization basic

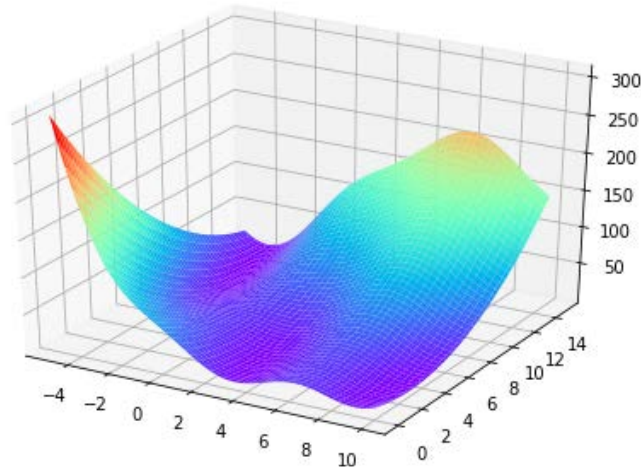
Optimize branin function

$$f(x_1, x_2) = a(x_2 - bx_1^2 + cx_1 - r)^2 + s(1 - t) \cos(x_1) + s$$

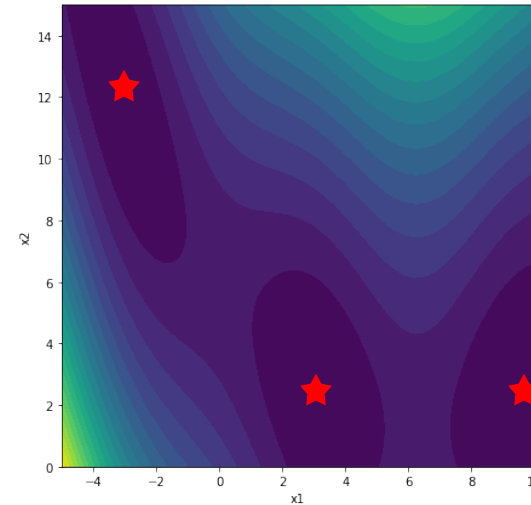
$$a = 1, b = \frac{5.1}{4\pi^2}, c = \frac{5}{\pi}, r = 6, s = 10, t = \frac{1}{8}\pi$$

$$x_1 \in [-5, 10], x_2 \in [0.15]$$

branin function



2D plot



Global Minimum: $f(x^*) = 0.3978$, at $x^* = (-\pi, 12.275), (\pi, 2.275), (9.42, 2.4)$

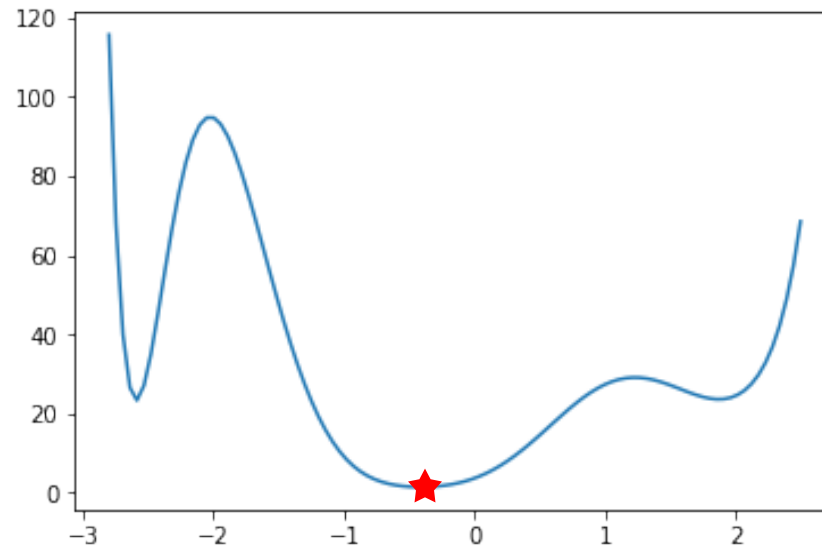
GpflowOpt Examples: 1. Bayesian optimization basic

Optimize polynomial function

$$f(x) = \sum_{n=1}^{11} c_n x^n,$$

$$c = [2, -4, -28, 62, 122, -256, -196, 140, 392, 240, 72]$$

$$x \in [-2.8, 2.5]$$



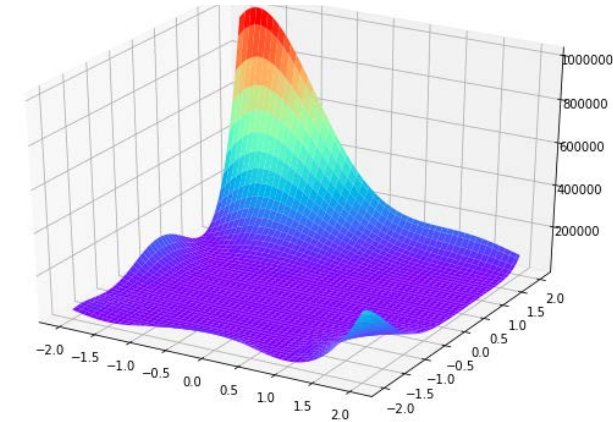
Global Minimum: $f(x^*) = 1.3855$, at $x^* = -0.4121$

GpflowOpt Examples: 2. BO with different acquisition functions

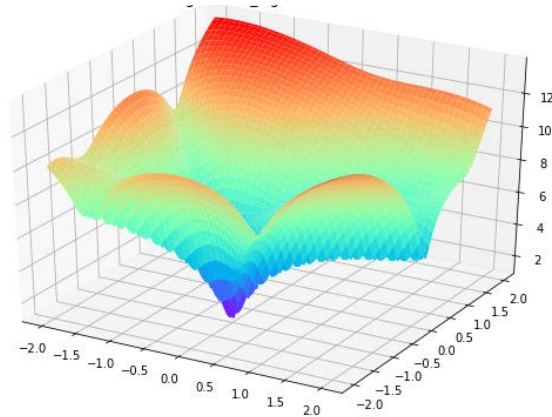
optimize goldstein function with various acquisition functions

$$f(\mathbf{x}) = [1 + (x_1 + x_2 + 1)^2(19 - 14x_1 + 3x_1^2 - 14x_2 + 6x_1x_2 + 3x_2^2)] \\ \times [30 + (2x_1 - 3x_2)^2(18 - 32x_1 + 12x_1^2 + 48x_2 - 36x_1x_2 + 27x_2^2)]$$

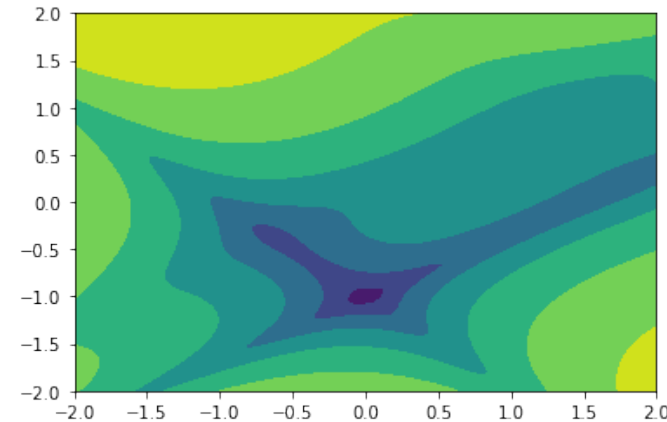
Goldstein function



Log Goldstein function



2D plot



Global Minimum: $f(x^*) = 3$, at $x^* = (0, -1)$

Log scale: $f(x^*) = 1.09$

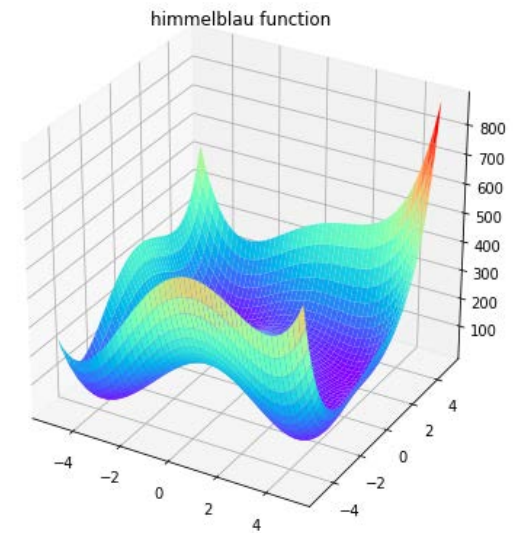
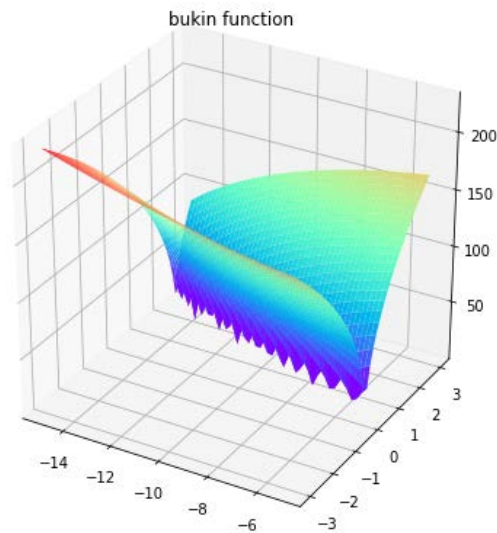
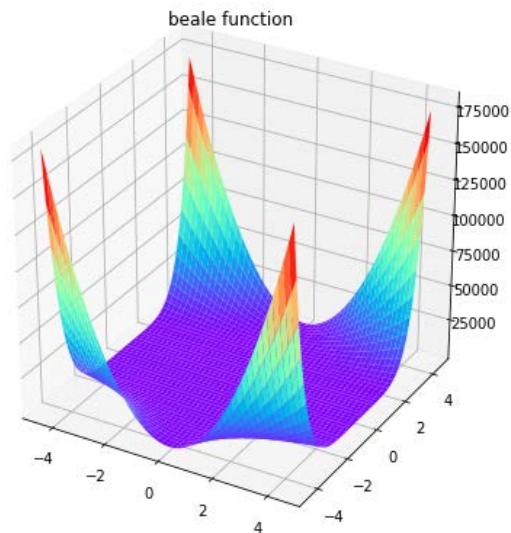
GpflowOpt Examples: 2. BO with different acquisition functions

Choose one of your favorite functions:

1. Beale function
2. Bukin function N.6
3. Himmelblau's function

https://en.wikipedia.org/wiki/Test_functions_for_optimization

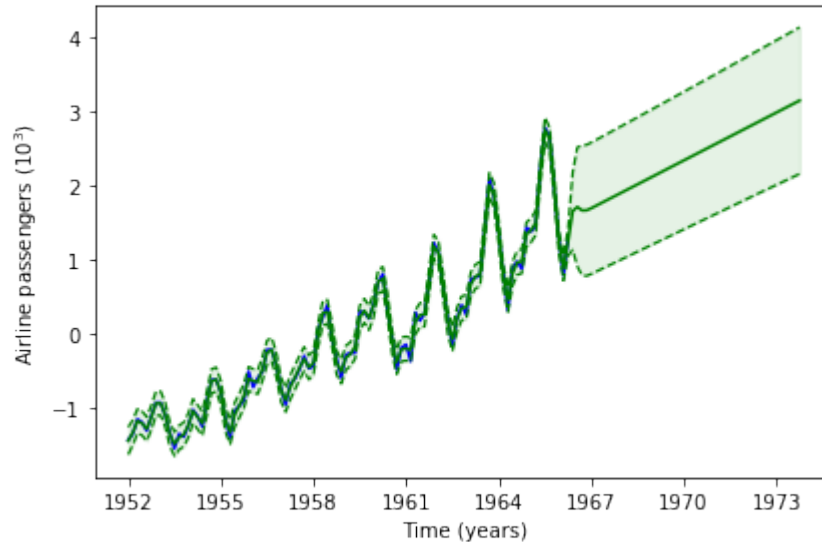
And compare BO results with PI / EI



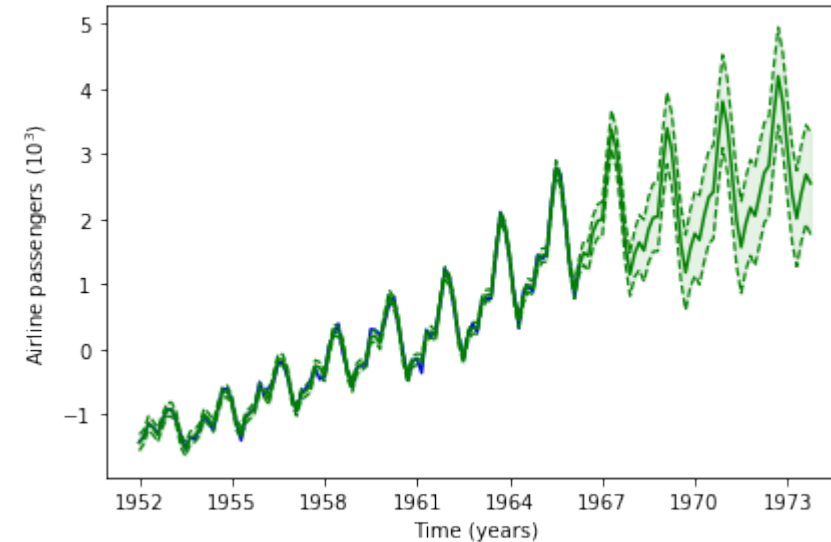
GpflowOpt Examples: 3. Bayesian optimization of hyper-parameters

Consider Airline passengers dataset

Prediction with **initial hyperparameters**



Prediction **with optimized hyperparameters**



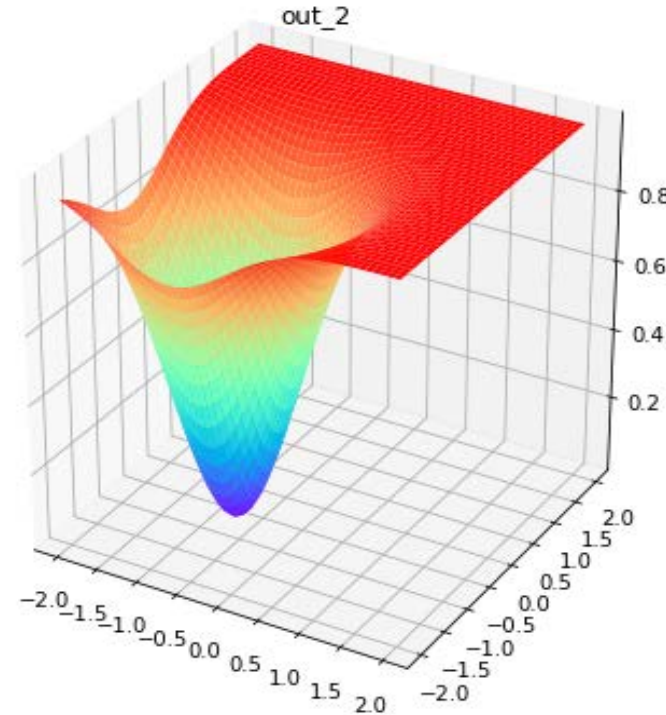
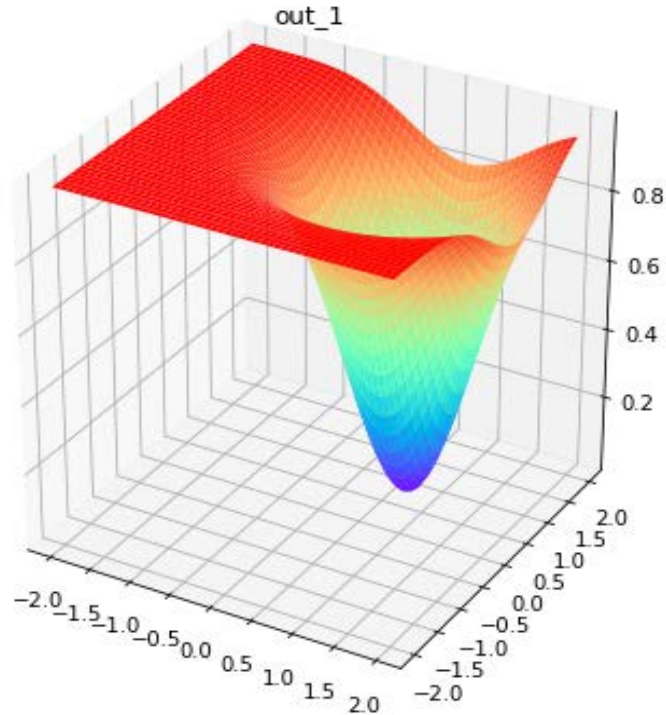
We will optimize the **initial values for the length-scales** of the RBF and the Cosine kernel

Practical Bayesian Optimization of Machine Learning algorithms, Snoek et al. 2012

<http://sanghyukchun.github.io/99/#gp-with-noisy-data>

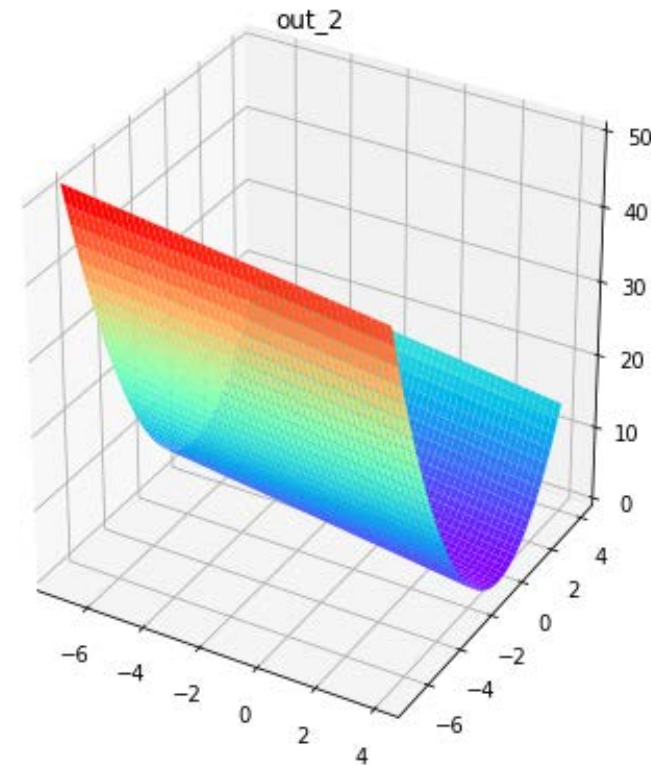
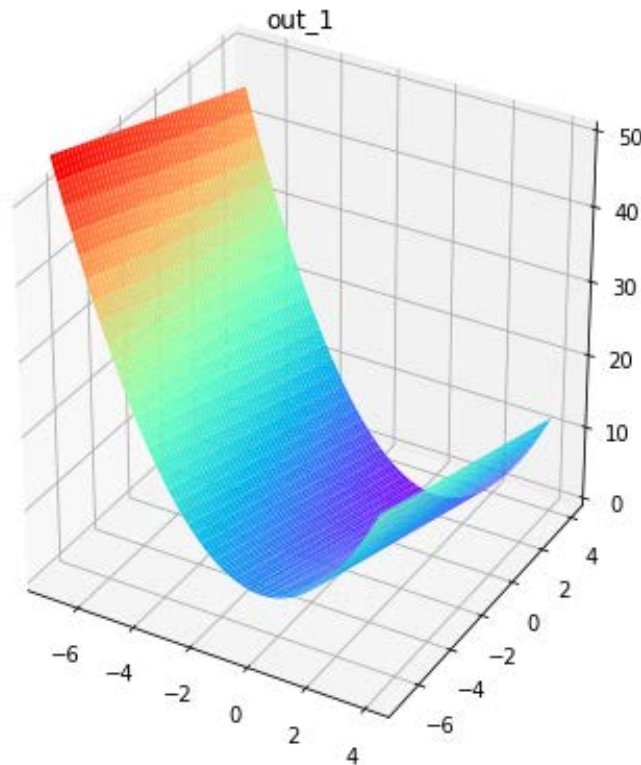
GpflowOpt Examples: 4. Multi-objective Bayesian optimization

Optimization of multi objective model using HVI acquisition function



GpflowOpt Examples: 4. Multi-objective Bayesian optimization

We will optimize multi-objective function shown as:



Bayesian optimization with unknown constraints

Motivation

- BO has been widely used. However this framework has not been extended to the inequality-constrained optimization setting, particularly the setting in which **evaluating feasibility is just as expensive as evaluating the objective**
- Optimize objective function while satisfying a certain constraints

$$\begin{aligned} & \max_x f(x) \\ \text{s. t. } & h_i(x) < a_i, \quad \text{for } i \in \{1, \dots, n\} \end{aligned}$$

- For example consider the following cases:
 - ✓ The goal of optimizing these hyper-parameters is to minimize test time, while constraining test accuracy (a parameter setting is only feasible if it achieves the same accuracy as the exact model)
 - ✓ In the experimental design setting, one may wish to maximize the yield of a chemical process, subject to the constraint that the amount of some unwanted byproduct produced is below a specific threshold.
 - ✓ In computer micro-architecture, fine-tuning the particular specifications of a CPU needs to be carefully balanced to optimize CPU speed, while keeping the power usage strictly within a pre-specified budget.

Motivation

- Extend the Bayesian optimization framework naturally to scenarios of optimizing an **expensive-to-evaluate function** under equally **expensive-to-evaluate constraints**.
- Extend Bayesian Optimization to incorporate inequality constraints, allowing problems of the form

$$\max_{c(x) \leq \lambda} f(x)$$

- ✓ $f(x)$ and $c(x)$ are the results of some expensive experiments
- ✓ When conduct the experiment, $f(x)$ and $c(x)$ are measured together.

Formulation

- Expected improvement was defined as

$$\text{EI}(x; \theta, \mathcal{D}) = \mathbb{E}[\max\{0, f(x) - f(x^+)\} | \mathcal{D}] = \mathbb{E}[I(x) | \mathcal{D}]$$

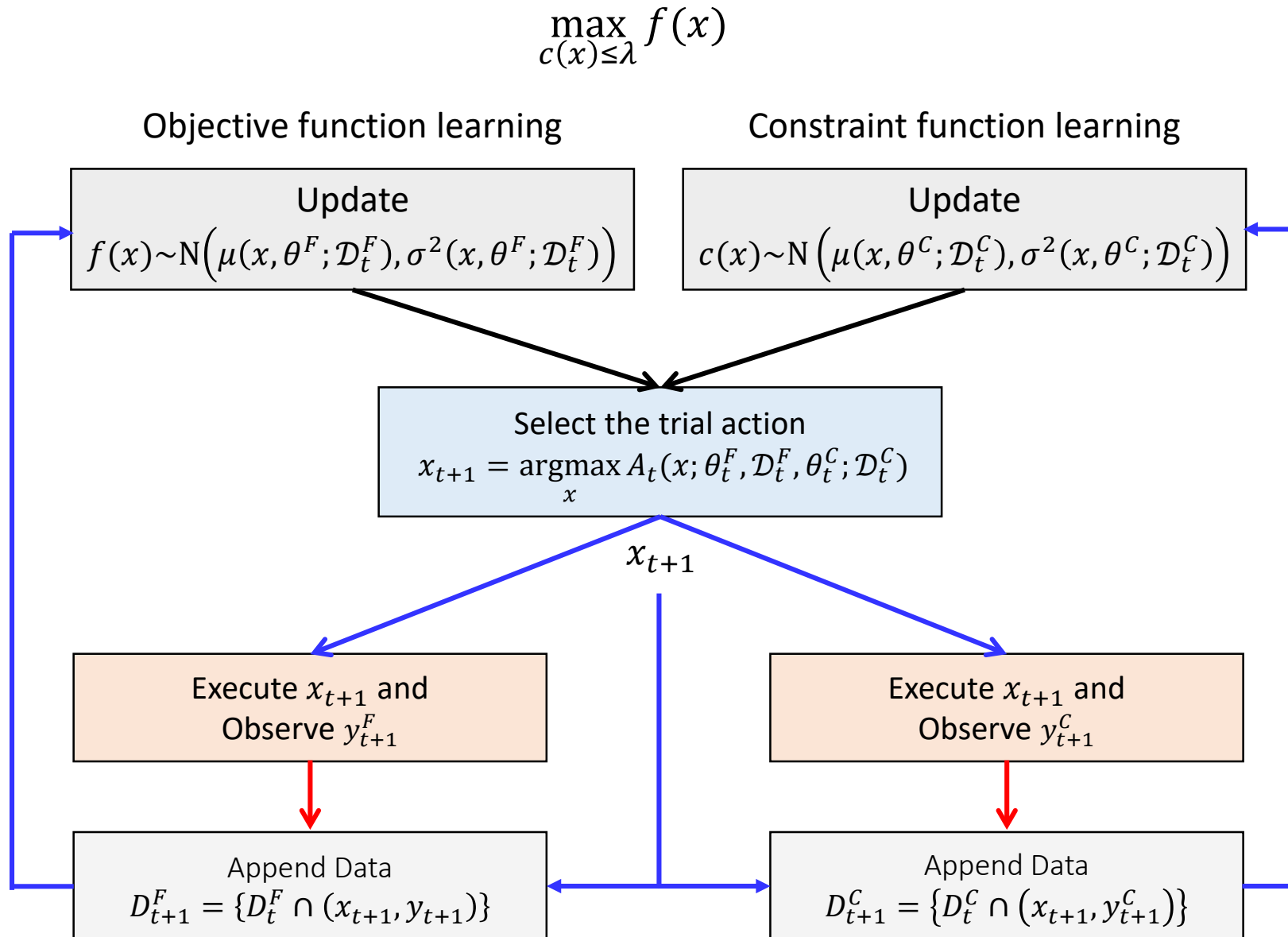
where $f(x^+)$ with $x^+ = \max_{x_i \in x_{1:t}} f(x_i)$ is the current estimated maximum, and $I(x) = \max\{0, f(x) - f(x^+)\}$ is the improvement function

- Adding inequality constraints to Bayesian optimization is most directly done via the EI acquisition function, which needs to be modified in two ways.
 - ✓ Augment definition of x^+ to be the feasible point with the maximum function value observed in the data set
 - ✓ Assign zero improvement to all infeasible point
- Constrained improvement function for a candidate x

$$I_c(x) = \Delta(x) \max\{0, f(x) - f(x^+)\} = \Delta(x) I(x)$$

where $\Delta(x) = \begin{cases} 1 & \text{if } c(x) \leq \lambda \\ 0 & \text{otherwise} \end{cases}$

Formulation



Formulation

$$f(x) \sim N(\mu(x, \theta^F; \mathcal{D}_t^F), \sigma^2(x, \theta^F; \mathcal{D}_t^F))$$

$$c(x) \sim N(\mu(x, \theta^C; \mathcal{D}_t^C), \sigma^2(x, \theta^C; \mathcal{D}_t^C))$$

- Constrained improvement function for a candidate x

$$I_C(x) = \Delta(x) \max\{0, f(x) - f(x^+)\} = \Delta(x) I(x)$$

where $\Delta(x) \in \{0, 1\}$ can be modeled as Bernoulli random variable with parameter

$$\mathbb{E}[\Delta(x)] = PF(x) = p(c(x) \leq \lambda) = \int_{-\infty}^{\lambda} p(c(x) | \mathcal{D}_t^C) dc(x)$$

- Because $c(x)$ is Gaussian modeled by GP, the quantity $PF(x)$ is a simple univariate Gaussian cumulative distribution function.

$Y \sim B(p)$: Bernoulli random variable

$$p_Y(Y = y) = p(y) = B(y|p) = p^y (1 - p)^{1-y}, \quad \text{for } y \in \{0, 1\}$$

$p \in [0, 1]$: Probability of success

$$\mathbb{E}[Y] = \sum_y yp(y) = 1p + 0(1 - p) = p$$

Formulation

$$f(x) \sim \mathcal{N}(\mu(x, \theta^F; \mathcal{D}_t^F), \sigma^2(x, \theta^F; \mathcal{D}_t^F))$$

$$c(x) \sim \mathcal{N}(\mu(x, \theta^C; \mathcal{D}_t^C), \sigma^2(x, \theta^C; \mathcal{D}_t^C))$$

- Constrained improvement function for a candidate x

$$I_C(x) = \Delta(x) \max\{0, f(x) - f(x^+)\} = \Delta(x) I(x)$$

where $\Delta(x) \in \{0, 1\}$ can be modeled as Bernoulli random variable with parameter

$$\mathbb{E}[\Delta(x)] = PF(x) = p(c(x) \leq \lambda) = \int_{-\infty}^{\lambda} p(c(x) | \mathcal{D}_t^C) dc(x)$$

➤ Because $c(x)$ is Gaussian modeled by GP, the quantity $PF(x)$ is a simple univariate Gaussian cumulative distribution function.

- Then, the expected constrained improvement can be computed as

$$\begin{aligned} EI_C(x) &= \mathbb{E}[I_C(x) | x, \mathcal{D}^F, \mathcal{D}^C] = \mathbb{E}[\Delta(x) I_C(x) | x, \mathcal{D}^F, \mathcal{D}^C] \\ &= \mathbb{E}[\Delta(x) | x, \mathcal{D}^C] \mathbb{E}[I_C(x) | x, \mathcal{D}^F] = PF(x) EI(x) \end{aligned}$$

where the third equality comes from the conditional independence of $c(x)$ and $f(x)$, given x .

Formulation for multiple inequality constraints

$$\begin{aligned} & \max_x f(x) \\ \text{s. t. } & h_i(x) < a_i, \quad \text{for } i \in \{1, \dots, n\} \end{aligned}$$

- Then, $\Delta(x) \in \{0, 1\}$ can be modeled as Bernoulli random variable with parameter

$$\begin{aligned} \mathbb{E}[\Delta(x)] &= PF(x) = p(\mathbf{c}(x) \leq \boldsymbol{\lambda}) \\ &= p(c_1(x) \leq \lambda_1, \dots, c_m(x) \leq \lambda_m) \\ &\approx \prod_{i=1}^m p(c_i(x) \leq \lambda_i) \end{aligned}$$

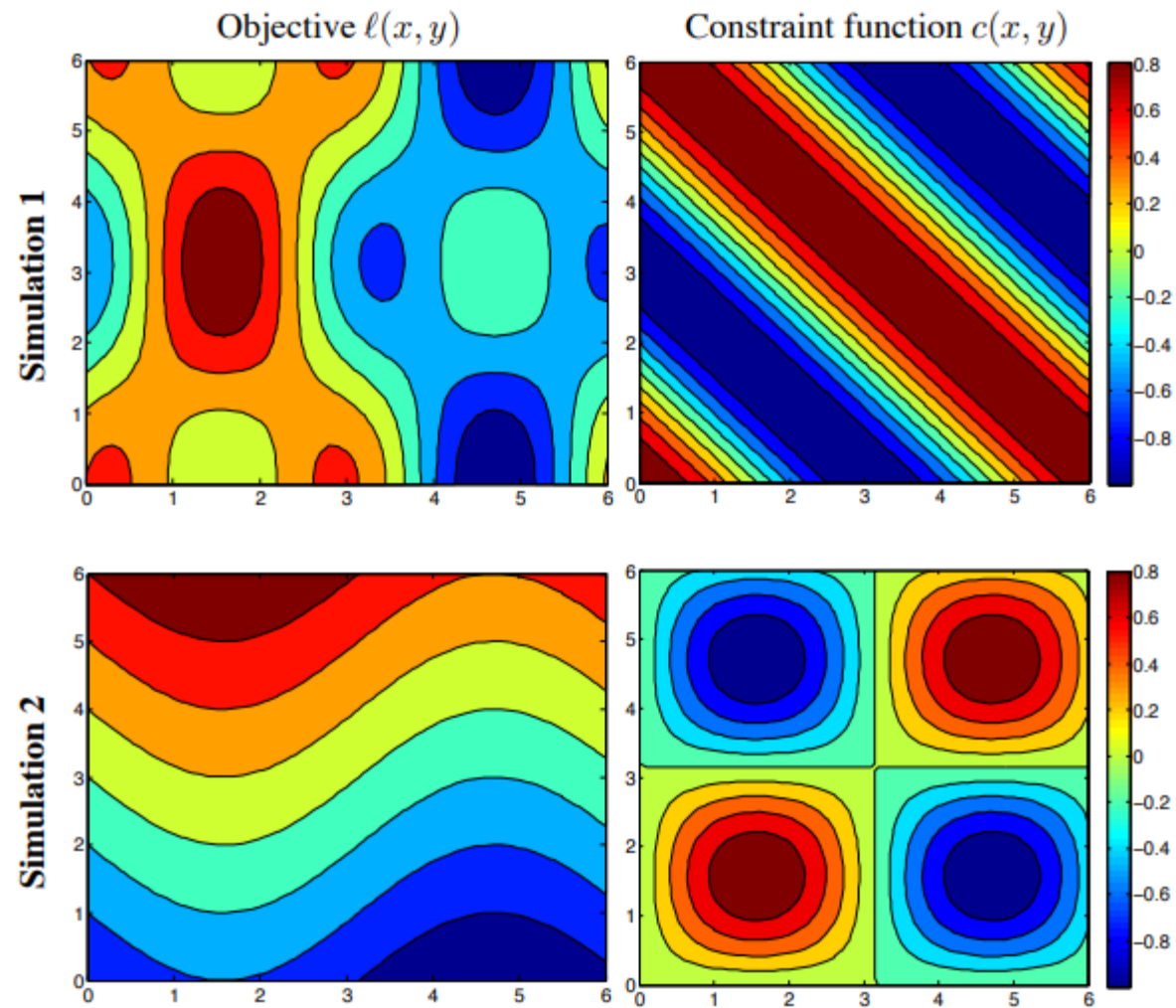
With

$$p(c_i(x) \leq \lambda_i) = \int_{-\infty}^{\lambda_i} p(c(x) | \mathcal{D}_t^C) dc(x)$$

- Then the expected constrained improvement can be computed as

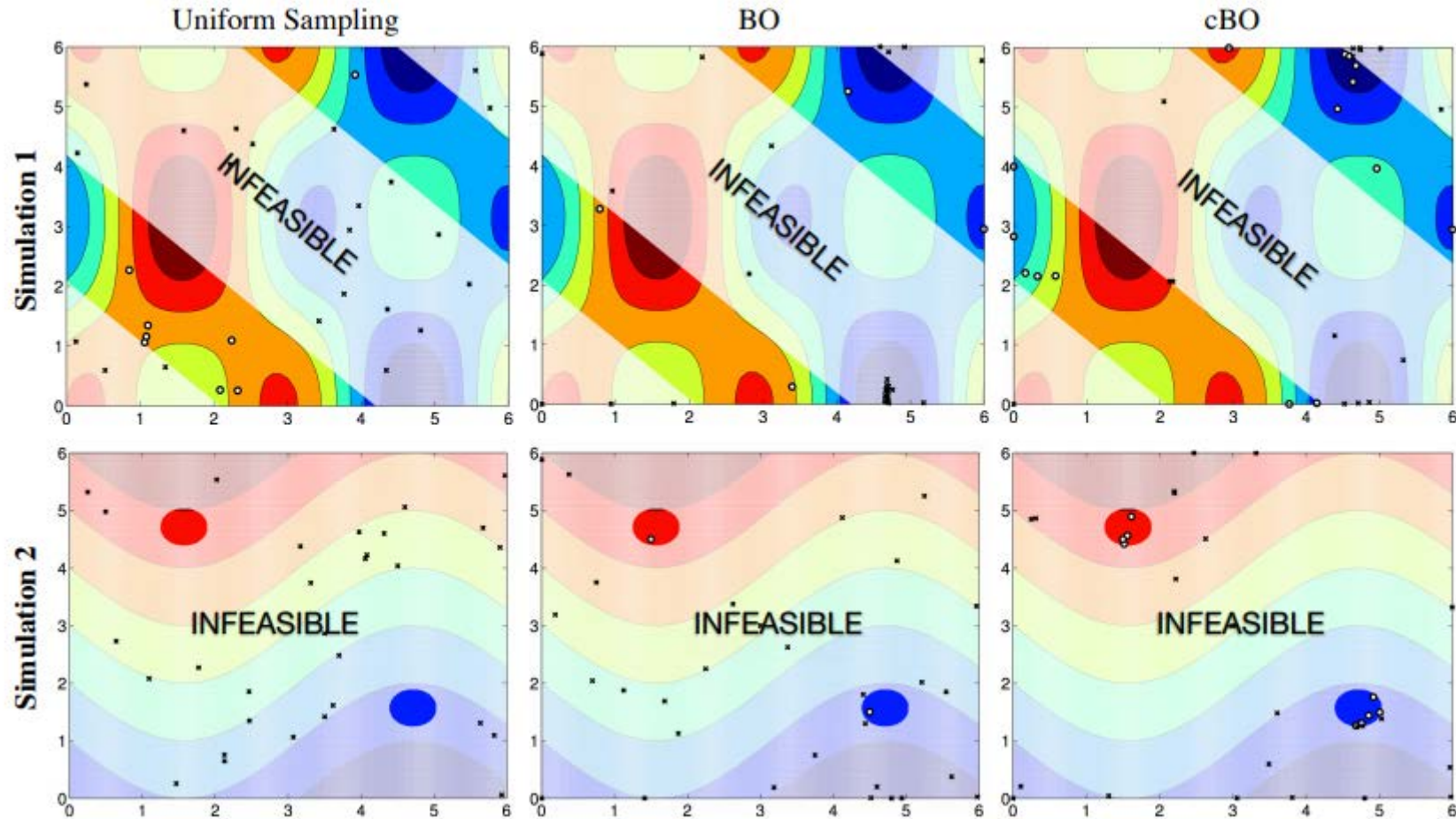
$$\begin{aligned} EI_C(x) &= E[I_C(x) | x, \mathcal{D}^F, \mathcal{D}^C] \\ &= E[\Delta(x) I_C(x) | x, \mathcal{D}^F, \mathcal{D}^C] \\ &= PF(x) EI(x) \end{aligned}$$

Simulation results



(Gardner et al., 2014, “Bayesian Optimization with Inequality Constraints”)

Simulation results

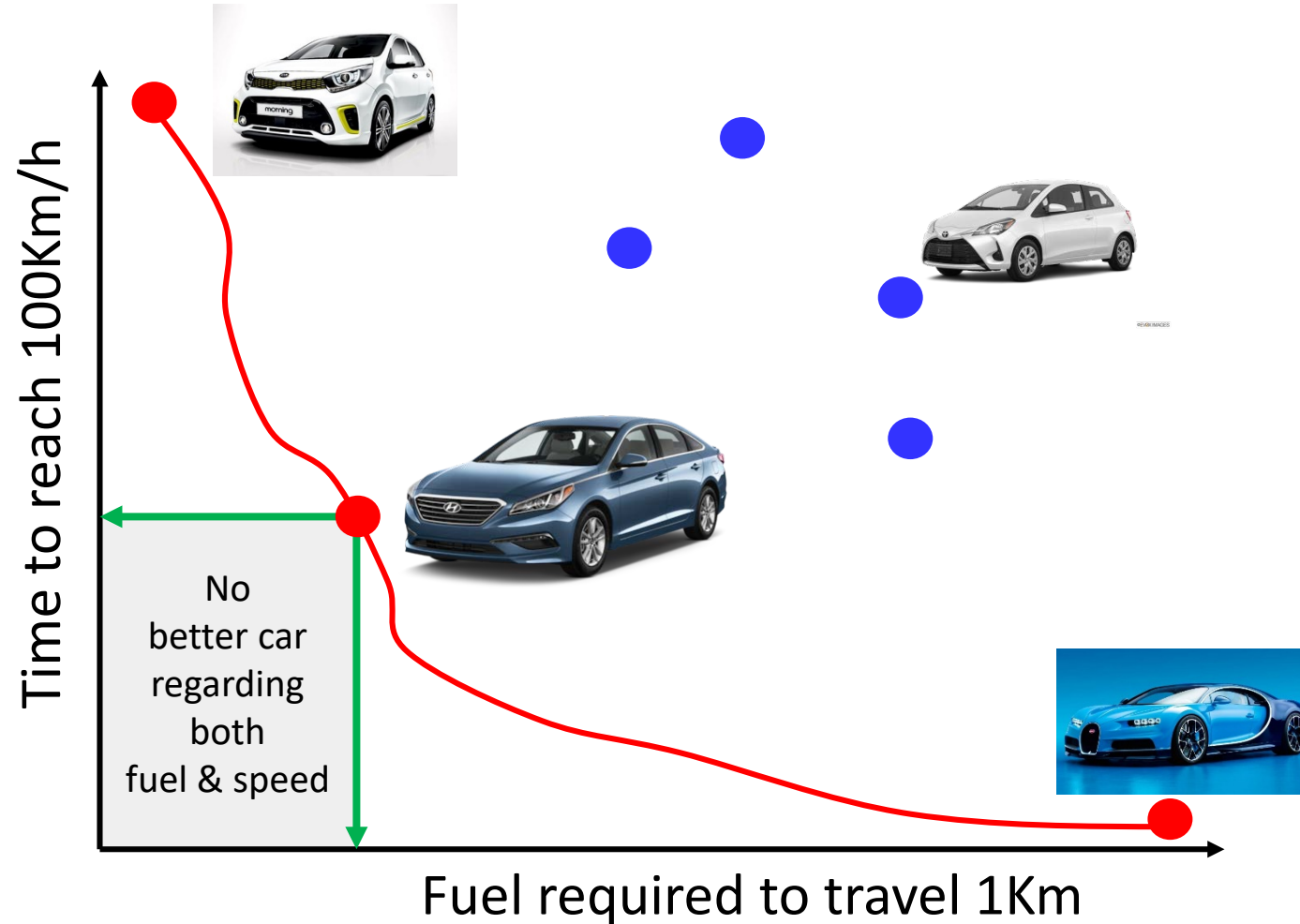


(Gardner et al., 2014, “Bayesian Optimization with Inequality Constraints”)

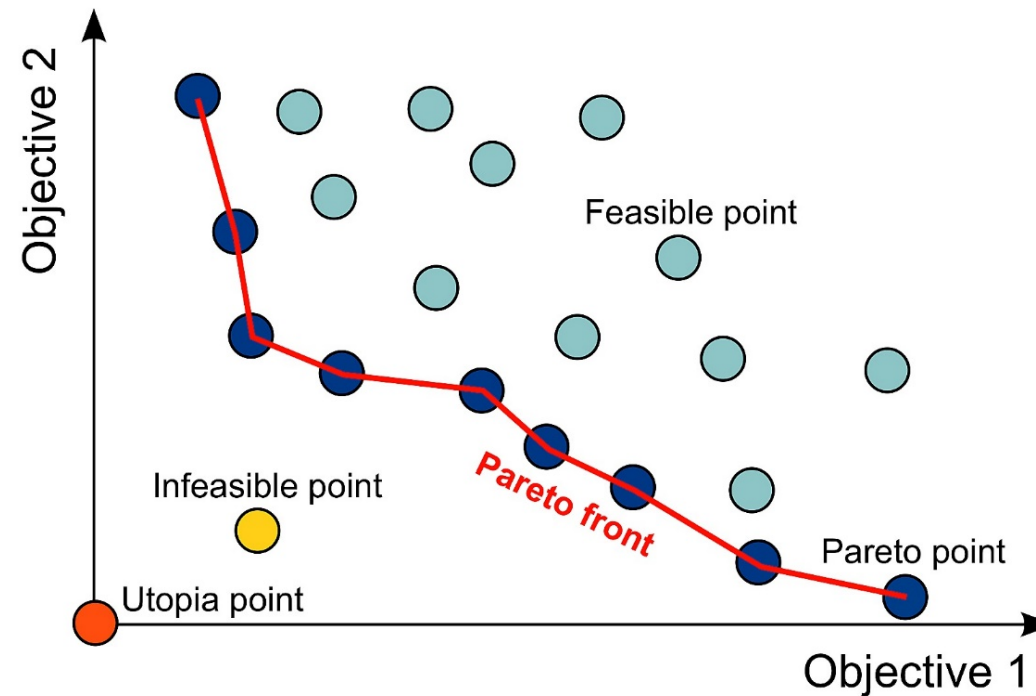
Multi-output Bayesian optimization (MOBO)

Motivation

- Develop the fastest car in the world and the most fuel-economic car in the world



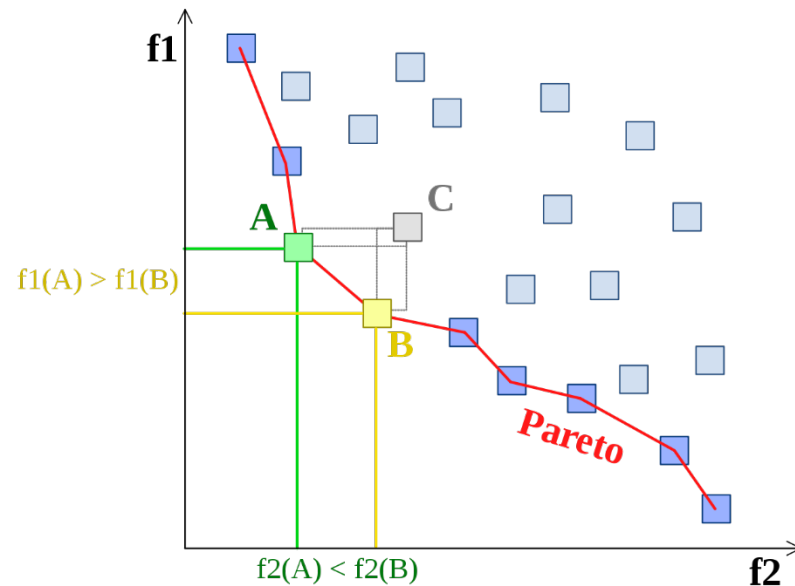
Pareto Frontier



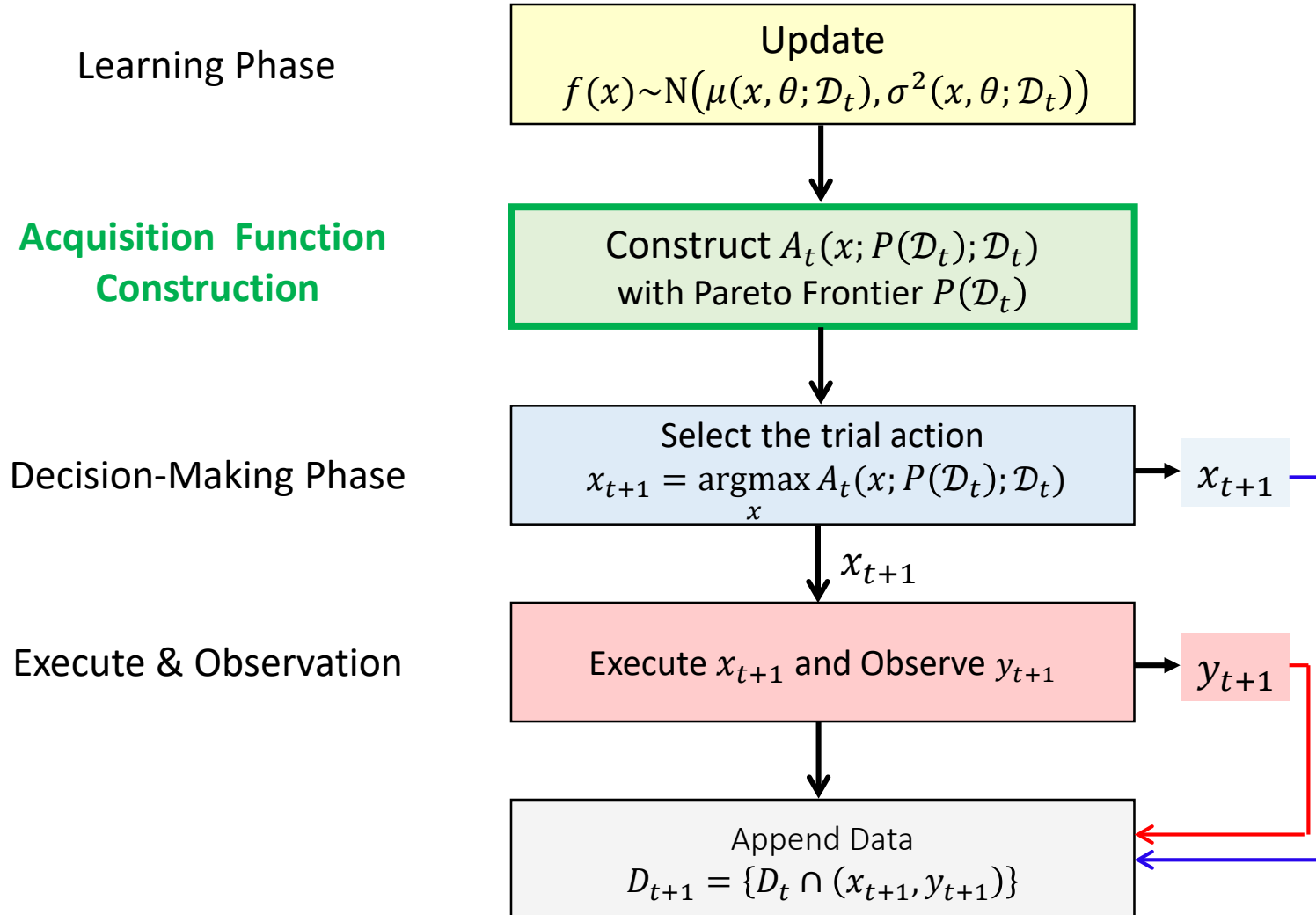
- **Pareto efficiency** or **Pareto optimality** is a state of allocation of resources from which it is impossible to reallocate so as to make any one individual or preference criterion better off without making at least one individual or preference criterion worse off.
- **Pareto frontier** is the set of all Pareto efficient allocations. It also is variously known as the **Pareto front** or **Pareto set**

Pareto equilibrium

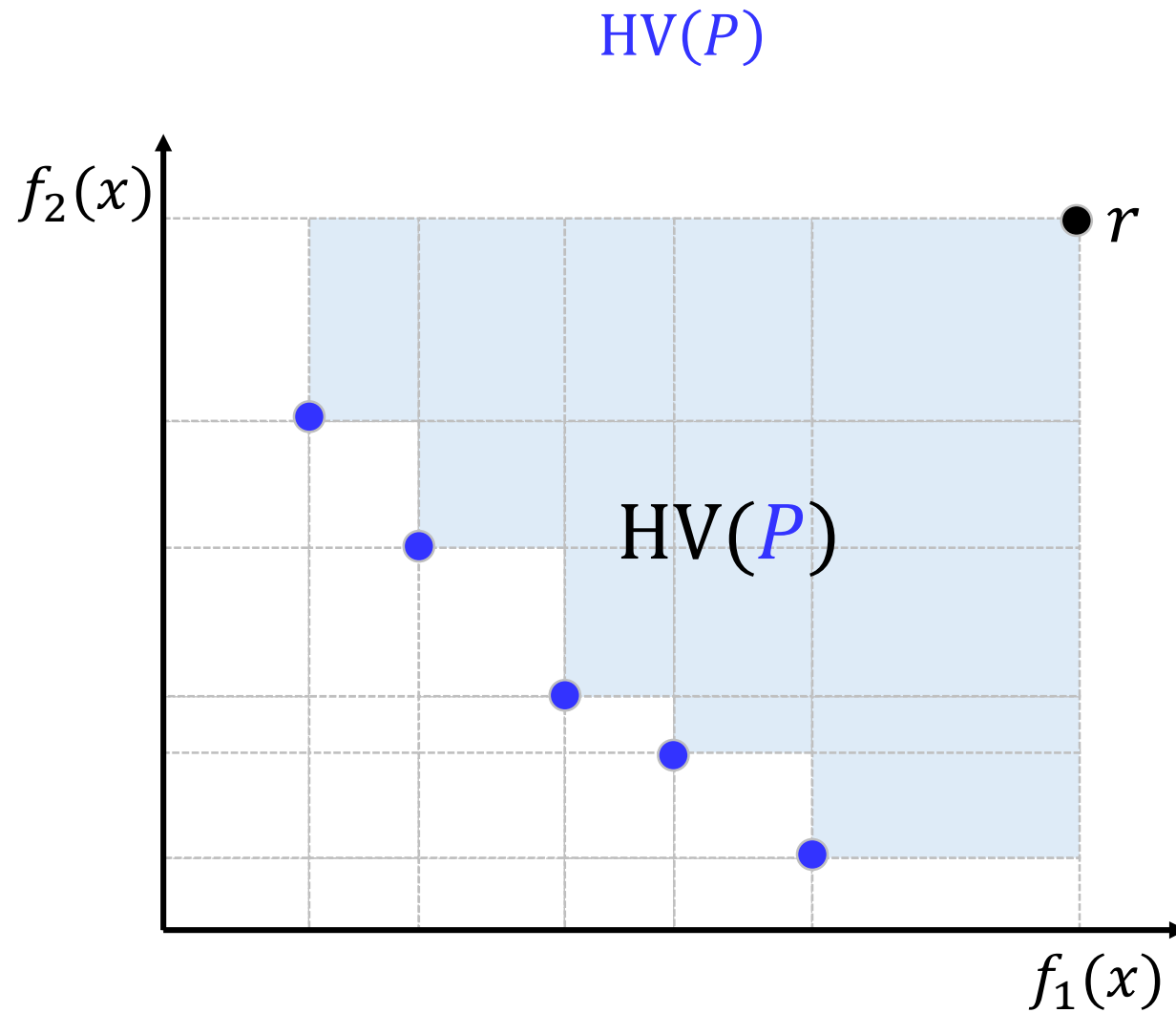
- Consider a system with function $f: \mathbb{R}^n \rightarrow \mathbb{R}^m$
 - X is a compact set of feasible decisions in the metric space $\mathbb{R}^n$
 - Y is the feasible set of criterion vectors in $\mathbb{R}^m$, such that
$$Y = \{y \in \mathbb{R}^m : y = f(x), x \in X\}$$
- A point $y'' \in \mathbb{R}^m$ is preferred to (strictly dominates) another point $y' \in \mathbb{R}^m$, written as $y'' \succ y'$.
- The Pareto frontier is thus expressed as
$$P(Y) = \{y' \in Y : \{y'' \in Y : y'' \succ y', y'' \neq y'\} = \emptyset\}$$



MOBO formulation & procedure

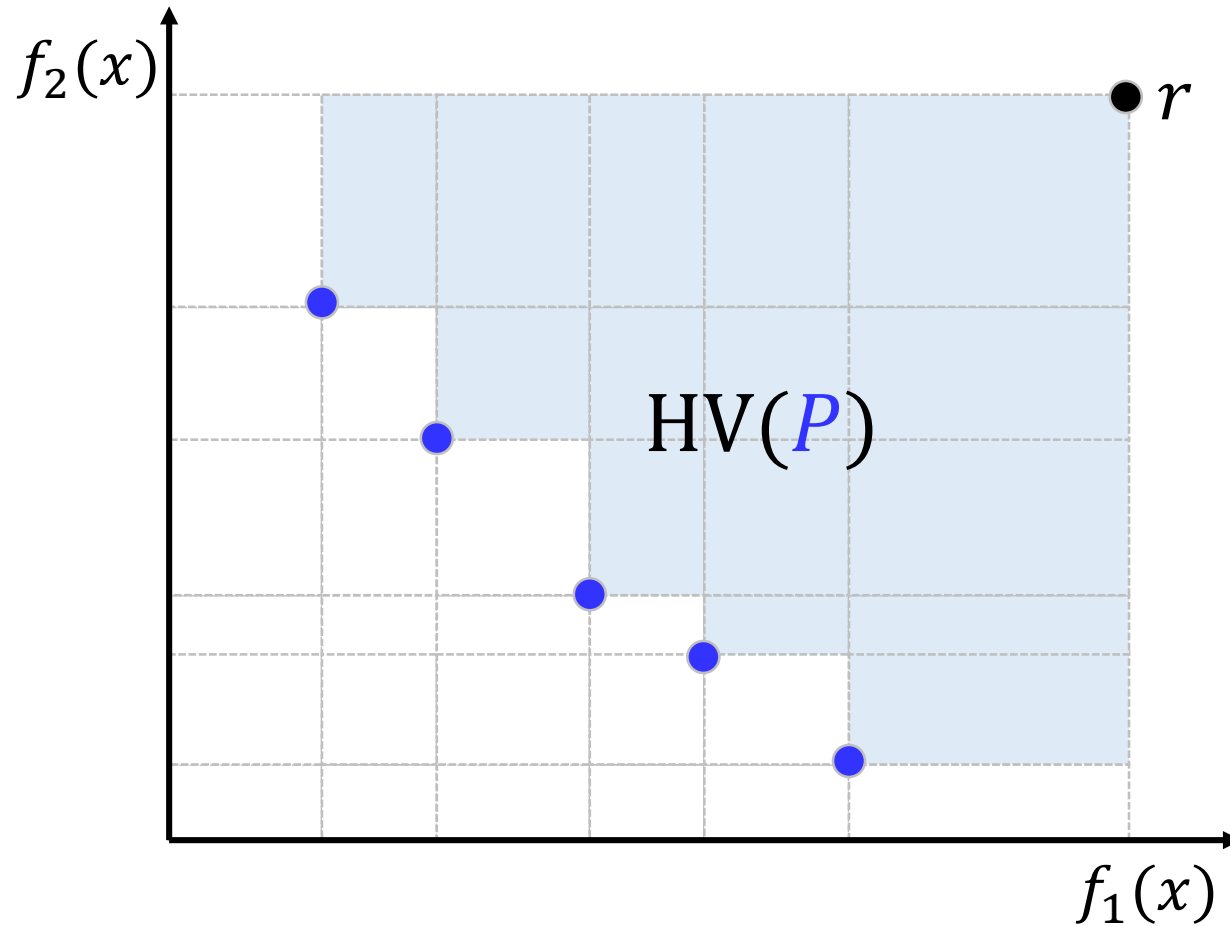


Defining acquisition function: Hypervolume function (HV)



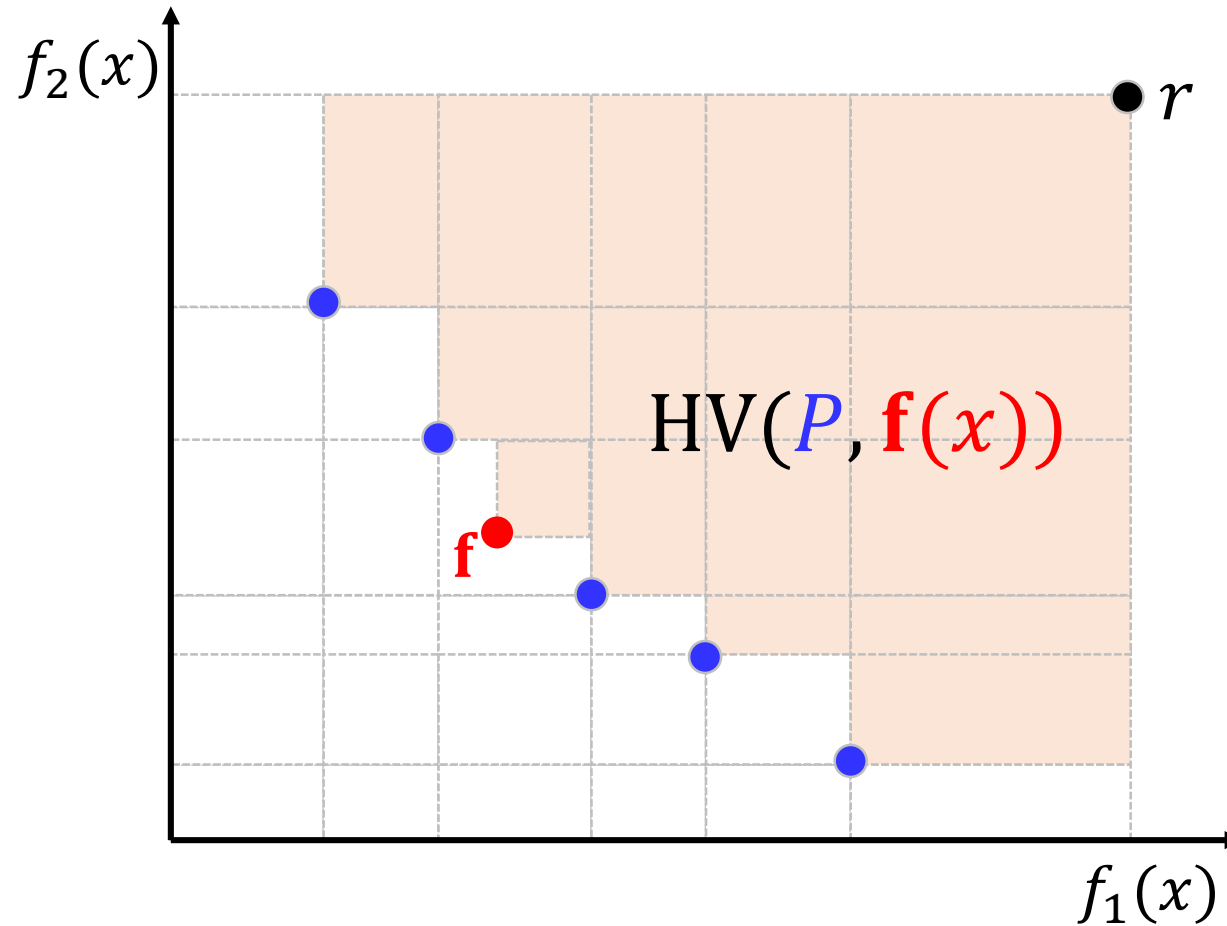
Defining acquisition function: Hypervolume improvement (HVI)

$$\text{HVI}(P, \mathbf{f}(x)) = \text{HV}(P, \mathbf{f}(x)) - \text{HV}(P)$$



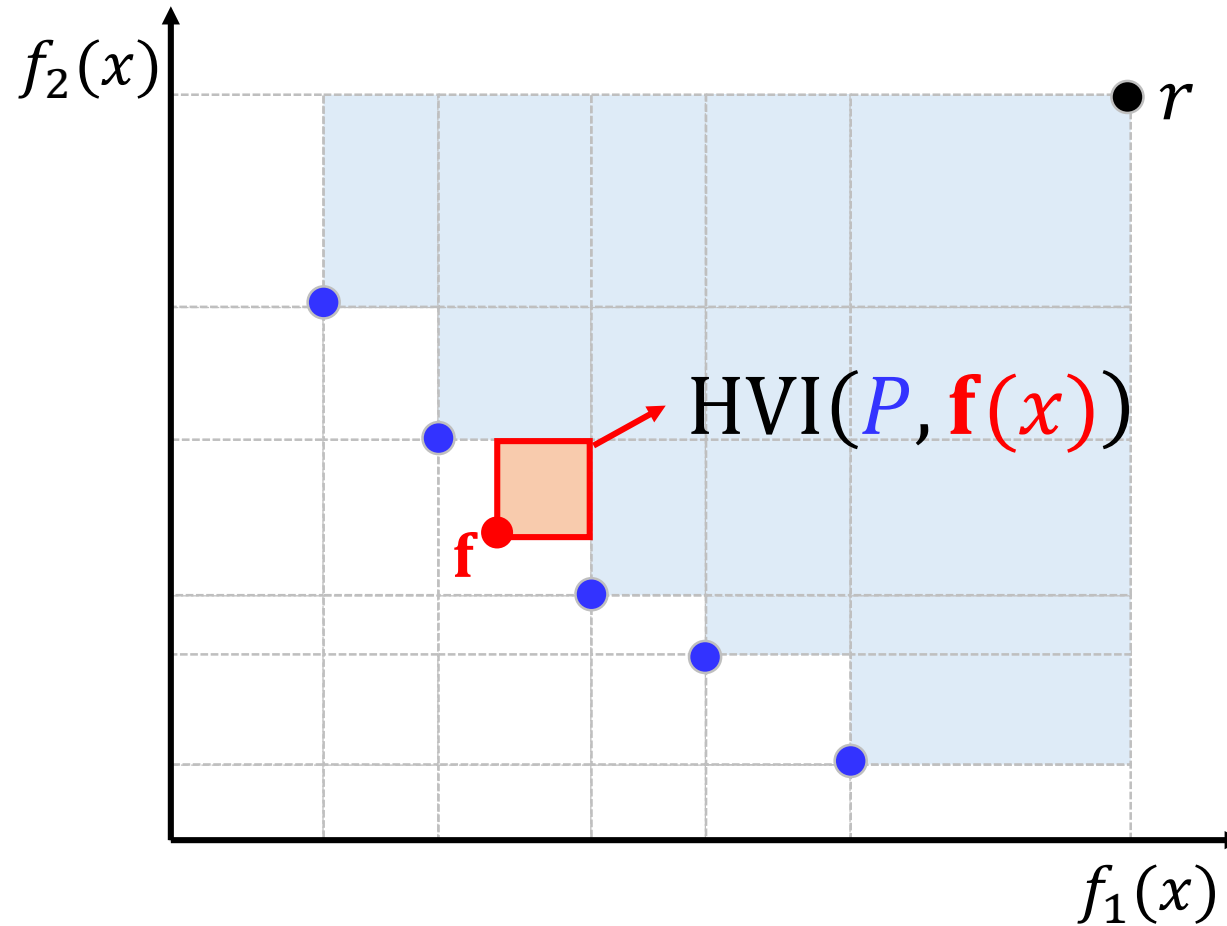
Defining acquisition function: Hypervolume improvement (HVI)

$$\text{HVI}(P, \mathbf{f}(x)) = \text{HV}(P, \mathbf{f}(x)) - \text{HV}(P)$$



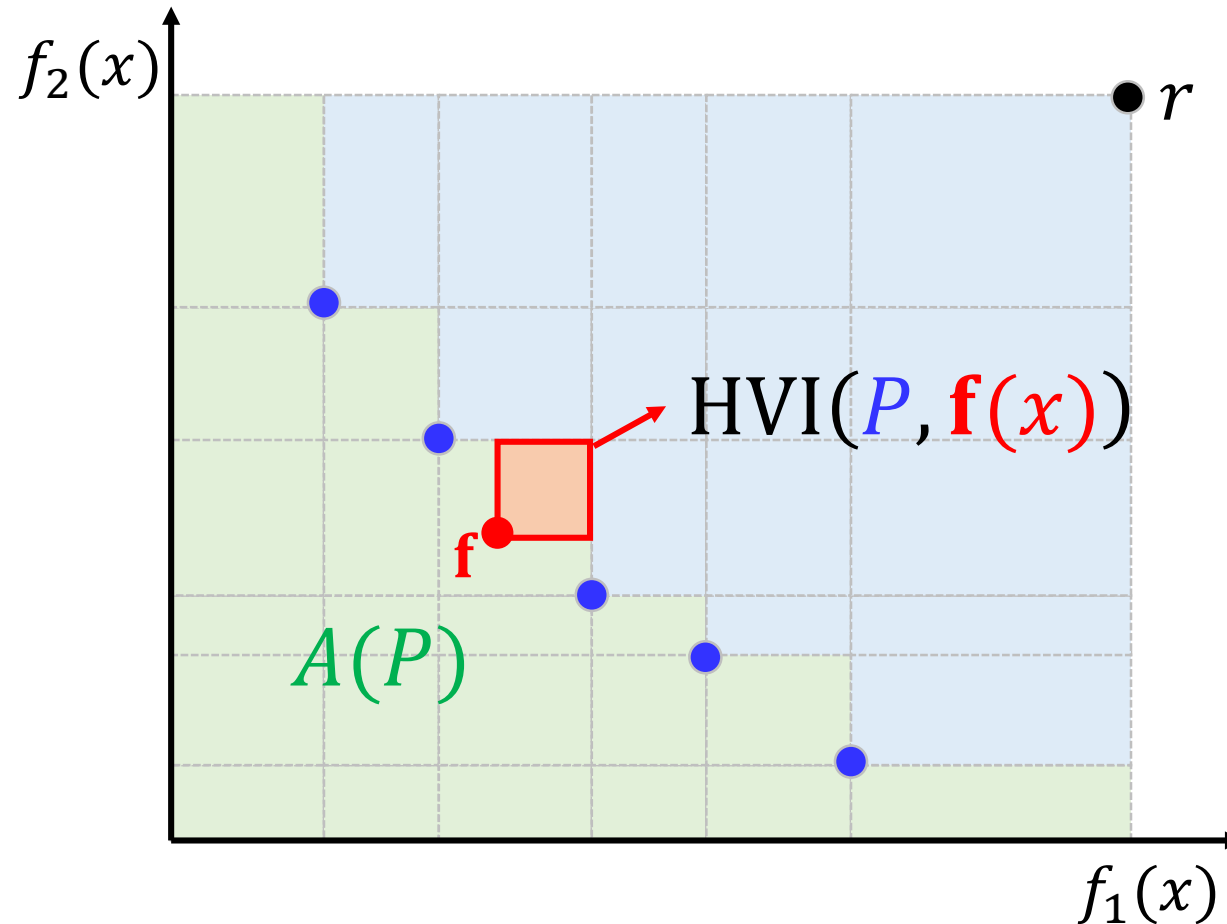
Defining acquisition function: Hypervolume improvement (HVI)

$$\text{HVI}(P, \mathbf{f}(x)) = \text{HV}(P, \mathbf{f}(x)) - \text{HV}(P)$$



Defining acquisition function: Hypervolume improvement (HVI)

$$\text{HVI}(P, \mathbf{f}(x)) = \text{HV}(P, \mathbf{f}(x)) - \text{HV}(P) \text{ for } \mathbf{f}(x) \in A(P)$$



$$\text{HVI}(P, \mathbf{f}(x)) \geq 0 \text{ for } \mathbf{f}(x) \in A(P), \text{ otherwise } \text{HVI}(P, \mathbf{f}(x)) = 0$$

Acquisition function for MOBO

- Probability of Hypervolume improvement (PHVI)
- Expected Hypervolume Improvement (EHVI)
- Hypervolume Probability Improvement (HVPI)

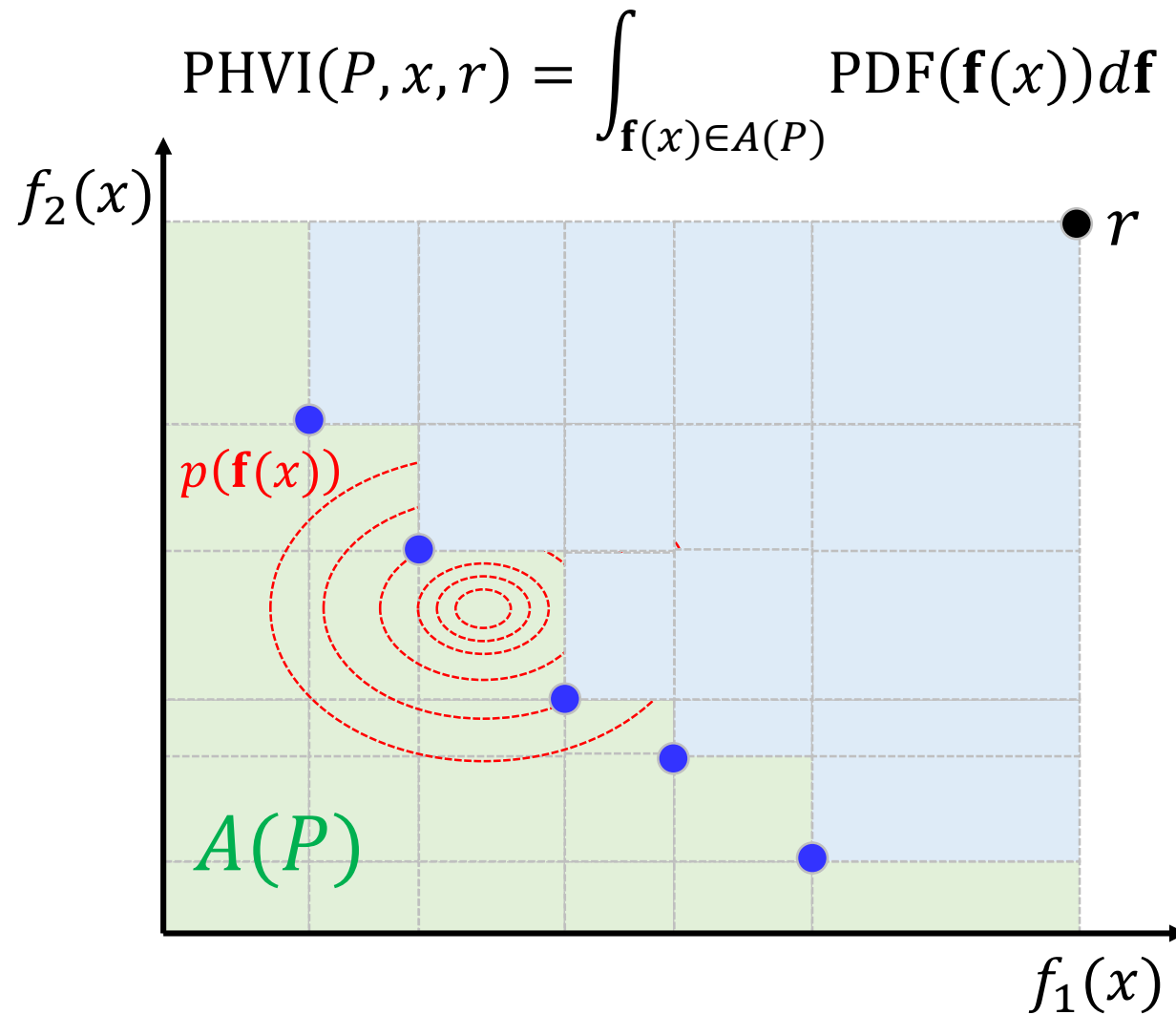
Probability of Hypervolume improvement function

- **Probability of Hypervolume improvement (PHVI)** can be computed by integrating the probability of the multi-dimensional function output $\mathbf{f}(x) = (f_1(x), \dots, f_m(x))$ over a non-dominated region $A(P)$ as

$$\text{PHVI}(P, x, r) = \int_{\mathbf{f} \in A(P)} p(\mathbf{f}(x)) d\mathbf{f}$$

- $\mathbf{f}(x) = (f_1(x), \dots, f_m(x))$ is m dimensional output vector
- $p(\mathbf{f}(x))$ is the probability density (PDF) of $\mathbf{f}(x)$
- $A(P)$ is the non-dominated region defined by the Pareto frontier P

Probability of Hypervolume improvement function



Probability of Hypervolume improvement function

- **Probability of Hypervolume improvement (PHVI)** can be computed by integrating the probability of the multi-dimensional function output $\mathbf{f}(x) = (f_1(x), \dots, f_m(x))$ over a non-dominated region $A(P)$ as

$$\text{PHVI}(P, x, r) = \int_{\mathbf{f} \in A(P)} p(\mathbf{f}(x)) d\mathbf{f}$$

- If we assume $f_j \sim \mathcal{GP}(m_j(\cdot), k(\cdot, \cdot))$, we can compute the posterior distribution on $f_j(x|\mathcal{D}) \sim \mathcal{N}(\mu_j(x), \sigma_j^2(x))$ for $j \in \{1, \dots, m\}$.
- Then we can compute $p(\mathbf{f}(x))$ and $\text{PHVI}(P, x, r)$ as

$$p(\mathbf{f}(x)) = \prod_{j=1}^m \phi_j(f_j(x))$$
$$\text{PHVI}(P, x, r) = \int_{\mathbf{f} \in A(P)} \prod_{j=1}^m \phi_j(f_j(x)) d\mathbf{f}$$

where $\mathbf{f}(x) = (f_1(x), \dots, f_m(x))$ can be also modeled as Multi-output GP as well

Expected Hypervolume Improvement (EHVI)

- **Expected Hypervolume Improvement (EHVI)** is a **generalized version of EI** for multi-objective function
- EHVI is computed as the expected value of the hypervolume improvement as

$$\begin{aligned}\text{EHVI}(P, x, r) &= \mathbb{E}[\max\{0, \text{HVI}(P, \mathbf{f}(x), r)\}] \\ &= \int_{\mathbf{f}(x) \in A(P)} \text{HVI}(P, \mathbf{f}(x), r) p(\mathbf{f}(x)) d\mathbf{f}\end{aligned}$$

- ✓ $\text{HVI}(P, \mathbf{f}(x), r)$ is defined **over non-dominated region $A(P)$** , therefore $\text{HVI}(P, \mathbf{f}(x), r) \geq 0$

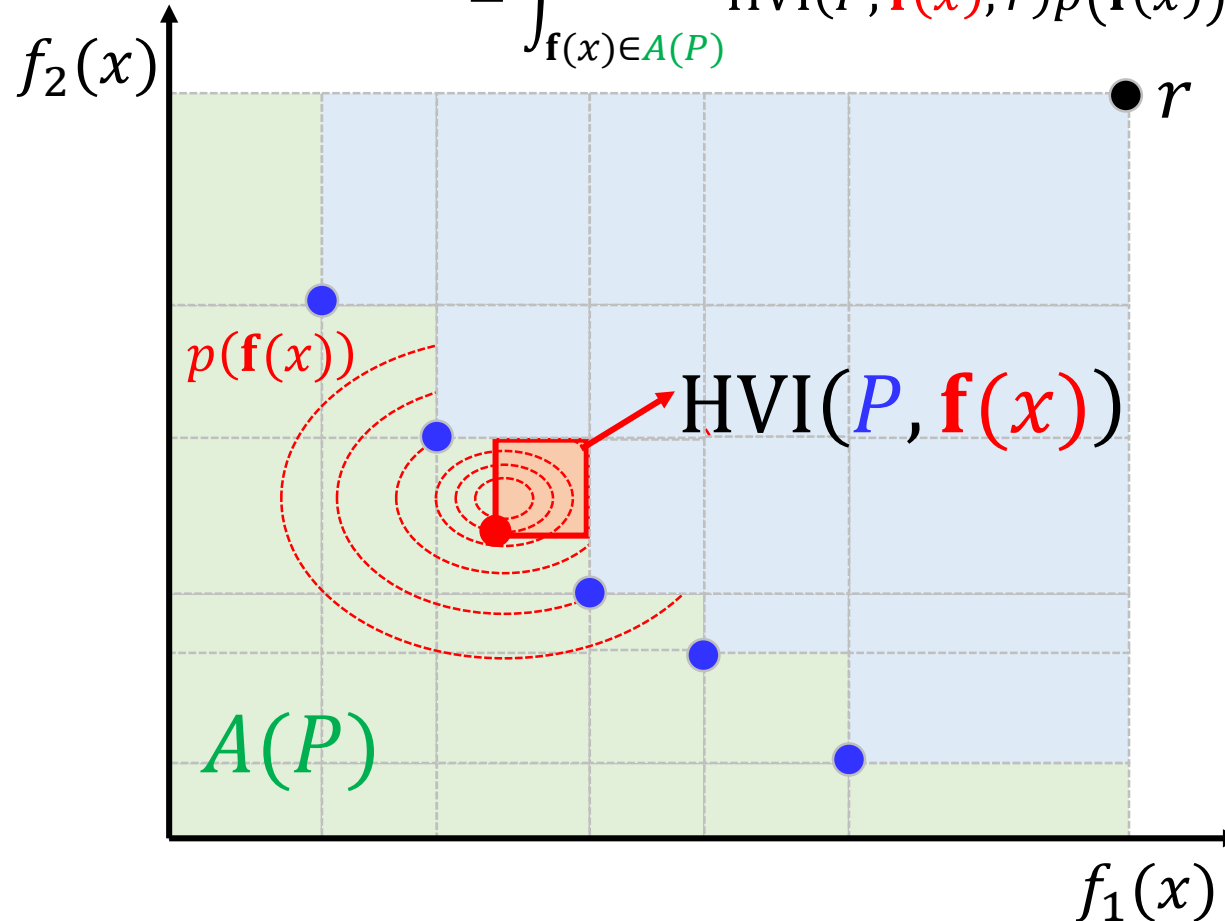
$$\max\{0, \text{HVI}(P, \mathbf{f}(x), r)\} = 0, \text{HVI}(P, \mathbf{f}(x), r)$$

- ✓ EHVI can be computed using Monte Carlo integral for high dimensional output
- ✓ For two dimensional outputs, it can be computed analytically [K. Yang et al., 2019]

Expected Hypervolume Improvement (EHVI)

- EHVI is computed as the expected value of the hypervolume improvement as

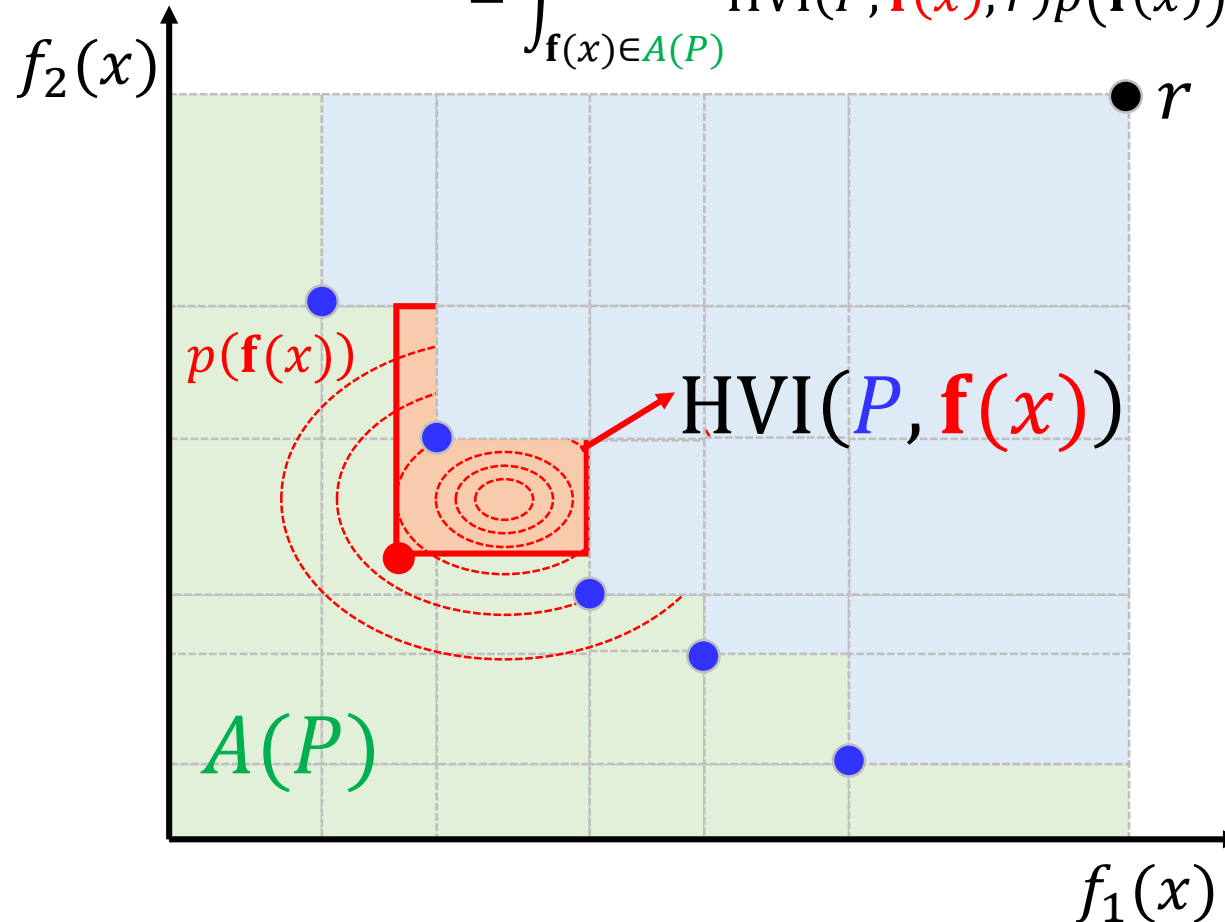
$$\begin{aligned}\text{EHVI}(P, x, r) &= \mathbb{E}[\max\{0, \text{HVI}(P, \mathbf{f}(x), r)\}] \\ &= \int_{\mathbf{f}(x) \in A(P)} \text{HVI}(P, \mathbf{f}(x), r) p(\mathbf{f}(x)) d\mathbf{f}\end{aligned}$$



Expected Hypervolume Improvement (EHVI)

- EHVI is computed as the expected value of the hypervolume improvement as

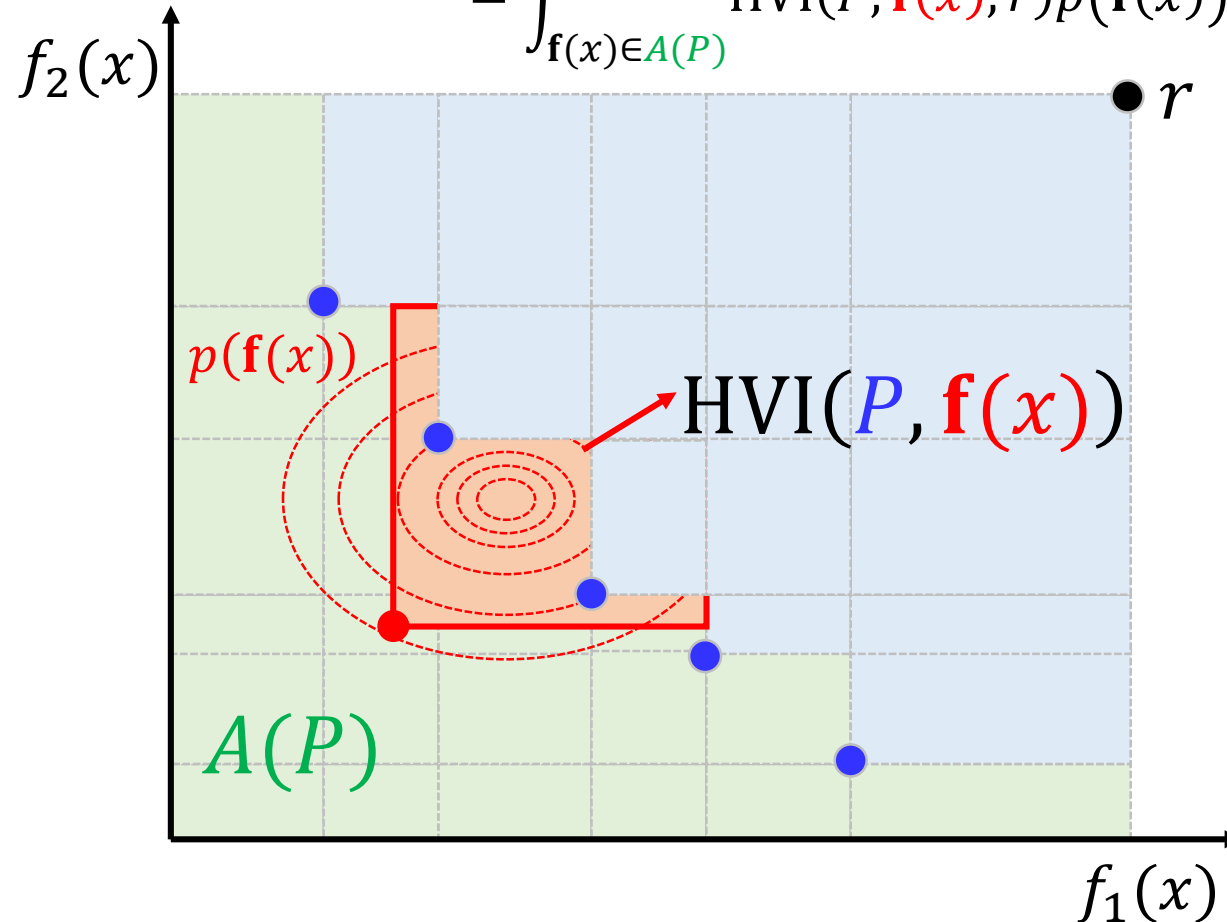
$$\begin{aligned}\text{EHVI}(P, x, r) &= \mathbb{E}[\max\{0, \text{HVI}(P, \mathbf{f}(x), r)\}] \\ &= \int_{\mathbf{f}(x) \in A(P)} \text{HVI}(P, \mathbf{f}(x), r) p(\mathbf{f}(x)) d\mathbf{f}\end{aligned}$$



Expected Hypervolume Improvement (EHVI)

- EHVI is computed as the expected value of the hypervolume improvement as

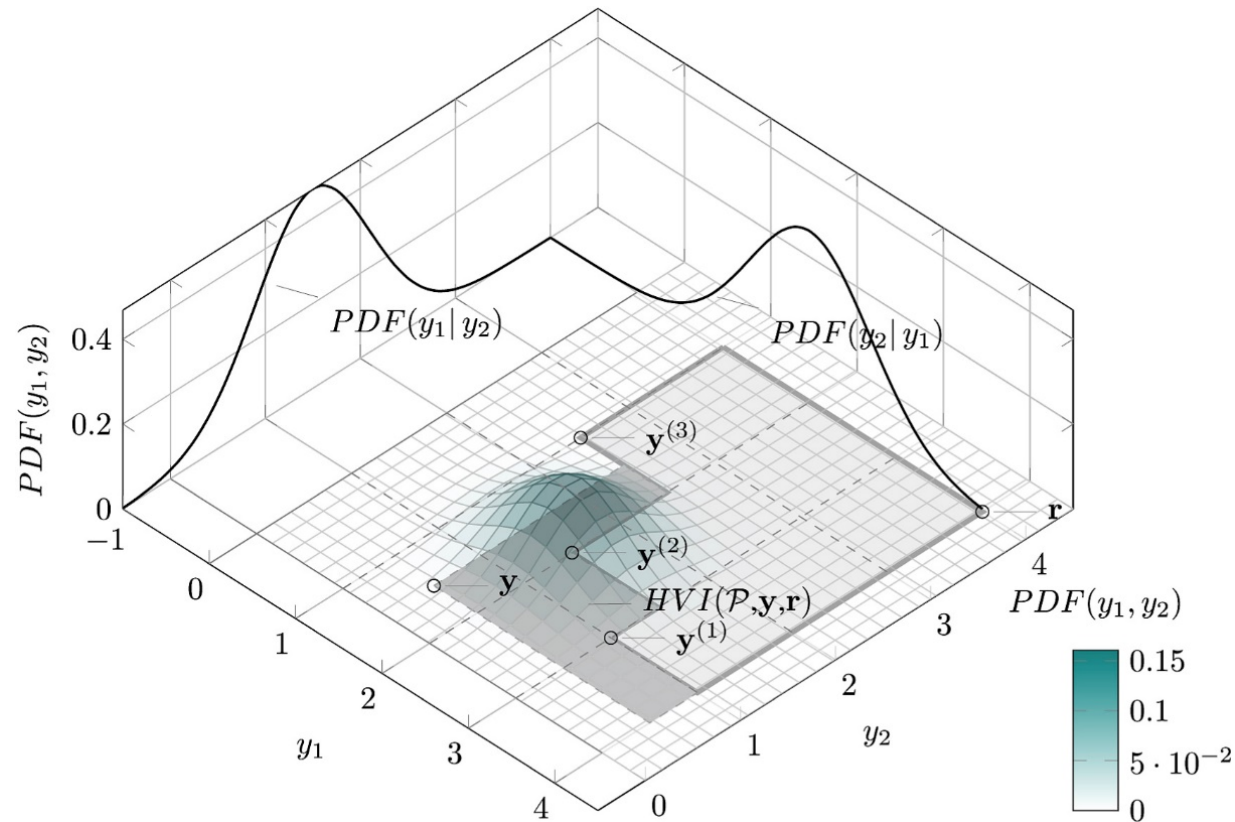
$$\begin{aligned}\text{EHVI}(P, x, r) &= \mathbb{E}[\max\{0, \text{HVI}(P, \mathbf{f}(x), r)\}] \\ &= \int_{\mathbf{f}(x) \in A(P)} \text{HVI}(P, \mathbf{f}(x), r) p(\mathbf{f}(x)) d\mathbf{f}\end{aligned}$$



Expected Hypervolume Improvement (EHVI)

- EHVI is computed as the expected value of the hypervolume improvement as

$$\begin{aligned}\text{EHVI}(P, x, r) &= \mathbb{E}[\max\{0, \text{HVI}(P, \mathbf{f}(x), r)\}] \\ &= \int_{\mathbf{f}(x) \in A(P)} \text{HVI}(P, \mathbf{f}(x), r) p(\mathbf{f}(x)) d\mathbf{f}\end{aligned}$$

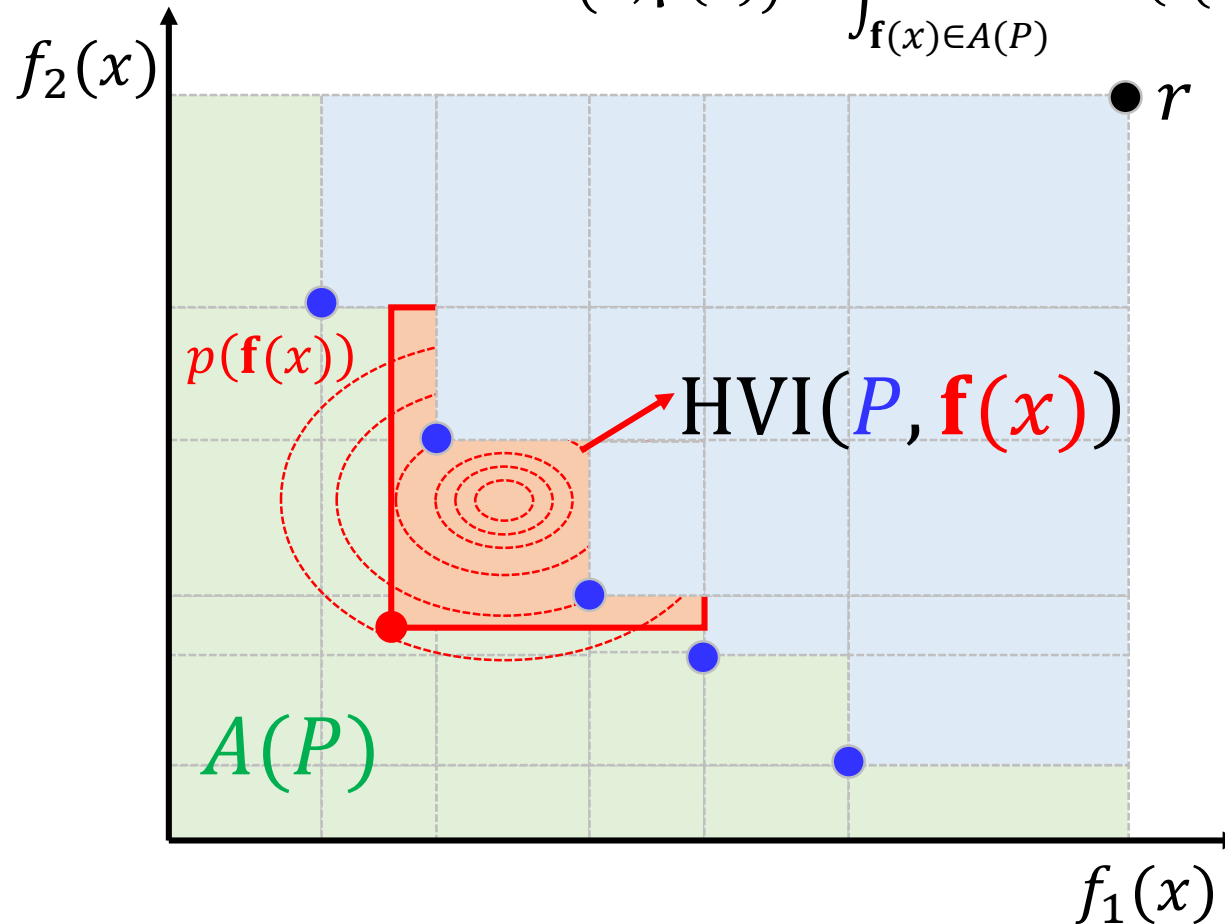


(Yang et al., “Multi-Objective Bayesian Global Optimization using expected hyper volume improvement gradient”)

Hypervolume Probability Improvement (HVPI)

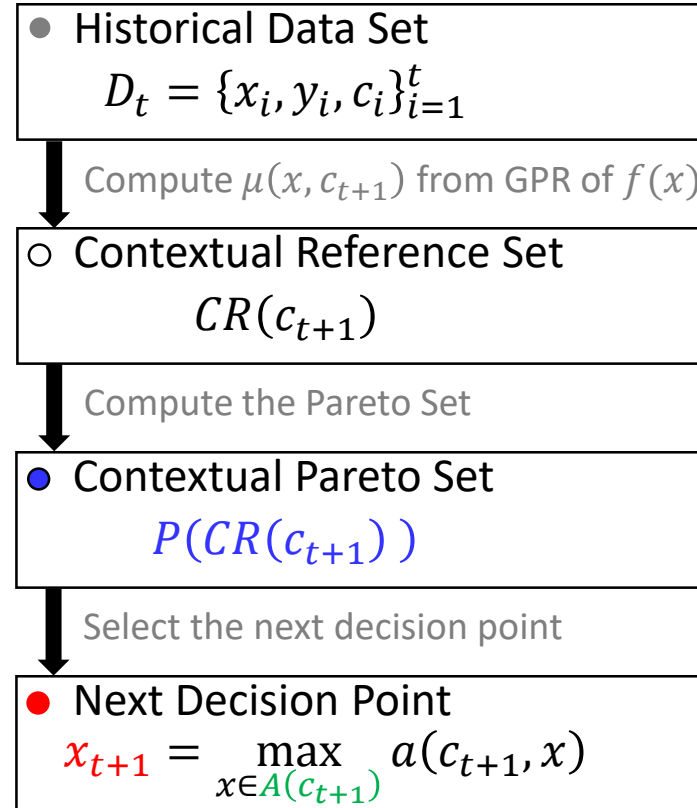
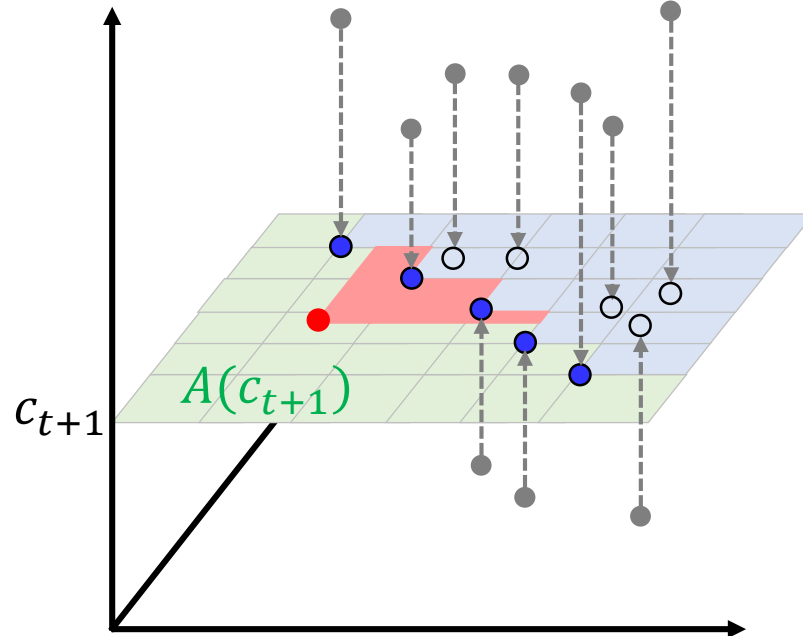
- **EHVI is difficult to compute.** Thus, as an approximate, HVPI is defined as the product between HVI at mean and PHVI

$$\begin{aligned}\text{HVPI}(P, x) &= \text{HVI}(P, \boldsymbol{\mu}(x)) \times \text{PHVI}(P, x) \\ &= \text{HVI}(P, \boldsymbol{\mu}(x)) \times \int_{\mathbf{f}(x) \in A(P)} \text{PDF}(\mathbf{f}(x)) d\mathbf{f}\end{aligned}$$



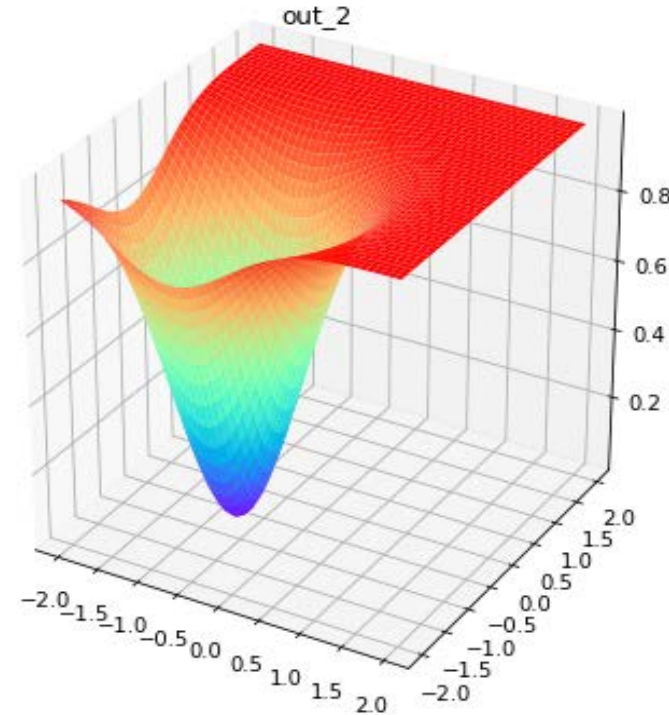
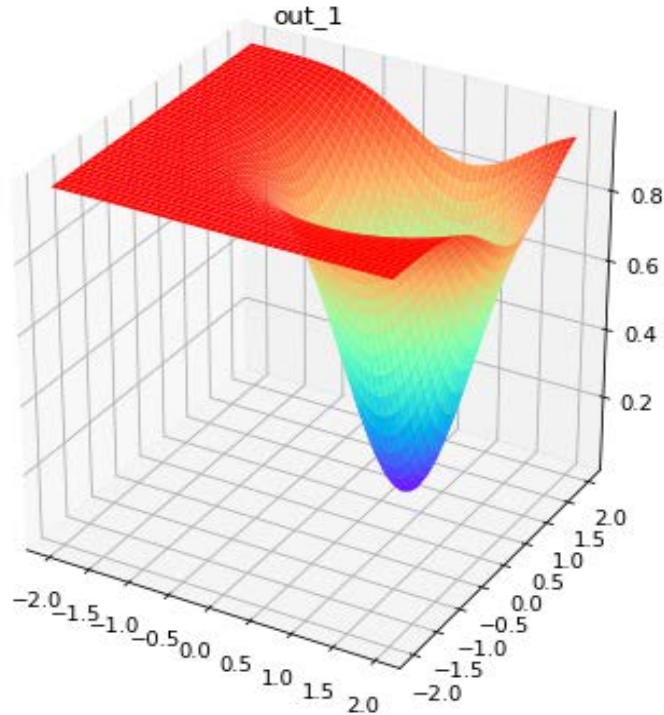
Contextual Multi-objective Bayesian optimization (CMOBO)

- Contextual Multi-objective Bayesian optimization
= Contextual Bayesian optimization + Multi-output Bayesian optimization



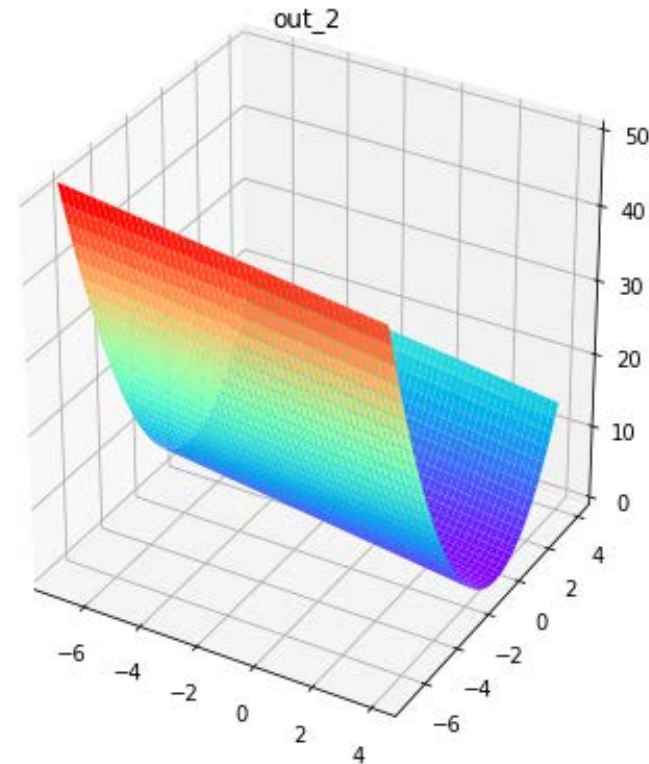
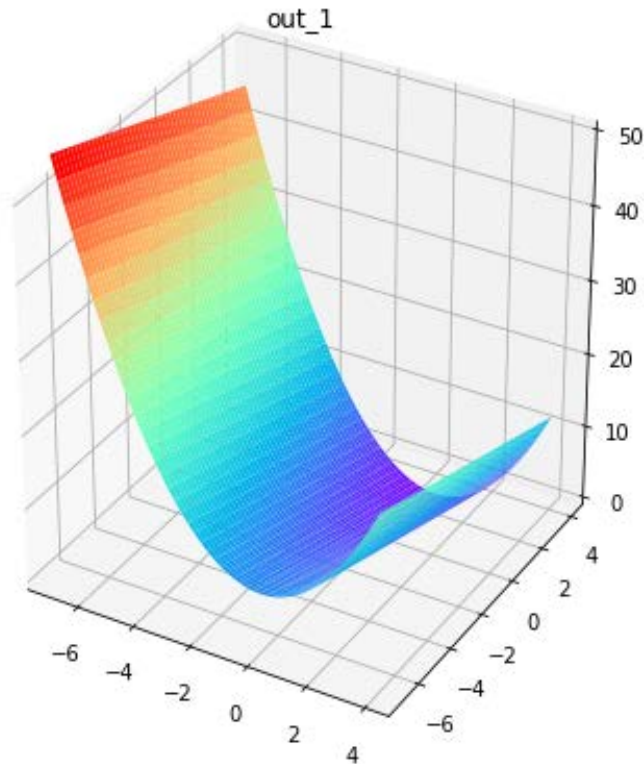
GPflowOpt examples: 4. Multi-objective Bayesian optimization

Optimization of multi objective model using HVI acquisition function



GPflowOpt examples: 4. Multi-objective Bayesian optimization

We will optimize multi-objective function shown as:



Bayesian optimization challenges

Bayesian optimization challenges

- Input dimension scalability
- The limited number of realization (number of evaluations) due to high computational cost
- The frequency of parameter update
- **Recommended solutions: combines with deep learning**

Input dimension scalability

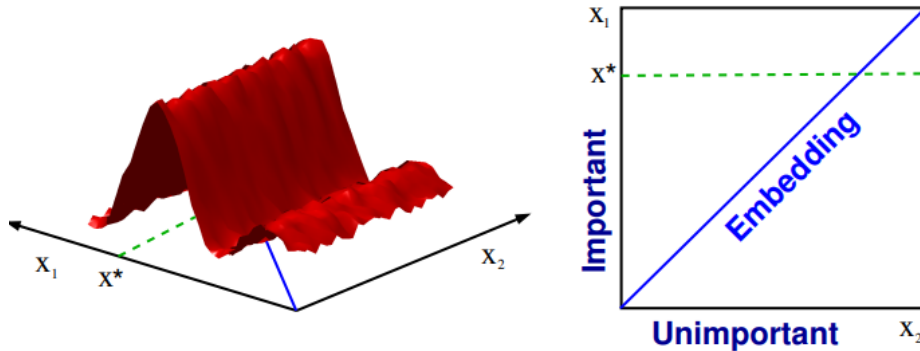
- Bayesian optimization (BO) is a powerful approach for seeking the global optimum of expensive black-box functions and has proven successful for fine tuning hyperparameters of machine learning models.
- The standard BO routine consists of two key steps:
 - 1) estimating the black-box function from data through a probabilistic surrogate model, usually a Gaussian process (GP), referred to as the response surface;
 - 2) maximizing an acquisition function that computes a score that trades off exploration and exploitation according to uncertainty and optimality of the response surface.

Input dimension scalability

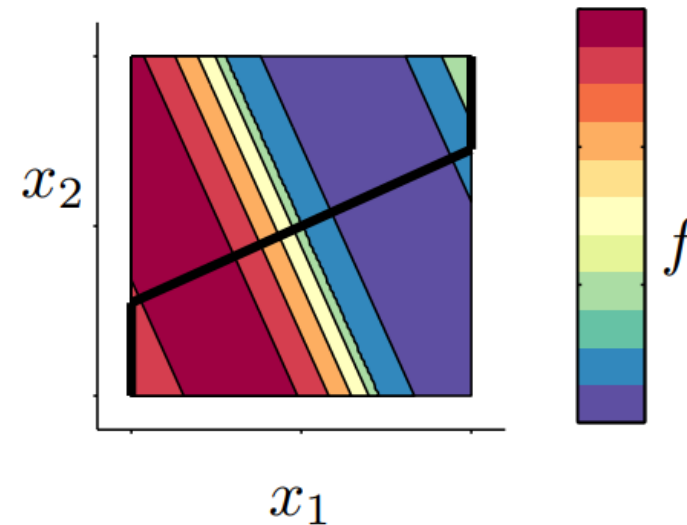
- As the dimensionality of the input space increases, these two steps become challenging.
 - 1) The sample complexity to ensure good coverage of inputs for learning the response surface is **exponential** in the number of dimensions
 - 2) With only a small evaluation budget the learned response surface and the resulting acquisition function are characterized by vast flat regions interspersed with highly non convex landscapes. This renders the maximization of the acquisition inherently hard.
- In practice, BO is typically limited to optimizing 10–20 parameters.
- To scale BO to high dimensions,
 - Some research makes structural assumptions on the decomposition of the objective
 - Some research exploits the intrinsic lower dimensionality of the problem, e.g. by using linear projections.

Input dimension scalability

- High-dimensional data often possesses a lower intrinsic dimensionality, which can also be exploited for optimization.
- A feature mapping can then be used to map the original D -dimensional data onto a $d \ll D$ dimensional manifold.



Wang et al., 2014, "Bayesian Optimization in High Dimensions via Random Embeddings"



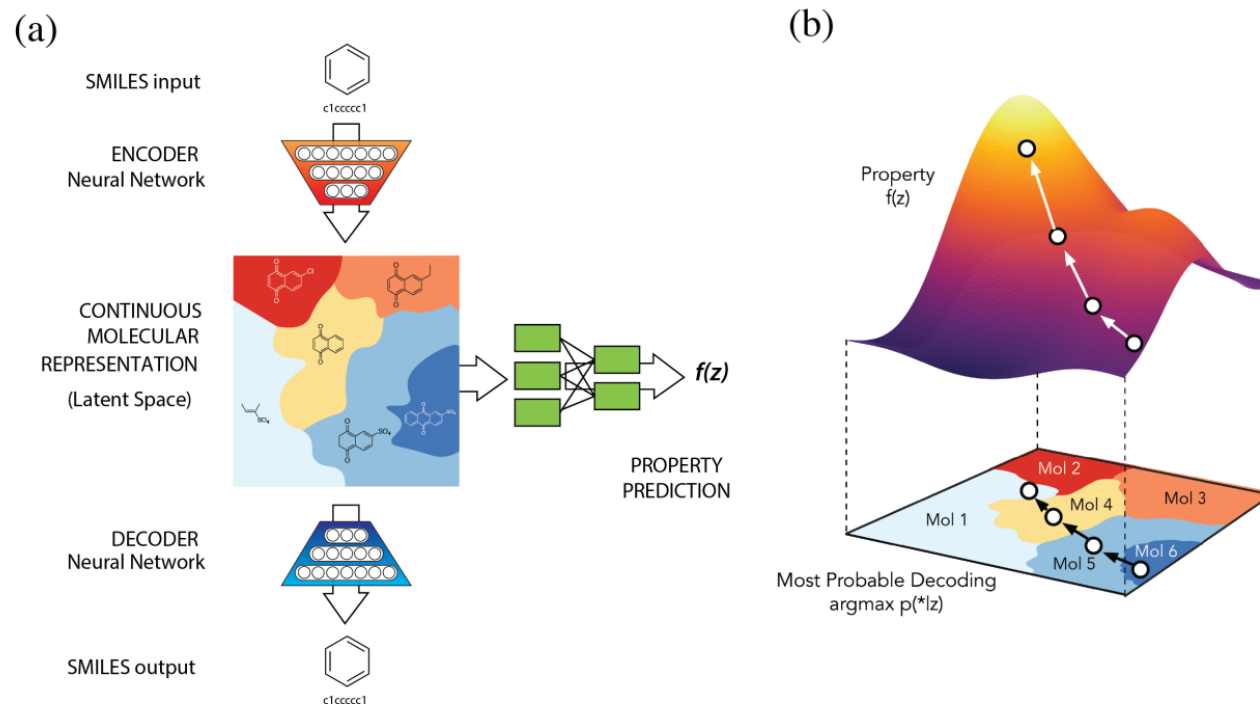
Garnett et al., 2013, "Active Learning of Linear Embeddings for Gaussian Processes"

- While these methods perform well in practice they are restricted to linear subspaces of the original domain.

Input dimension scalability

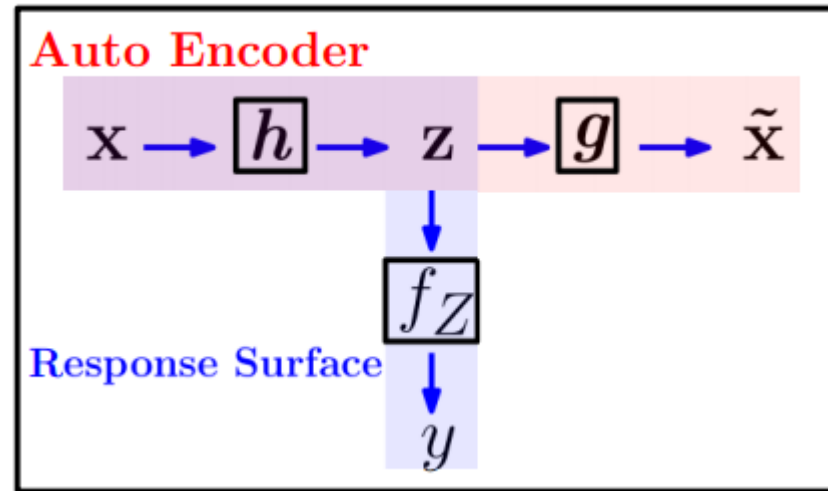
- **Bayesian optimization with nonlinear embeddings**
- A low-dimensional data representation is learned with variational autoencoder (VAEs) and the optimization can be conducted in the latent space.

VAE-BO



Input dimension scalability

- **Bayesian optimization with nonlinear embeddings**
- A low-dimensional data representation is learned with variational autoencoder (VAEs) and the optimization can be conducted in the latent space.



- Learns a nonlinear feature mapping $\mathbf{h}: R^D \rightarrow R^d$ to reduce the dimensionality of the inputs.
- Learn reconstruction mapping $\mathbf{g}: R^d \rightarrow R^D$ based on GPs to evaluate the true objective function.

Data point number scalability

- Bayesian optimization is typically limited to a few thousand evaluations.
 - Yet, reliable search and estimation for complex functions in very high-dimensional spaces may well require more evaluations.
 - With the increasing availability of parallel computing resources, large number of function evaluations are possible if the underlying approach can leverage the parallelism.
 - Comparing to the millions of evaluations possible (and needed) with local methods like stochastic gradient descent, **the scalability of global Bayesian optimization leaves large room for desirable progress.**

Data point number scalability

- Snoek et al., 2015, “Scalable Bayesian optimization using deep neural networks”

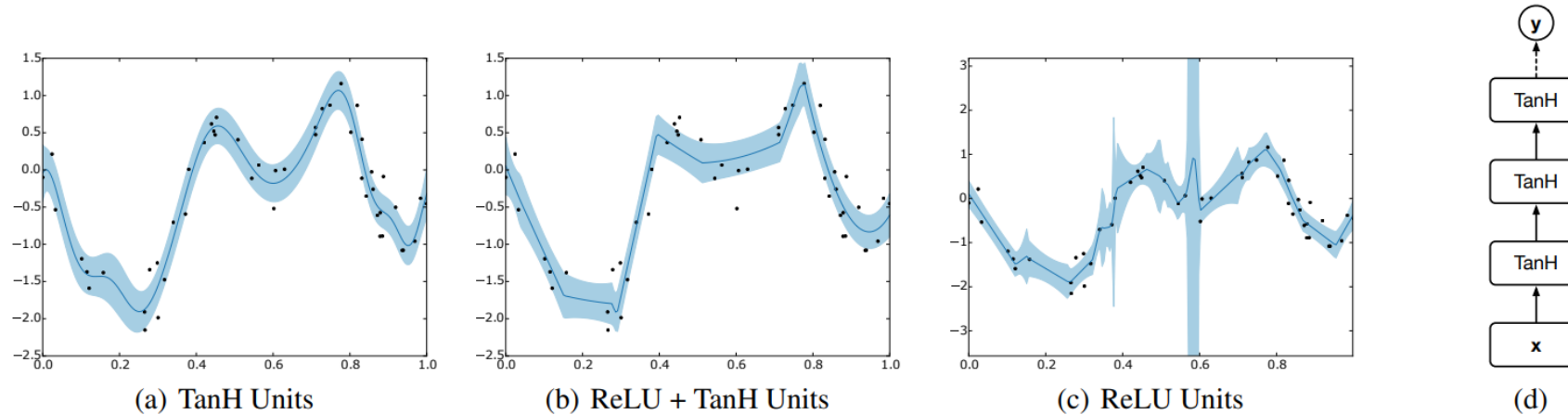


Figure 2. A comparison of the predictive mean and uncertainty learned by the model when using 2(a) only tanh, 2(c) only rectified linear (ReLU) activation functions or 2(b) ReLU's but a tanh on the last hidden layer. The shaded regions correspond to standard deviation envelopes around the mean. The choice of activation function significantly modifies the basis functions learned by the model. Although the ReLU, which is the standard for deep neural networks, is highly flexible, we found that its unbounded activation can lead to extremely large uncertainty estimates. Subfigure 2(d) illustrates the overall architecture of the DNGO model. Dashed lines correspond to weights that are marginalized.

- Use of neural networks as an alternative to GPs to model distributions over functions.
 - adaptive basis function regression with a neural network as the parametric form performs competitively with state-of-the-art GP-based approaches
 - scales linearly with the number of data rather than cubically.

Data point number scalability

- McIntire et al., 2016, “Sparse Gaussian Processes for Bayesian Optimization ”
 - Use online Sparse GP to update the kernel matrix for every new data point as

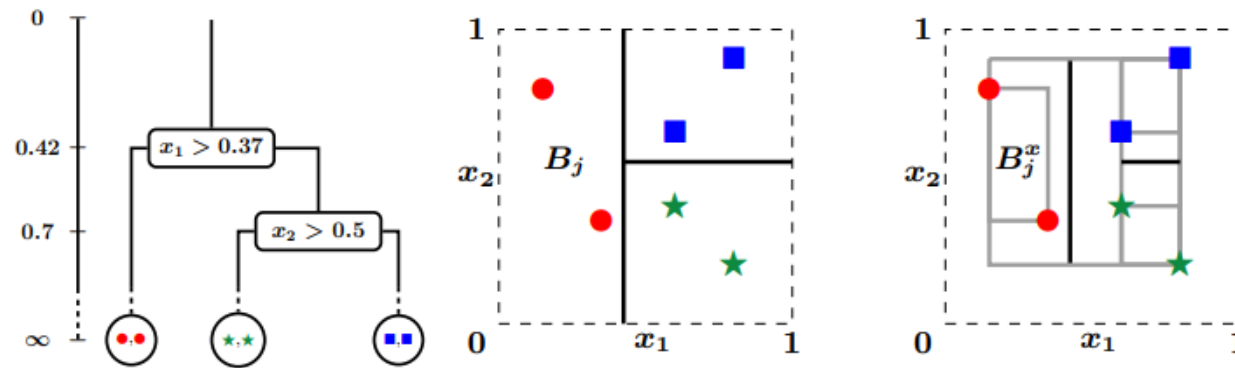
$$\begin{aligned} K_{\mathcal{I}} = Q^{-1} &\Rightarrow K_{\mathcal{I}}^{(r)} = (Q^{(r)} - \frac{1}{q^*} Q^* Q^{*\top})^{-1} \\ &\Rightarrow \hat{Q} = Q^{(r)} - \frac{1}{q^*} Q^* Q^{*\top} . \end{aligned}$$

Algorithm 1 Online GP Update

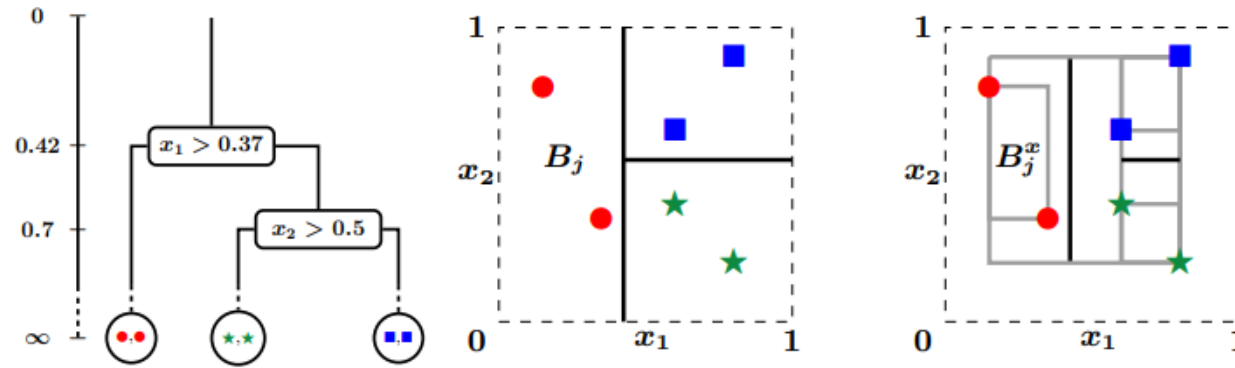
- 1: **Input:** data point x , output y
 - 2: **Persistent:** Inducing variables $X_{\mathcal{I}}$, GP model parameters
 - 3: Assess novelty γ of point x .
 - 4: **if** $\gamma < \epsilon_{tol}$ **then**
 - 5: Perform sparse update without expanding the model.
 - 6: **else**
 - 7: Perform full update, adding x to $X_{\mathcal{I}}$ and extending GP model parameters.
 - 8: **end if**
 - 9: **if** Model size exceeds m **then**
 - 10: Score inducing inputs $X_{\mathcal{I}}$ on impact of removal.
 - 11: Remove the lowest-scoring element of $X_{\mathcal{I}}$; update the GP model to minimize the impact of removal.
 - 12: **end if**
-

Data point number scalability

- Wang et al., 2015, “Batched Large-scale Bayesian Optimization in High-dimensional Spaces”
 - Use ensemble Bayesian optimization (EBO) to address three current challenges in BO simultaneously:
 - ✓ large-scale observations;
 - ✓ high dimensional input spaces; and
 - ✓ selections of batch queries that balance quality and diversity.
 - The key idea of EBO is to operate on an ensemble of additive Gaussian process models, each of which possesses a randomized strategy to divide and conquer



Data point number scalability



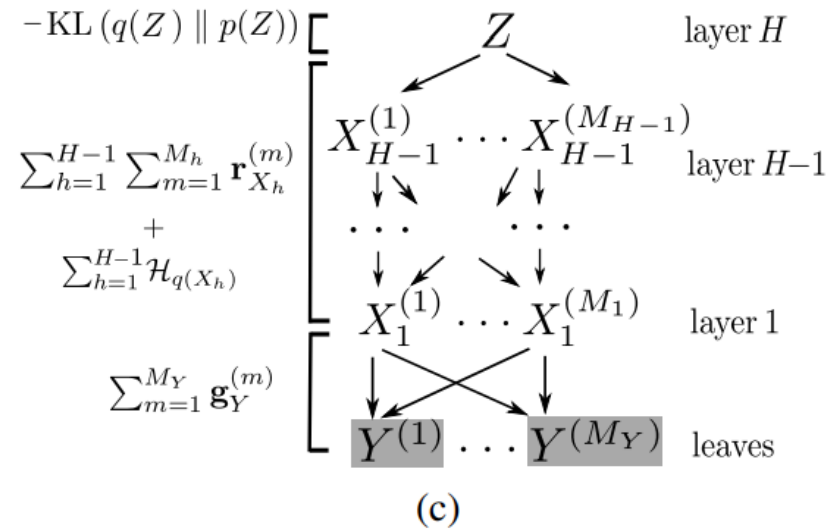
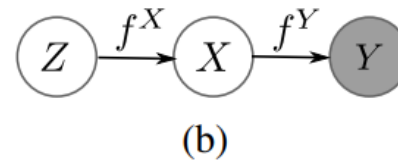
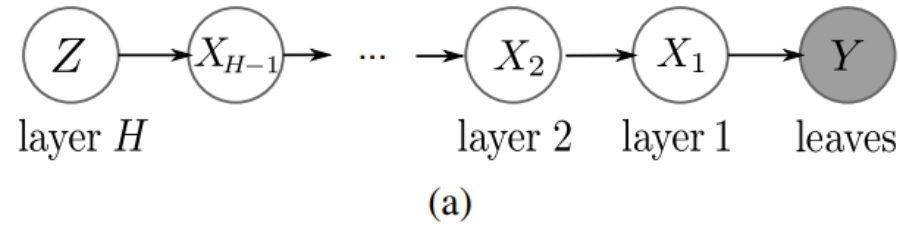
- Ensemble Bayesian Optimization (EBO) Procedure
 - Partitioning the input space via a Mondrian process
 - EBO uses a (stochastic) series of Mondrian trees to partition the input space
 - Learning a local TileGP via Gibbs sampling
 - Fit local GPs to the segmented data set
 - Acquisition functions and filtering
 - Select the next sampling points and filter good samples with good outcomes
 - learn the kernel parameters of a GP locally, and aggregate these parameters.

The frequency of parameter update

- How often we need to update the parameters for the GPR surrogate model?
 - If the new sample does not provide any information to the current surrogate model, the new sample does not have to be included to the training data set, thus new parameters optimization is not required
 - Measuring the magnitude of information is key issue
 - GPR hyperparameters can also be optimized in a form of online gradient ascent

Gaussian process + Deep learning

- Damianou et al., 2013, “Deep Gaussian Processes ”



Gaussian process + Deep learning

- Dai et al., 2016, “VARIATIONAL AUTO-ENCODED DEEP GAUSSIAN PROCESSES”

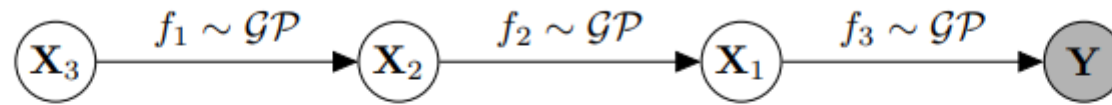
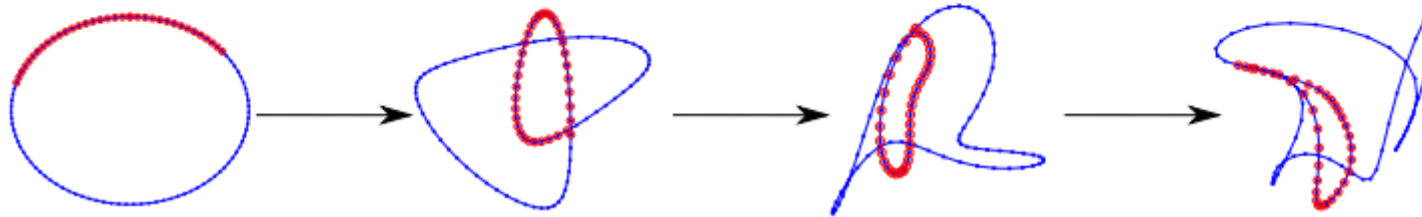
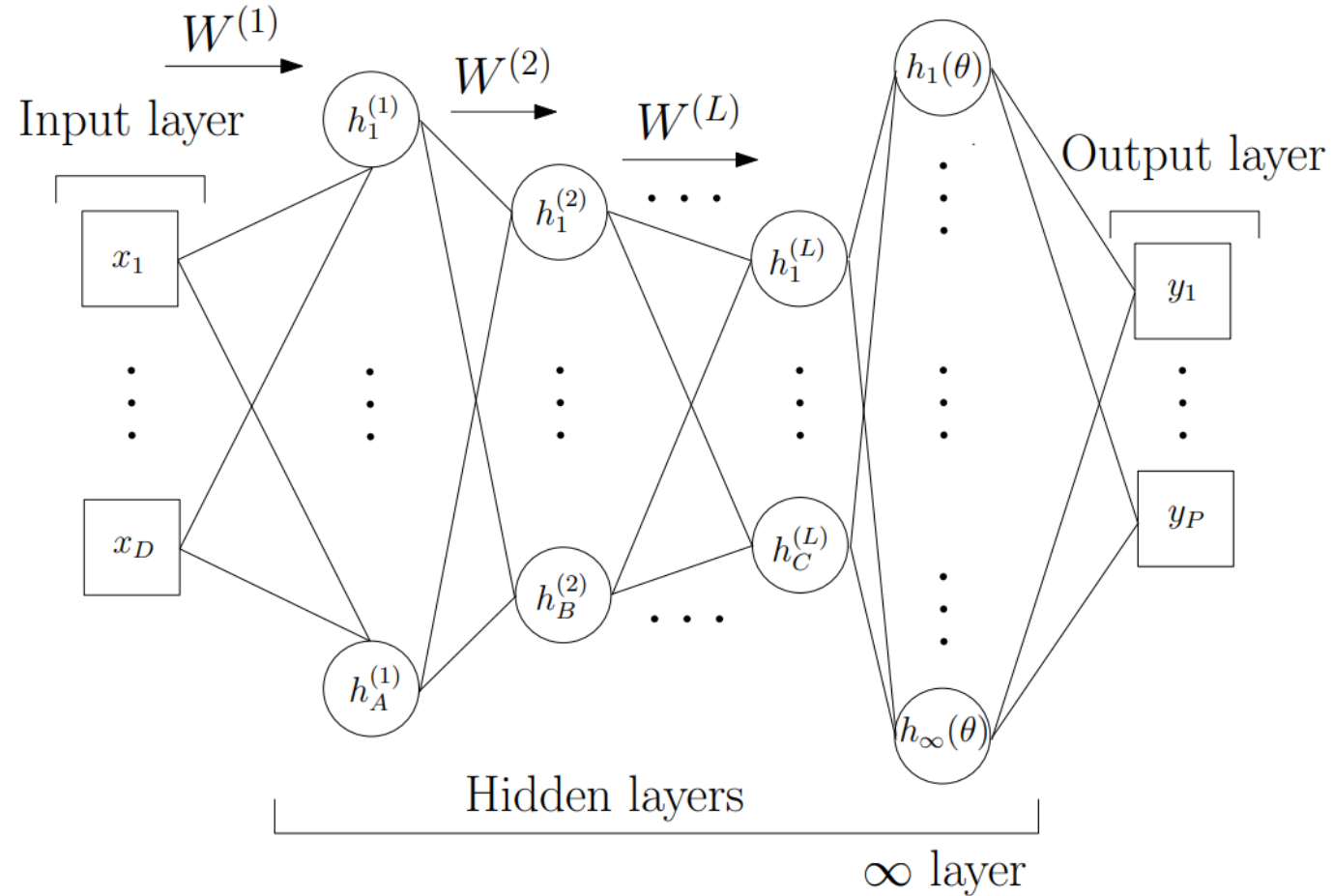


Figure 1: A deep Gaussian process with two hidden layers.



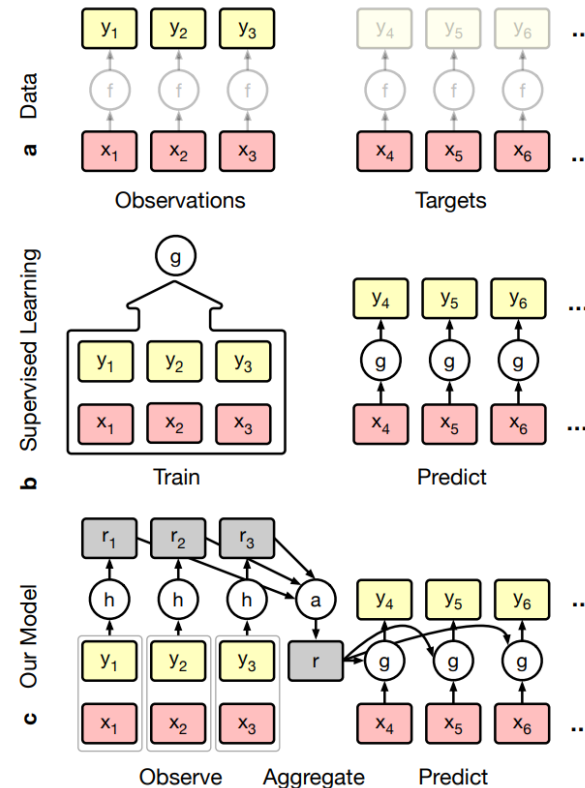
Gaussian process + Deep learning

- Wilson et al., 2016, “Deep Kernel Learning”



Gaussian process + Deep learning

- Garnelo et al., 2019, “Conditional Neural Process”



- Conditional Neural Process (CNPs) are inspired by the flexibility of stochastic processes such as GPs, but are structured as neural networks and trained via gradient descent.
- CNPs make accurate predictions after observing only a handful of training data points, yet scale to complex functions and large datasets.